

Keyestudio 4DOF Robot Mechanical Arm Kit for Arduino DIY



CONTENT GUIDE

1. Kit Overview.....	1
2. Kit Features.....	1
3. Part List.....	3
4. Assembly Guide.....	12
Step1: Begin with the Baseplate Assembly.....	12
Step2: Assemble Arm Middle Parts.....	31
Step3: Assemble the Claw Servo Plate.....	68
Step4: Final Assembly.....	83
Step5: Assemble the Joystick Control Plate.....	91
5. Robot Arm Projects.....	97
Project 1: Getting Started with ARDUINO.....	97
1) UNO R3 Control Board.....	97
2) Example Use of ARDUINO IDE.....	126
Project 2: Joint Rotation and Pin Control.....	136
1) Joint Rotation and Servo Angle Settings.....	136
2) Pin Control.....	137
Project 3: JoyStick Controlled Robot Arm.....	139
1) Servo Control.....	139
2) Read the JoyStick Value.....	148
3) Dual-JoyStick Controlling.....	152
4) Add Memory Function.....	162

Project 4: Bluetooth Controlled Robot Arm.....	207
1) Principle of Bluetooth Control.....	207
2) Bluetooth Control Key Test.....	209
3) Bluetooth Controlling Arm.....	214
Project 5: PS2 Controlled Robot Arm (Extension).....	222
1) PS2 Joypad Key Test.....	222
2) PS2 Joypad Control.....	231
3) PS2 Controlling Posture Memory.....	243
4) PS2 Controlling Posture Memory And Loop.....	260
6. More Resources:.....	278

1. Kit Overview

How to DIY a robotic arm to complete multiple movements? Cool. With this keystudio robotic arm kit, you are able to DIY your own controllable mechanical arm using ARDUINO microcontroller. It uses Keystudio UNO R3 (Black) Main Control Board and 2 JoyStick modules to control the angle degree of 4 servos.

When DIY this 4DOF robot arm kit, you could get everything needed for arm installation and debugging.

There are 3 controlling methods as follows:

- 1) Controlling through Wired JoyStick (included in the kit);
- 2) Phone Bluetooth Controlling (note: HC-06 Bluetooth module Not Included, only provide the test code for Bluetooth and APP for Android phone);
- 3) Wireless PS2 JoyStick Control (PS2 JoyStick module Not Included, we only provide the test code.)

You are able to get all related information in the Arm kit.

Take your brain on an inspiring journey through the world of programming. Get started now!

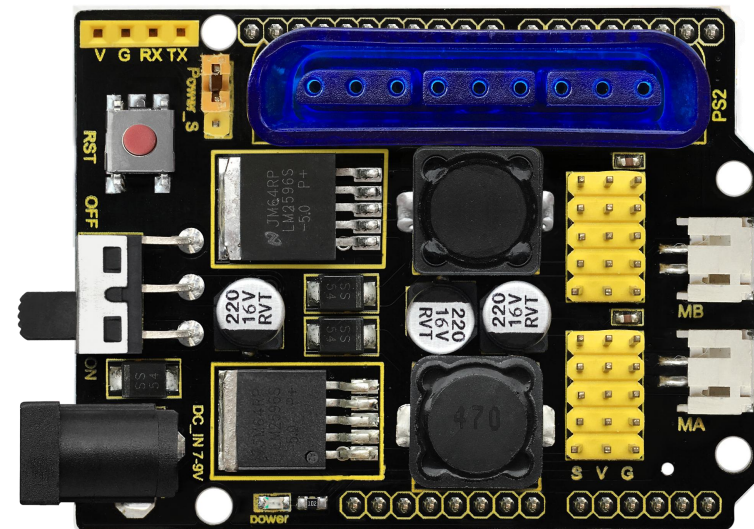
2. Kit Features

You can check out these features:

- Detailed installation instructions
- Detailed debugging methods, starting Arduino from entry.
- Three controlling methods: Wired JoyStick Control; Phone Bluetooth Control; Wireless PS2 JoyStick Control.

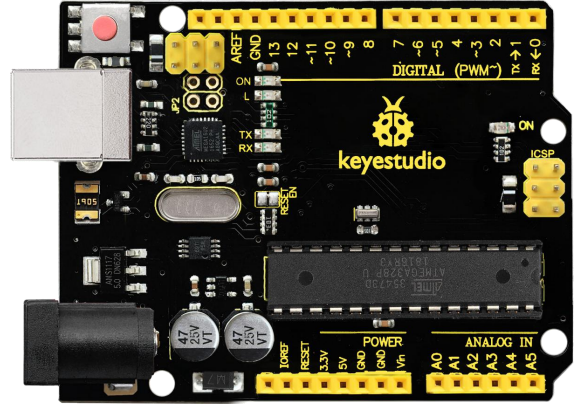
The parameters of keystudio TB6612FNG motor/servo drive expansion board are as follows:

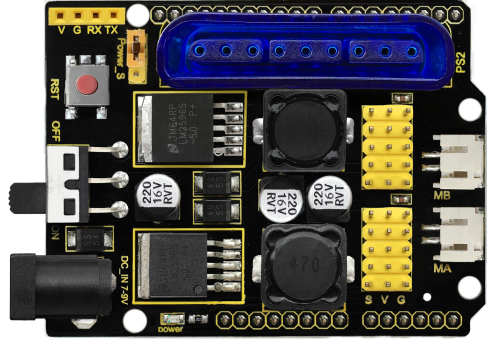
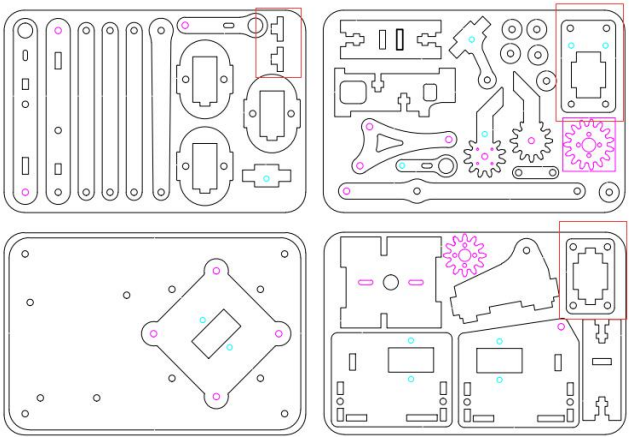
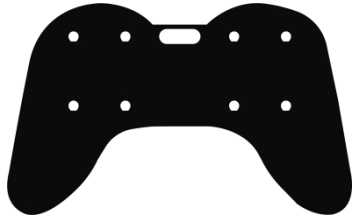
- VIN voltage: VIN = DC 7-15V
- VIN current: 5A
- Two-way 5V output: 5V/3A
- TB6612FNG: VIN input DC 7-15V; average drive current 1.2A; peak current 3.2A
- PS2 interface: compatible with Sony PS2 receiver, can be plugged directly into the expansion board.
- Dimensions: 73*53.34mm

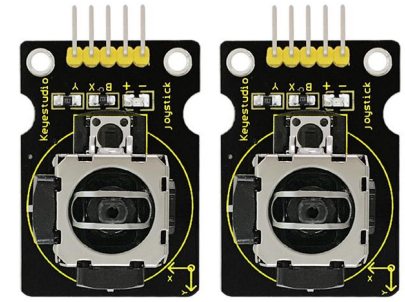


3. Part List

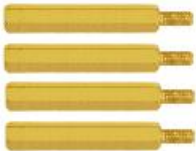
You can see a pretty beautiful packaging box for the arm kit, and inside the packaging you will find all the parts and screws listed below.

No.	Item	Quantity	Picture
1	Keyestudio UNO R3 (Black) Main Control Board	1	

2	keyestudio TB6612FNG motor/servo drive shield	1	
3	MeArm T=3MM Black Acrylic panels	1	
4	MeArm T=3MM Black Acrylic handle	1	

5	MeArm Black ABS Cylindrical holder Diameter 42mm	1	
6	Black 180° Servo	4	
7	Keyestudio Bluetooth Module-(HC-06)	1	
8	Keyestudio Joystick Module	2	

9	3D PS2 Joystick cap	2	
10	Yellow-black handle 3*40MM Phillips Screwdriver	1	
11	Galvanized M2+M3 2mm thin double-headed small wrench	1	
12	M3*6MM round-head cross screw	12	
13	M3*10MM round-head cross screw	22	

14	M3*14MM flat-head screw	2	
15	M3*12MM round-head cross screw	12	
16	M3*24+6MM copper pillar	4	
17	M3*6mm+6mm copper pillar	10	
18	M3 Stainless steel Hex Nut	22	

19	M3 hexagon Lock Nut	24	
20	M1.2x5MM Phillips tapping screw	8	
21	M2x5MM Phillips tapping screw	10	
22	M3 304 Stainless steel flat washer 3*7*0.5MM	10	

23	M2x8MM Phillips tapping screw	2	
24	M3*16MM flat-head screw	2	
25	Male to female 10CM jumper wire	4	
26	female to female 50CM jumper wire	10	

27	Black 3*100MM cable ties	7	
28	18650 dual-cell battery case with plug lead 15CM	1	

Note: Peel the plastic film off the board first when installing the smart car. To be honest, we never intend to send wood to you.

4. Assembly Guide

Follow the assembly steps below to build your own robot arm, believe you will be full of delight to experience the robot arm DIY. If still confused, you can refer to the assembly video.

Step1: Begin with the Baseplate Assembly

(1) Firstly, you should prepare the components as follows:

- M3*30+5MM single-pass copper pillar *4
- M3*6mm+6mm single-pass copper pillar *4
- M3 Hex Nut *8
- Black Acrylic plate *1

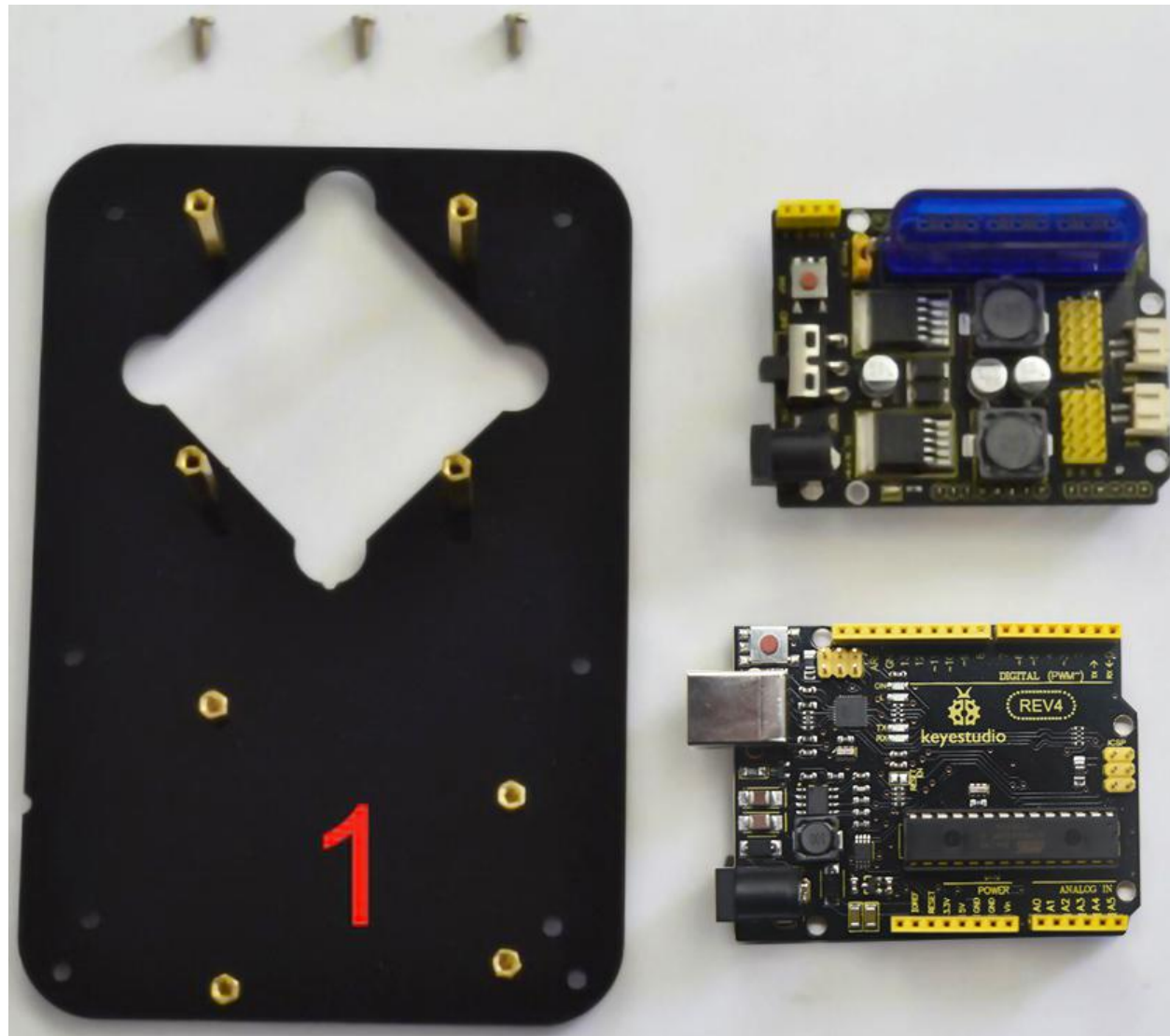


Then, screw the copper pillars with M3 hex nuts on the black Acrylic baseplate.



(2) Then install the control board, and prepare the components as follows:

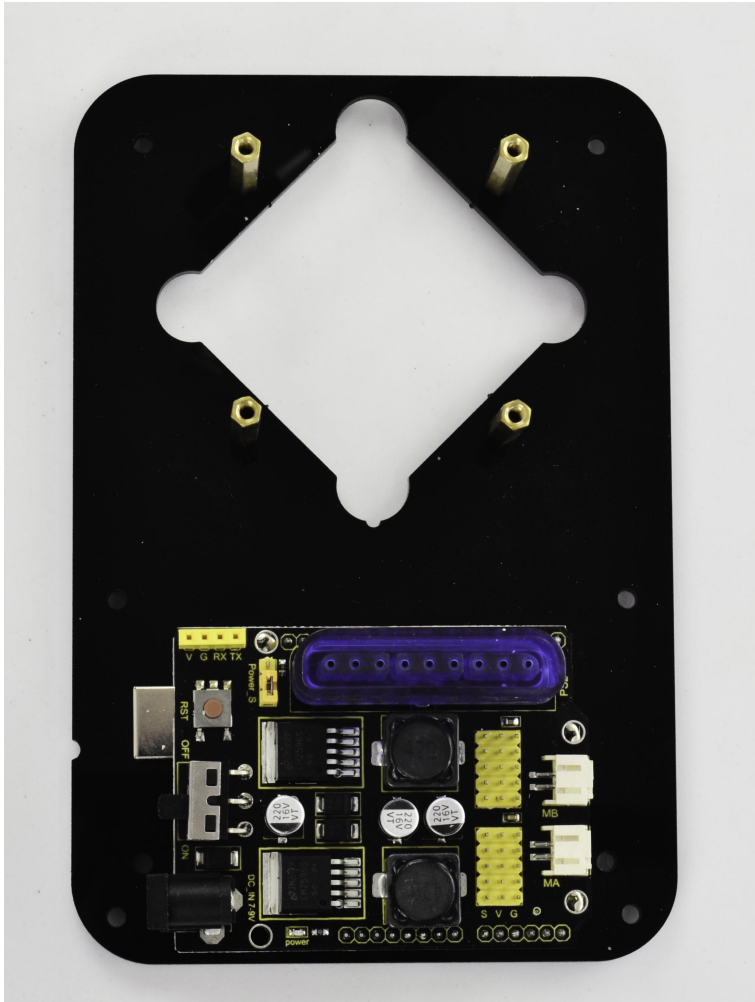
- M3*6MM round-head screw *3
- Keyestudio UNO R3 board *1
- keyestudio TB6612FNG motor shield *1



Firstly, screw the UNO R3 board on the pillar using three M3*6MM round-head screws.

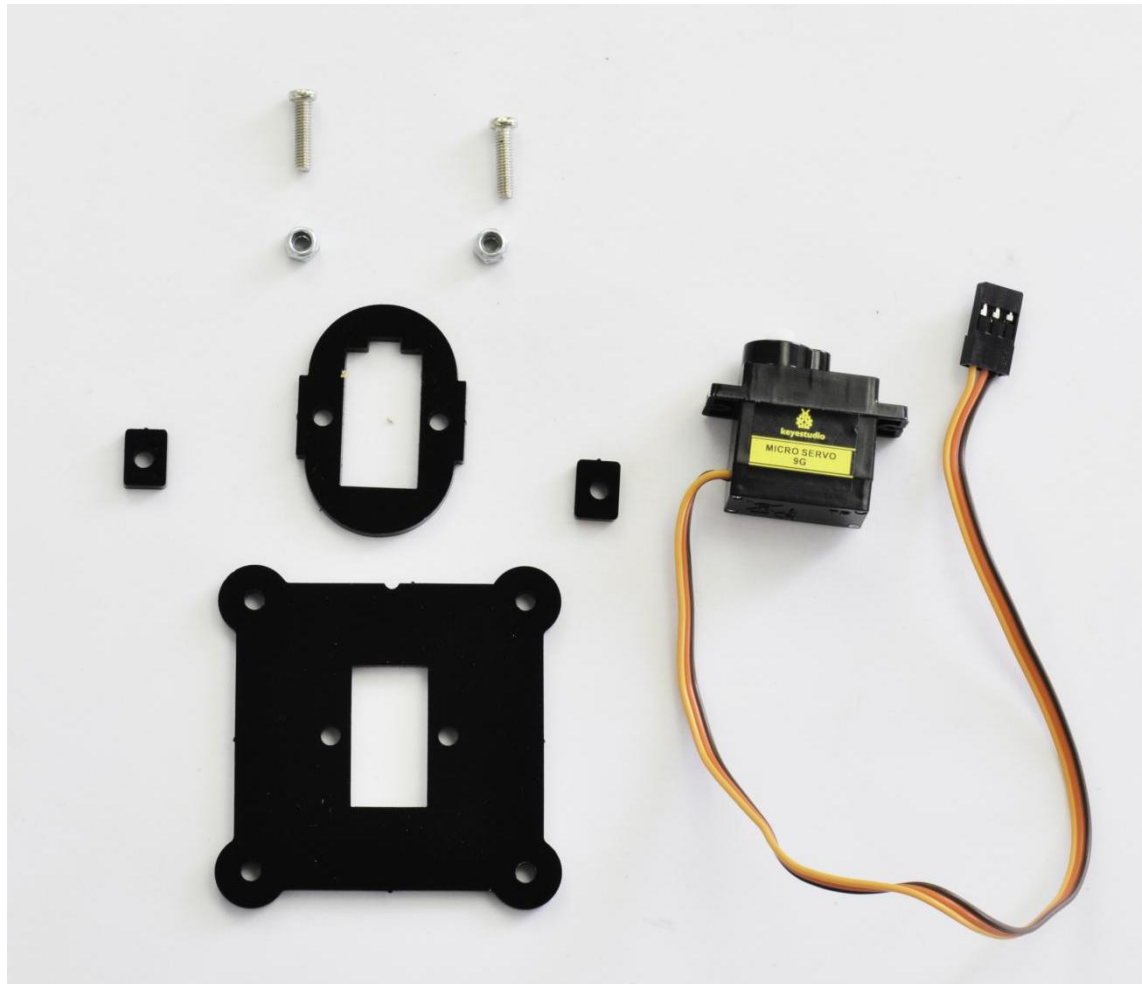


Then stack the motor drive shield onto the UNO R3 board.



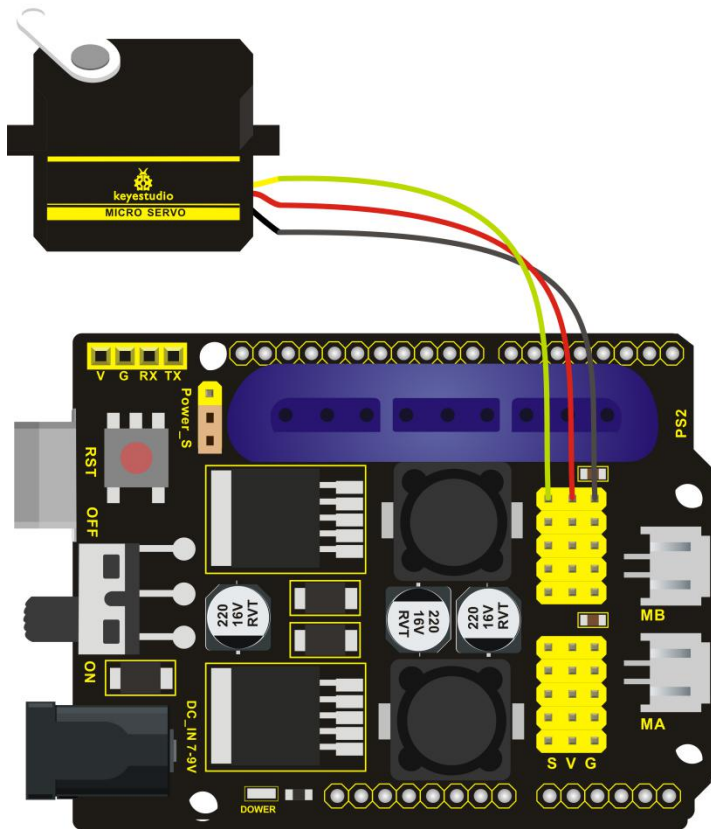
(3) Completed the above assembly, let's mount the Pivot Servo Plate onto the base.

- M3*12MM round-head screw *2
- M3 hex lock Nut *2
- Black 180° servo *1
- Acrylic plate * 4



Note: before install the servo, should set the servo angle to 80 degrees.

To set the servo angle, first connect the servo to **A0** of motor shield, upload the code below to UNO R3 board, powered on, press the reset button, servo will rotate to 80°.

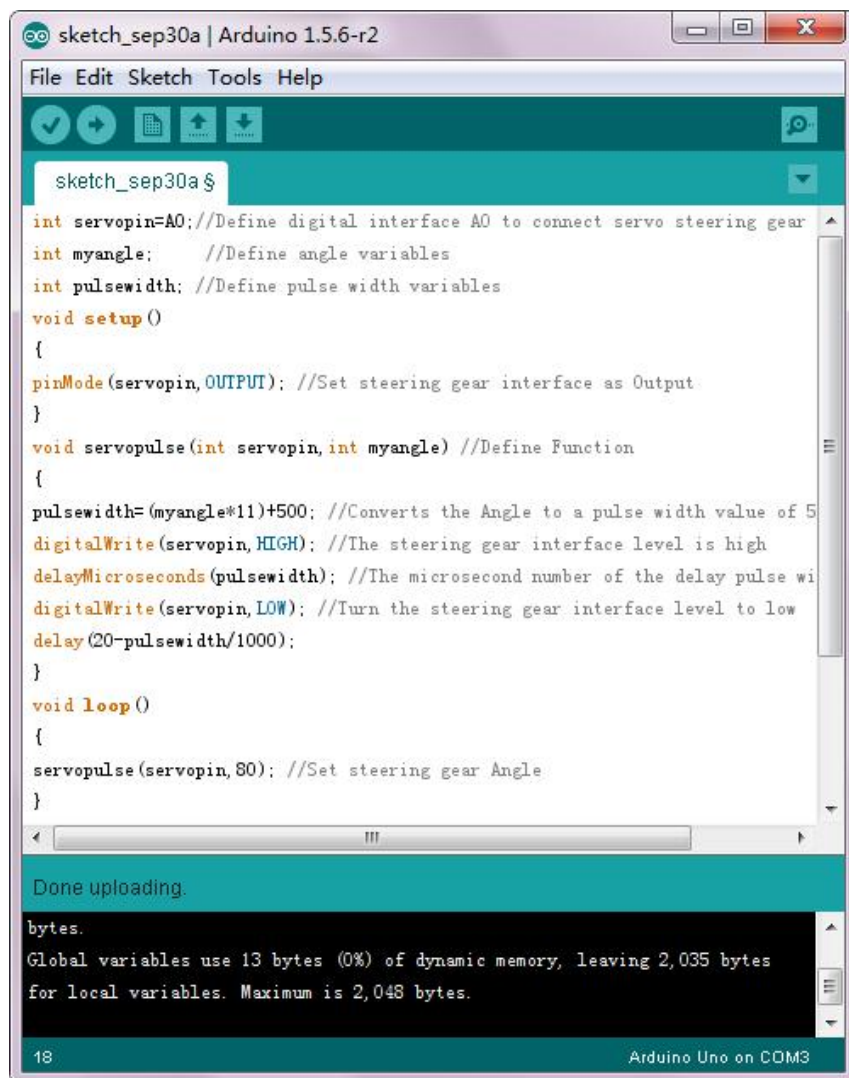


Code for 80° Servo:

```
*****  
  
int servopin=A0;//Define digital interface A0 to connect servo steering gear signal line  
int myangle;      //Define angle variables  
int pulsewidth; //Define pulse width variables  
void setup()  
{  
  pinMode(servopin,OUTPUT); //Set steering gear interface as Output  
}  
  
void servopulse(int servopin,int myangle) //Define Function  
{  
  pulsewidth=(myangle*11)+500; //Converts the Angle to a pulse width value of 500 - 2480  
  digitalWrite(servopin,HIGH); //The steering gear interface level is high  
  delayMicroseconds(pulsewidth); //The microsecond number of the delay pulse width value  
  digitalWrite(servopin,LOW); //Turn the steering gear interface level to low  
  delay(20-pulsewidth/1000);  
}  
  
void loop()  
{  
  servopulse(servopin,80); //Set steering gear Angle  
}  
  
//0 Degree Code:  
//    servopulse(servopin,0);
```

```
//80 Degree Code:
//    servopulse(servopin, 80);
//180 Degree Code:
//    servopulse(servopin, 180);

*****
```

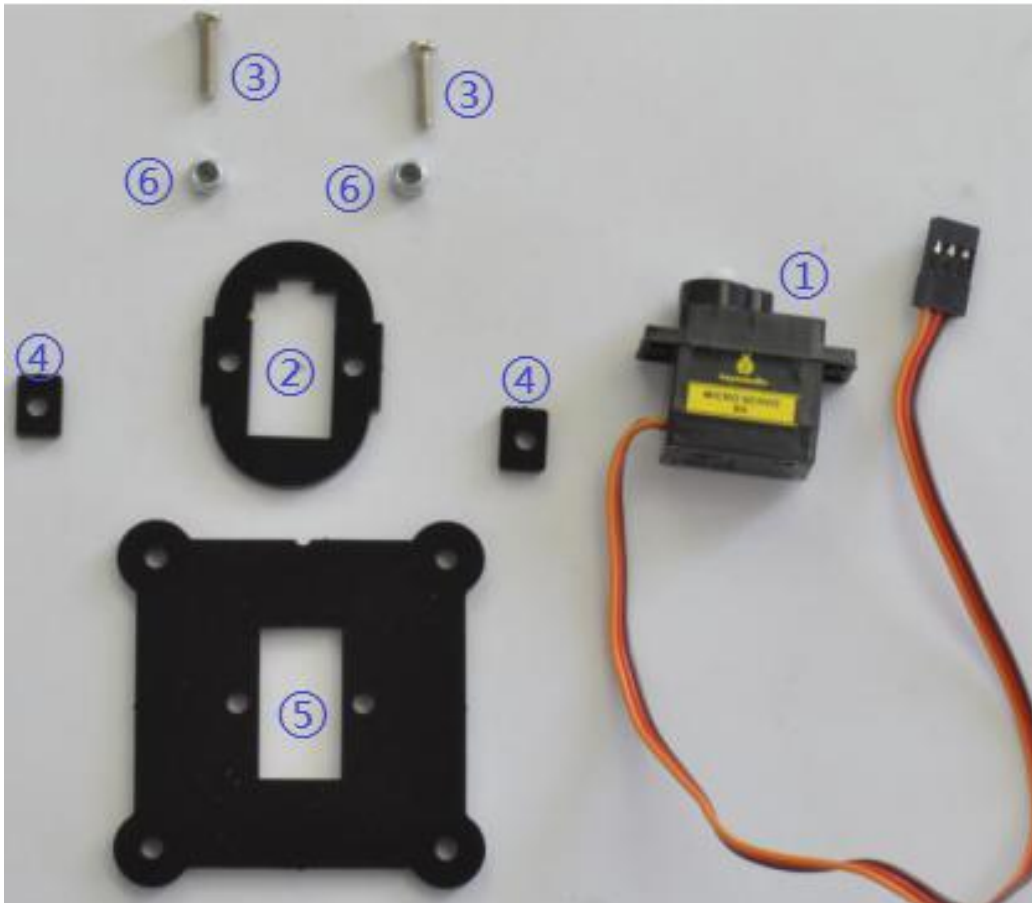


```
sketch_sep30a | Arduino 1.5.6-r2
File Edit Sketch Tools Help
sketch_sep30a$
int servopin=AO; //Define digital interface AO to connect servo steering gear
int myangle;      //Define angle variables
int pulsewidth;  //Define pulse width variables
void setup()
{
  pinMode(servopin, OUTPUT); //Set steering gear interface as Output
}
void servopulse(int servopin, int myangle) //Define Function
{
  pulsewidth=(myangle*11)+500; //Converts the Angle to a pulse width value of 5
  digitalWrite(servopin, HIGH); //The steering gear interface level is high
  delayMicroseconds(pulsewidth); //The microsecond number of the delay pulse wi
  digitalWrite(servopin, LOW); //Turn the steering gear interface level to low
  delay(20-pulsewidth/1000);
}
void loop()
{
  servopulse(servopin, 80); //Set steering gear Angle
}
Done uploading.
bytes.
Global variables use 13 bytes (0%) of dynamic memory, leaving 2,035 bytes
for local variables. Maximum is 2,048 bytes.
18 Arduino Uno on COM3
```

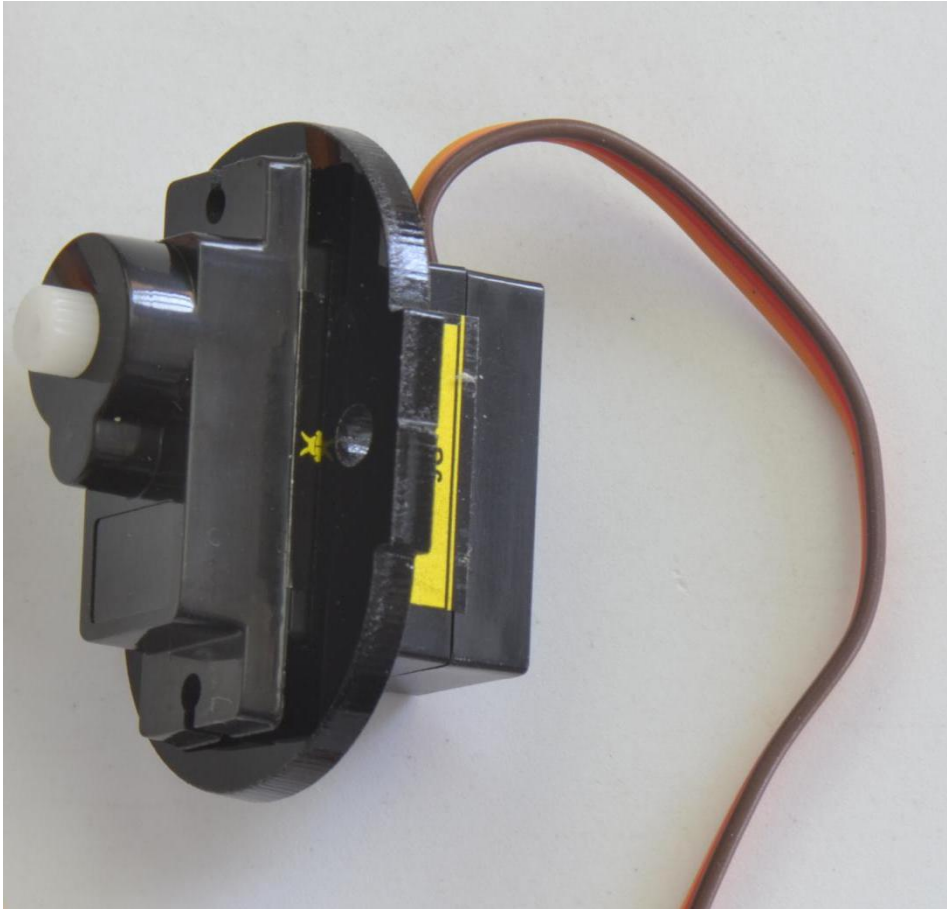
Notice:

Set well the servo angle and complete the below servo base plate assembly, power off the servo to avoid the angle error and make sure the servo can rotate freely. Don't over-tighten the screws.

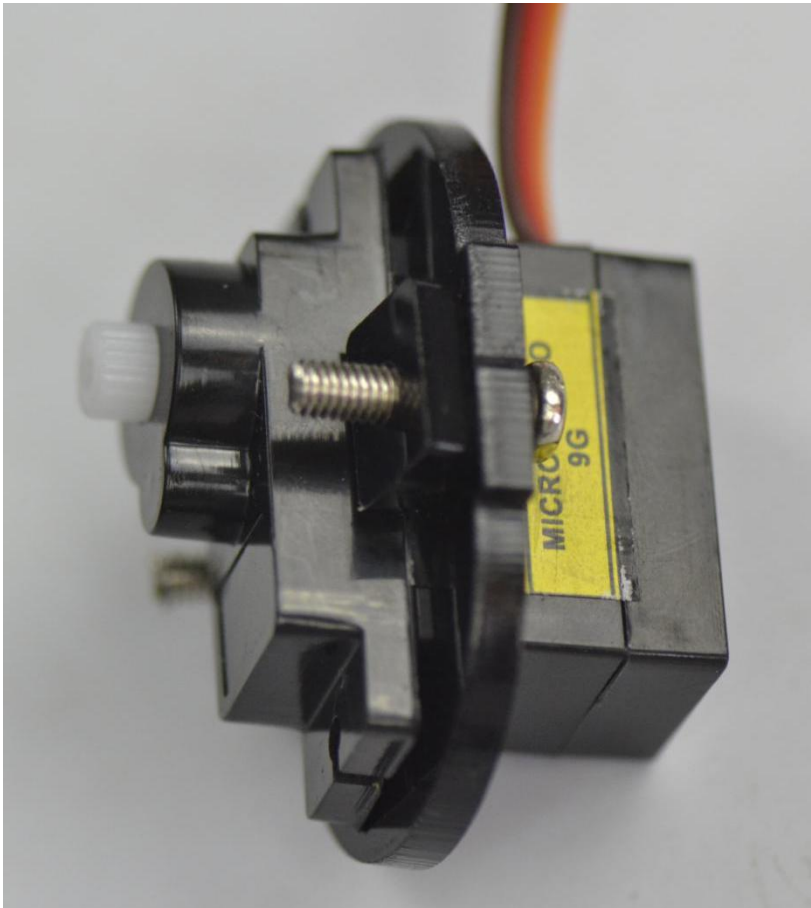
Adjusted well the servo motor, start to install the **Servo Base Plate**. Follow the marks.



Firstly mount the acrylic plate② to the servo motor.



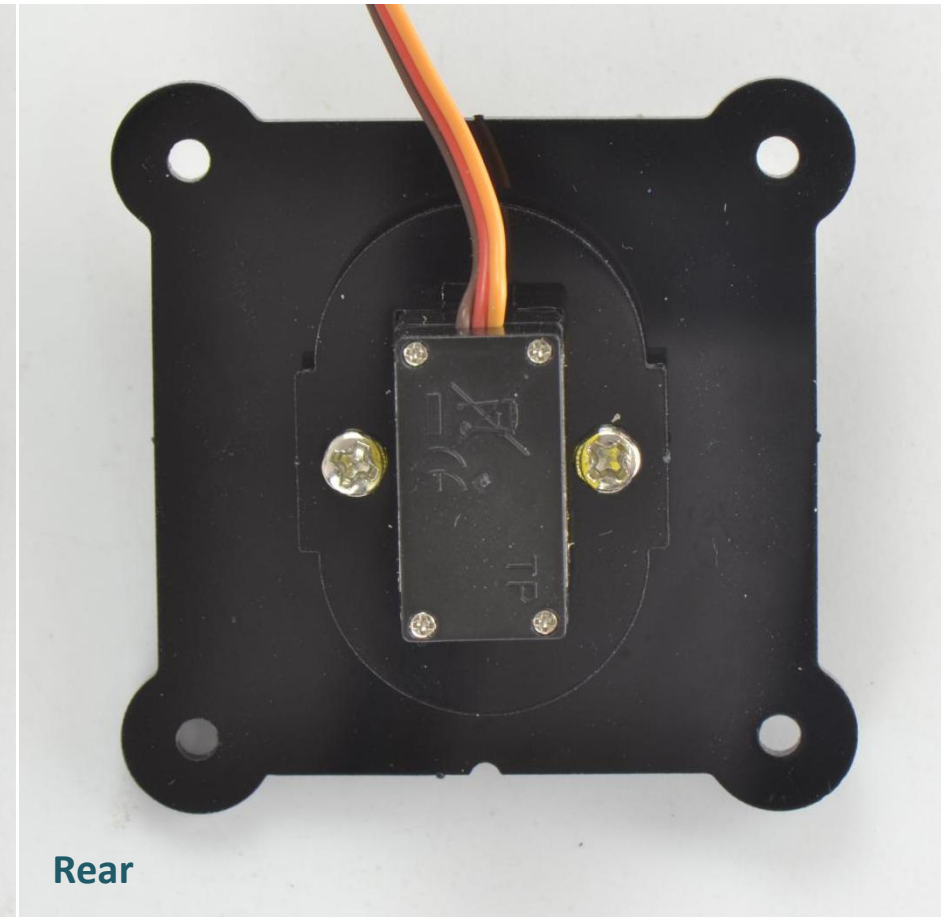
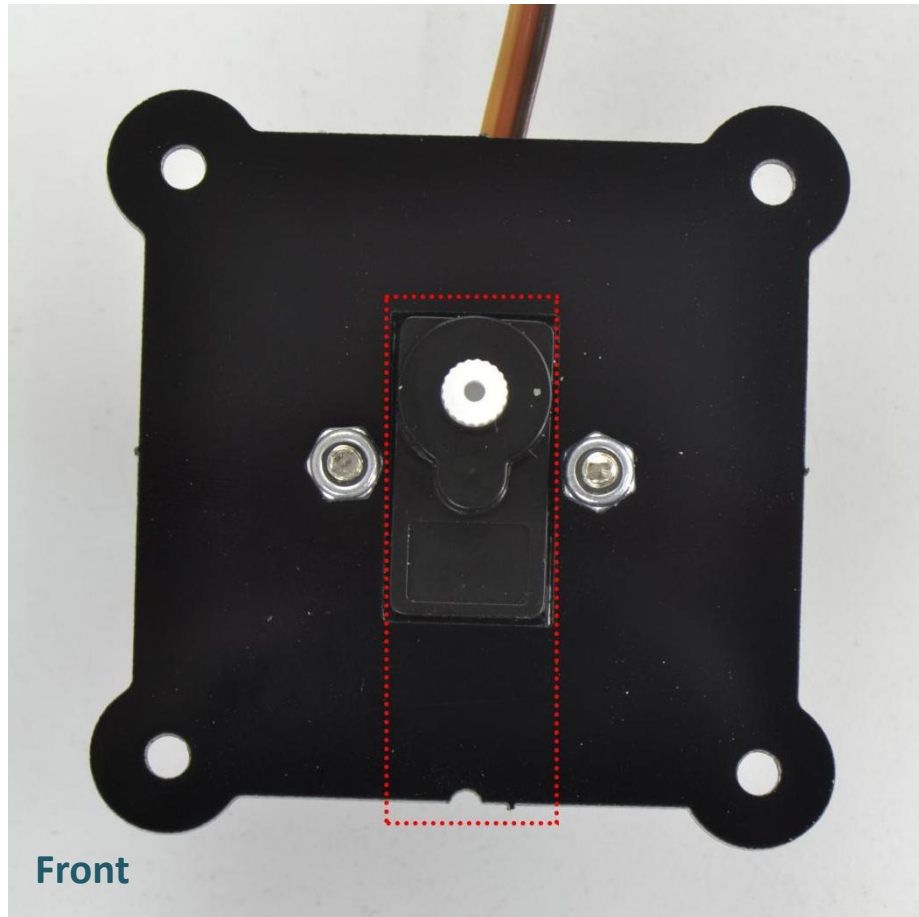
Then mount the two acrylic plates ④ to the servo using two M3*12MM round-head screws.



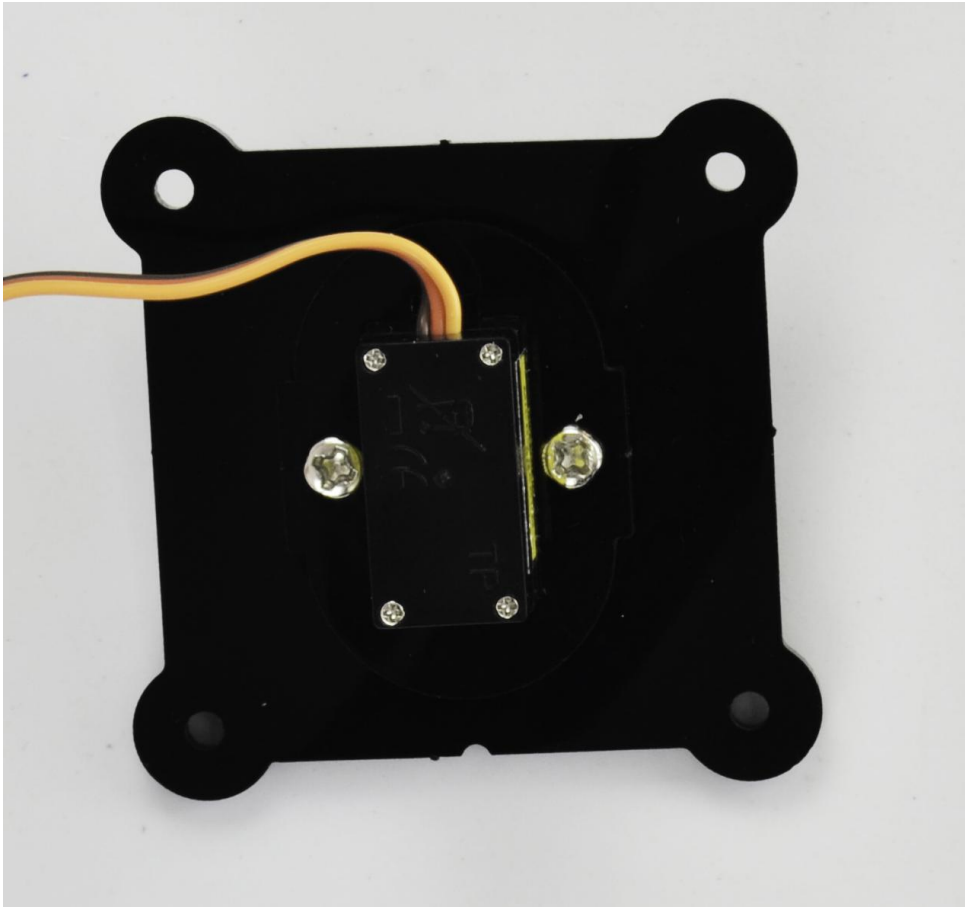
After that, screw the acrylic plate^⑤ to the servo using two M3 hex lock Nuts^⑥.



You should get the detailed view shown below (pay close attention to red mark).

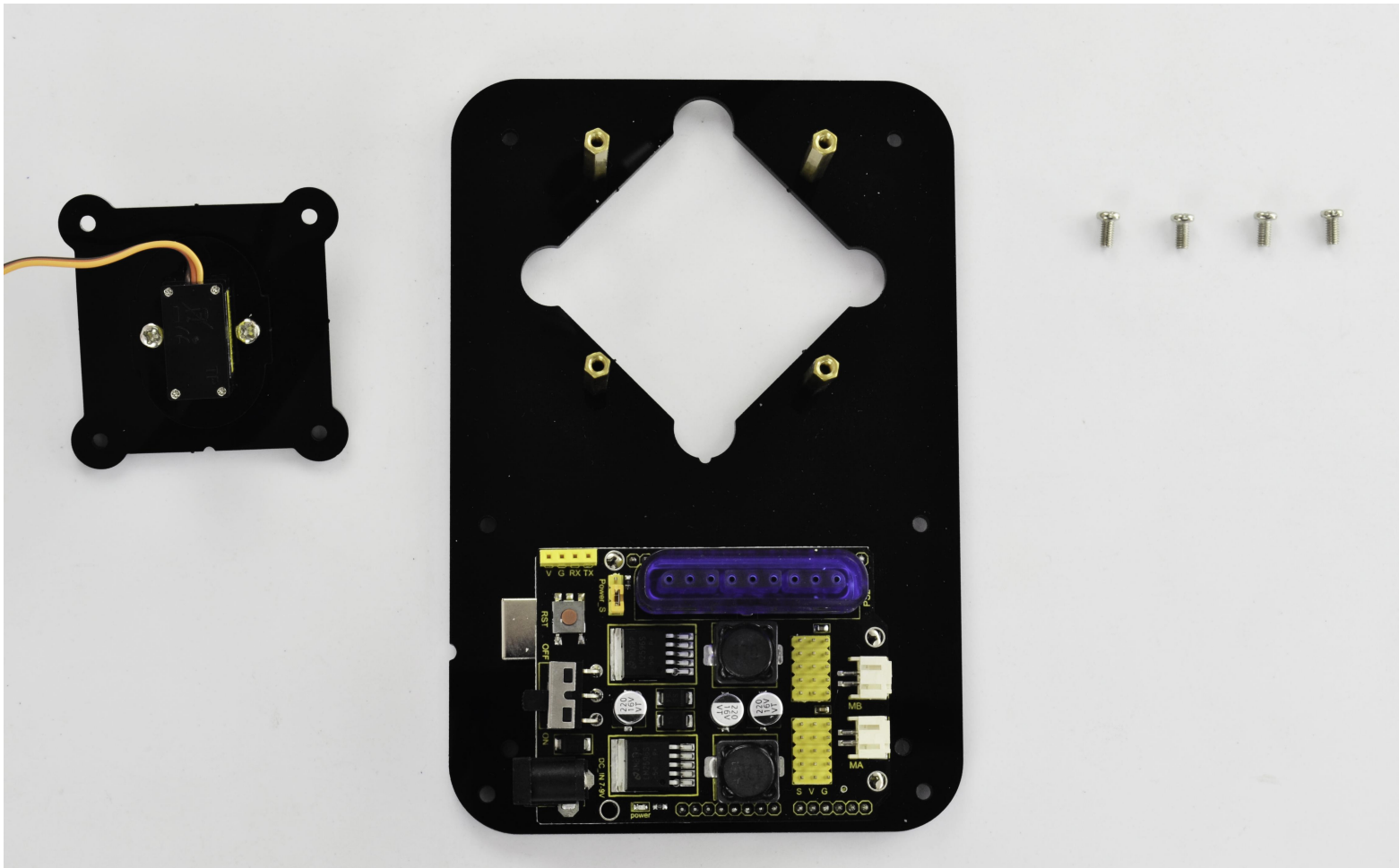


You should get the complete **Servo Base Part**.

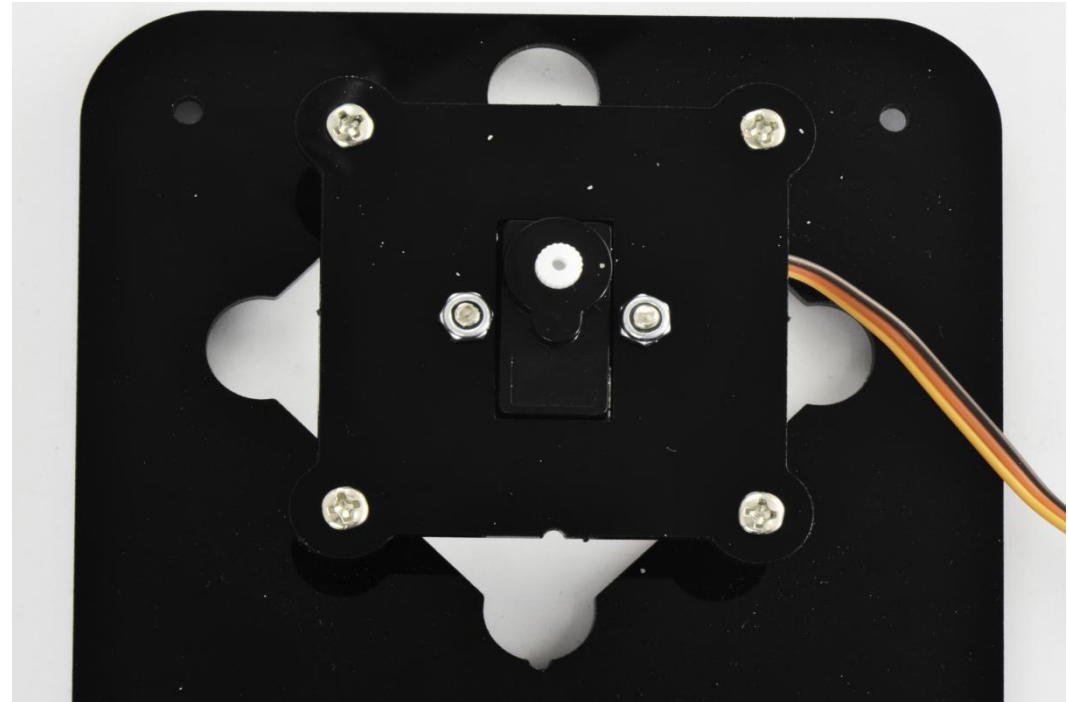


(4) Now you should install the Servo Base part to the Base Plate:

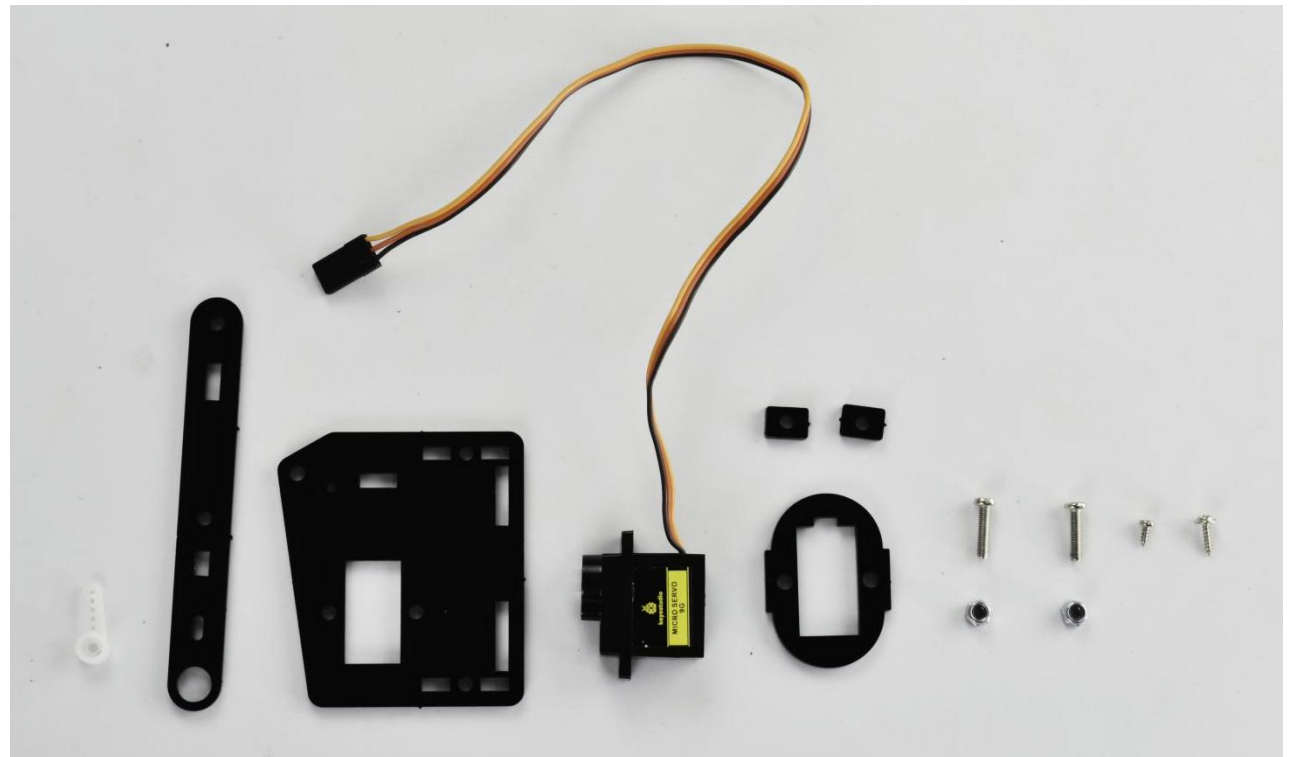
- M3*6MM round-head hex screw *4



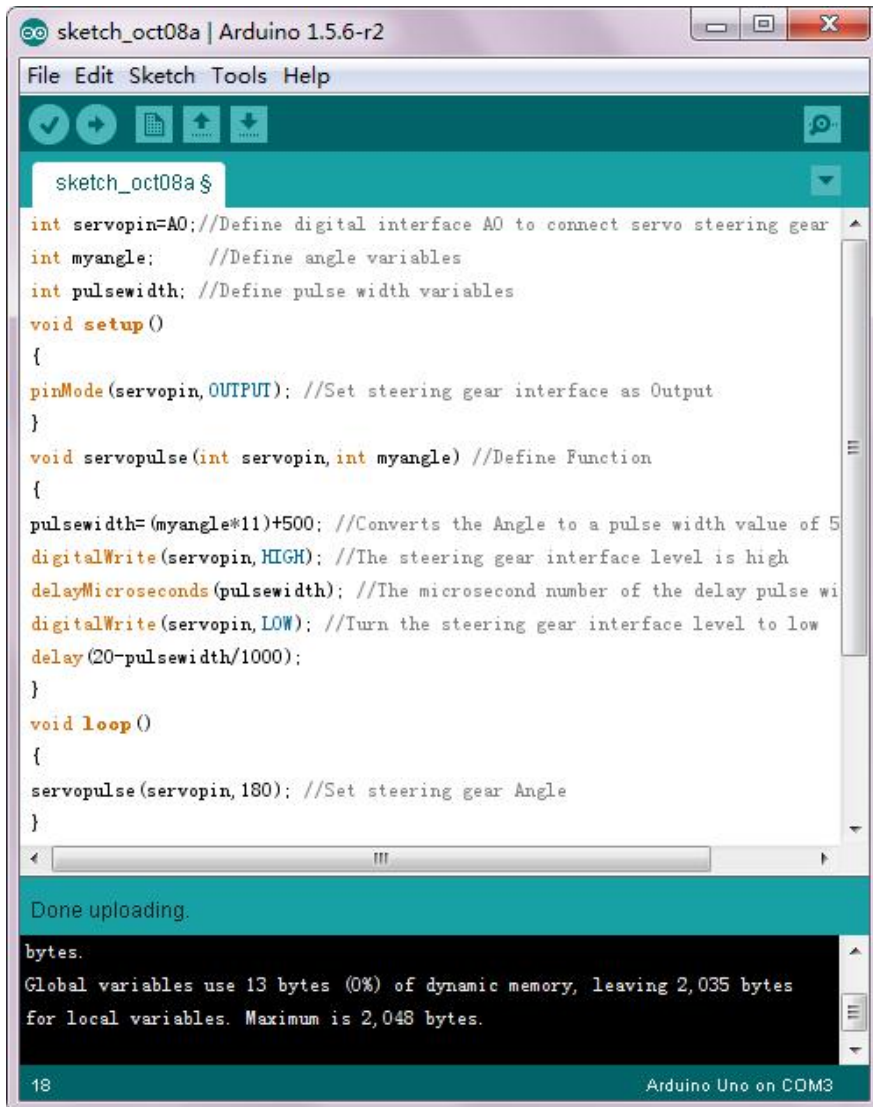
Install the Servo plate to the Base Plate using four M3*6MM round-head hex screws.



- M3*12MM round-head screw *2
- M3 hex lock nut *2
- M2x5MM Phillips tapping screw *1
- M2x8MM Phillips tapping screw *1
- Black 180° servo *1
- White servo mount *1
- Acrylic plate *5



Note: before install the servo plate, should set the servo angle to 180 degrees. The method is the same as 80° servo settings mentioned above. You just need to change the `servopulse(servopin,80)` into `servopulse(servopin,180)` in the code.



```
sketch_oct08a $
int servopin=A0; //Define digital interface A0 to connect servo steering gear
int myangle;      //Define angle variables
int pulsewidth;  //Define pulse width variables
void setup()
{
  pinMode(servopin,OUTPUT); //Set steering gear interface as Output
}
void servopulse(int servopin,int myangle) //Define Function
{
  pulsewidth=(myangle*11)+500; //Converts the Angle to a pulse width value of 5
  digitalWrite(servopin,HIGH); //The steering gear interface level is high
  delayMicroseconds(pulsewidth); //The microsecond number of the delay pulse wi
  digitalWrite(servopin,LOW); //Turn the steering gear interface level to low
  delay(20-pulsewidth/1000);
}
void loop()
{
  servopulse(servopin,180); //Set steering gear Angle
}
```

Done uploading.

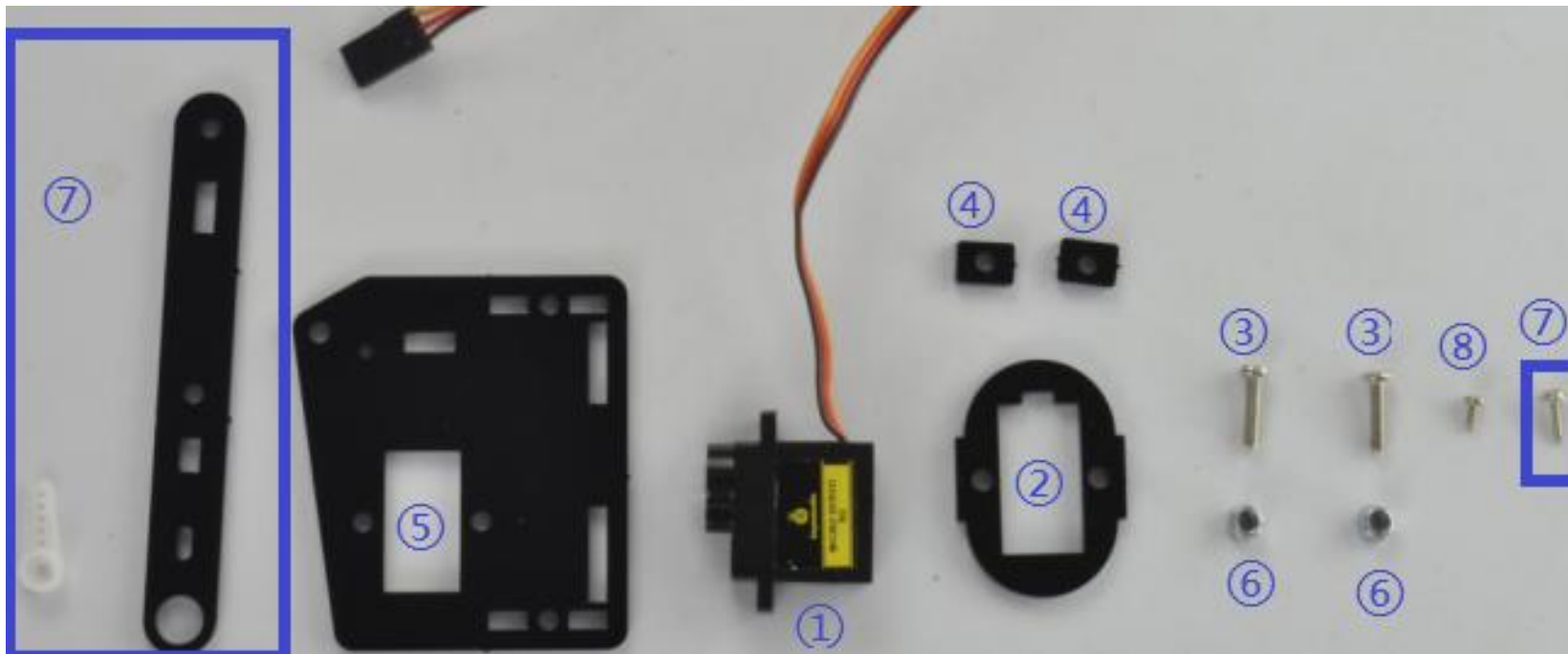
bytes.

Global variables use 13 bytes (0%) of dynamic memory, leaving 2,035 bytes for local variables. Maximum is 2,048 bytes.

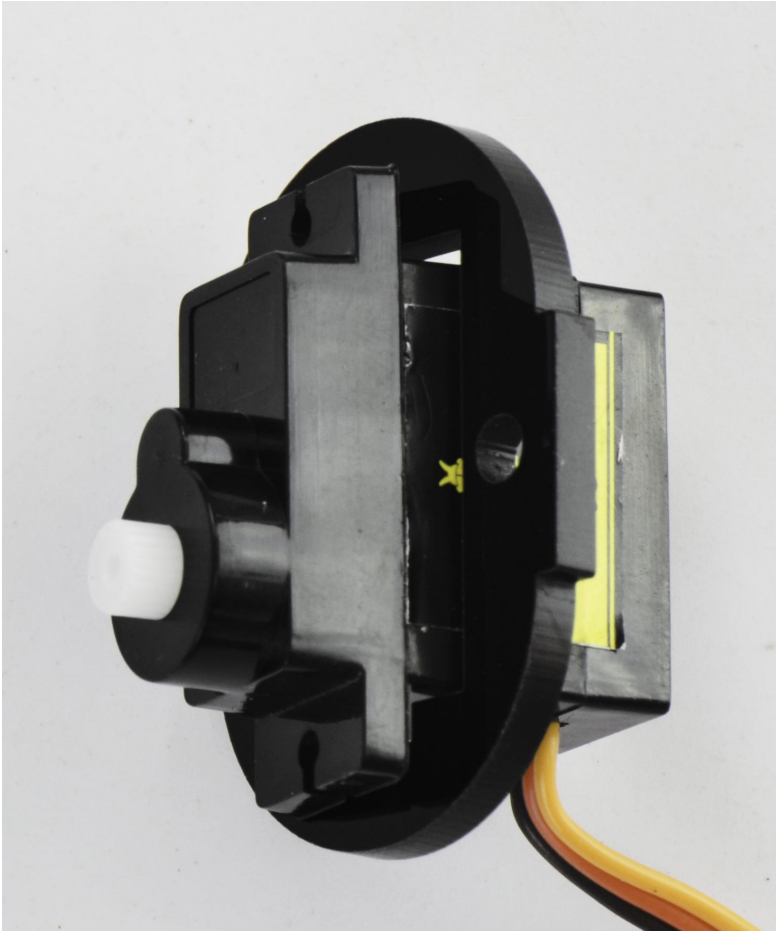
18 Arduino Uno on COM3

Now assemble the Left Arm Servo Plate as below.

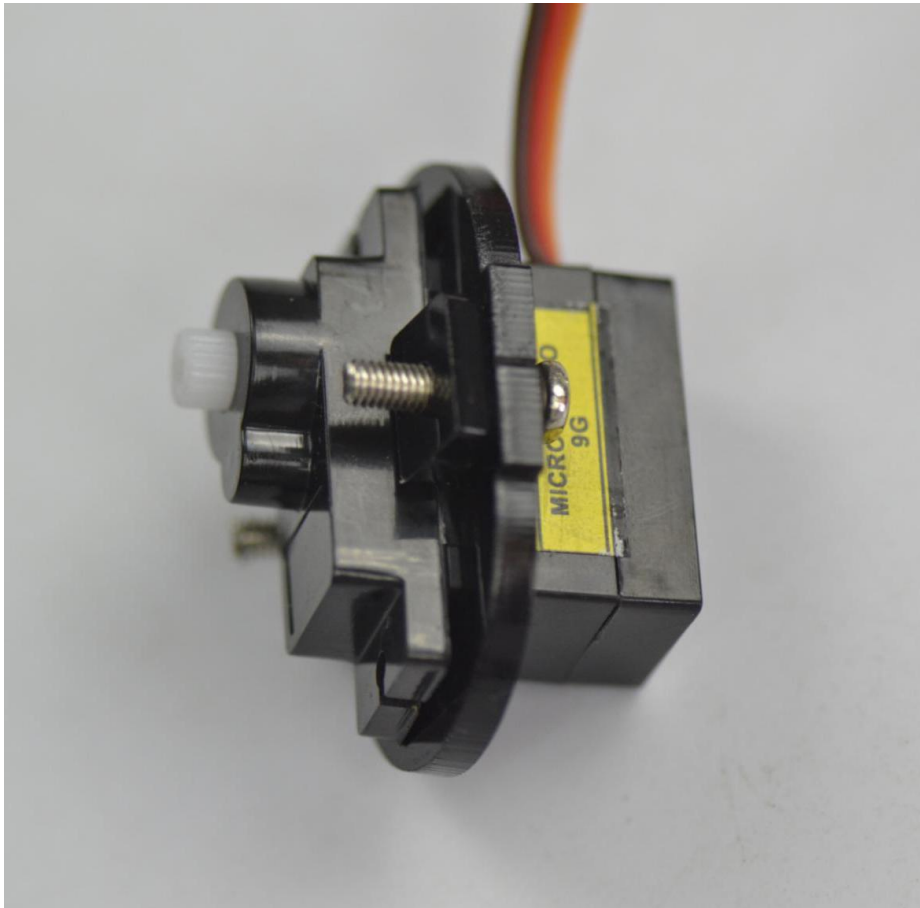
(pay close attention to servo direction, rocker connected to servo and marked position)



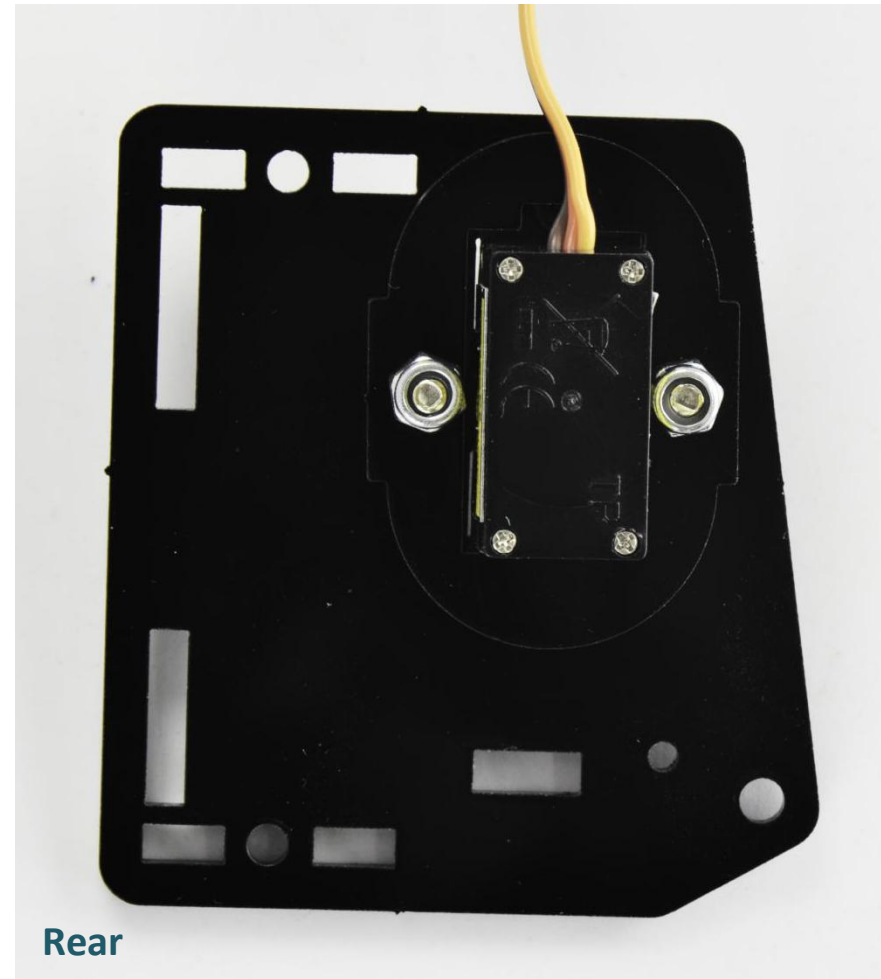
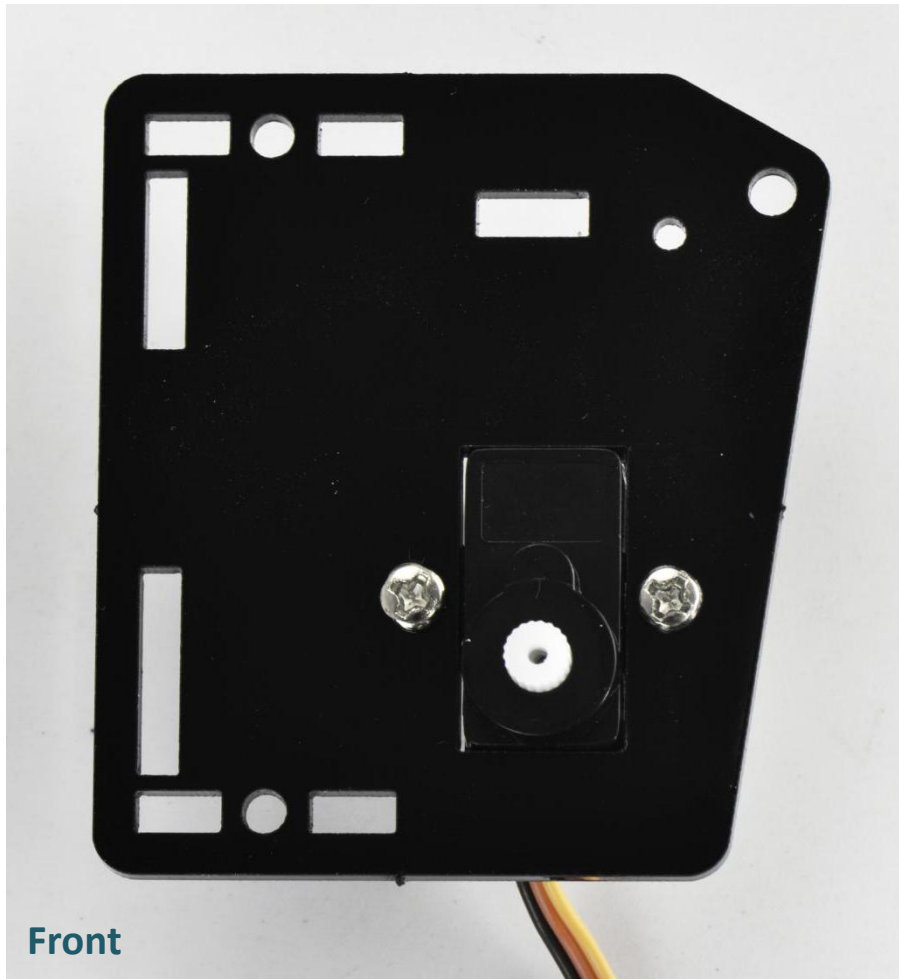
Firstly mount the acrylic plate② to the Left servo motor.



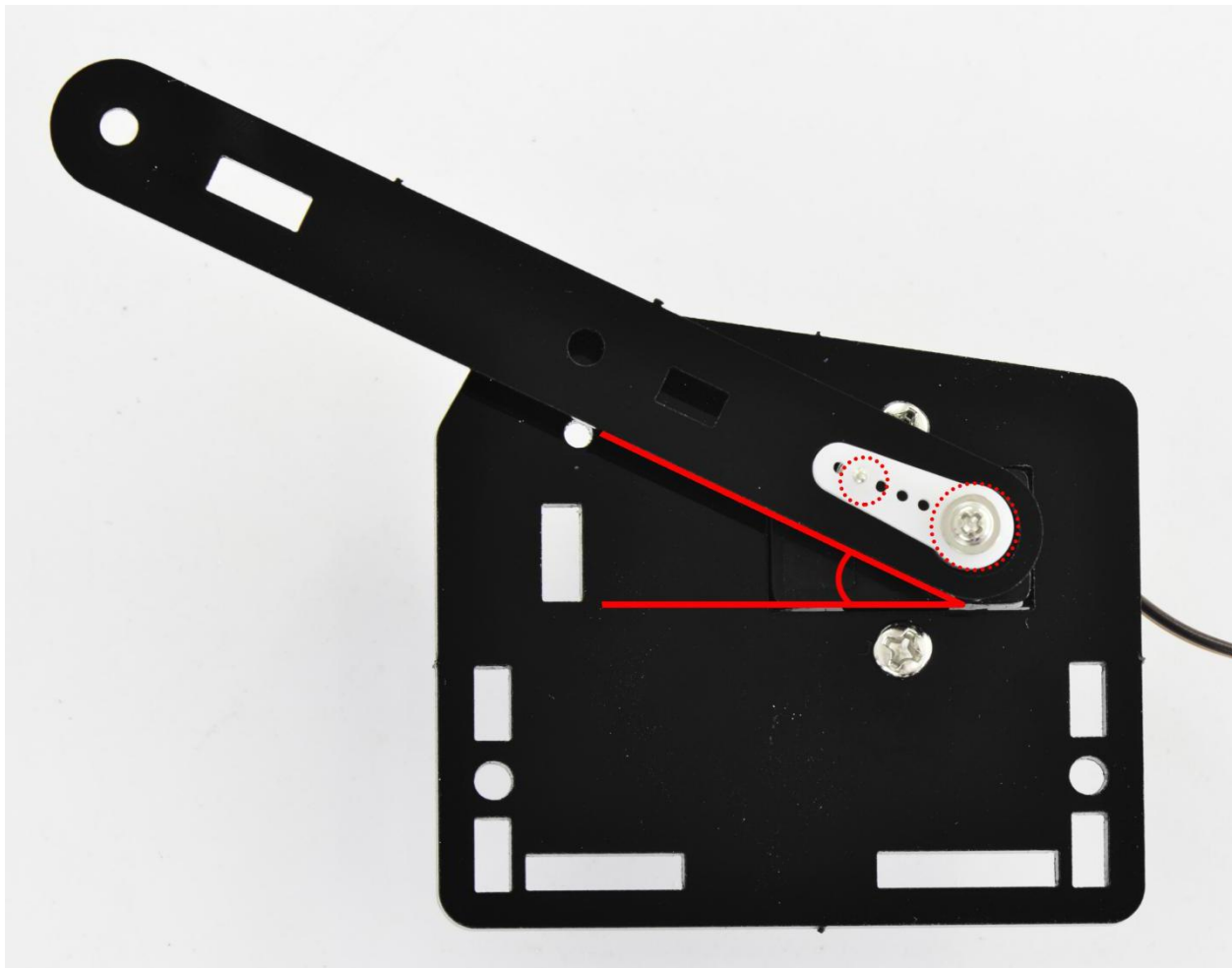
Second, mount the two acrylic plates④ to the servo using two M3*12MM round-head screws③. You can get the Left servo plate shown below.



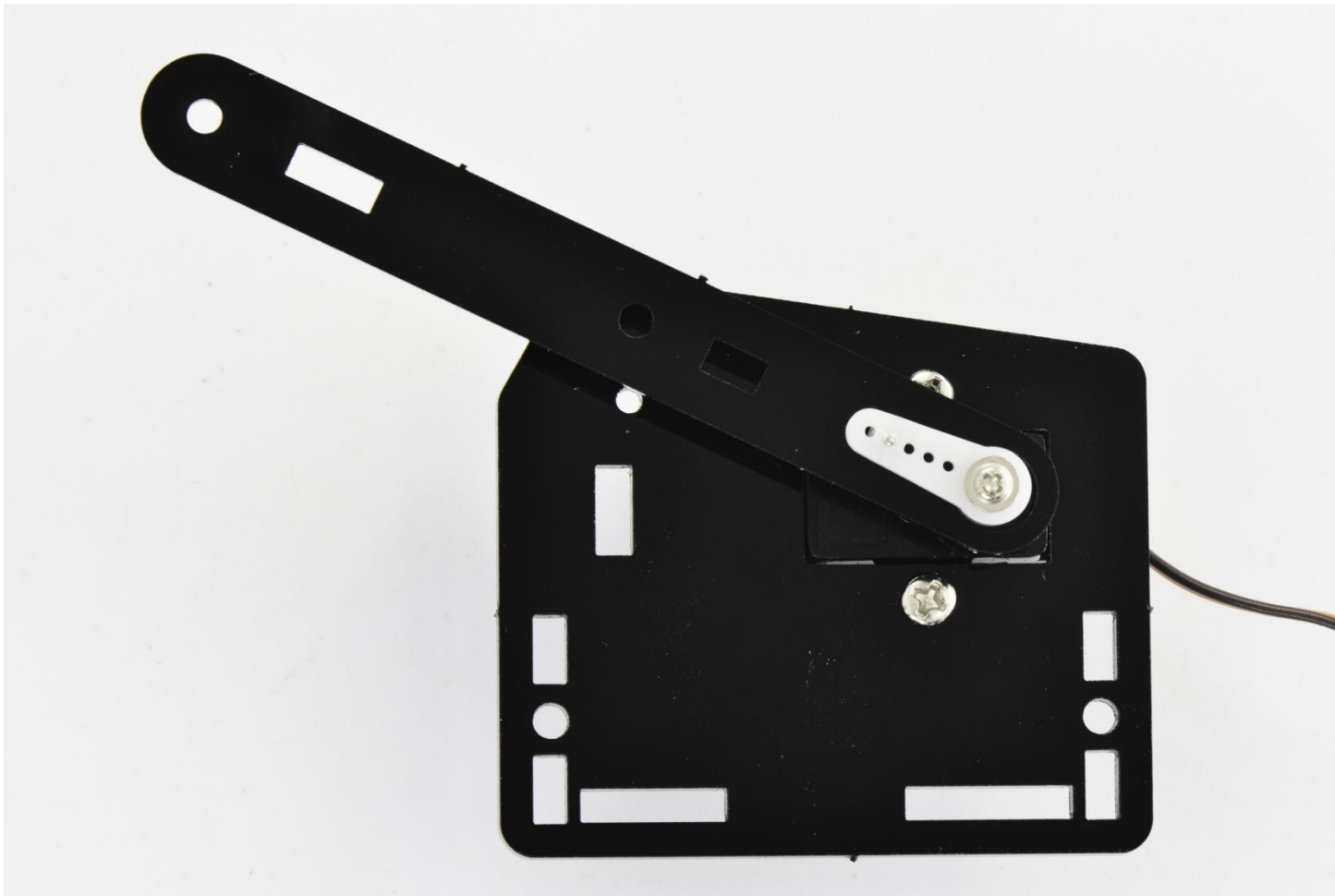
Then screw the Left servo plate to acrylic plate⑤ using two M3 hex lock Nuts⑥. Get the Left servo base plate shown below.



After that, screw the acrylic plate joint and white servo mount⑦ to the Left servo base plate using a M2x8MM tapping screw and a M2x5MM tapping screw⑧.

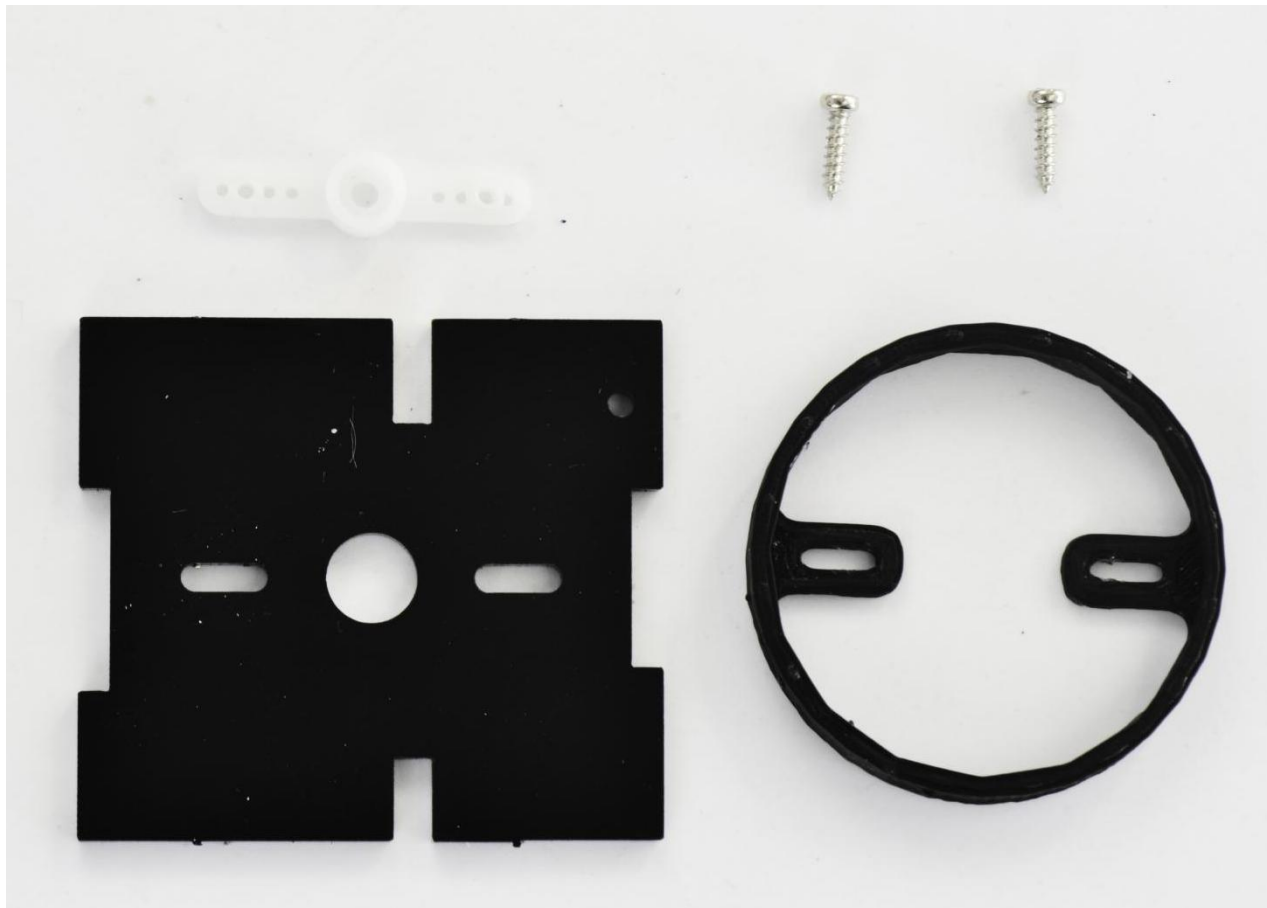


Finally you should get the complete Left Arm Servo Plate. Shown below.

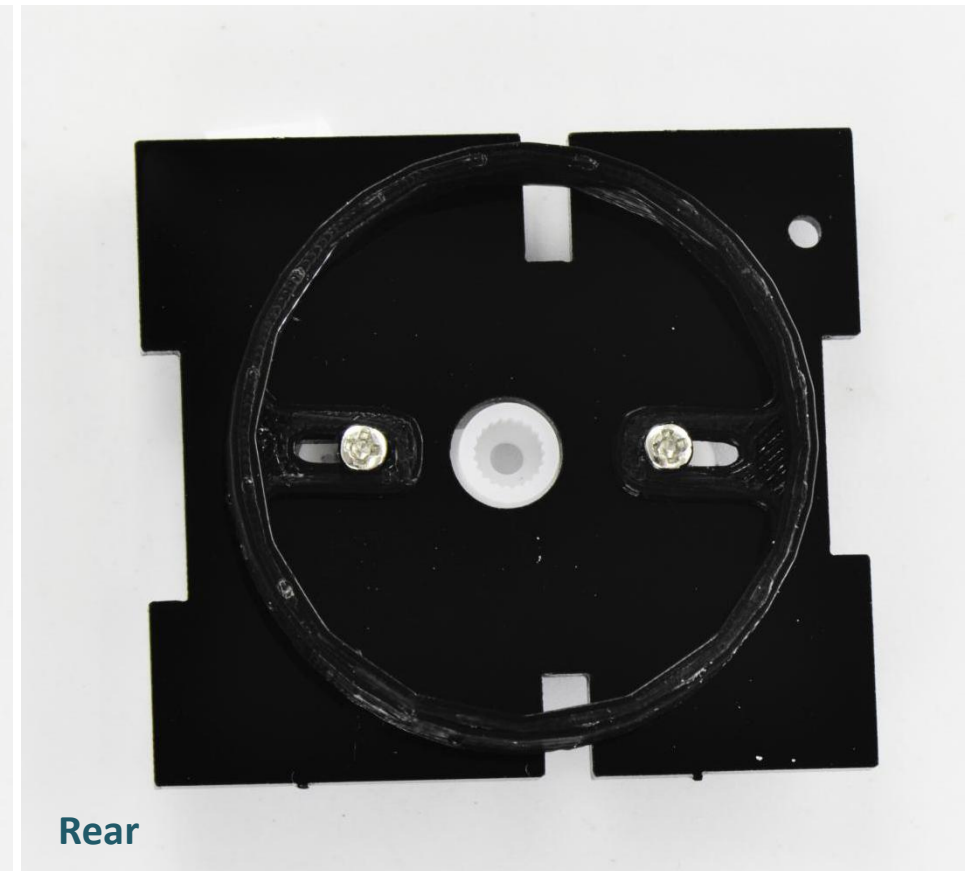


(2) We will now attach a servo mount to the **Arm Bottom Plate**.

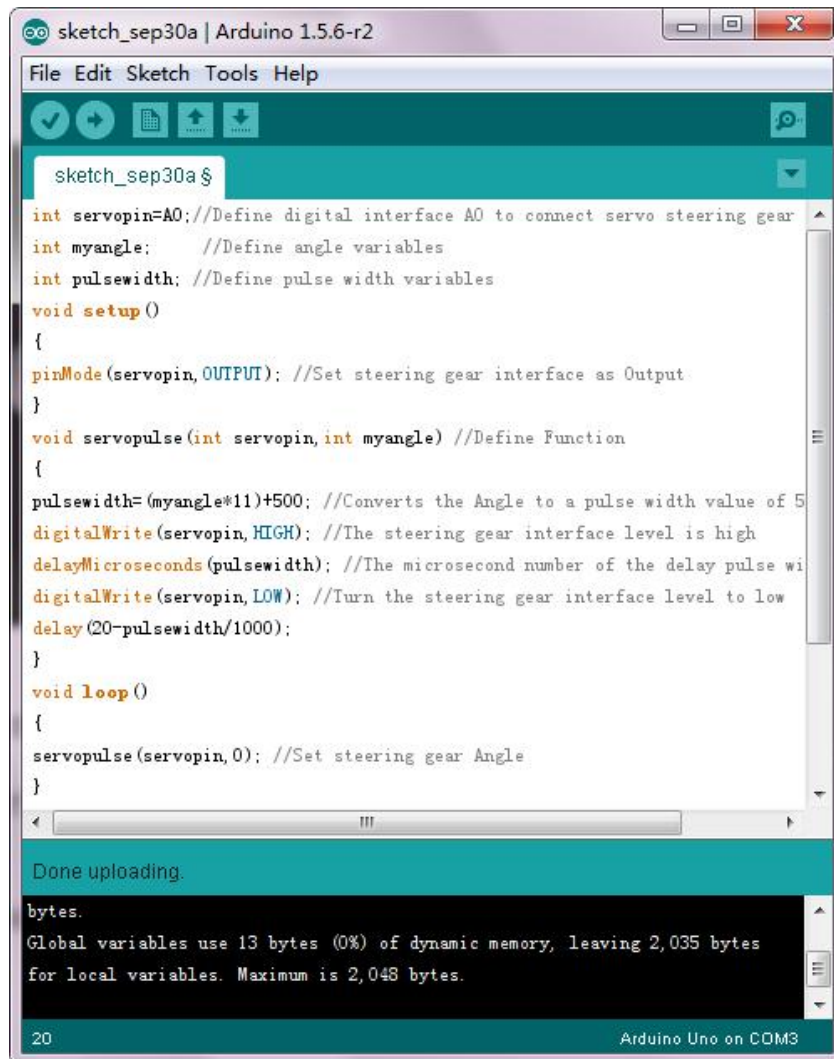
- M2x8MM Phillips tapping screw *2
- White servo mount *1
- Acrylic bottom plate *1
- Black Cylindrical holder *1



Fix the white servo mount and black Cylindrical holder on the acrylic plate using two M2x8MM Phillips tapping screws.



-



```

sketch_sep30a | Arduino 1.5.6-r2
File Edit Sketch Tools Help

sketch_sep30a $

int servopin=A0; //Define digital interface A0 to connect servo steering gear
int myangle; //Define angle variables
int pulsewidth; //Define pulse width variables
void setup()
{
  pinMode(servopin, OUTPUT); //Set steering gear interface as Output
}
void servopulse(int servopin, int myangle) //Define Function
{
  pulsewidth=(myangle*11)+500; //Converts the Angle to a pulse width value of 5
  digitalWrite(servopin, HIGH); //The steering gear interface level is high
  delayMicroseconds(pulsewidth); //The microsecond number of the delay pulse wi
  digitalWrite(servopin, LOW); //Turn the steering gear interface level to low
  delay(20-pulsewidth/1000);
}
void loop()
{
  servopulse(servopin, 0); //Set steering gear Angle
}

Done uploading.
bytes.
Global variables use 13 bytes (0%) of dynamic memory, leaving 2,035 bytes
for local variables. Maximum is 2,048 bytes.

20 Arduino Uno on COM3

```

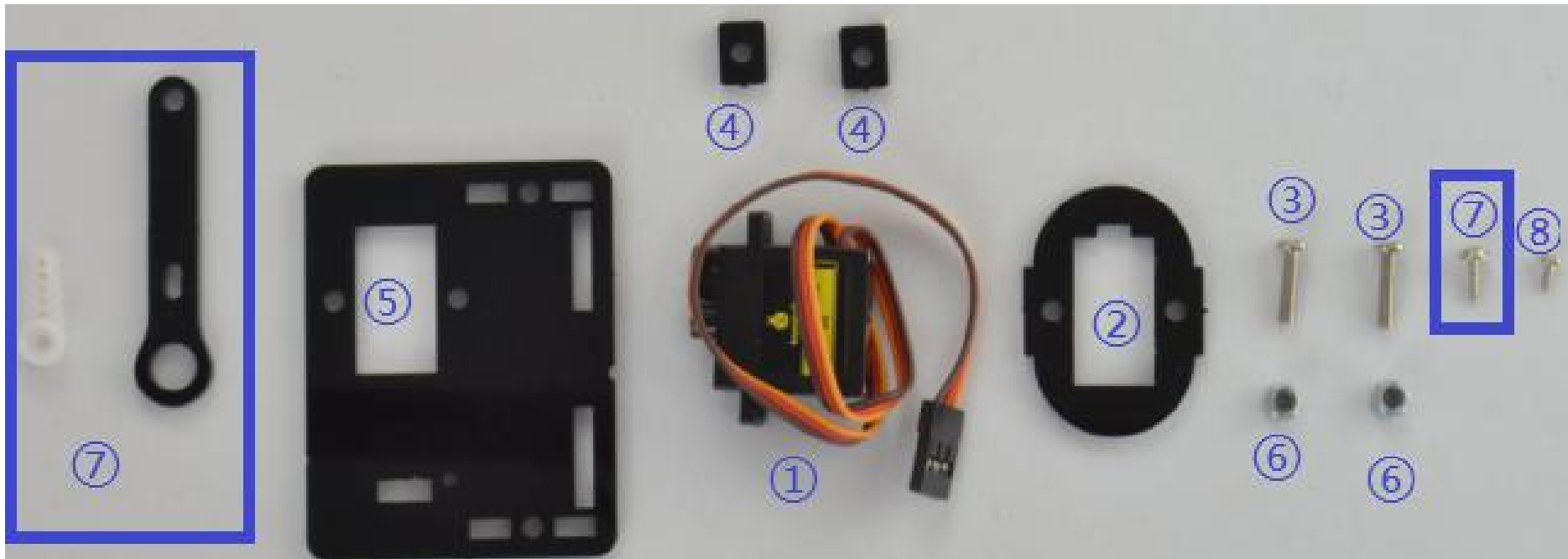
Note: before install the servo, should set the servo angle to 0 degrees.

The method is the same as 80° servo settings mentioned above.

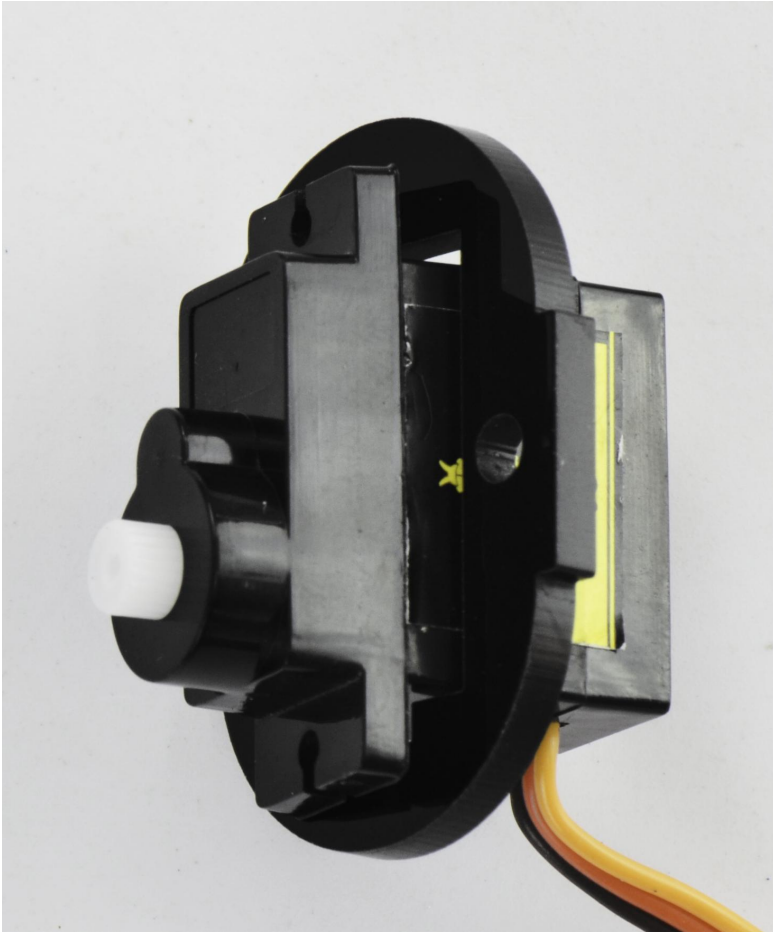
You just need to change the `servopulse(servopin,80)` into `servopulse(servopin,0)` in the code.

Now assemble the Right Arm Servo Plate as below.

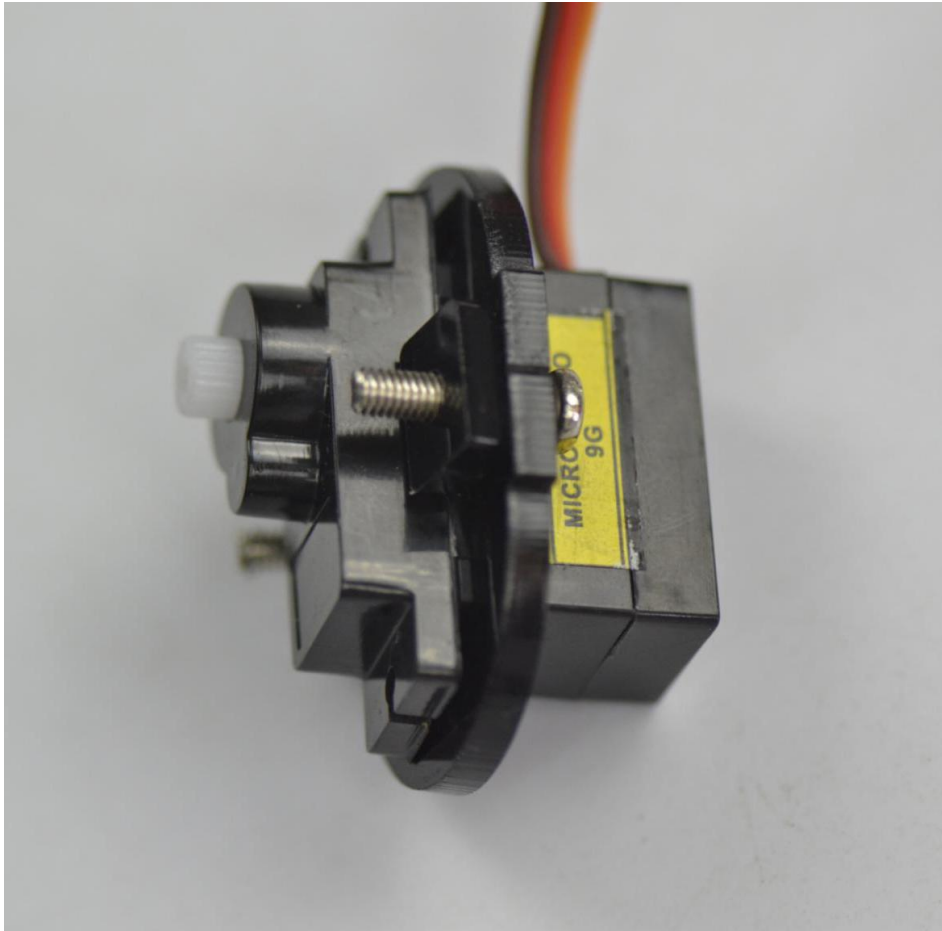
(pay close attention to servo direction, rocker connected to servo and marked position)



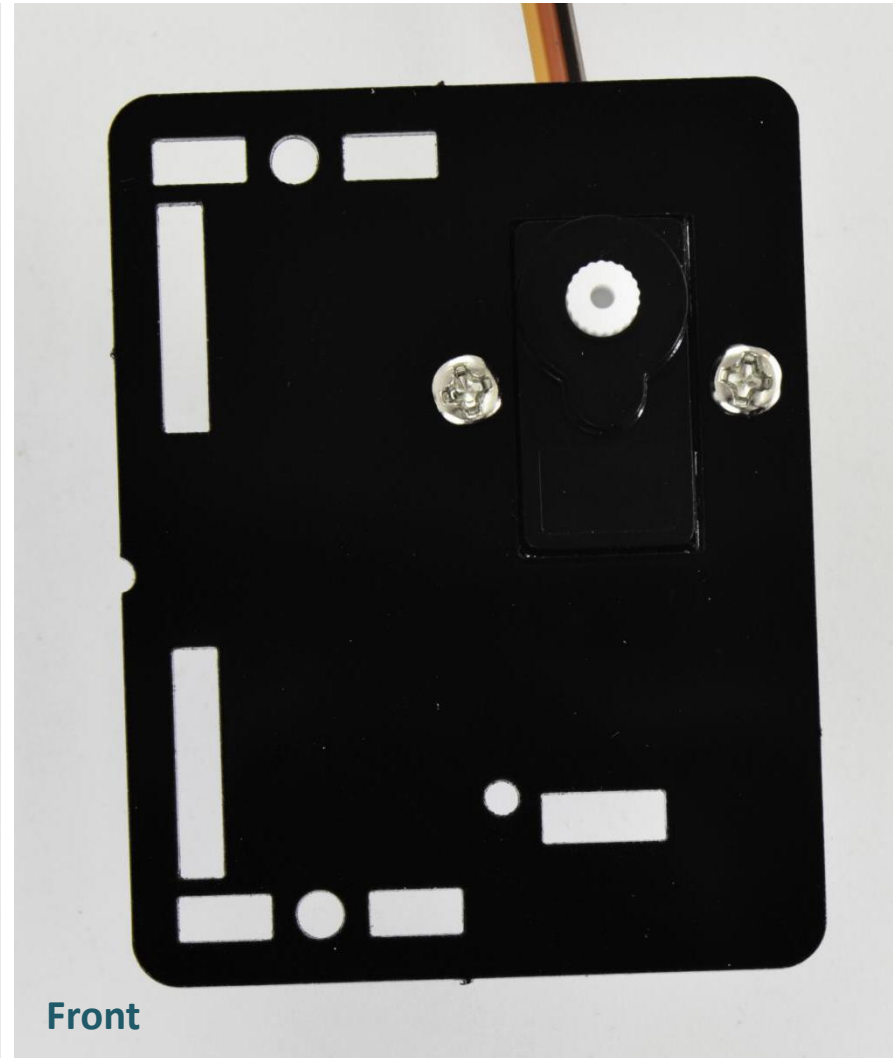
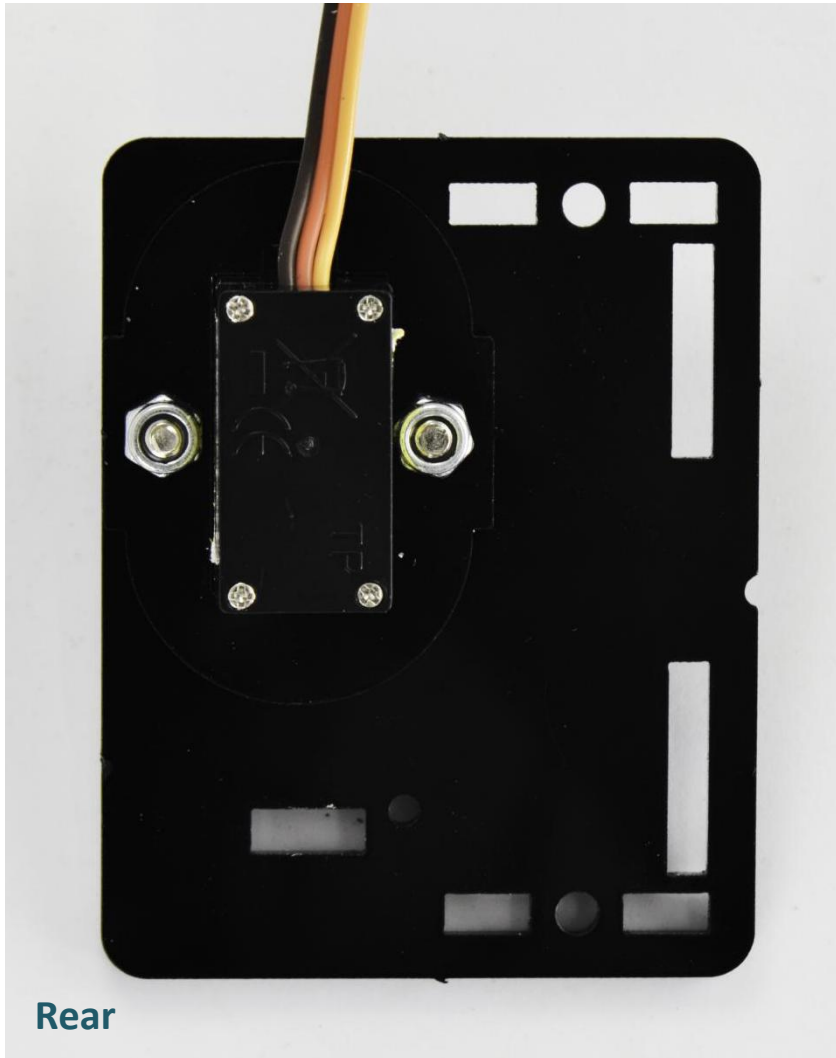
Firstly mount the acrylic plate② to the Right servo motor.



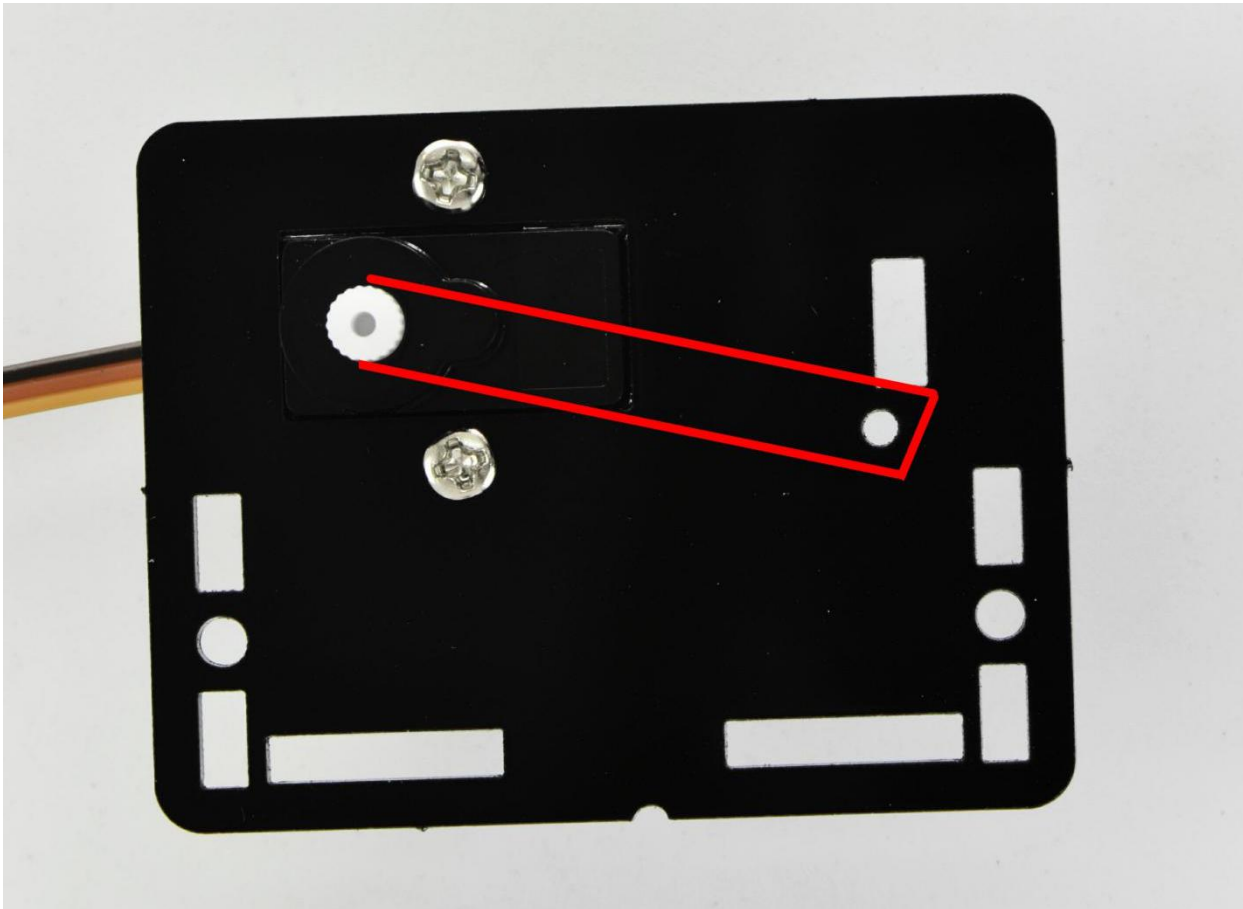
Second, mount the two acrylic plates④ to the servo using two M3*12MM round-head screws③.

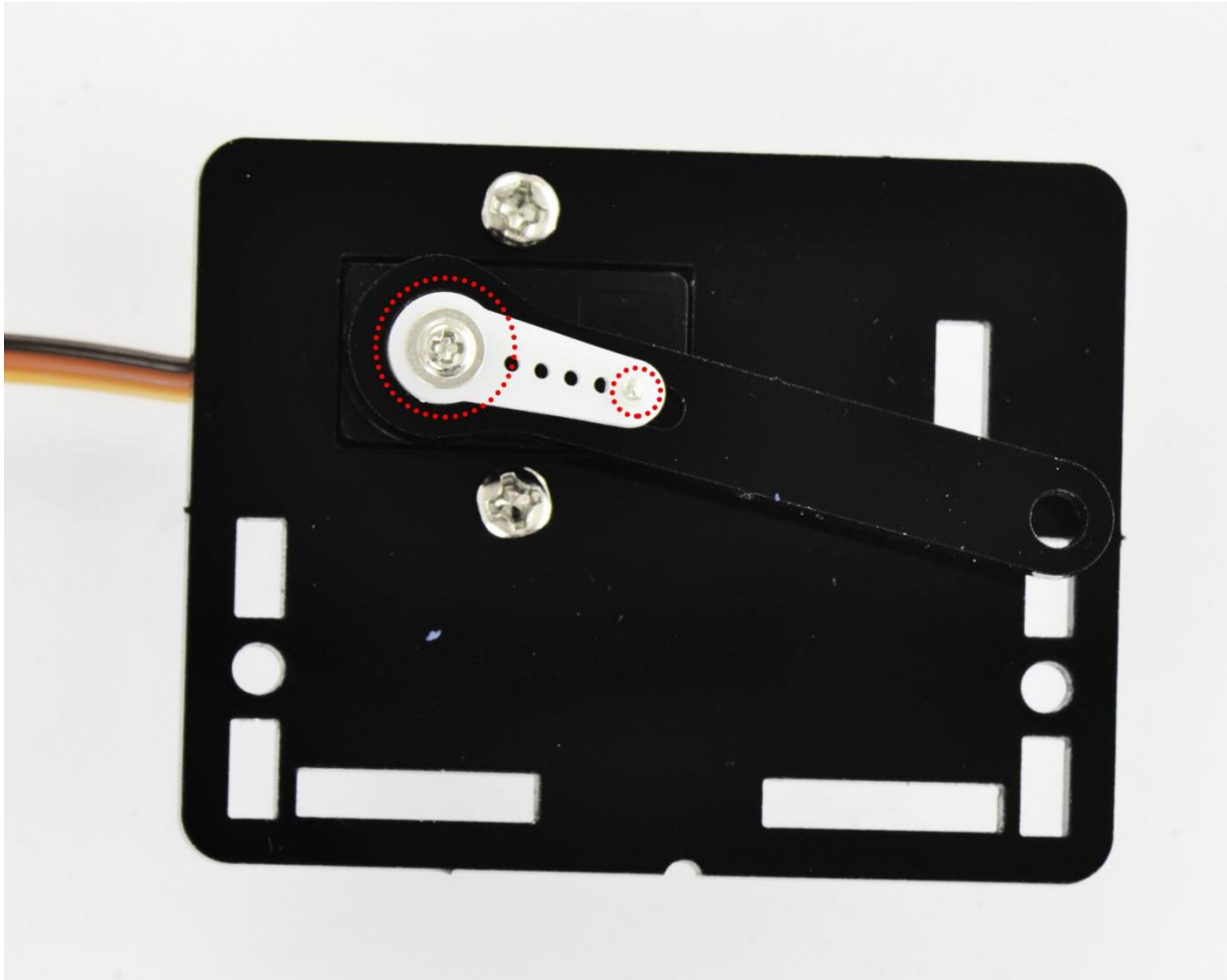


Then screw the Right servo plate to acrylic plate⑤ using two M3 hex lock Nuts⑥.



After that, screw the short acrylic plate joint and white servo mount⑦ to the Right servo motor using a M2x8MM tapping screw and a M2x5MM tapping screw⑧.



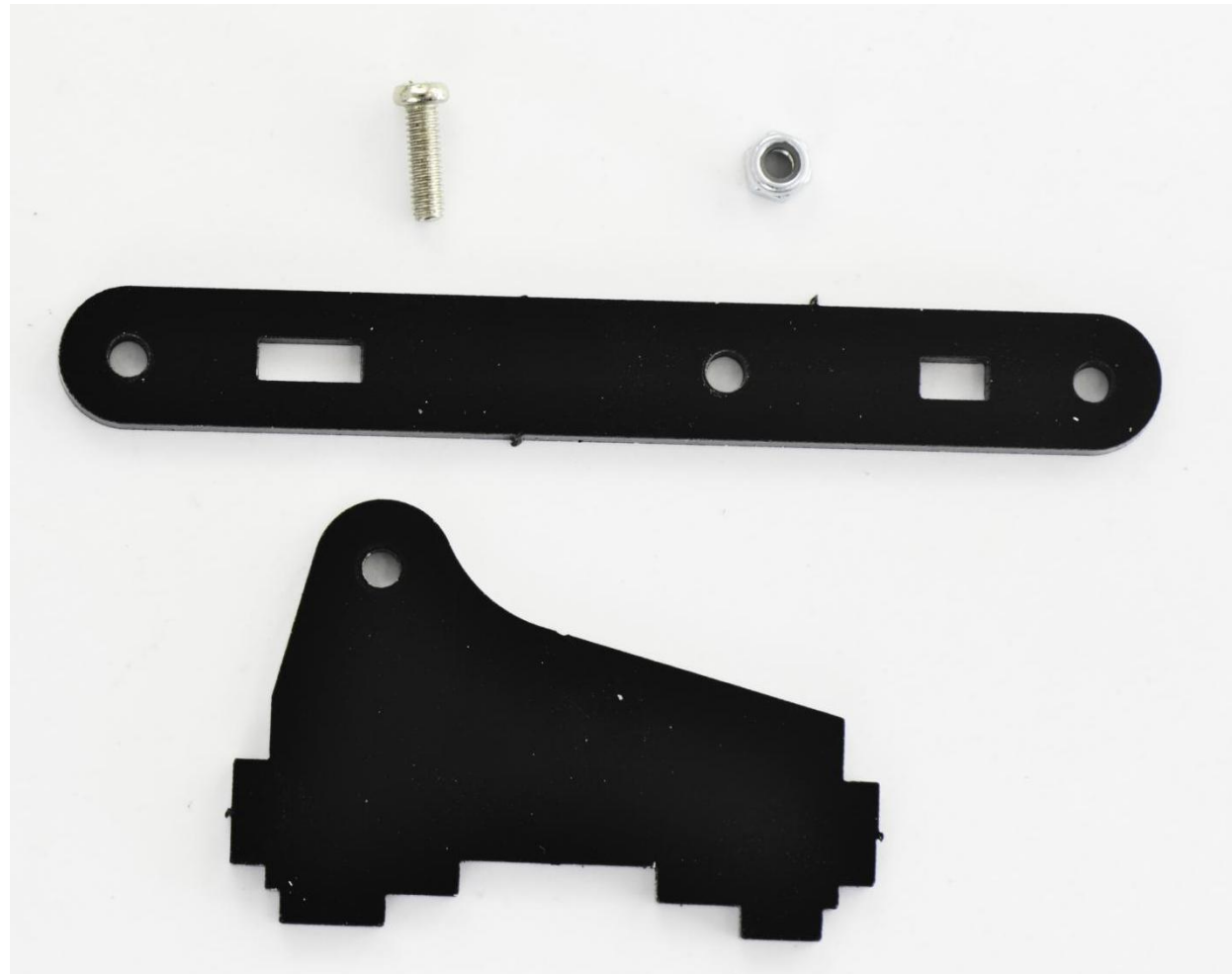


Finally you should get the complete Left Servo Plate. Shown below.

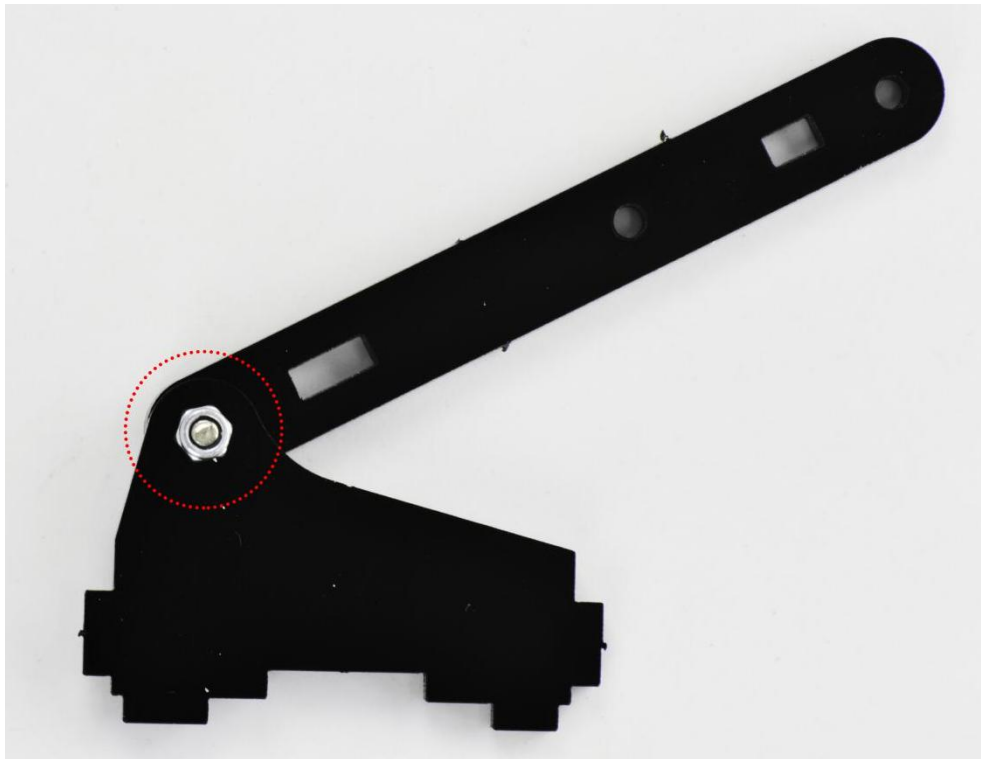


(4) Next install the **Servo Wrist Joint**.

- M3*10MM round-head screw *1
- M3 hex lock nut *1
- Acrylic plate*2

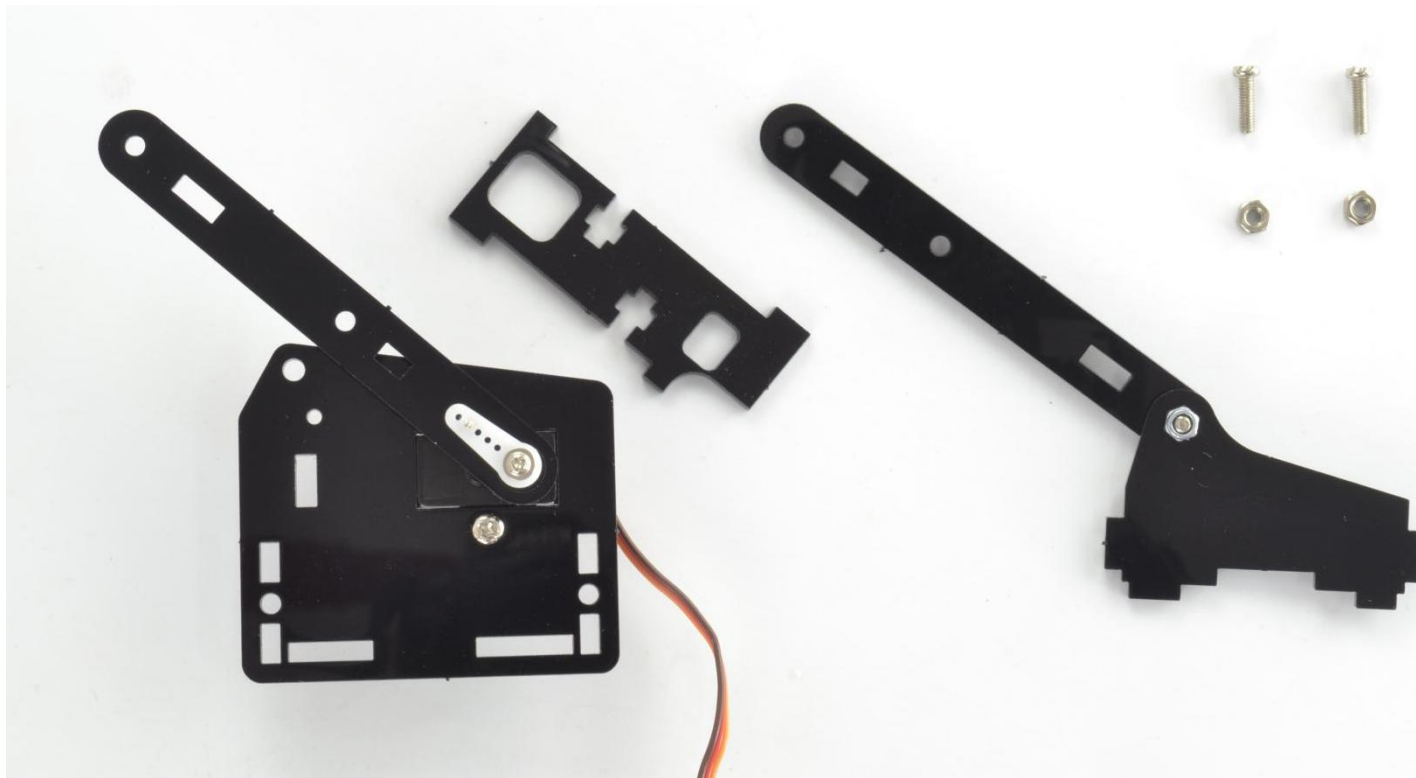


Fix the two Acrylic plates together using a M3*10MM screw and a M3 lock nut.
You can get the Wrist Joint Plate shown below.

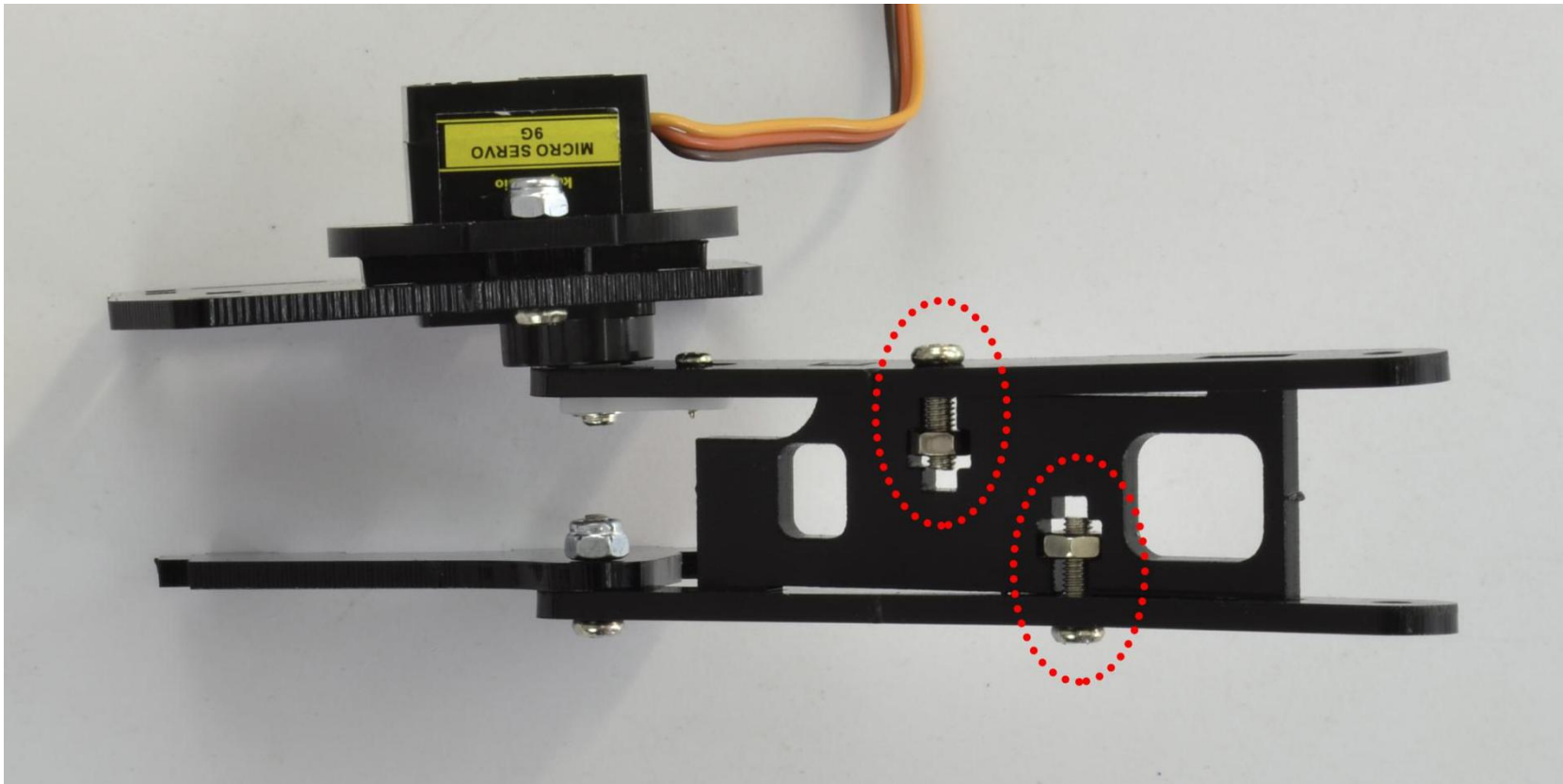


(5) The above parts are installed well. Mount the Wrist Joint and Left Arm Servo Plate together.

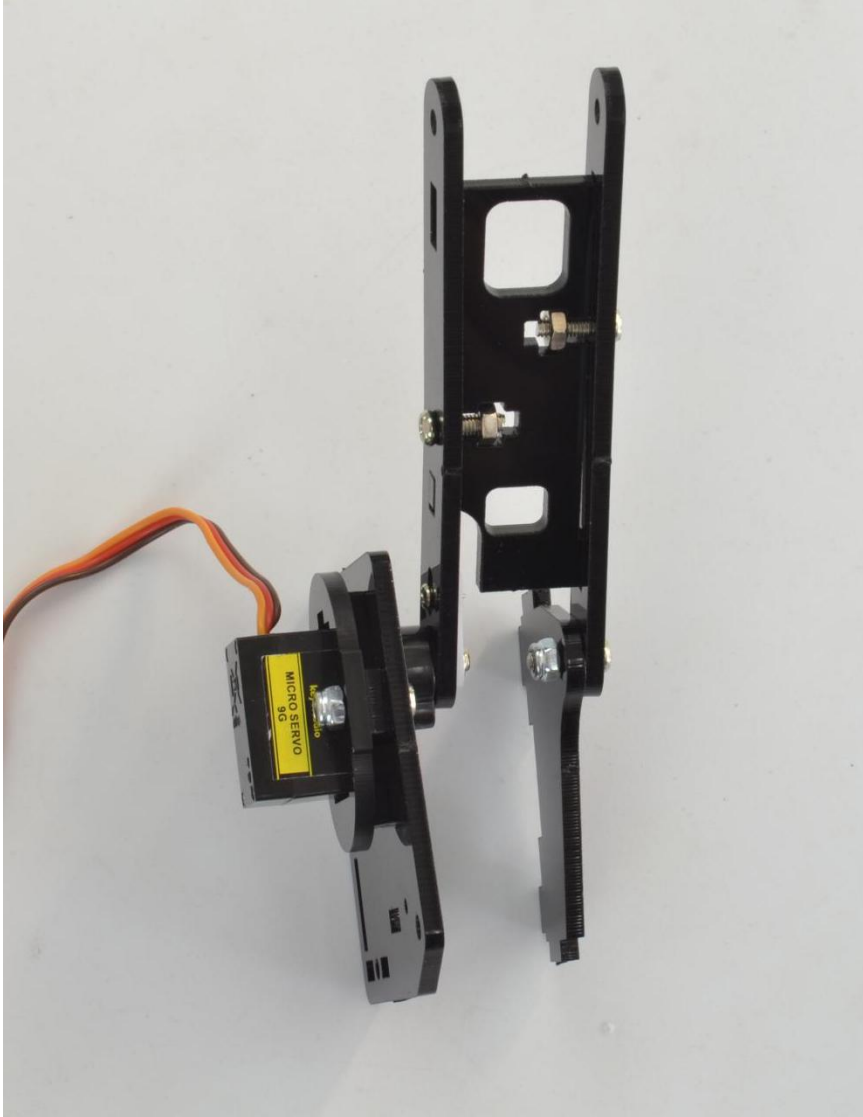
- M3*10MM round-head screw *2
- M3 hex nut *2
- Acrylic plate*1
- Left Arm Servo Plate
- Wrist Joint Plate



Install both Left Arm Servo Plate and Wrist Joint plate to a single acrylic plate using two M3 hex nuts and two M3*10MM round-head screws.

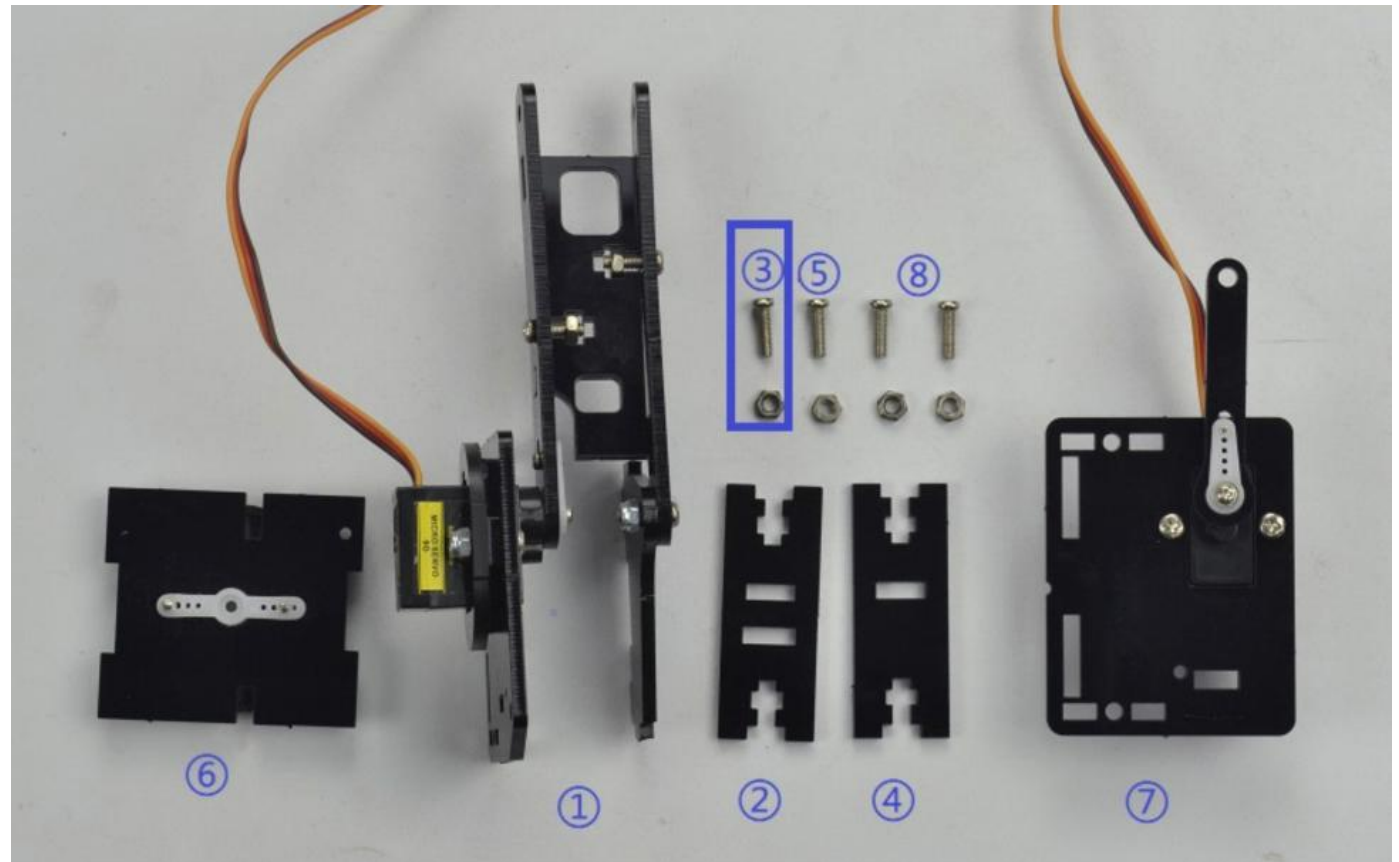


You should get the complete parts shown below.

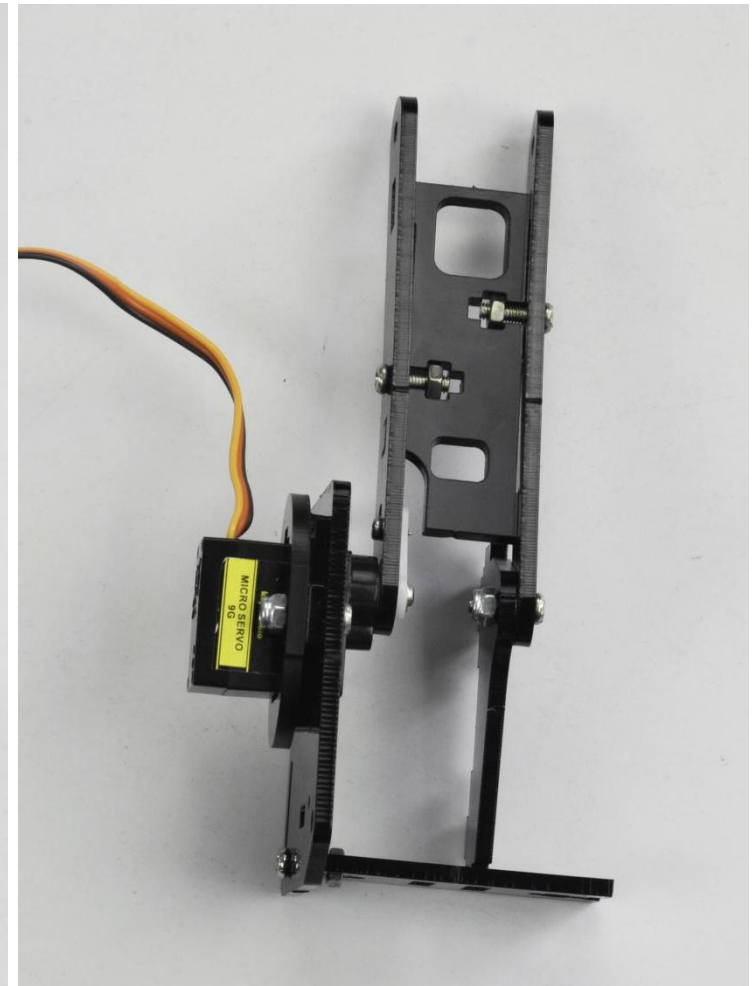
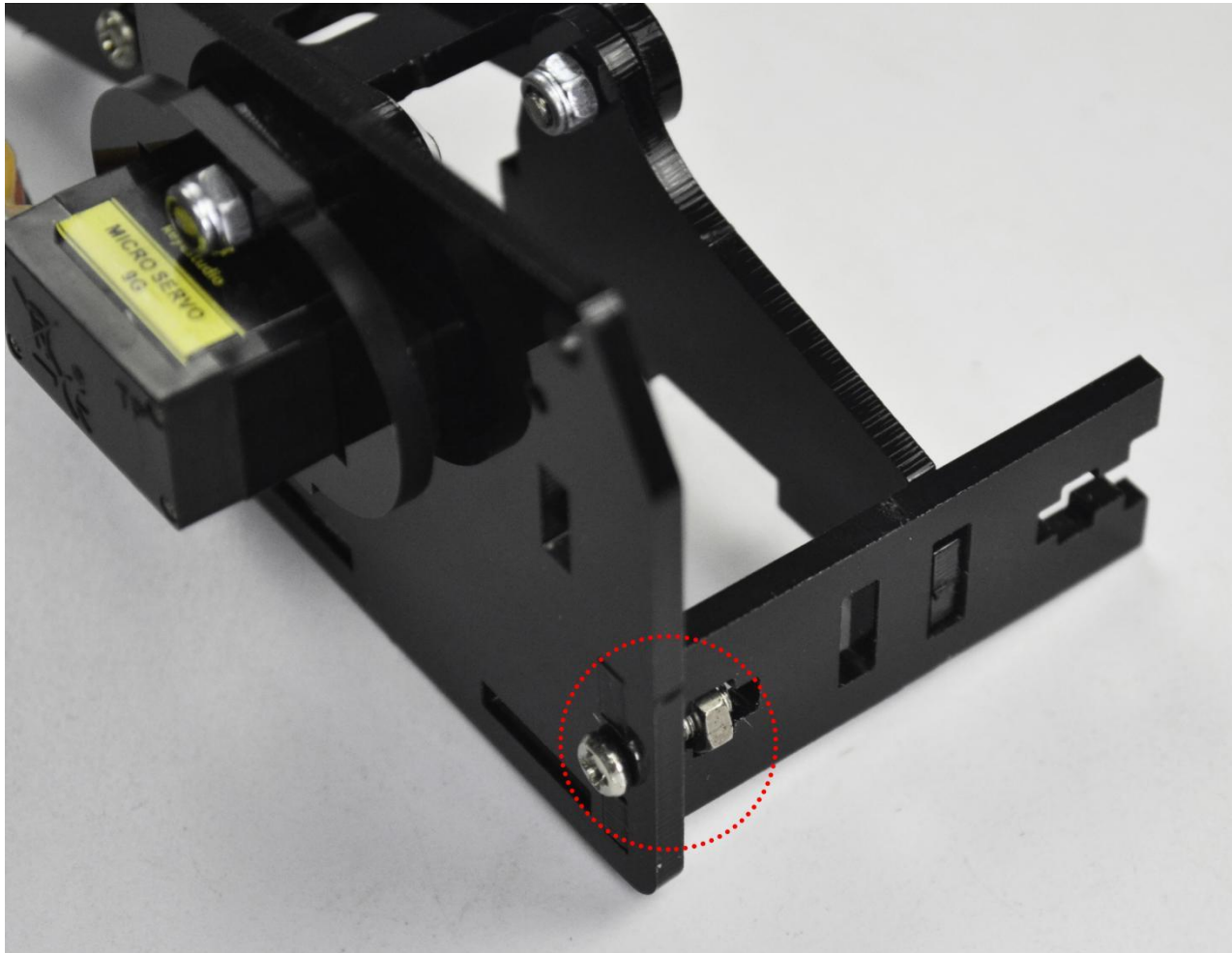


(6) Complete the whole **Arm Base Parts**.

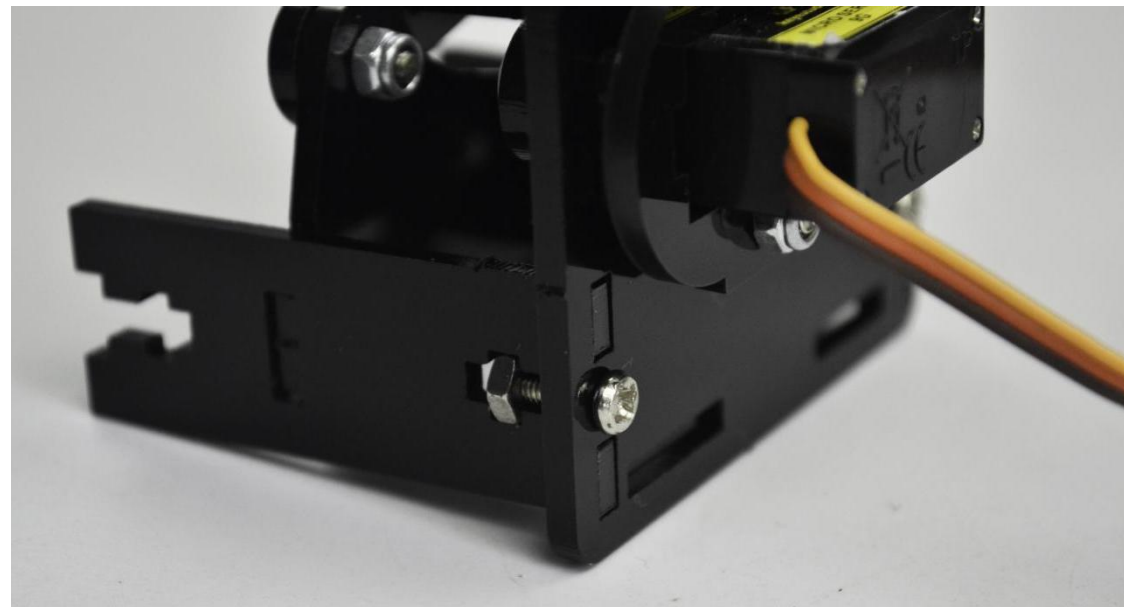
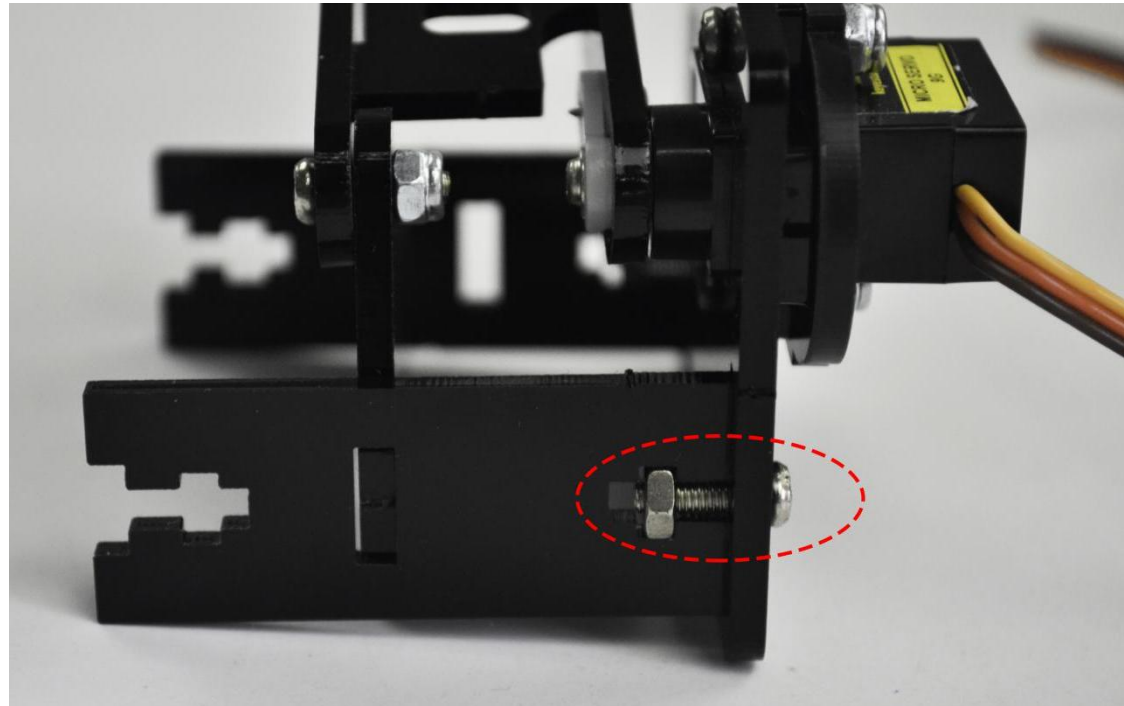
- M3*10MM round-head screw *4
- M3 hex nut *4
- Acrylic plate *2
- Arm Bottom Plate⑥
- Left Arm Servo Plate①
- Right Arm Servo Plate⑦



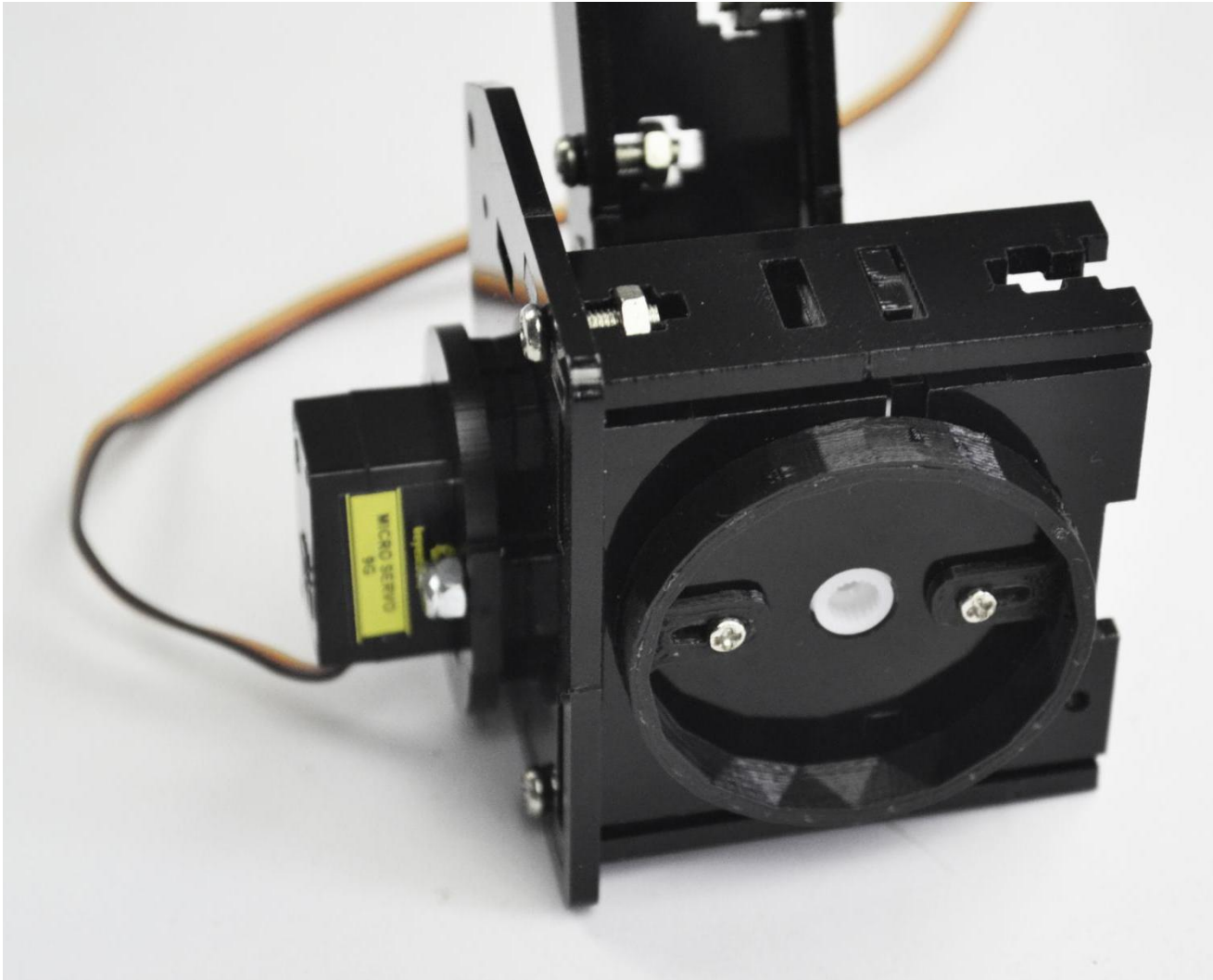
Mount the Acrylic plate② to Left Arm Servo Plate① using a M3*10MM round-head screw and a M3 hex nut③.



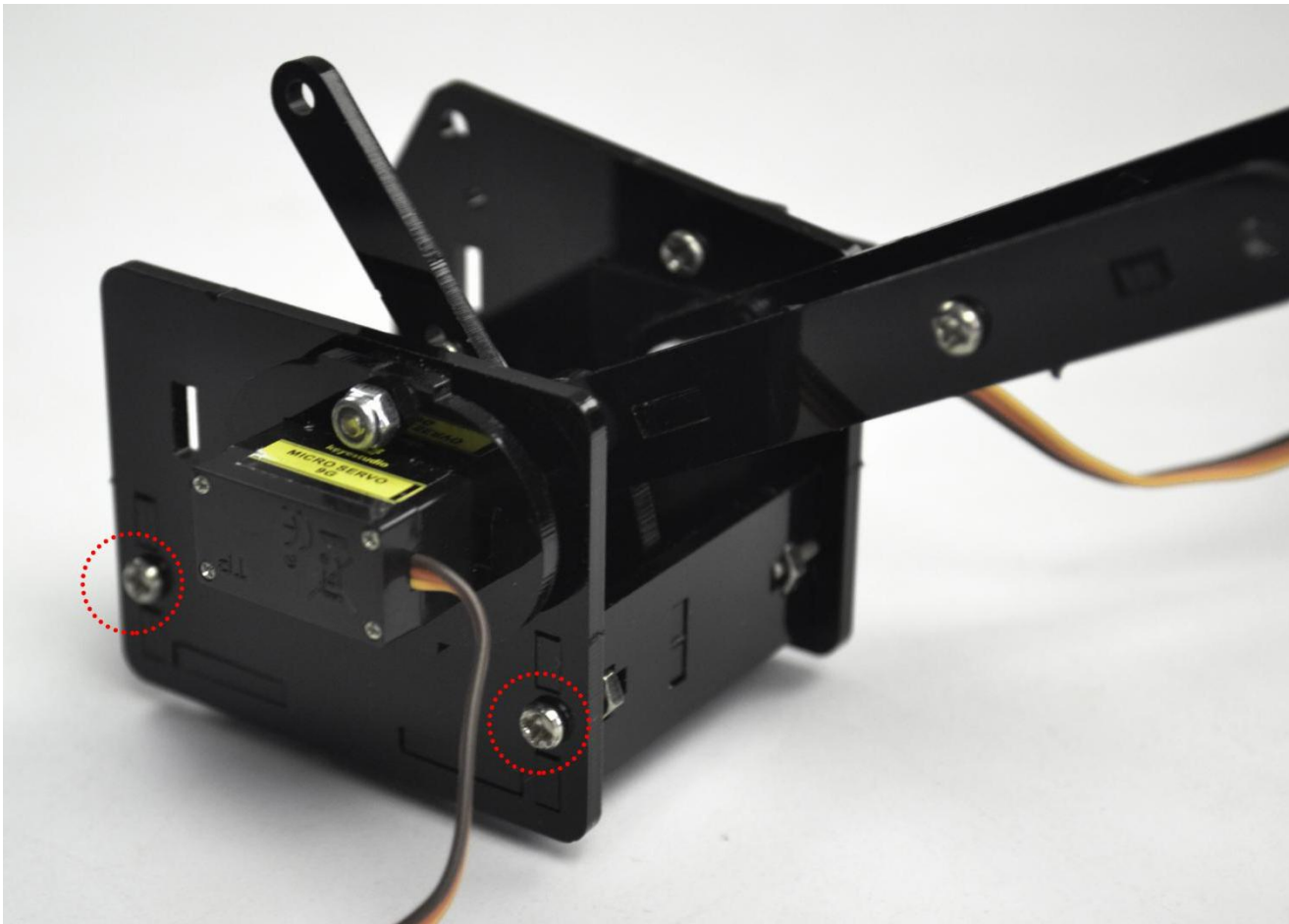
Then screw another Acrylic plate④ to Left Arm Servo Plate① using a M3*10MM round-head screw and a M3 hex nut.



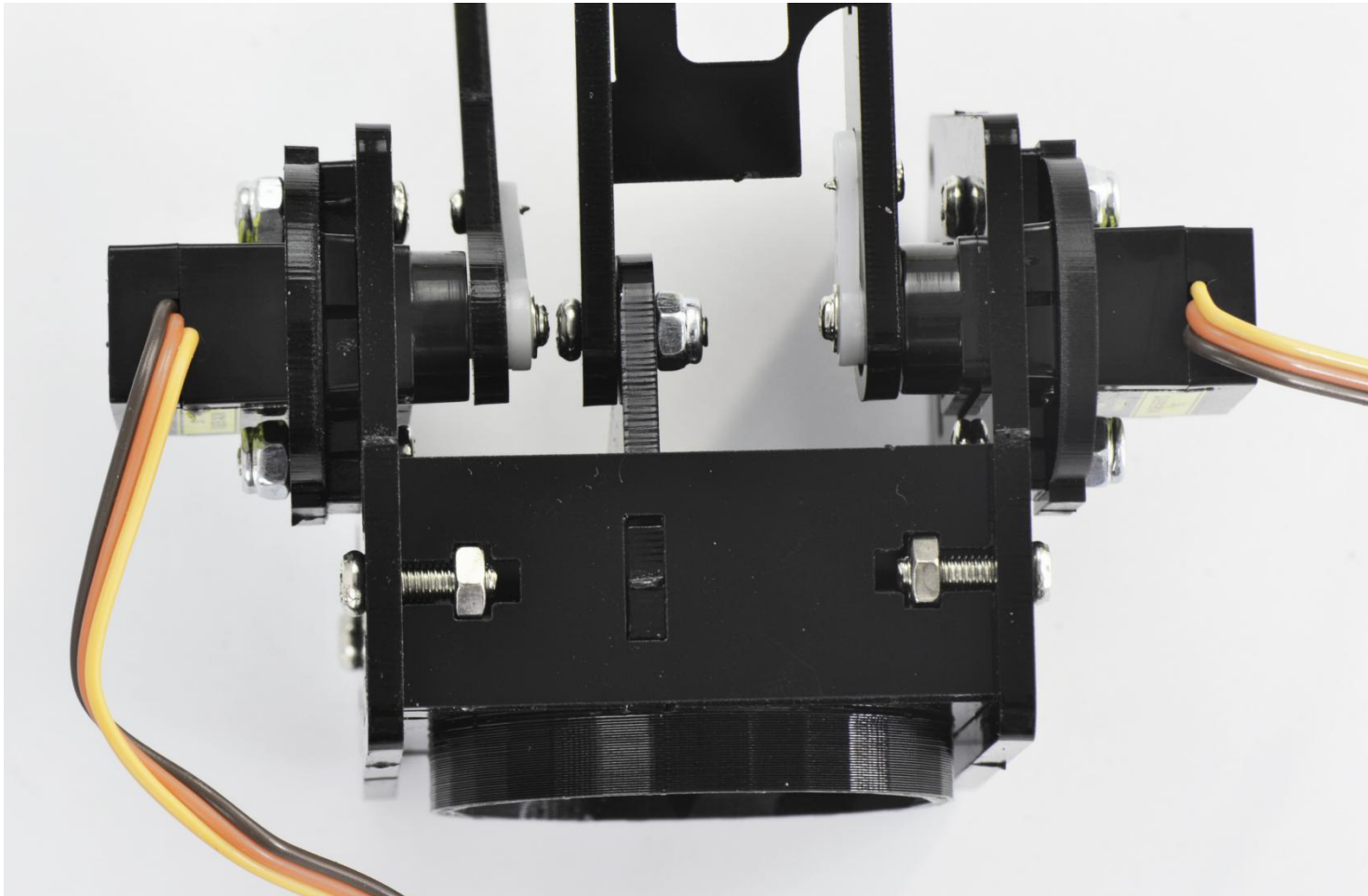
After that, snap the Arm Bottom Plate⑥ between the both side plate of Arm BasePlate.

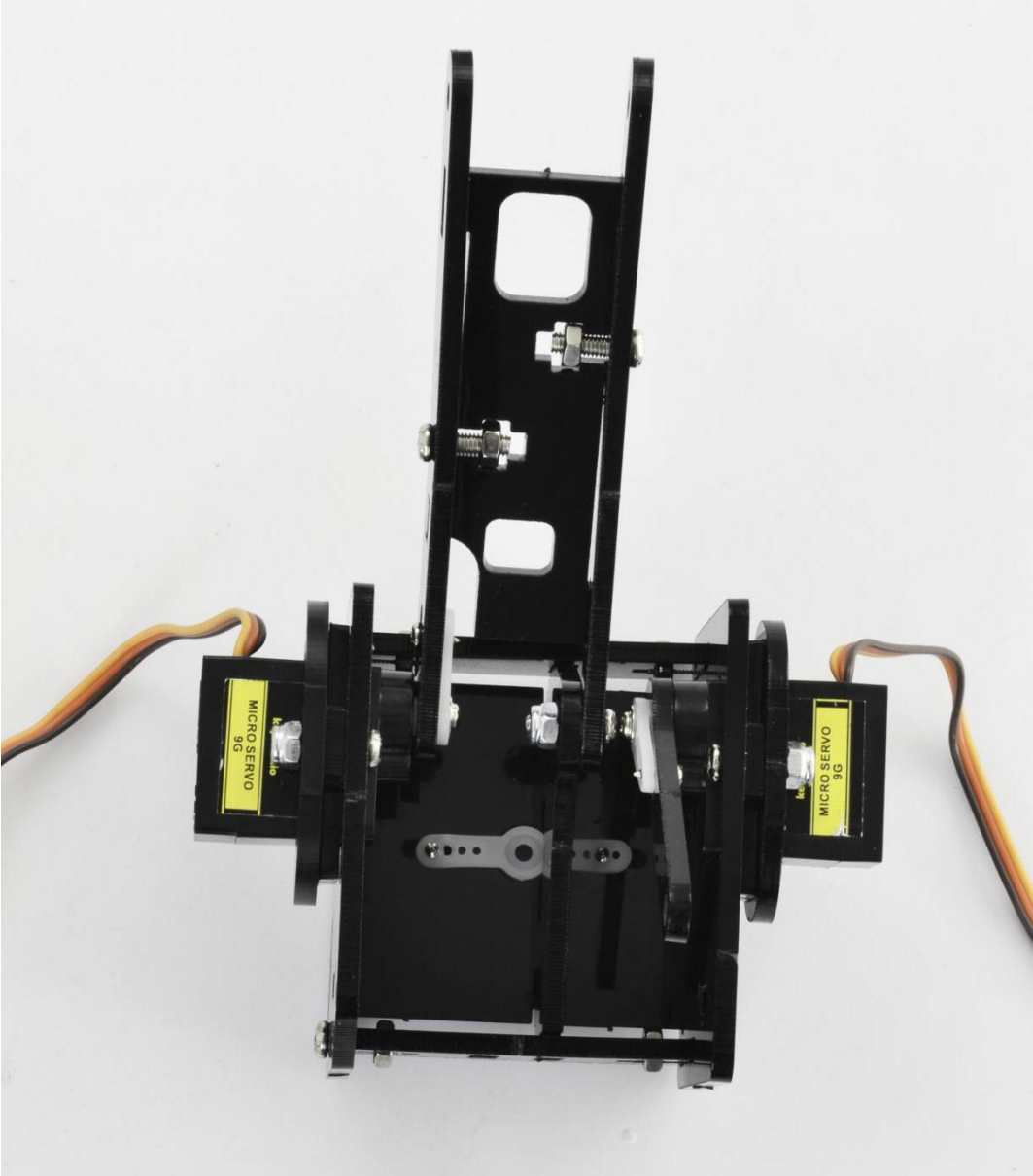


At last, mount the Right Arm Servo Plate ⑦ to the BasePlate using two M3*10MM round-head screws and two M3 hex nuts.



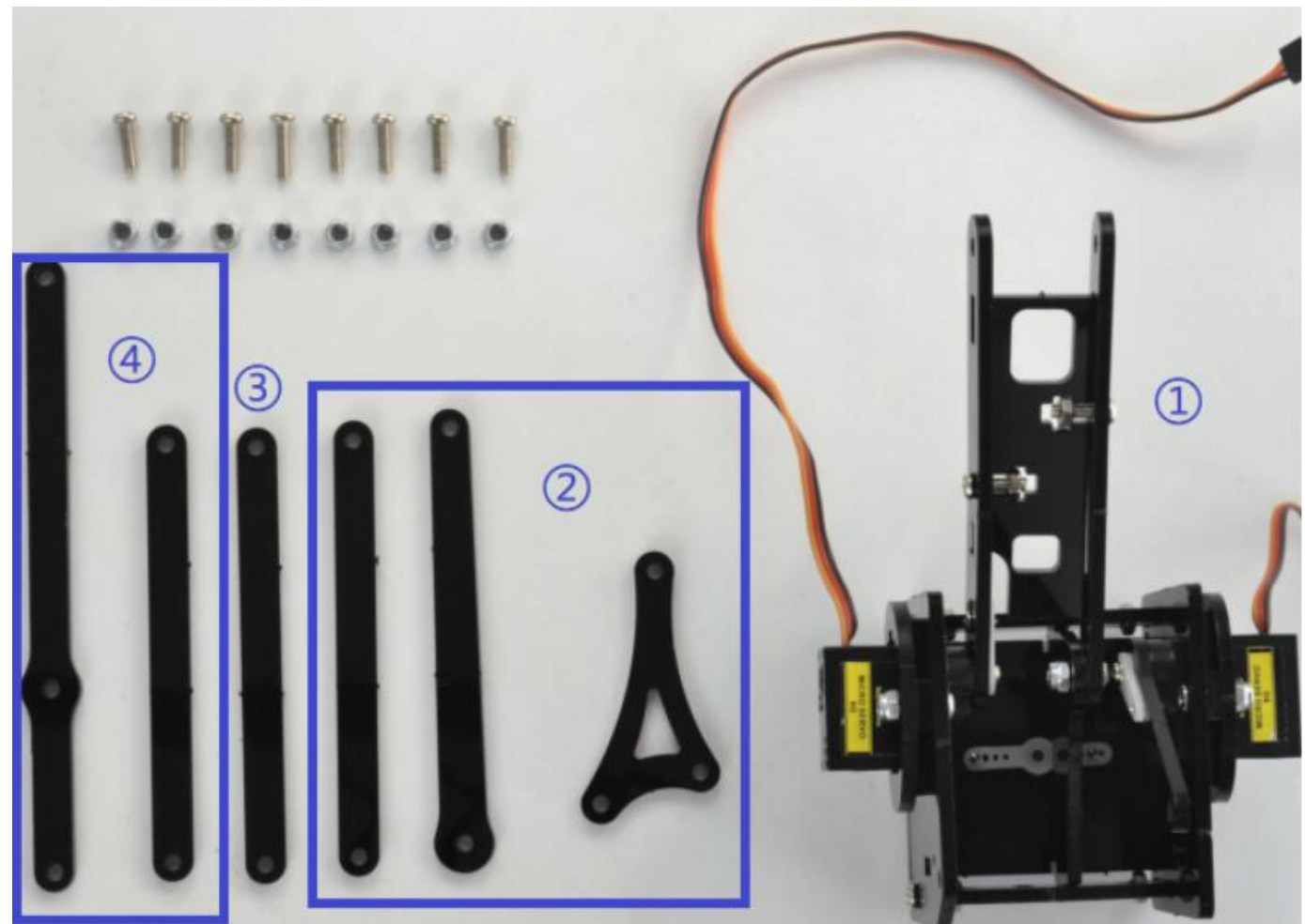
With this step completed, the Arm Base Parts is really starting to take shape!





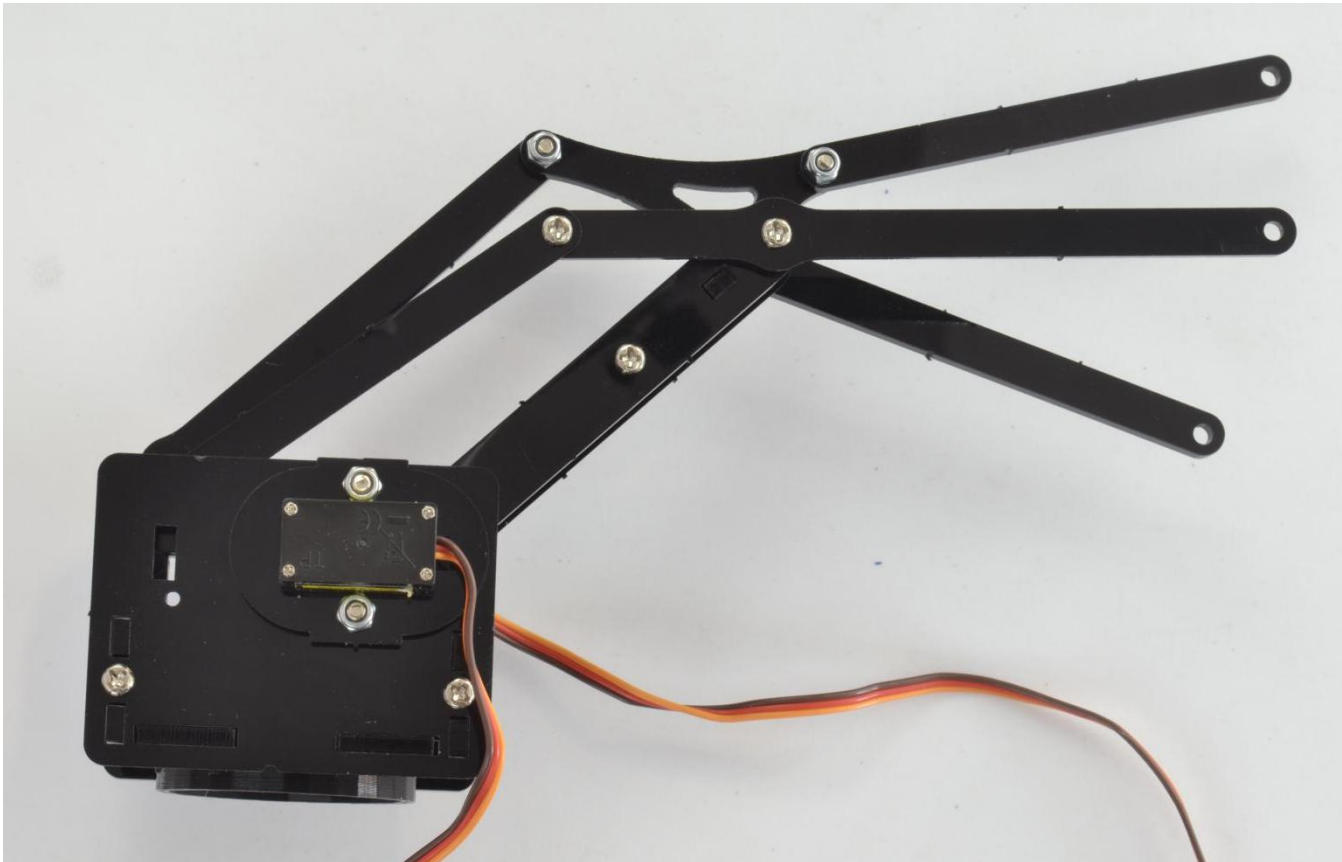
(7) Now we should install the Wrist Joint Connector between the Left and the Right Wrist Joint.

- M3*10MM round-head screw *7
- M3*12MM round-head screw *1
- M3 hex lock nut *8
- Acrylic plate*6
- Arm Base Parts①



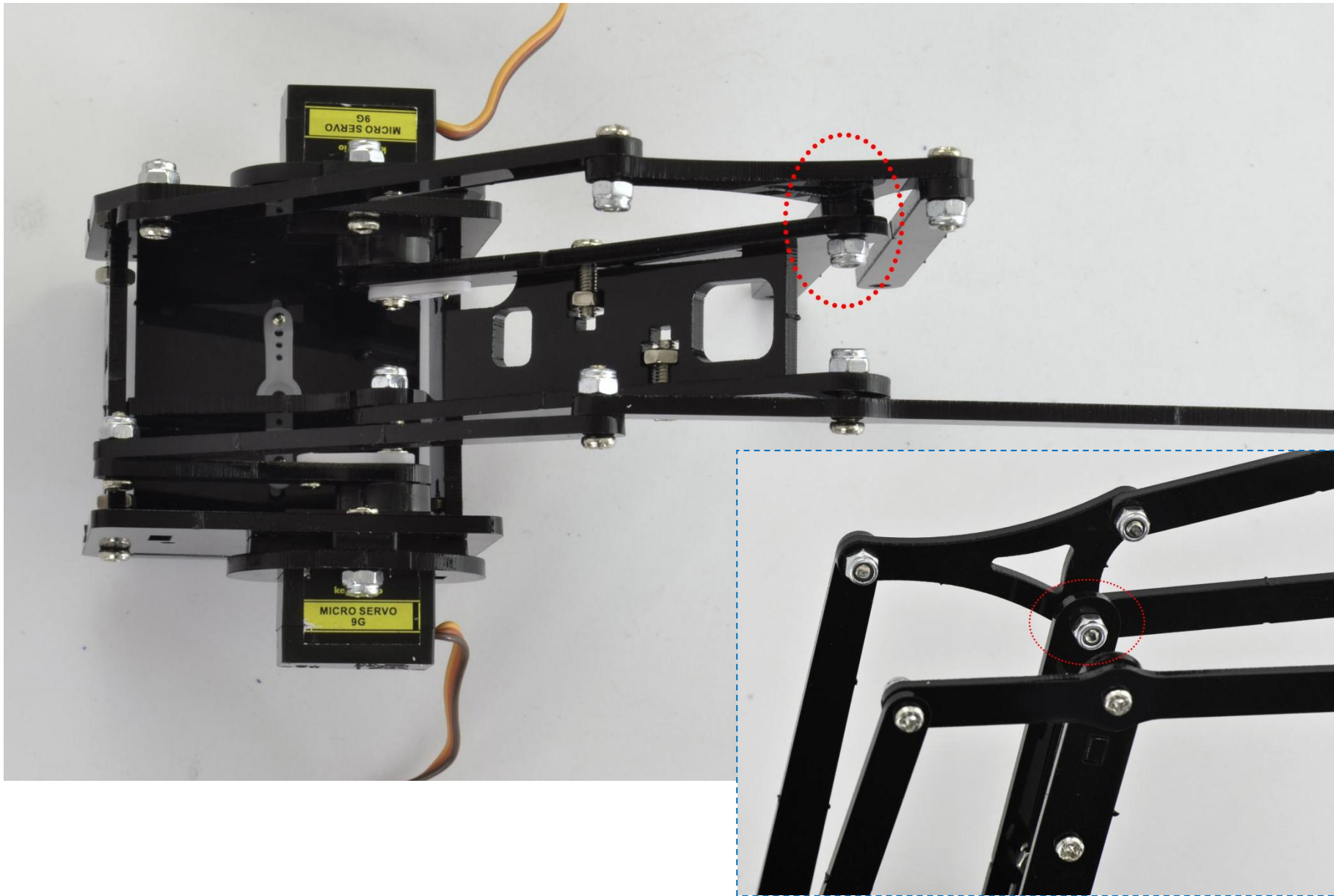
Fix well the Acrylic plates ②③④.



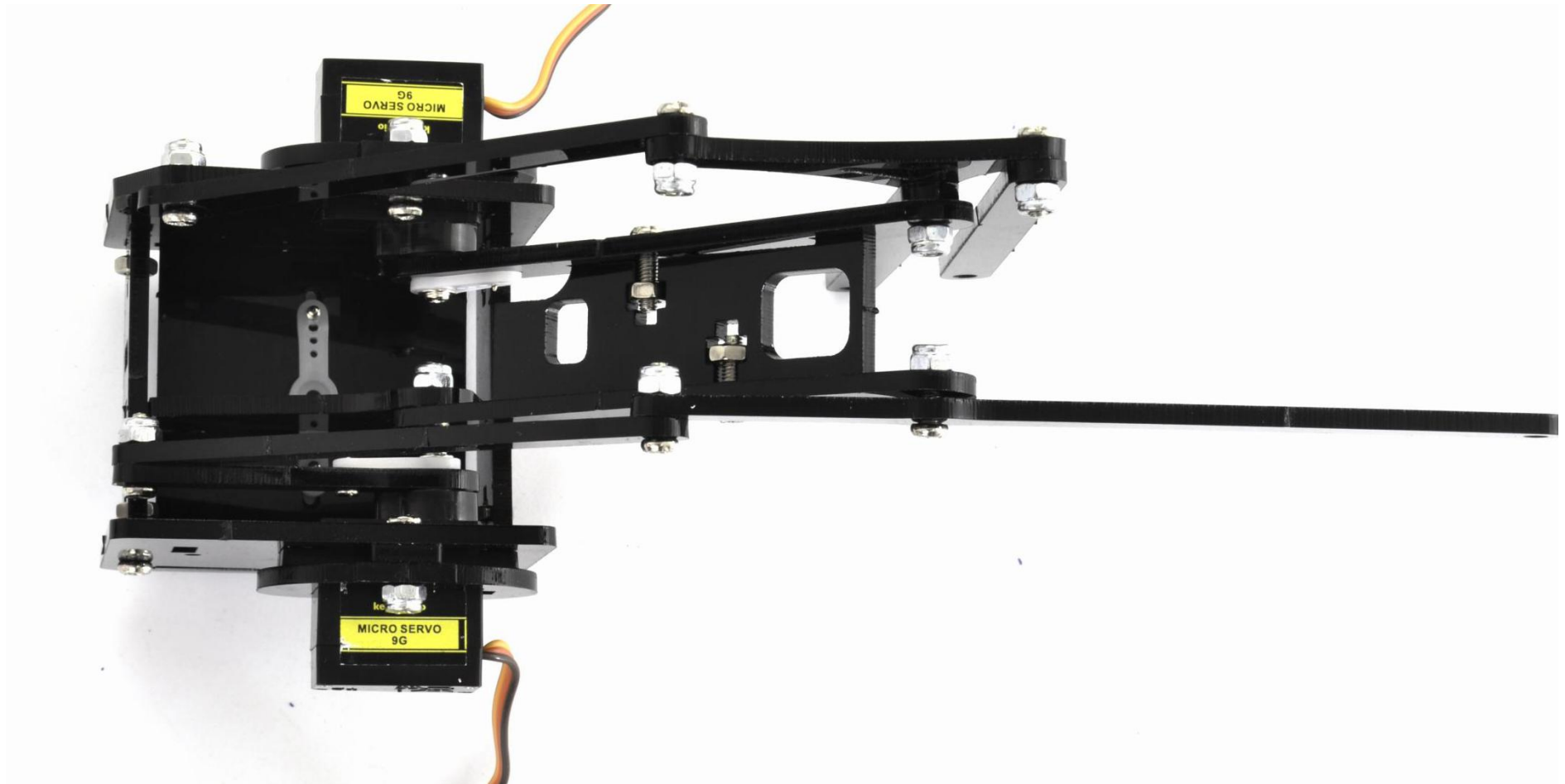


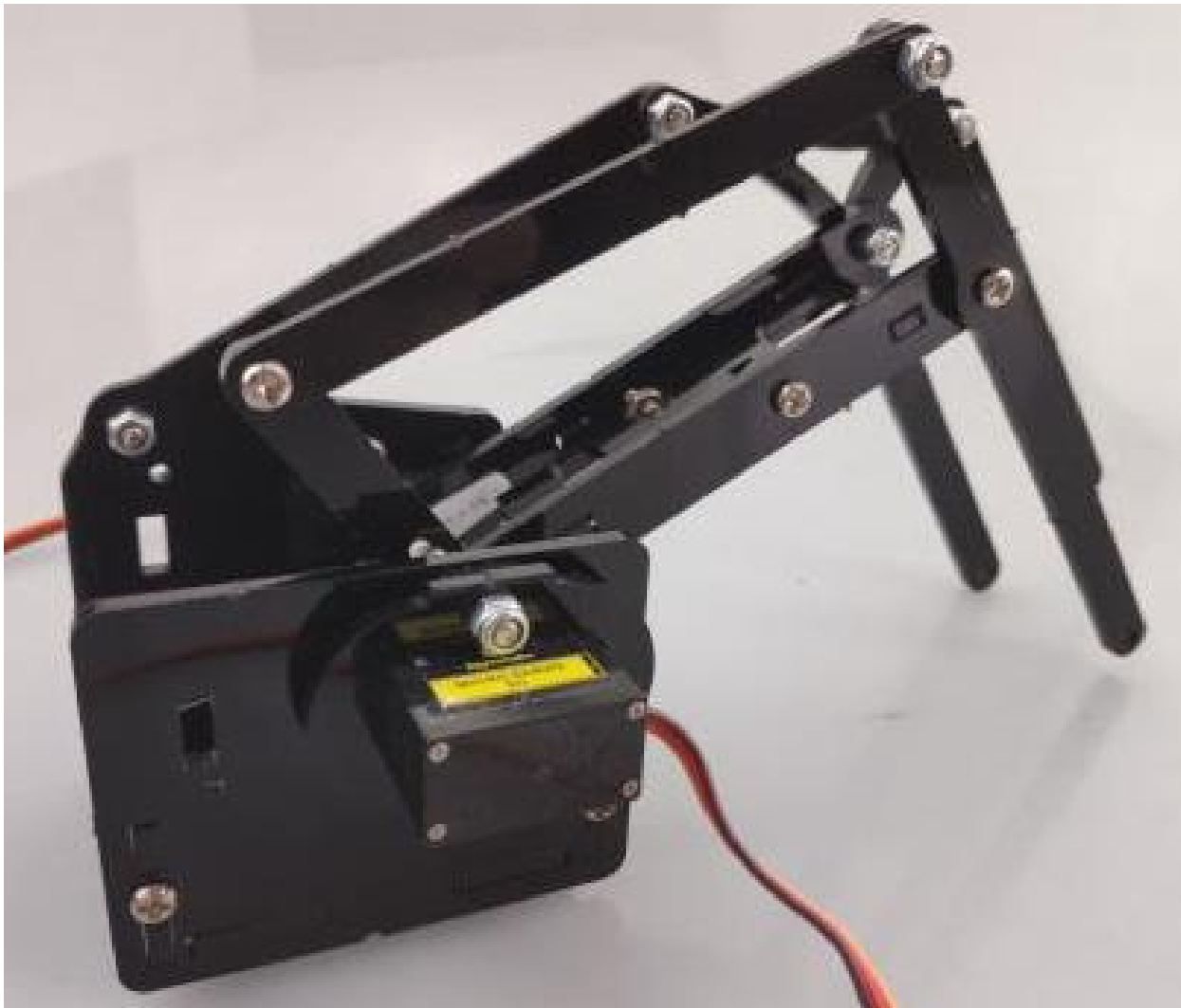
Then screw them to Arm Base Parts① with 7 M3*10MM screws, a M3*12MM screw and 8 M3 lock nuts.
Don't over-tighten the screws as they serve as pivot points to allow the robot arm to move.

(Note: only red circle there use the M3*12MM screw, other wrist joints all use M3*10MM screw.)



Installed well.

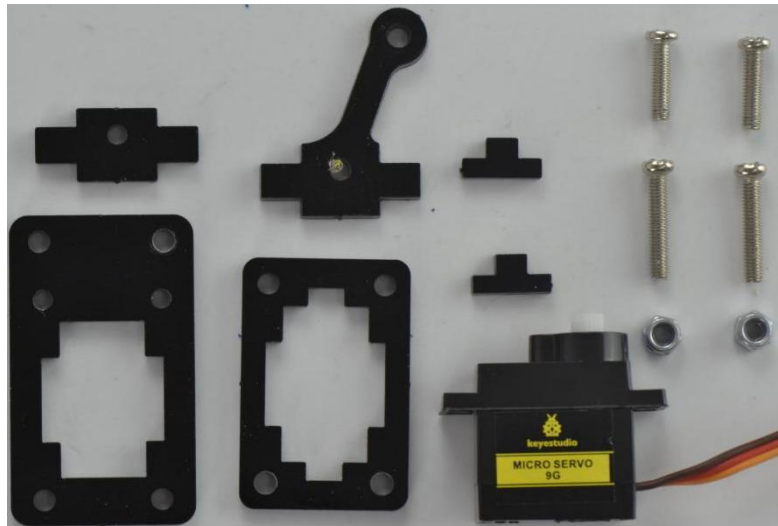


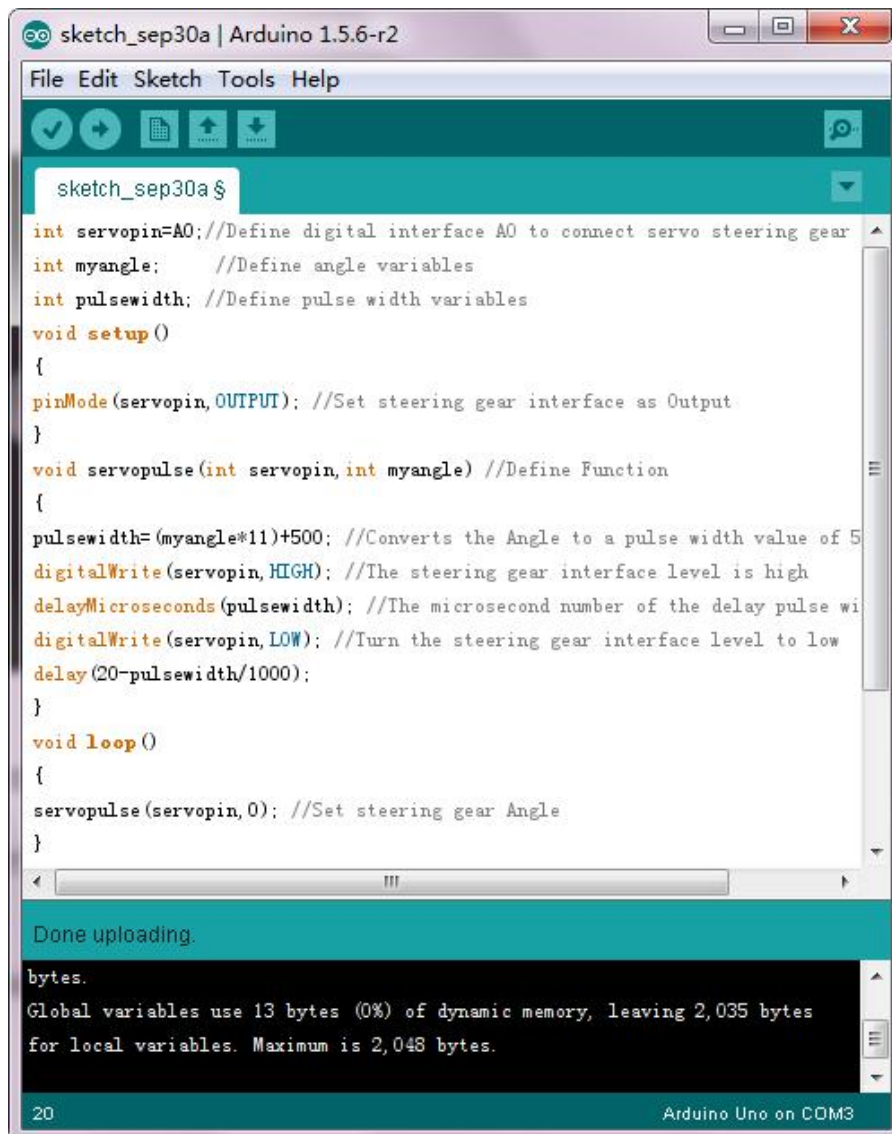


Step3: Assemble the Claw Servo Plate

(1) We are now on the final stretch, it's time to work on the claw.

- M3*16MM flat-head screw *2
- M3*14MM flat-head screw *2
- M3 hex lock nut *4
- Black 180° servo *1
- Acrylic plate*6





```

sketch_sep30a $
int servopin=A0; //Define digital interface A0 to connect servo steering gear
int myangle;      //Define angle variables
int pulswidth;   //Define pulse width variables
void setup()
{
  pinMode(servopin, OUTPUT); //Set steering gear interface as Output
}
void servopulse(int servopin, int myangle) //Define Function
{
  pulswidth=(myangle*11)+500; //Converts the Angle to a pulse width value of 5
  digitalWrite(servopin, HIGH); //The steering gear interface level is high
  delayMicroseconds(pulswidth); //The microsecond number of the delay pulse wi
  digitalWrite(servopin, LOW); //Turn the steering gear interface level to low
  delay(20-pulswidth/1000);
}
void loop()
{
  servopulse(servopin, 0); //Set steering gear Angle
}

```

Done uploading.

bytes.
Global variables use 13 bytes (0%) of dynamic memory, leaving 2,035 bytes
for local variables. Maximum is 2,048 bytes.

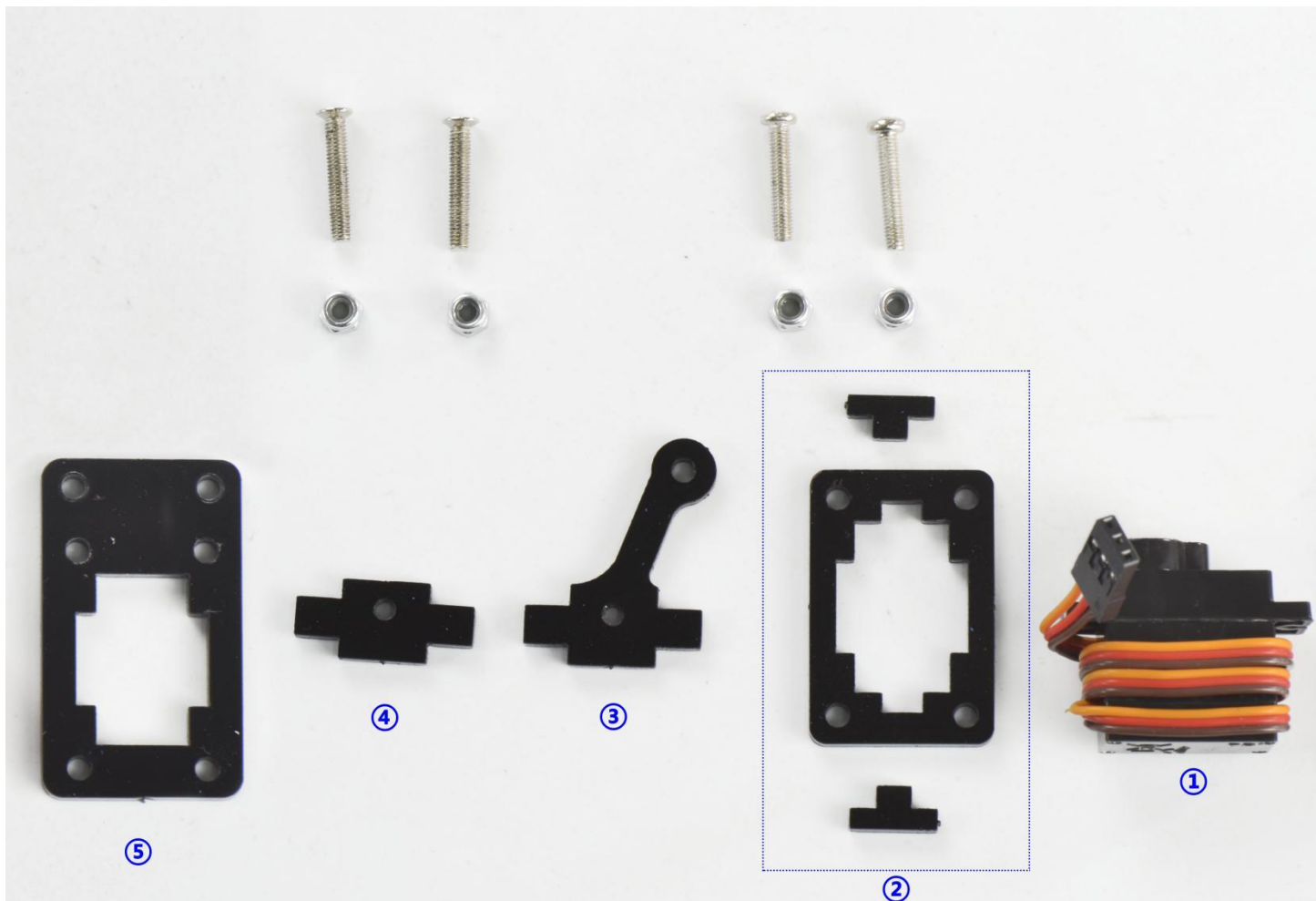
20 Arduino Uno on COM3

Note: before install the claw servo motor, should set the servo angle to 0 degrees.

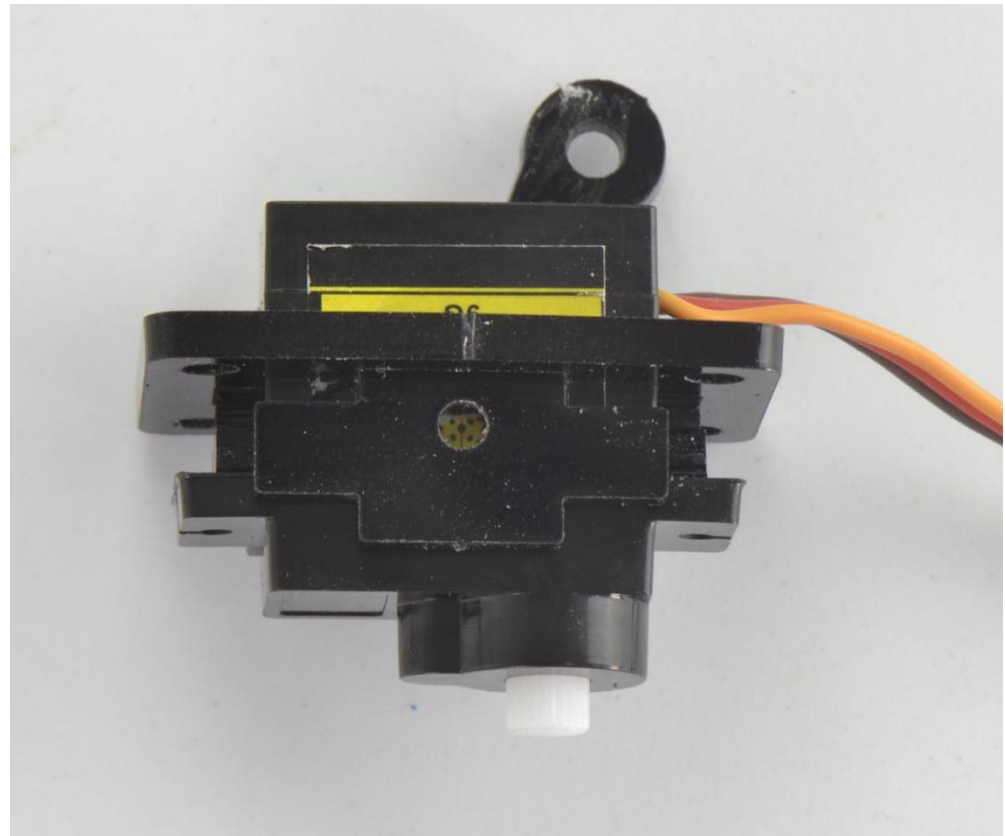
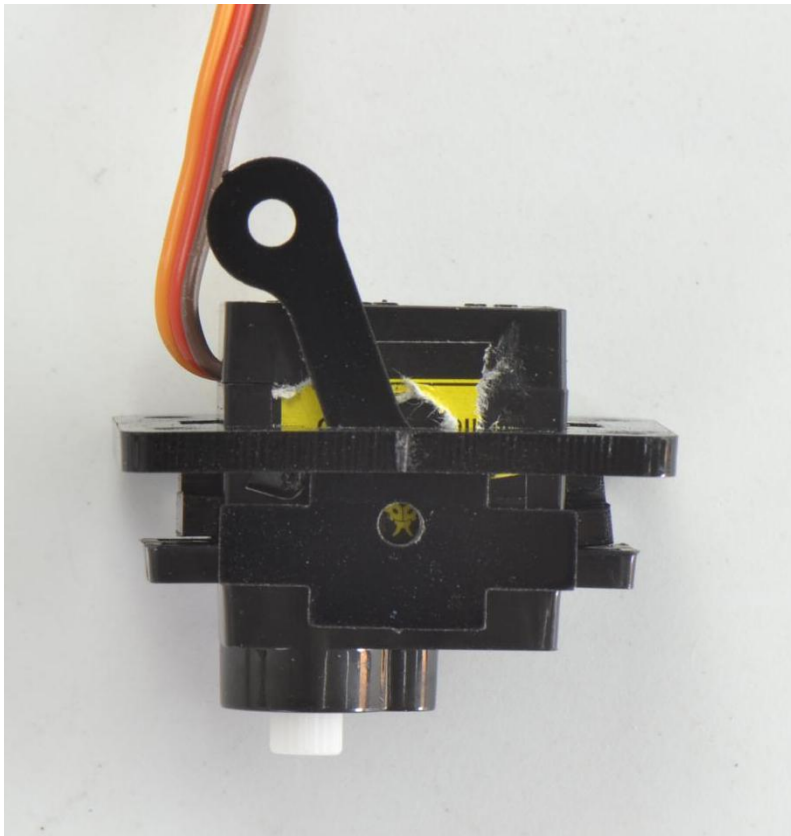
The method is the same as 80° servo settings mentioned above.

You just need to change the `servopulse(servopin,80)` into `servopulse(servopin,0)` in the code.

We will start by mounting the **Claw Servo Motor Plate**.

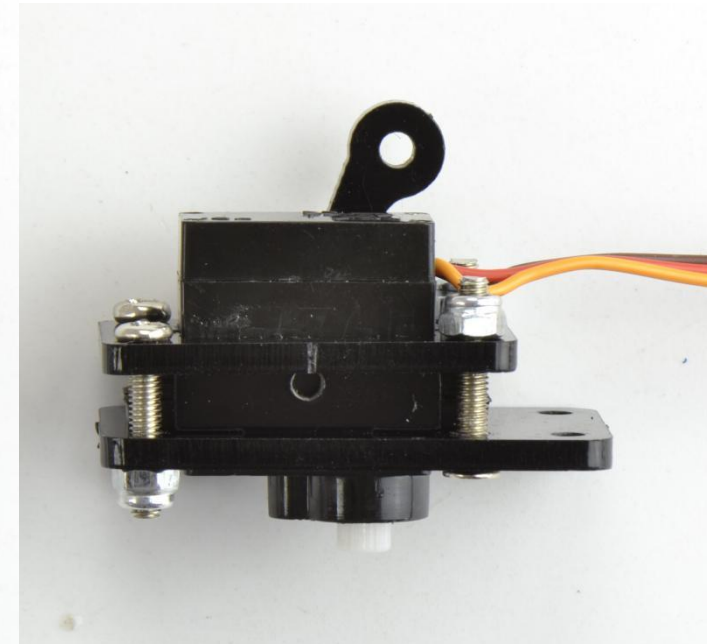
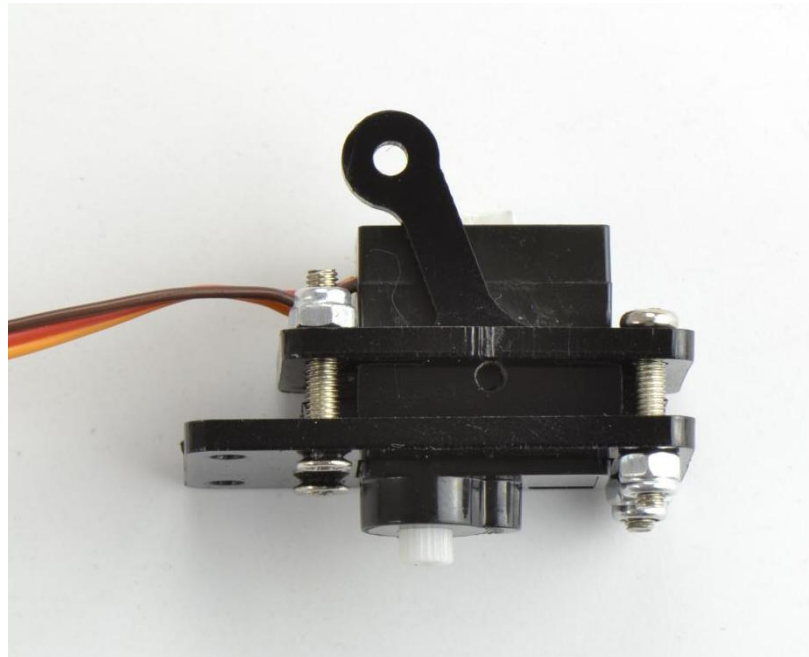
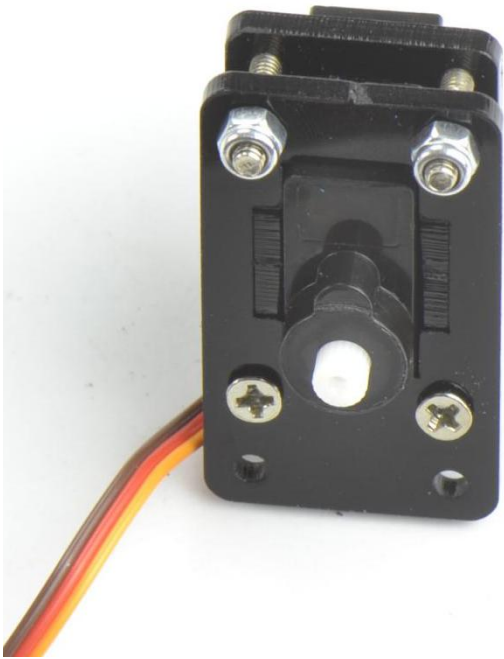


Fix well the Acrylic Plates②③④, and insert the servo motor. Get the servo plate shown below.



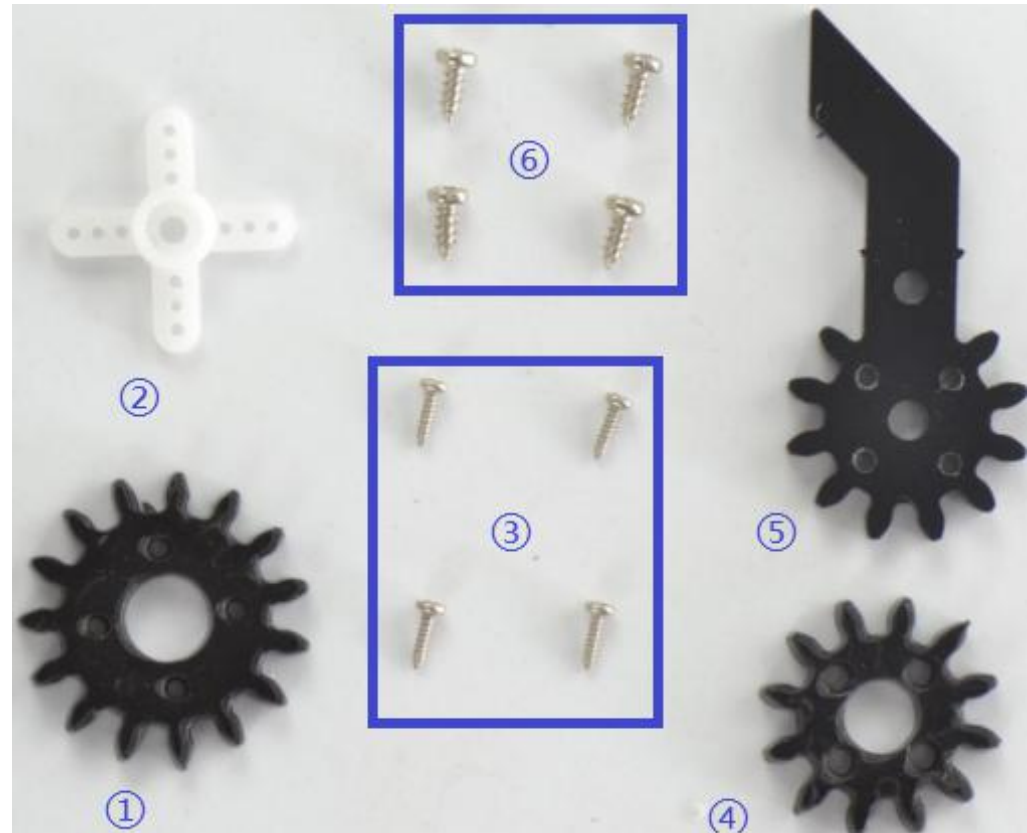
Then tighten a Acrylic Plate⑤ to the above servo plates using two M3*16MM flat-head screws and two

M3*16MM round-head screws and four M3 hex nuts.



(2) Next is to install the **Gripper Plate**.

- M1.2x5MM Phillips tapping screw *4
- M2x5MM Phillips tapping screw *4
- White servo mount *1
- Gripper Acrylic plate *3



Firstly fix the white servo mount② onto acrylic plate① using four M1.2x5MM tapping screws③.

Front view



Rear view

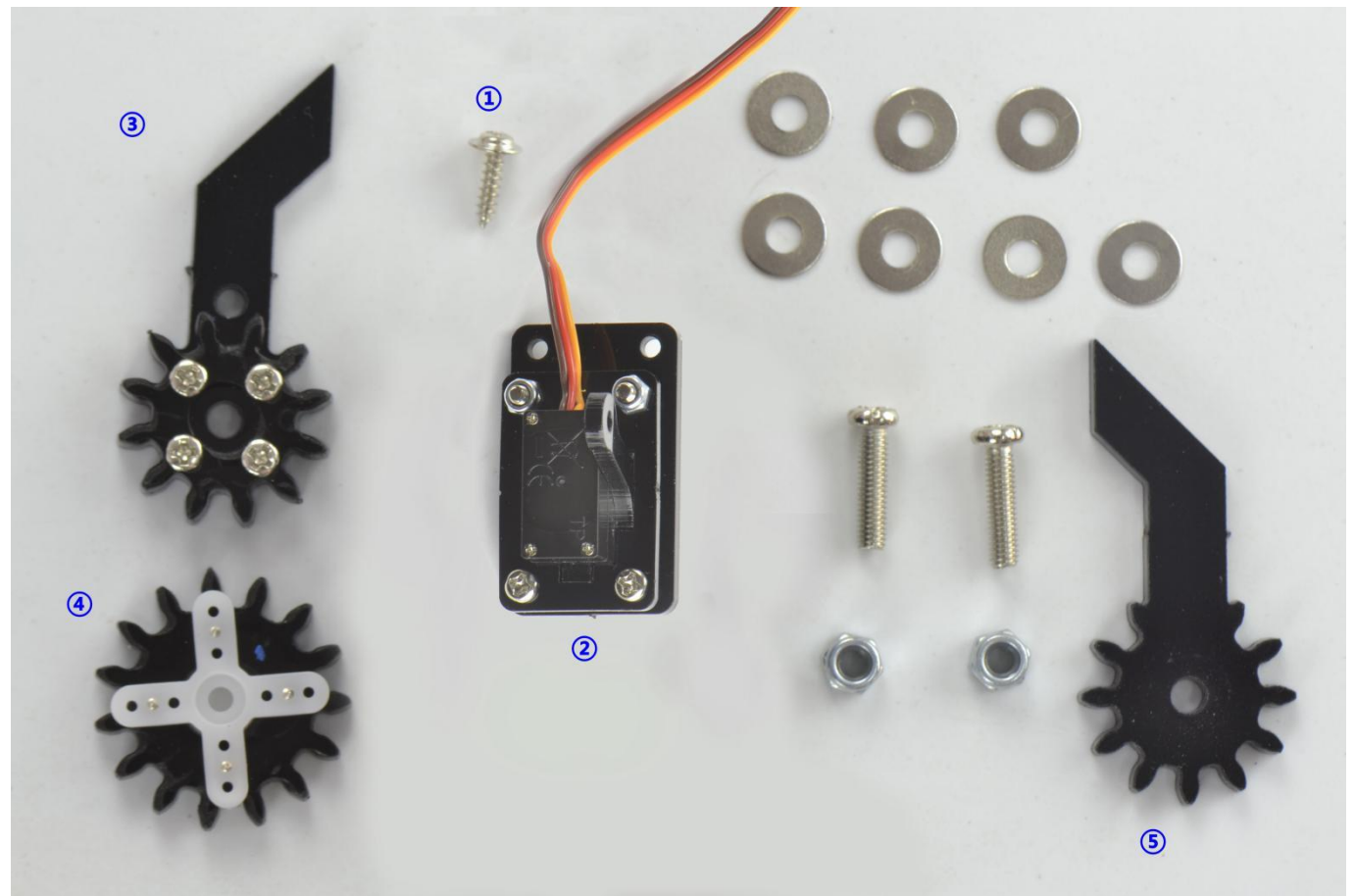


Then fix the acrylic plate④ onto another longer acrylic plate⑤ using four M2x5MM tapping screws.

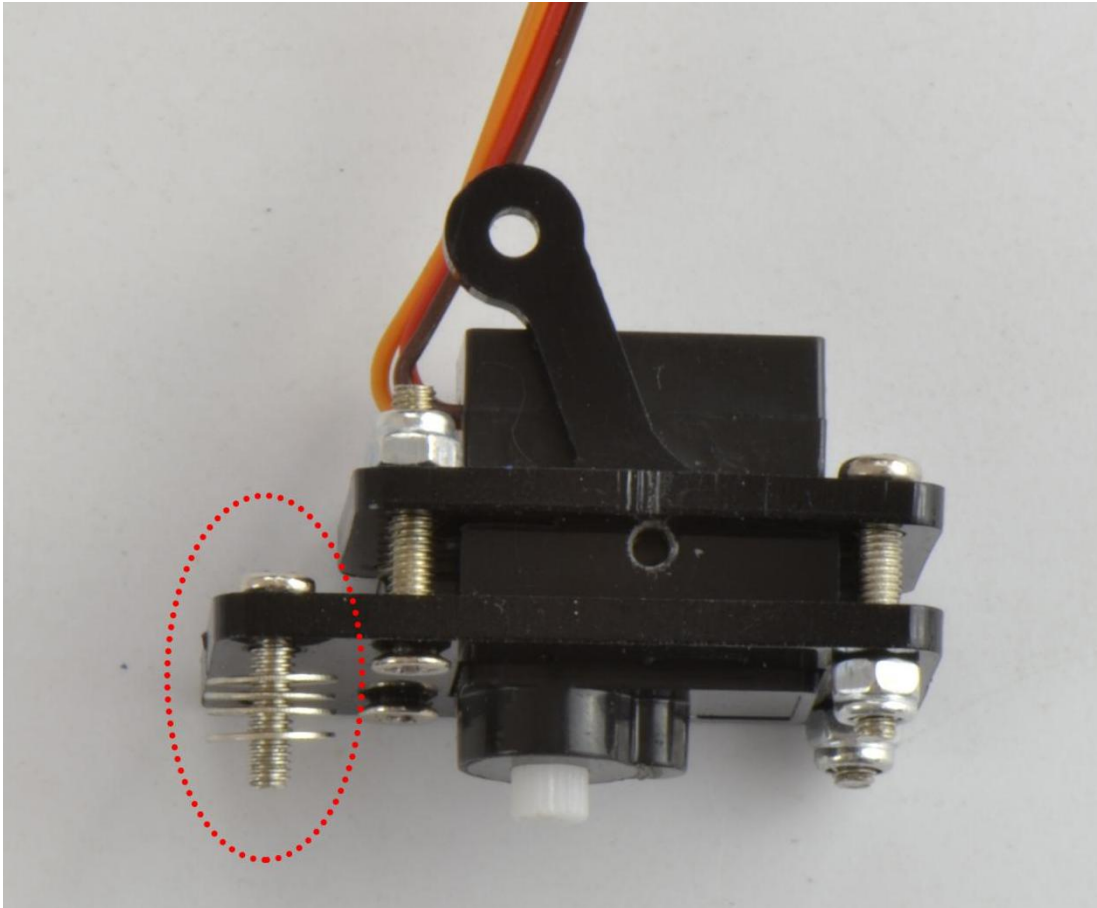


(3) Mount the grippers onto the claw servo plate.

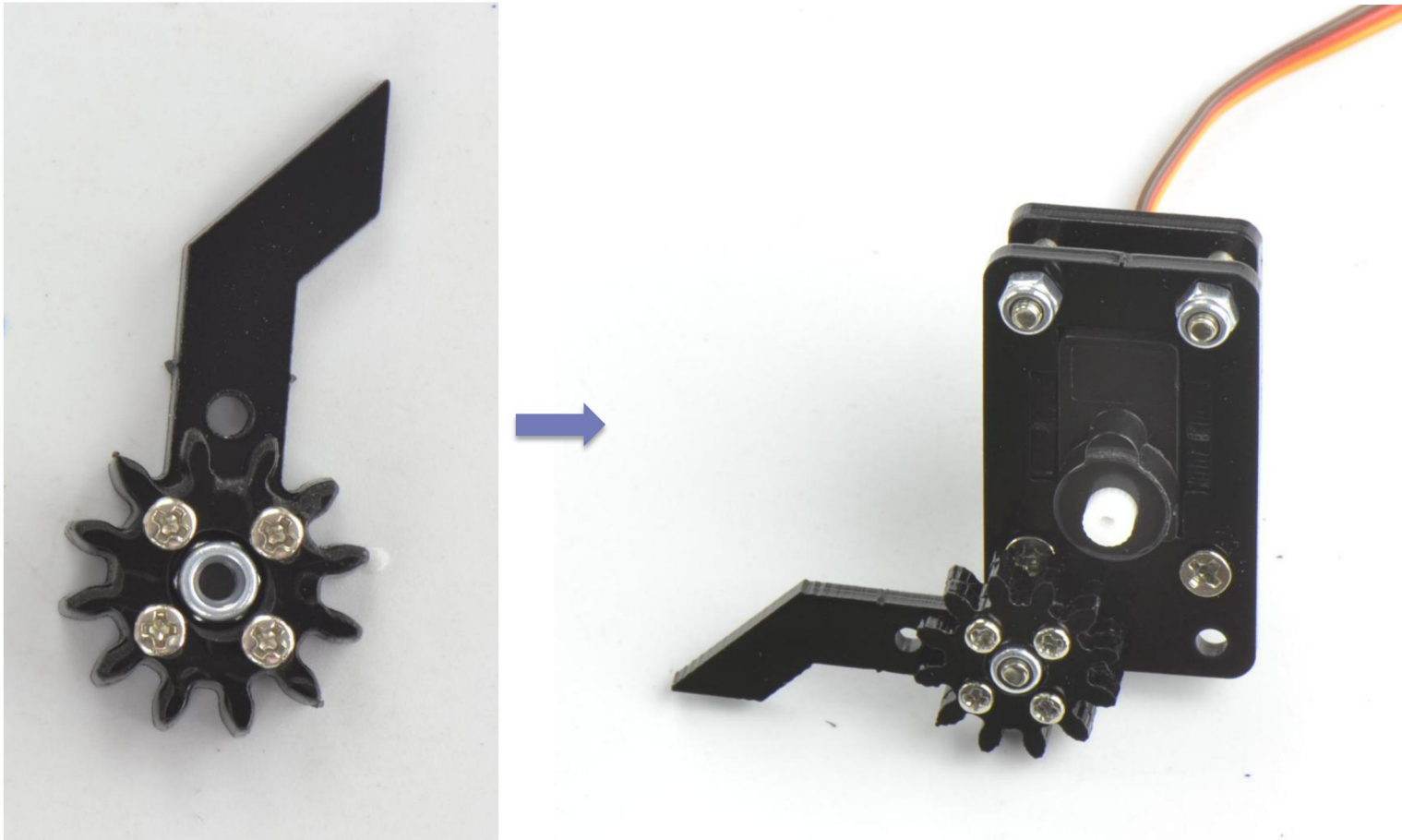
- M3*12MM round-head screw *2
- M3 hex lock nut *2
- M3 flat washer *7
- Servo screw *1 ①
- Claw servo plate ②
- Gripper plate ③
- Gripper plate ④
- Gripper Acrylic plate*1 ⑤



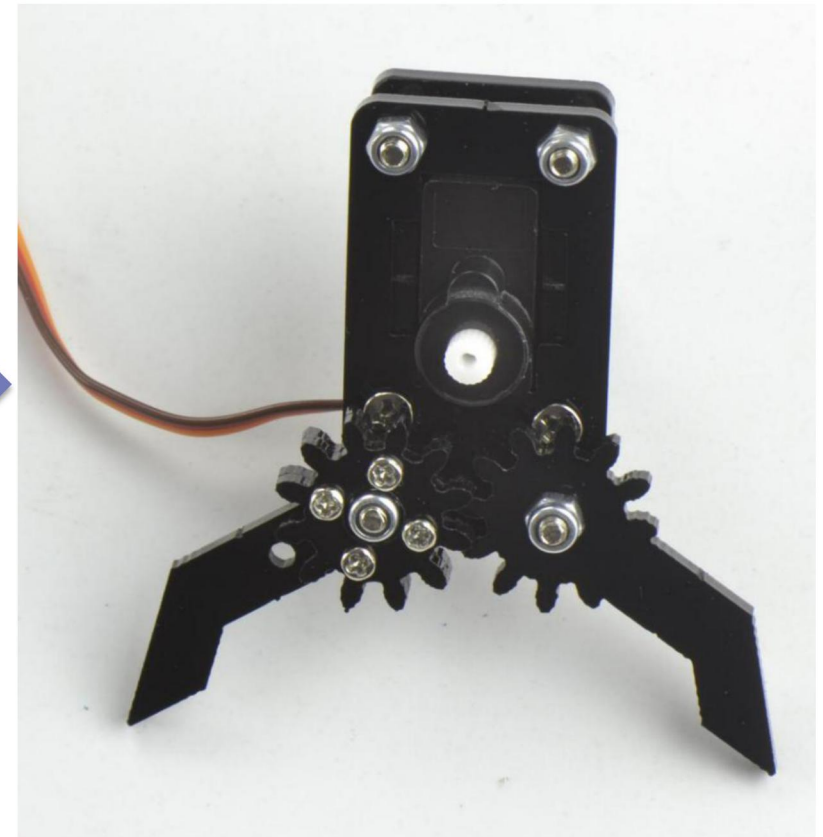
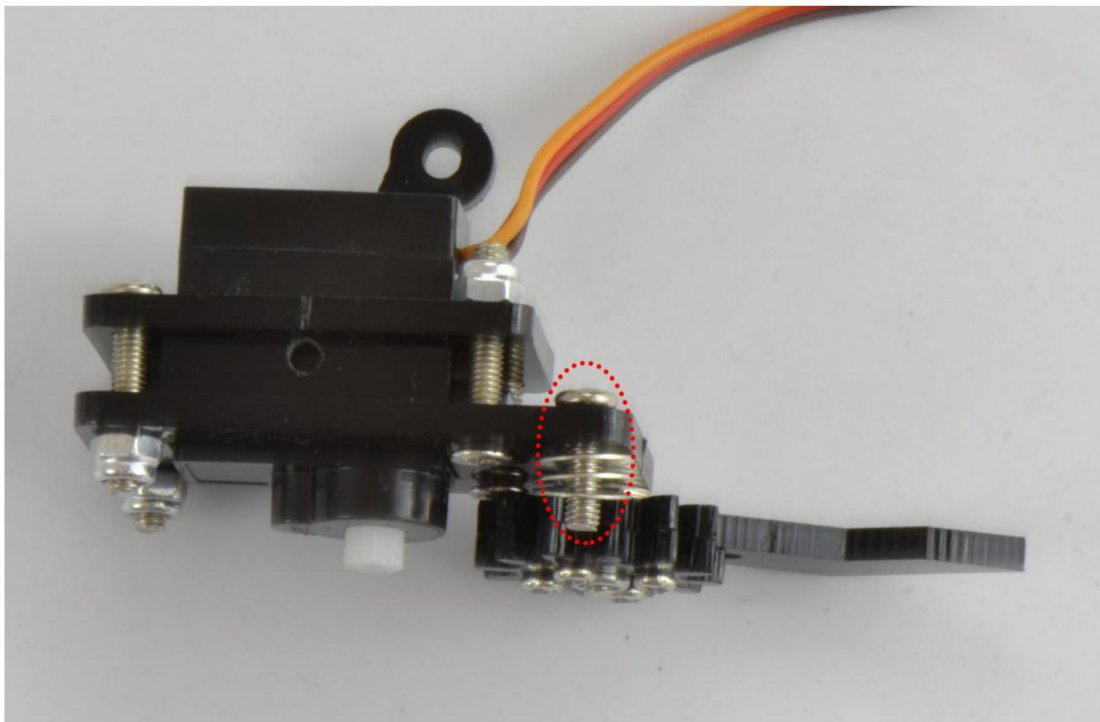
Place the four flat washers onto a M3*12MM round-head screw connected to the Claw servo plate.



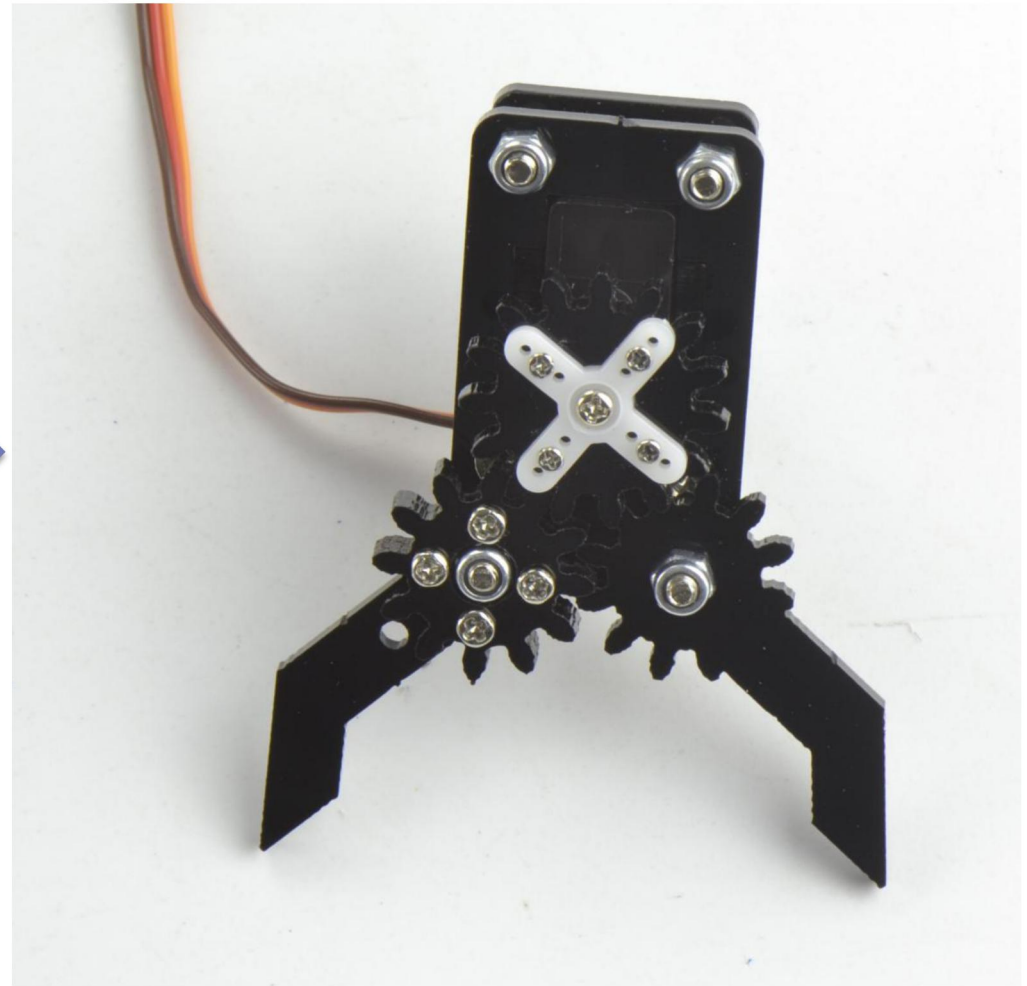
Then fix a M3 hex lock nut to the Gripper plate③. Next, place the Gripper plate onto Claw servo plate.



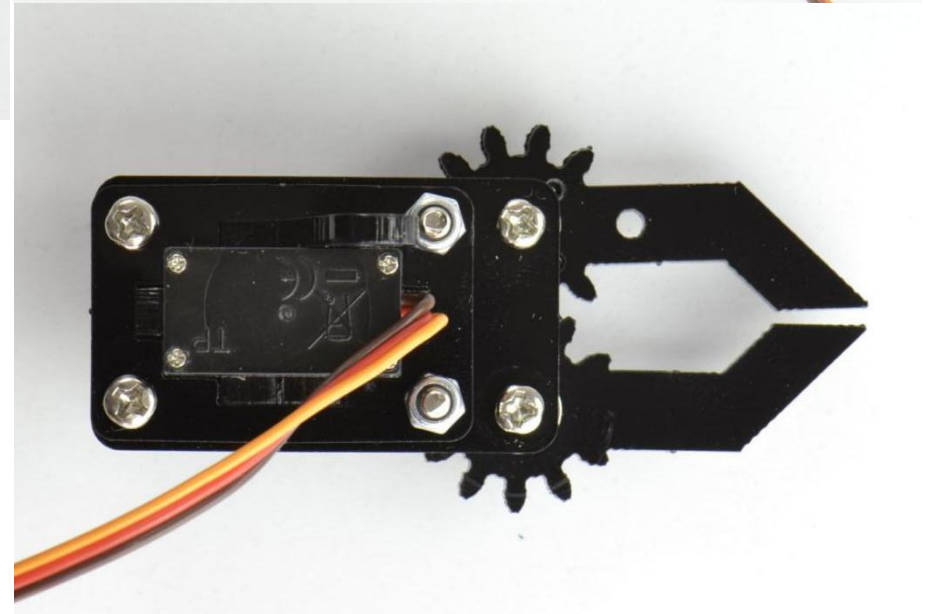
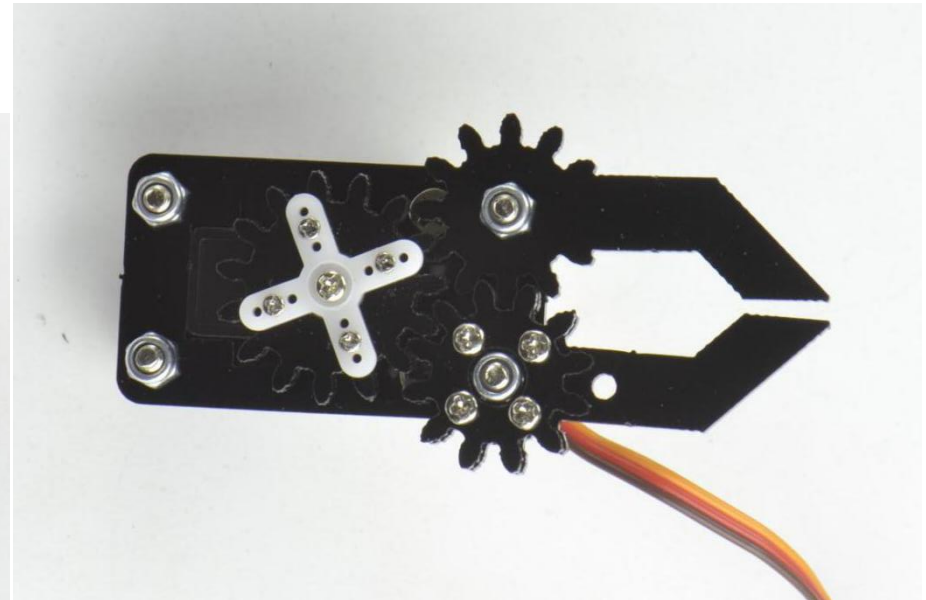
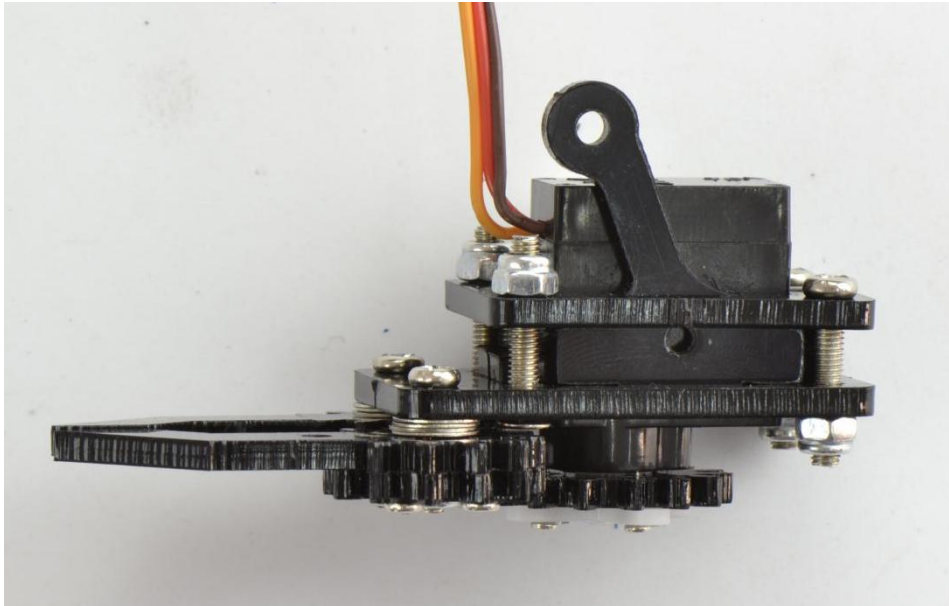
After that, place another Gripper plate⑤ onto Claw servo plate using 3 flat washers and a M3*12MM round-head screw.



Finally, screw the Gripper plate④ onto the shaft of servo motor with a servo screw.



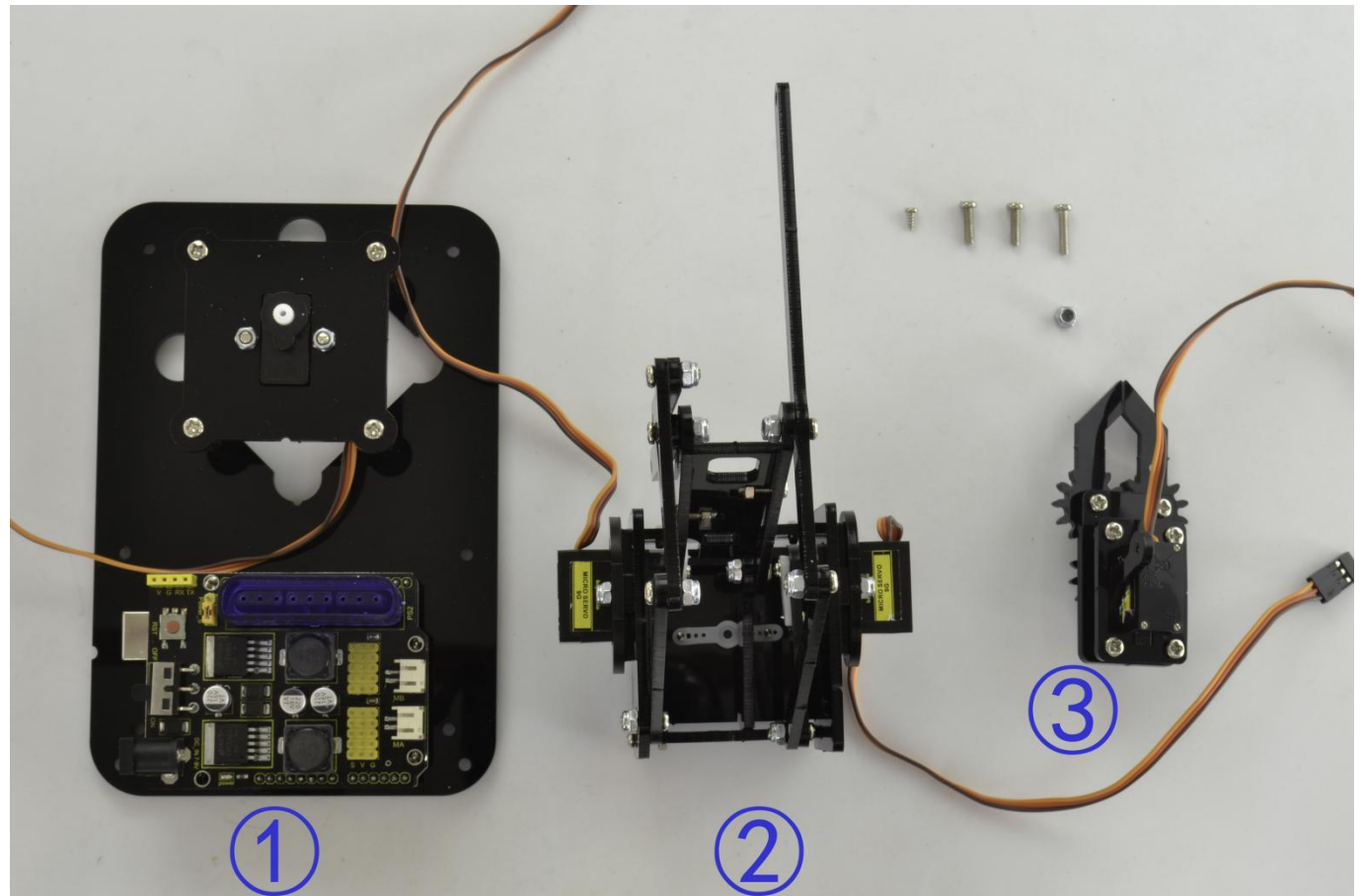
Claw servo plate completed!



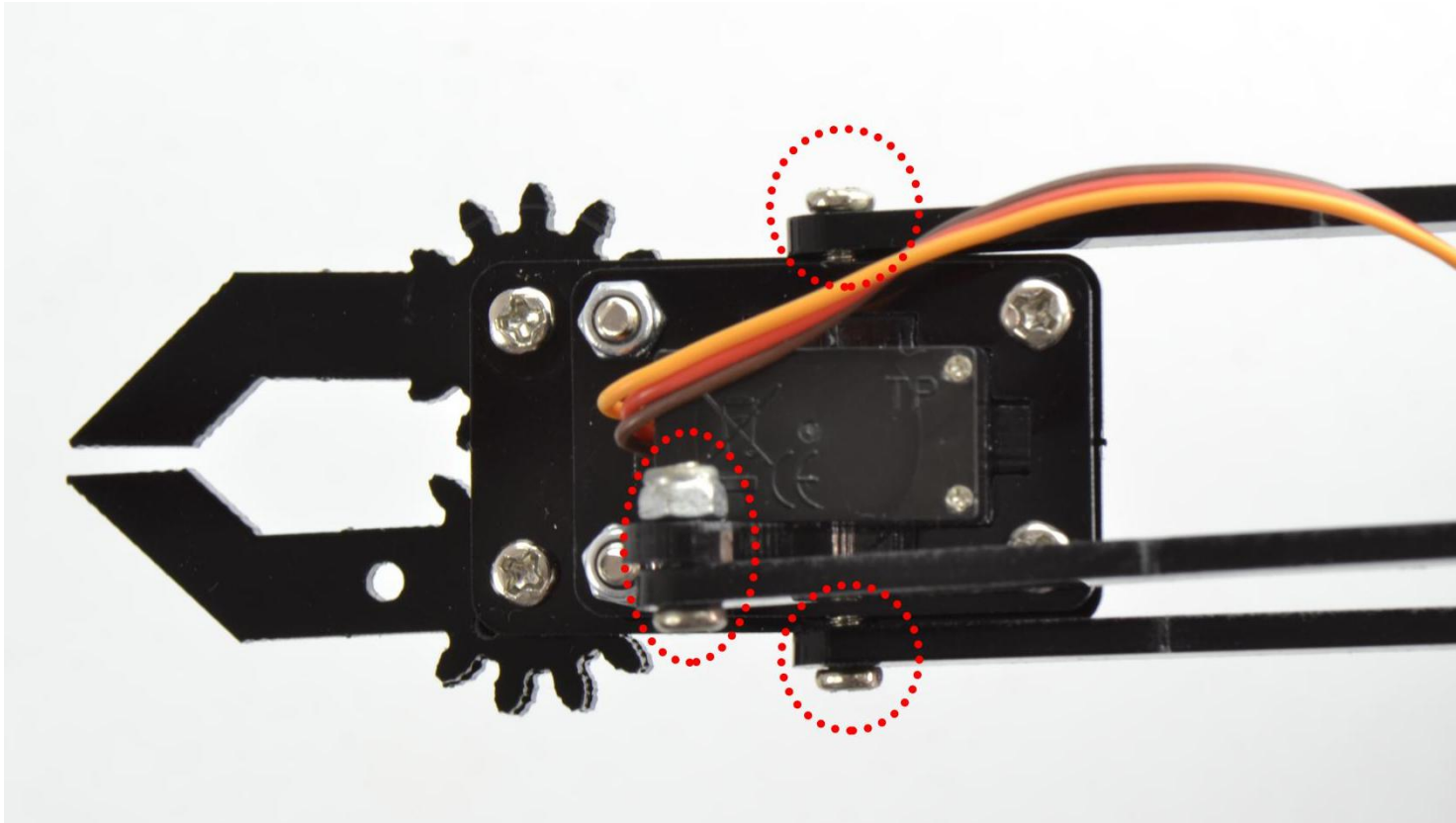
Step4: Final Assembly

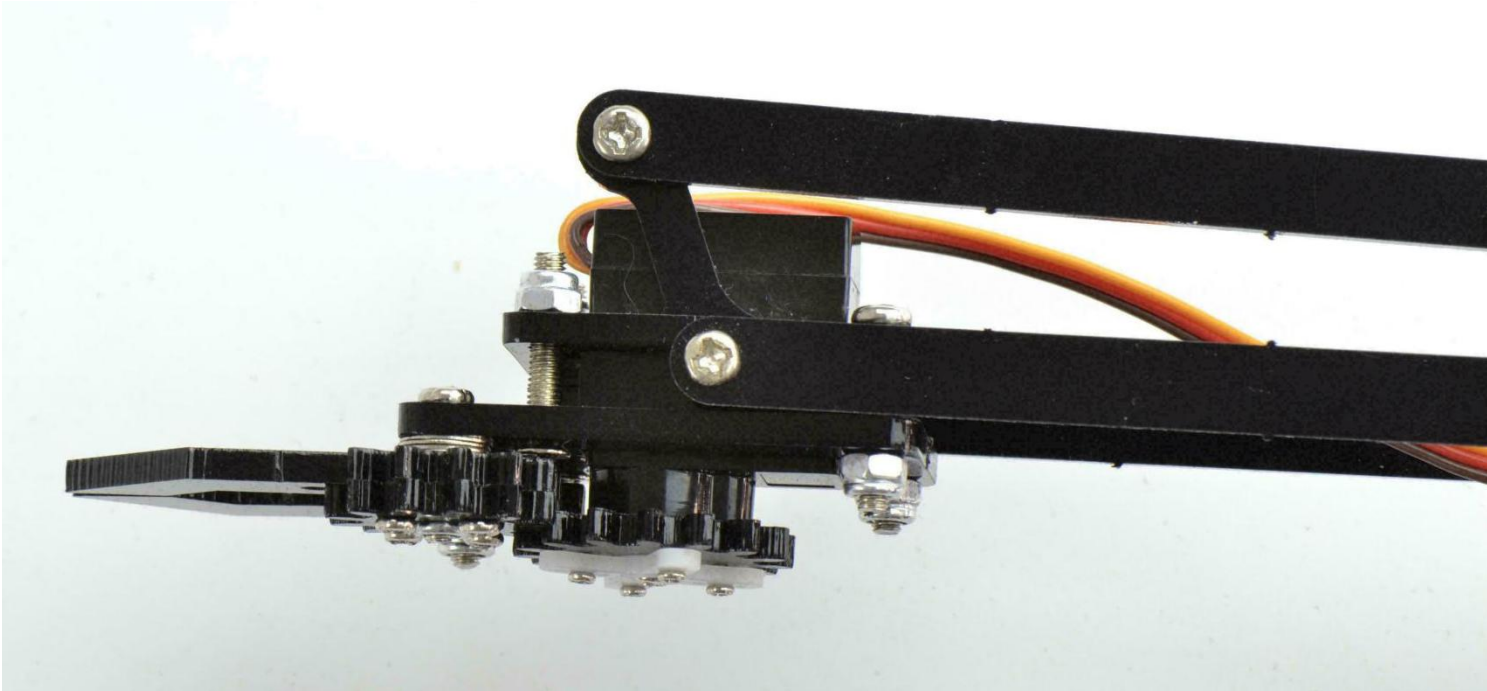
(1) Now we will assemble all the finished parts together

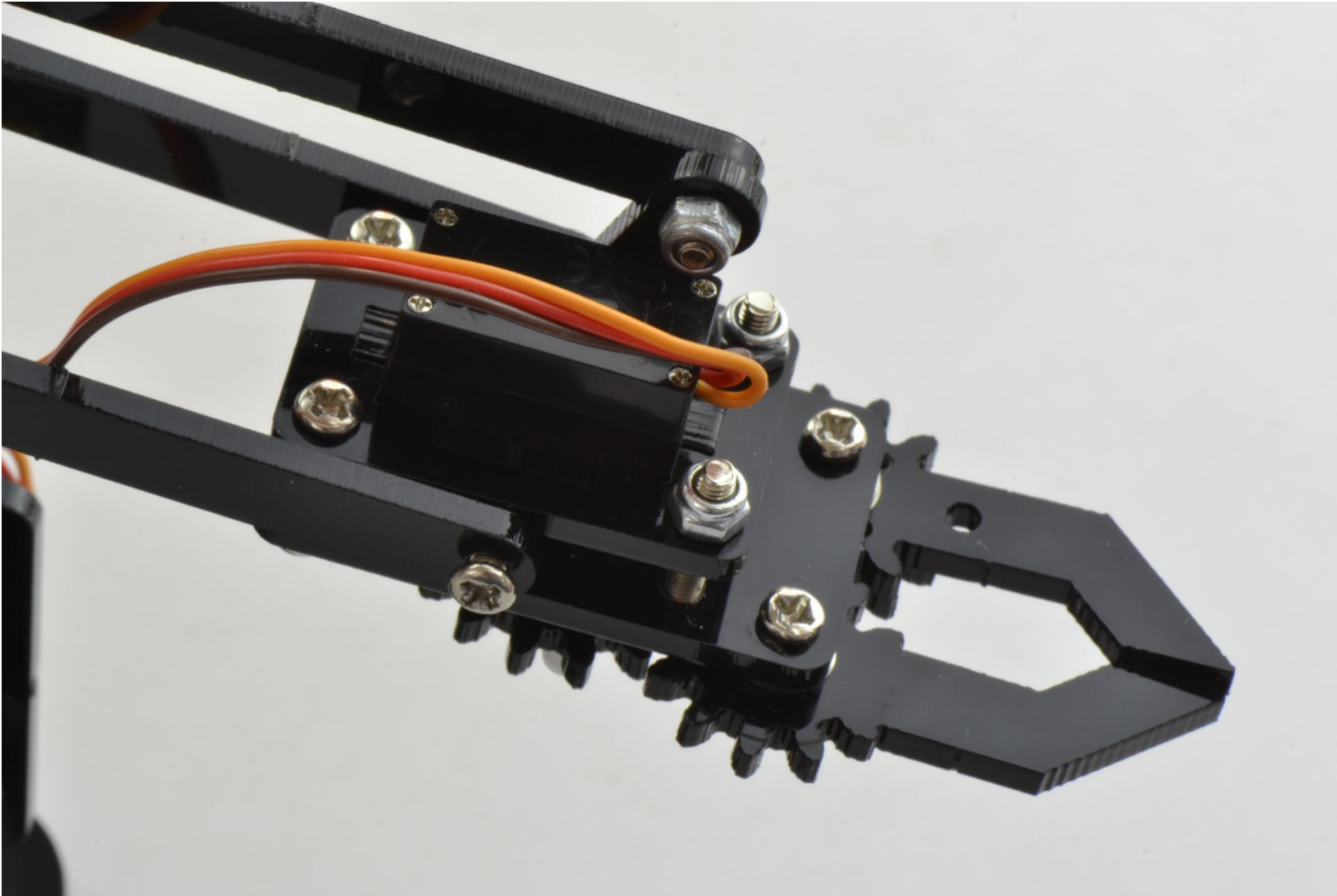
- M3*10MM round-head screw *3
- M3 hex lock nut *1
- M2x5MM tapping screw *1
- Servo Base Plate①
- Arm Middle Parts②
- Claw Servo Plate③



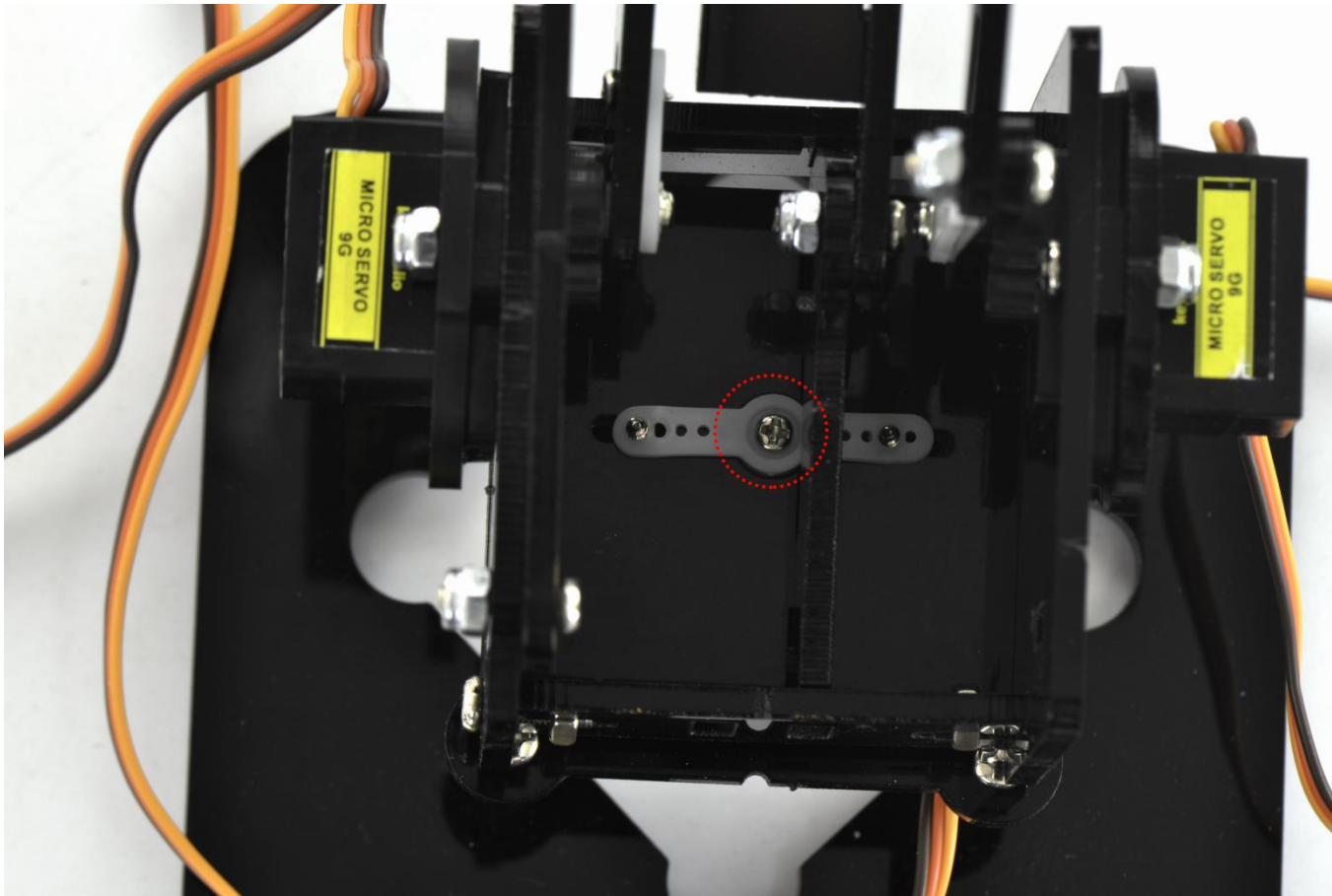
Mount the Claw Servo Plate③ to Arm Middle Parts② using three M3*10MM round-head screws and a M3 hex lock nut.

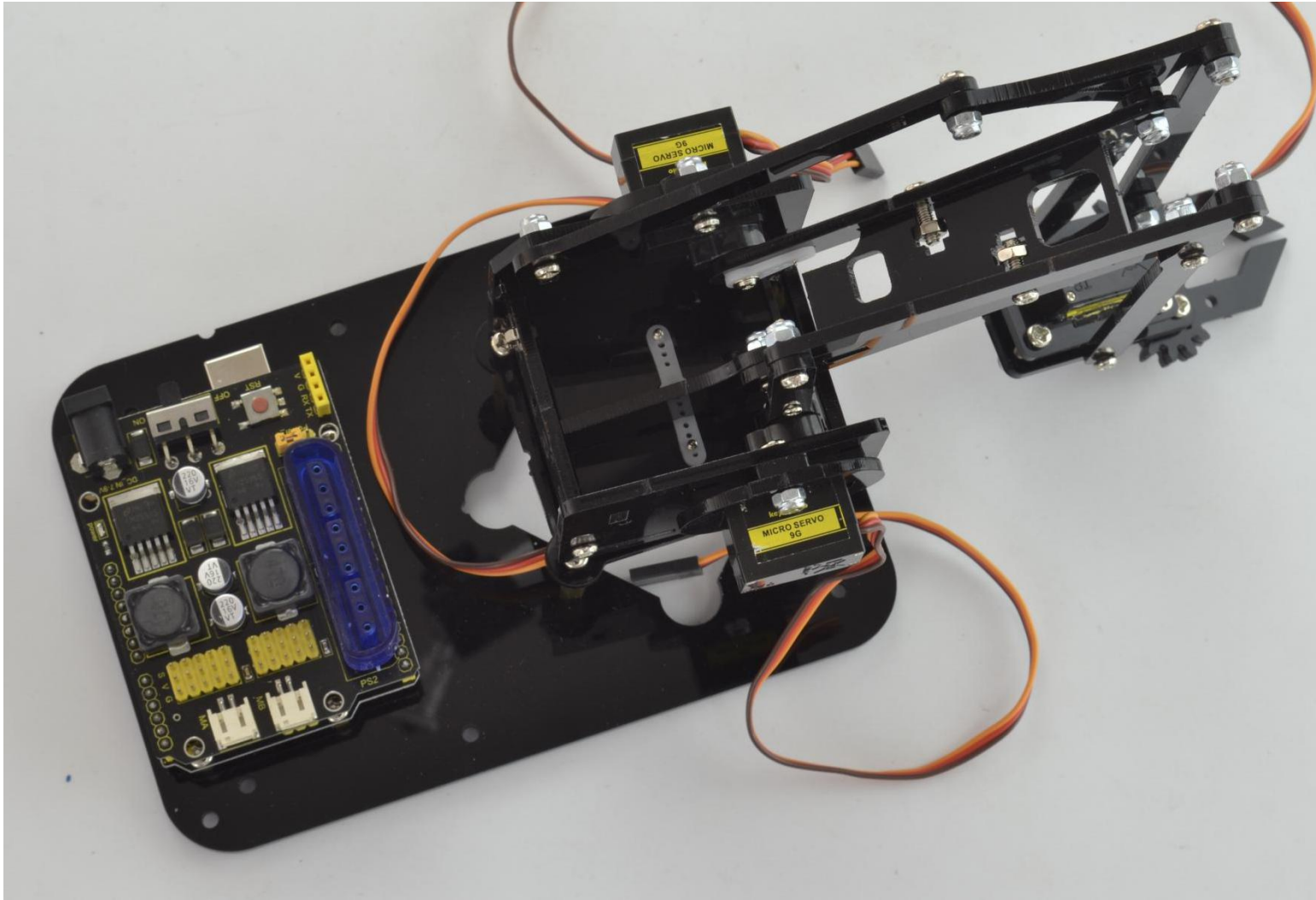






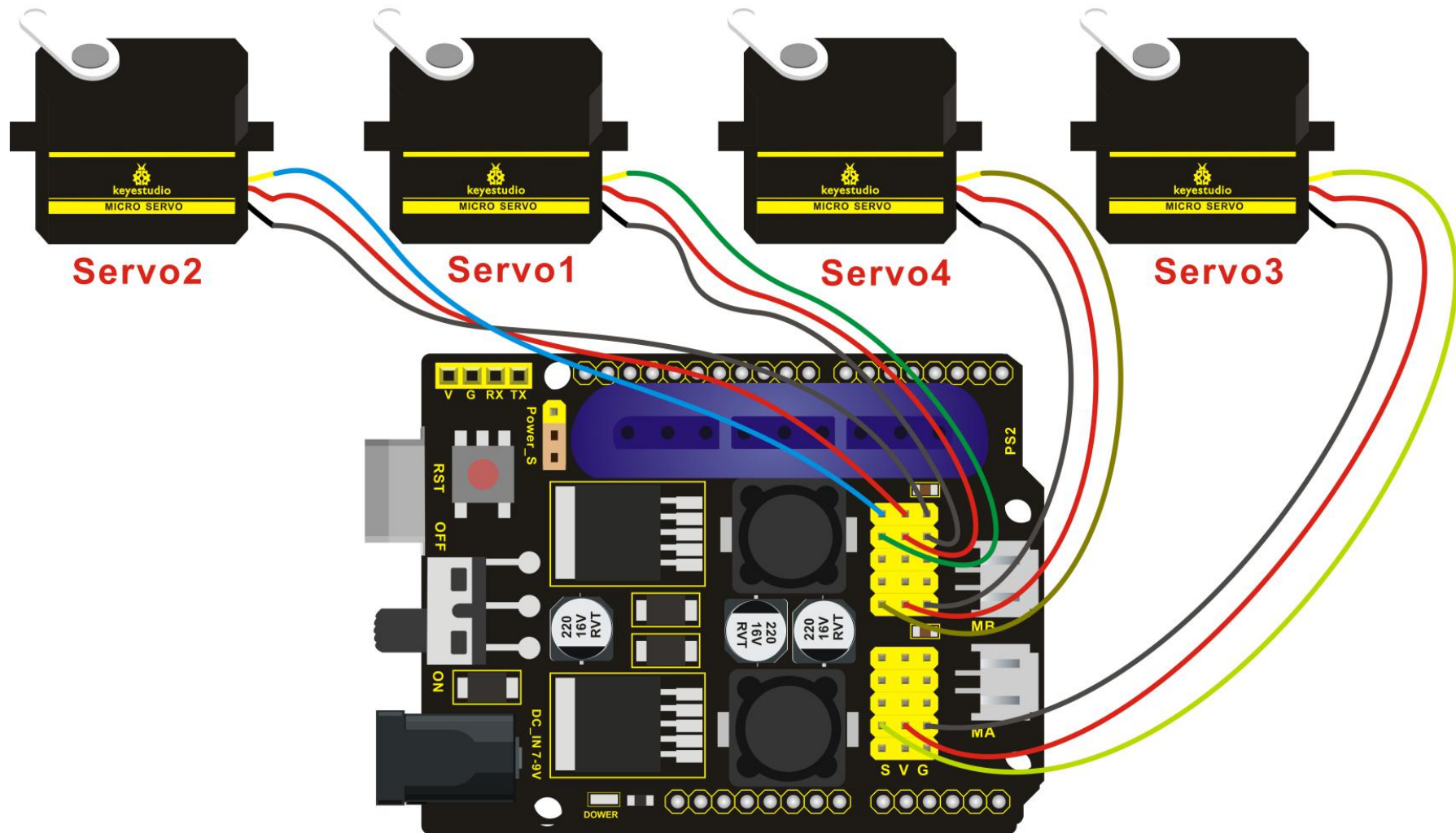
After that, screw them to Servo Base Plate① using a M2x5MM tapping screw.

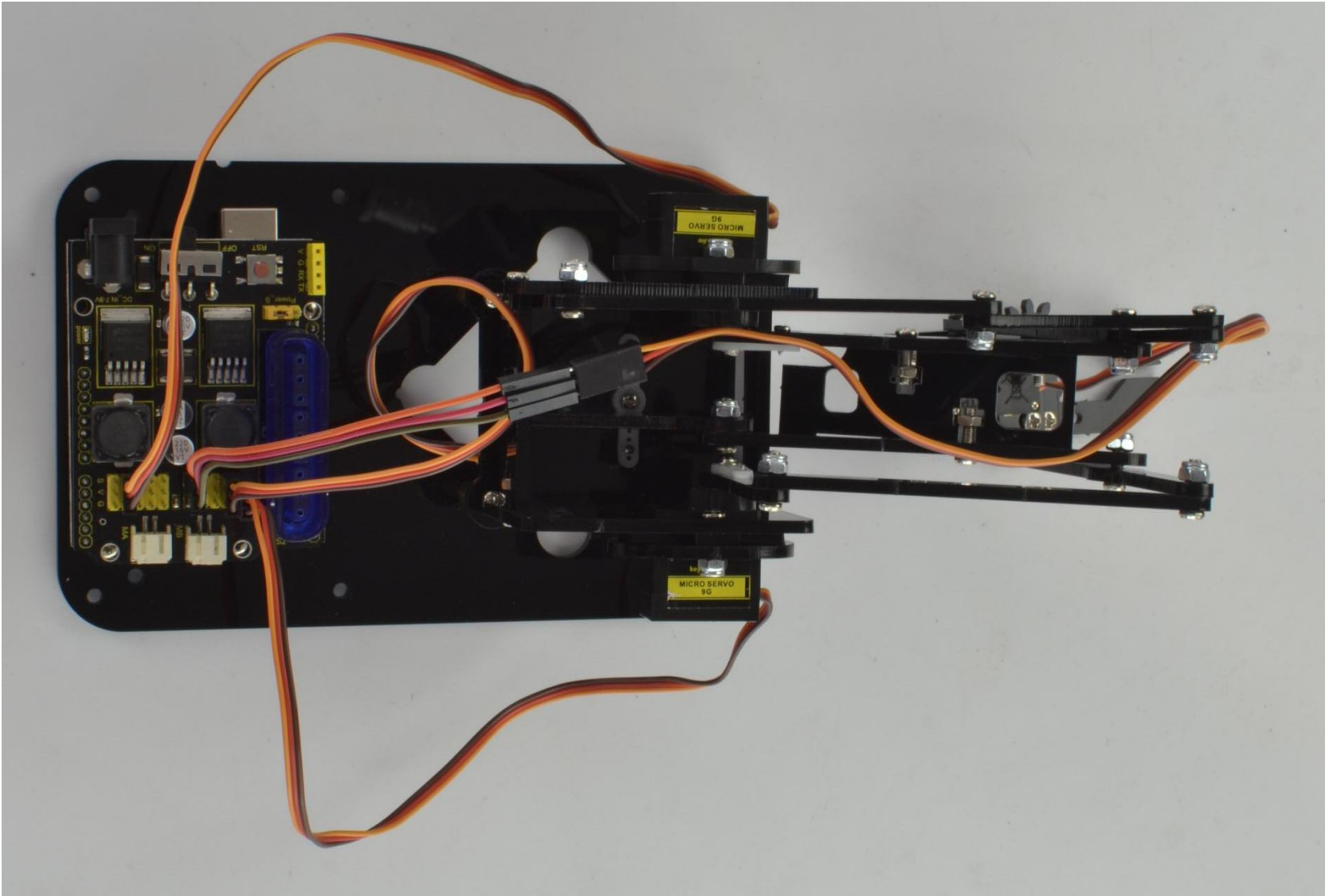




The robot arm kit is nearly complete! It's now time to connect the servo to drive shield.

Hookup Guide:





Step5: Assemble the Joystick Control Plate

(1) The last step is to assemble the Joystick control plate

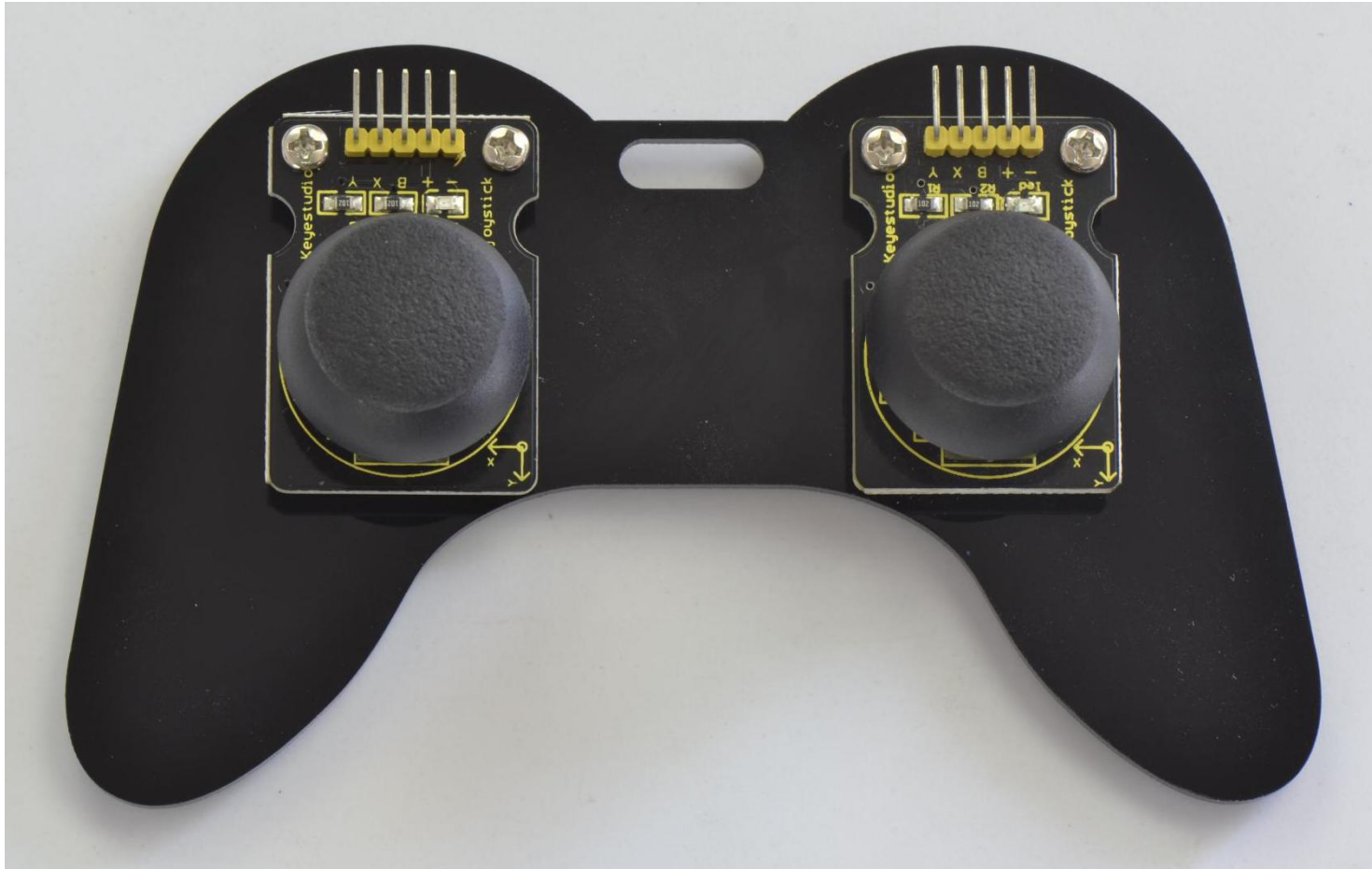
- M3*6MM round-head screw *4
- M3*6+6mm single-pass copper pillar *6
- M3 hex nut *6
- Joystick module *2
- Black Joystick plate *1



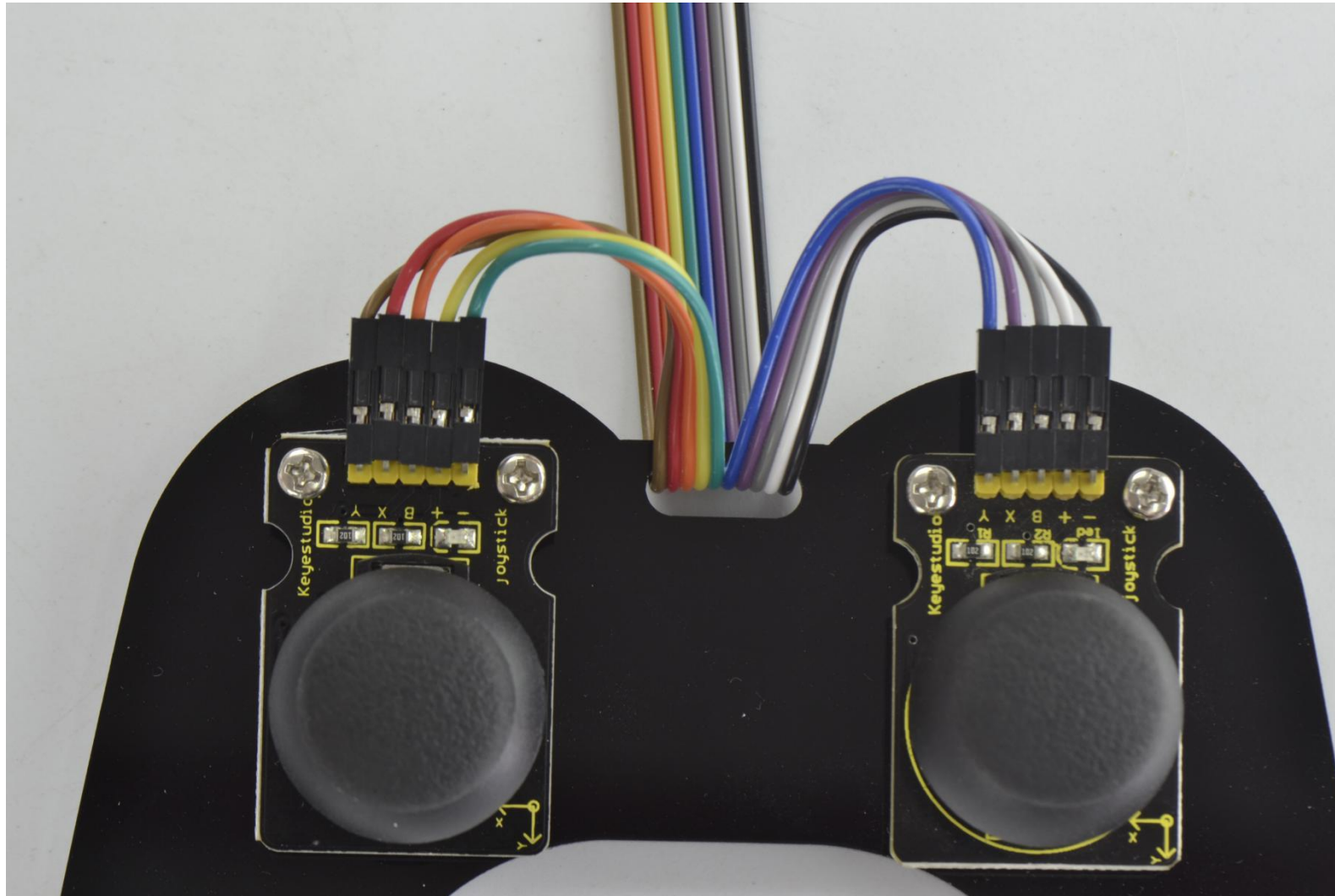
Mount 6 single-pass copper pillars onto Joystick plate with 6 M3 hex nuts.



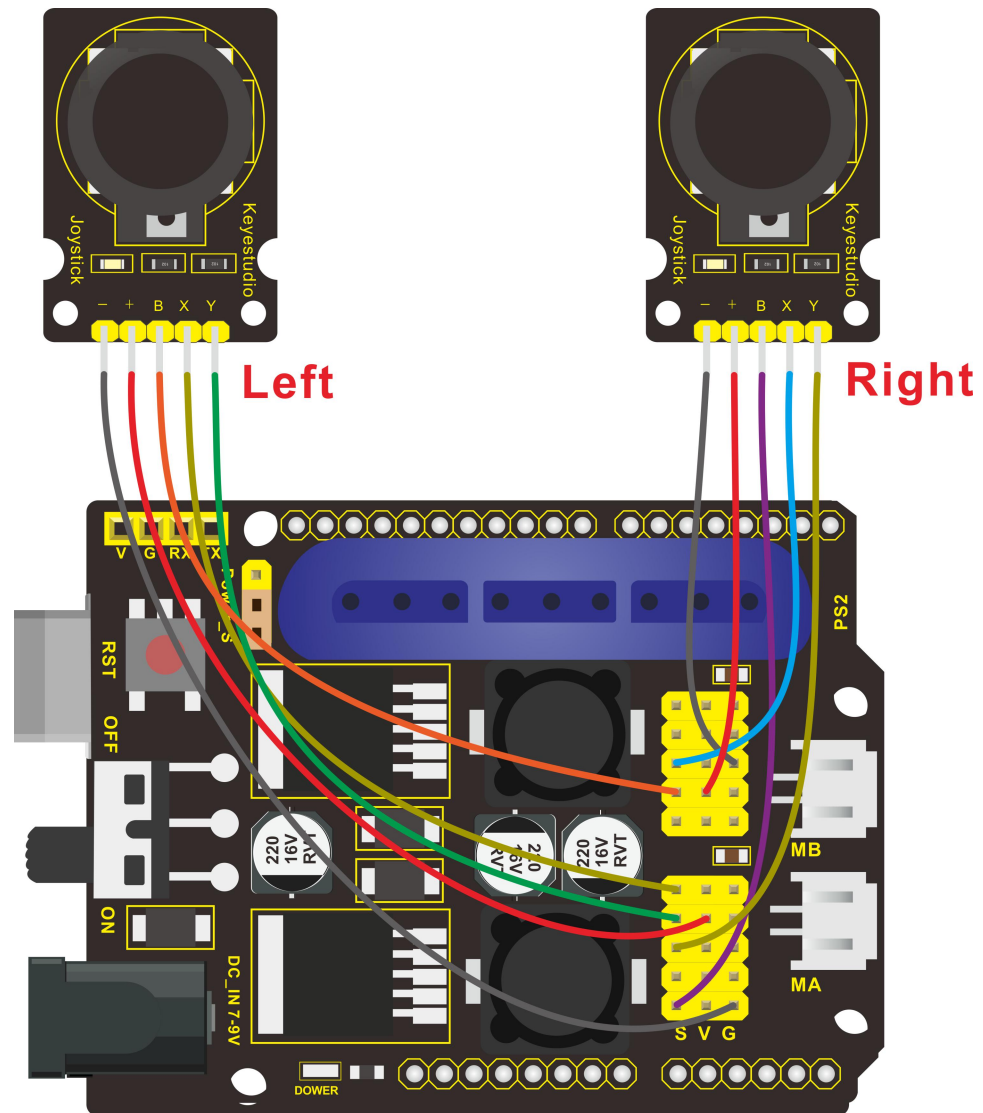
Then screw the two Joystick modules onto the Acrylic plate using four M3*6MM round-head screws.



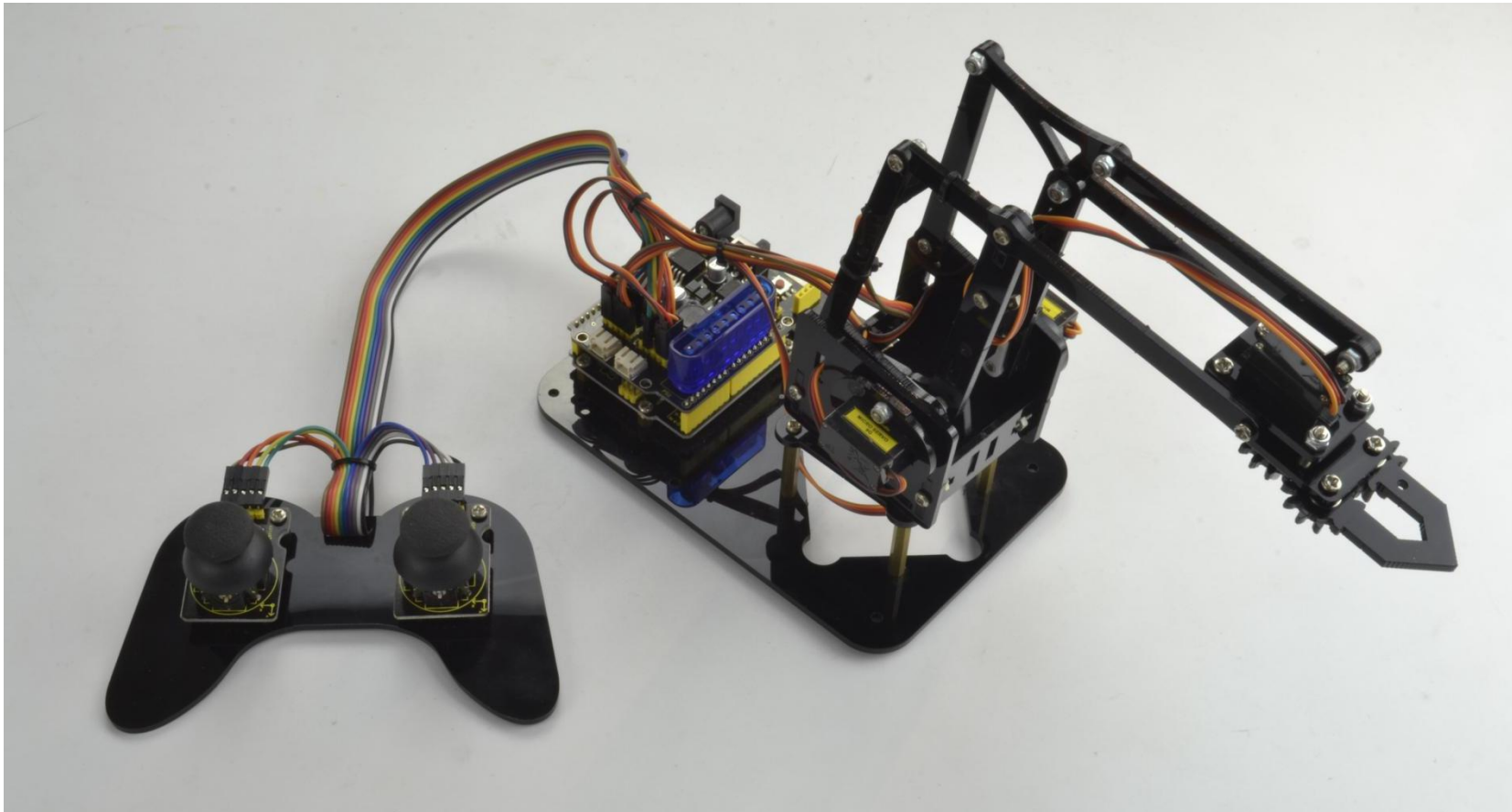
(2) Finally connect the Joystick module to drive shield using female-to-female jumper wires.



Hookup Guide:



Congrats! Your robot arm kit is complete!



Done wiring, power and program everything up to give your robot arm kit a try!

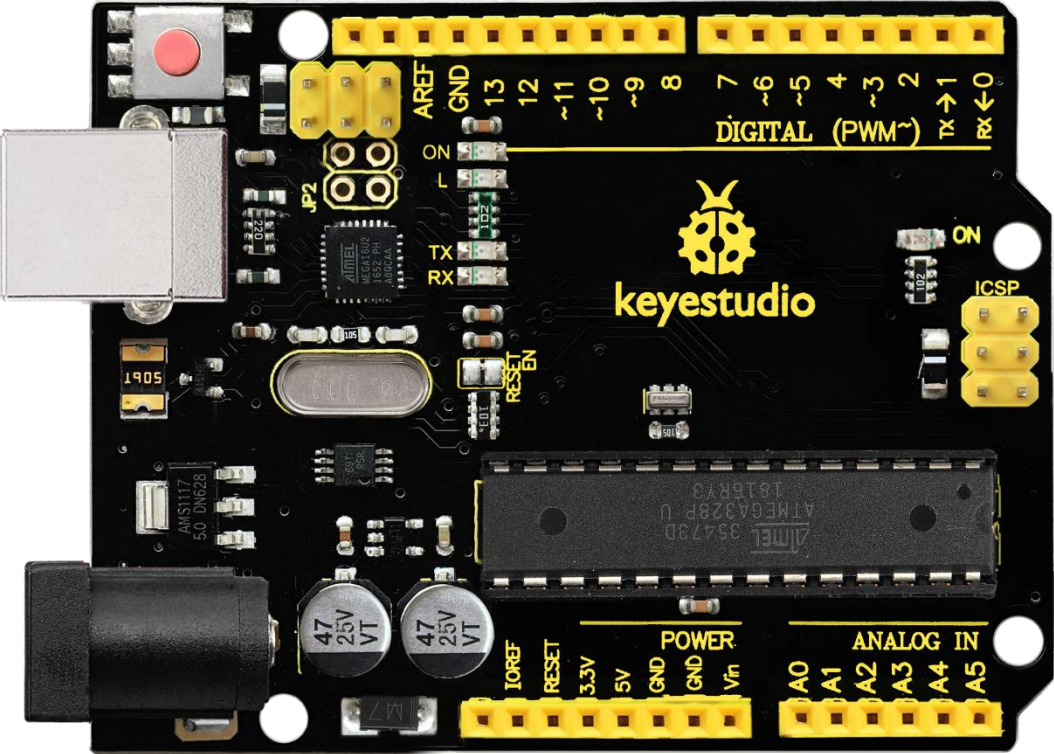
5. Robot Arm Projects

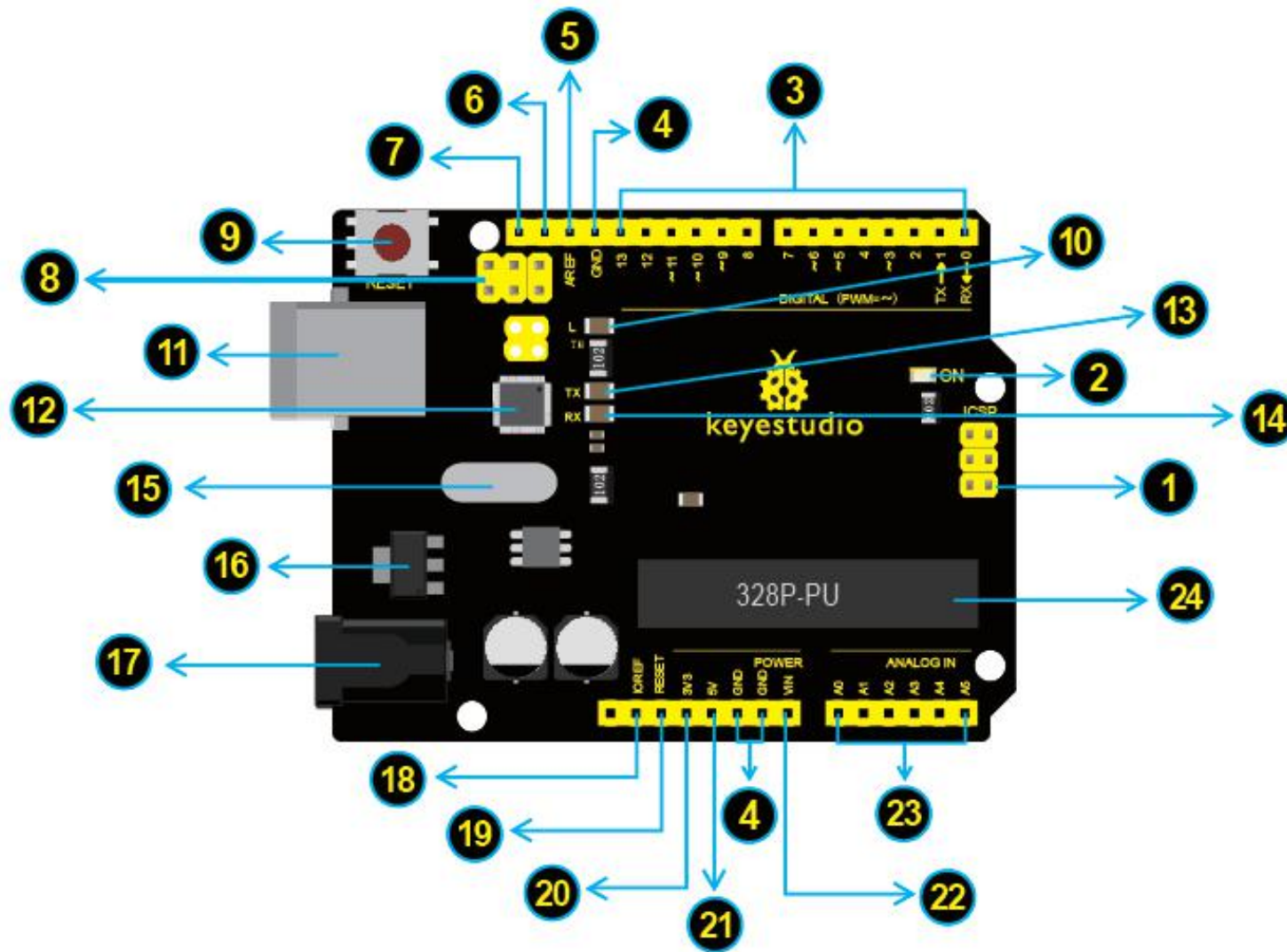
Project 1: Getting Started with ARDUINO

1) UNO R3 Control Board

When it comes to using the UNO R3 as core of our robot, the UNO R3 is the best board to get started with electronics and coding. If this is your first experience tinkering with the platform, the UNO R3 is the most robust board you can start playing with.

Well, let's at first have a look at this UNO R3 board.





	ICSP (In-Circuit Serial Programming) Header In most case, ICSP is the AVR, an Arduino micro-program header consisting of MOSI, MISO, SCK, RESET, VCC, and GND. It is often called the SPI (serial peripheral interface) and can be considered an "extension" of the output. In fact, slave the output devices under the SPI bus host. When connecting to PC, program the firmware to ATMEGA328P-PU.
	Power LED Indicator Powering the Arduino, LED on means that your circuit board is correctly powered on. If LED is off, connection is wrong.

	Digital I/O Keyestudio UNO R3 (Black) Main Control Board has 14 digital input/output pins (of which 6 can be used as PWM outputs). These pins can be configured as digital input pin to read the logic value (0 or 1). Or used as digital output pin to drive different modules like LED, relay, etc. The pin labeled “~” can be used to generate PWM.
	GND (Ground pin headers) Used for circuit ground
	AREF Reference voltage (0-5V) for analog inputs. Used with analogReference() .
	SDA IIC communication pin
	SCL IIC communication pin

	ICSP (In-Circuit Serial Programming) Header In most case, ICSP is the AVR, an Arduino micro-program header consisting of MOSI, MISO, SCK, RESET, VCC, and GND. Connected to ATMEGA 16U2-MU. When connecting to PC, program the firmware to ATMEGA 16U2-MU.
	RESET Button You can reset your Keyestudio UNO R3 (Black) Main Control Board, for example, start the program from the initial status. You can use the RESET button.
	D13 LED There is a built-in LED driven by digital pin 13. When the pin is HIGH value, the LED is on, when the pin is LOW, it's off.
	USB Connection Keyestudio UNO R3 (Black) Main Control Board can be powered via USB connector. All you needed to do is connecting the USB port to PC using a USB cable.

	ATMEGA 16U2-MU USB to serial chip, can convert the USB signal into serial port signal.
	TX LED Onboard you can find the label: TX (transmit) When Keystudio UNO R3 (Black) Main Control Board communicates via serial port, send the message, TX led flashes.
	RX LED Onboard you can find the label: RX(receive) When Keystudio UNO R3 (Black) Main Control Board communicates via serial port, receive the message, RX led flashes.
	Crystal Oscillator How does Arduino calculate time? by using a crystal oscillator. The number printed on the top of the Arduino crystal is 16.000H9H. It tells us that the frequency is 16,000,000 Hertz or 16MHz.

	Voltage Regulator Convert an external input DC7-12V voltage into DC 5V, then switch DC 5V to the processor and other components. Output DC 5V, the drive current is 2A.
	DC Power Jack Keyestudio UNO R3 (Black) Main Control Board can be supplied with an external power DC7-12V from the DC power jack.
	IOREF Used to configure the operating voltage of microcontrollers. Use it less.
	RESET Header Connect an external button to reset the board. The function is the same as reset button (labeled 9)

	Power Pin 3V3 A 3.3 volt supply generated by the on-board regulator. Maximum current draw is 50 mA.
	Power Pin 5V Provides 5V output voltage
	Vin You can supply an external power input DC7-12V through this pin to Keyestudio UNO R3 (Black) Main Control Board.
	Analog Pins Keyestudio UNO R3 (Black) Main Control Board has 6 analog inputs, labeled A0 through A5. These pins can read the signal from analog sensors (such as humidity sensor or temperature sensor), and convert it into the digital value that can read by microcontrollers) Can also used as digital pins, A0=D14, A1=D15, A2=D16, A3=D17, A4=D18,

A5=D19.

Microcontroller

Each Keystudio UNO R3 (Black) Main Control Board has its own microcontroller. You can regard it as the brain of your board.



The main IC (integrated circuit) on the Arduino is slightly different from the panel pair. Microcontrollers are usually from ATMEL. Before you load a new program on the Arduino IDE, you must know what IC is on your board. This information can be checked at the top of IC.

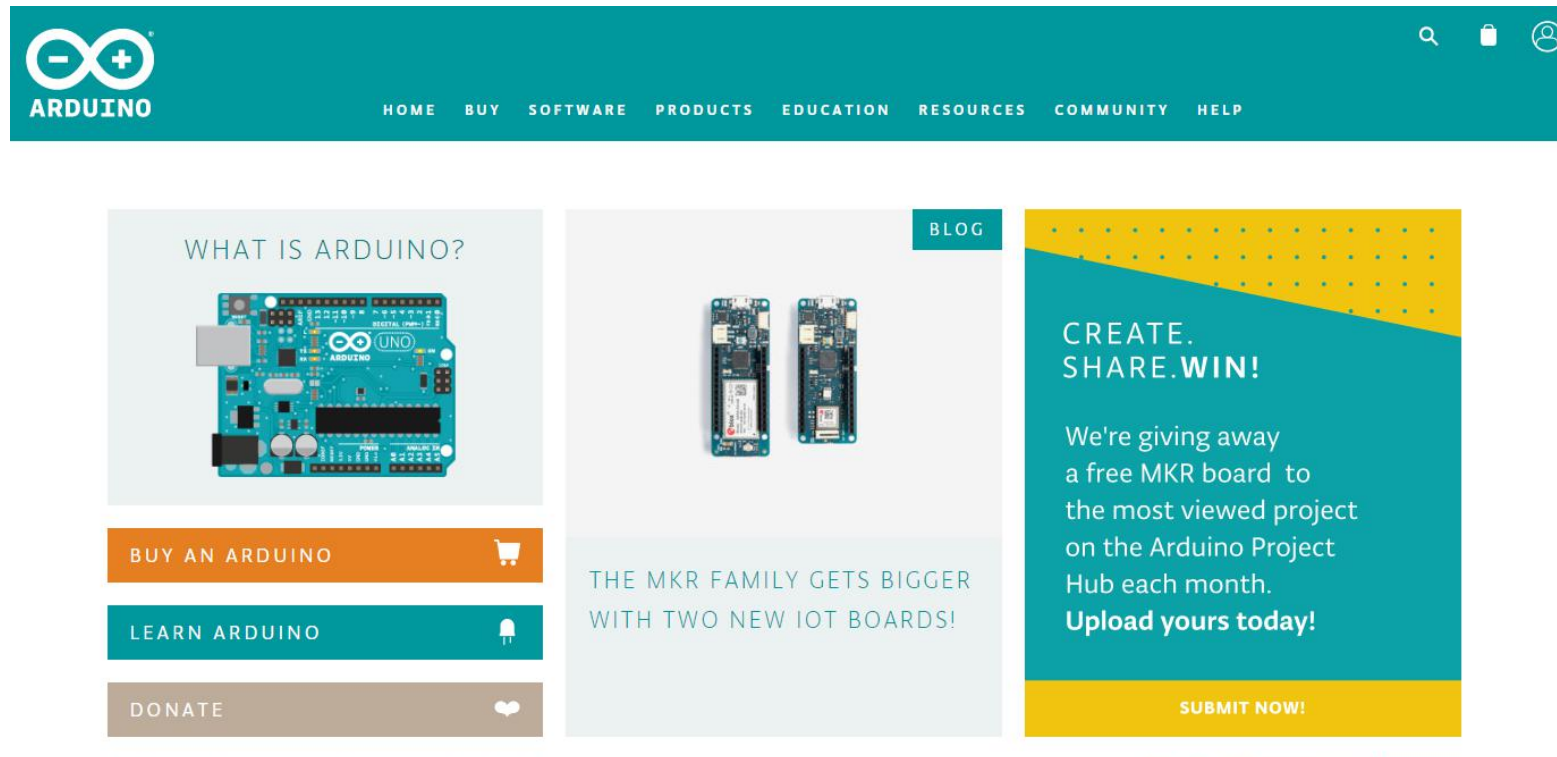
Installing Arduino IDE

When you get the control board, first you should install the Arduino software and driver.

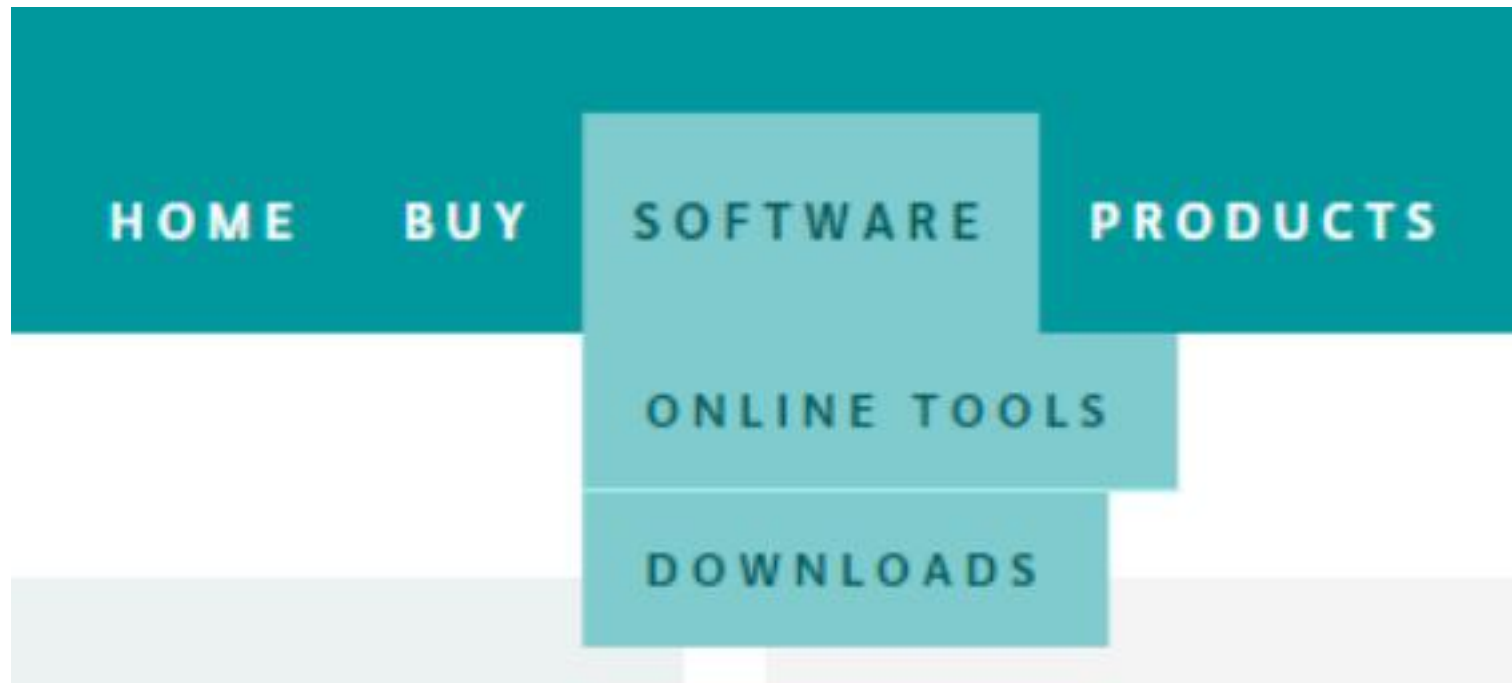
You can see all the Arduino software versions from the link below:

<https://www.arduino.cc/en/Main/OldSoftwareReleases#1.5.x>

Or you can browse the ARDUINO website at this link, <https://www.arduino.cc>, pop up the following interface.



Then click the SOFTWARE on the browse bar, you will have two options ONLINE TOOLS and DOWNLOADS.



Click DOWNLOADS, it will appear the latest software version of ARDUINO 1.8.5 shown as below.

Download the Arduino IDE



ARDUINO 1.8.5

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing and other open-source software.

This software can be used with any Arduino board. Refer to the [Getting Started](#) page for Installation instructions.

Windows Installer, for Windows XP and up
Windows ZIP file for non admin install

Windows app Requires Win 8.1 or 10
[Get](#) 

Mac OS X 10.7 Lion or newer

Linux 32 bits
Linux 64 bits
Linux ARM

[Release Notes](#)
[Source Code](#)
[Checksums \(sha512\)](#)

In this software page, on the right side you can see the version of development software for different operating systems. So ARDUINO has a rather powerful compatibility. You should download the software that is compatible with the operating system of your computer.

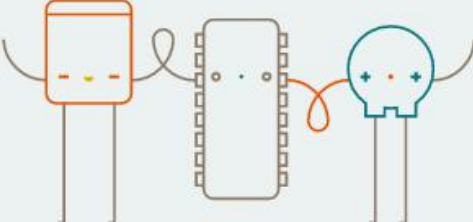
In our project, we will take WINDOWS system as an example here. There are also two options under Windows system, one is installed version, the other is non-installed version.

For simple installed version, first click Windows Installer, you will get the following page.



Contribute to the Arduino Software

Consider supporting the Arduino Software by contributing to its development. (US tax payers, please note this contribution is not tax deductible). [Learn more on how your contribution will be used.](#)



SINCE MARCH 2015, THE ARDUINO IDE HAS BEEN DOWNLOADED **24,353,248** TIMES. (IMPRESSIVE!) NO LONGER JUST FOR ARDUINO AND GENUINO BOARDS, HUNDREDS OF COMPANIES AROUND THE WORLD ARE USING THE IDE TO PROGRAM THEIR DEVICES, INCLUDING COMPATIBLES, CLONES, AND EVEN COUNTERFEITS. HELP ACCELERATE ITS DEVELOPMENT WITH A SMALL CONTRIBUTION! REMEMBER: OPEN SOURCE IS LOVE!

\$3 **\$5** **\$10** **\$25** **\$50** **OTHER**

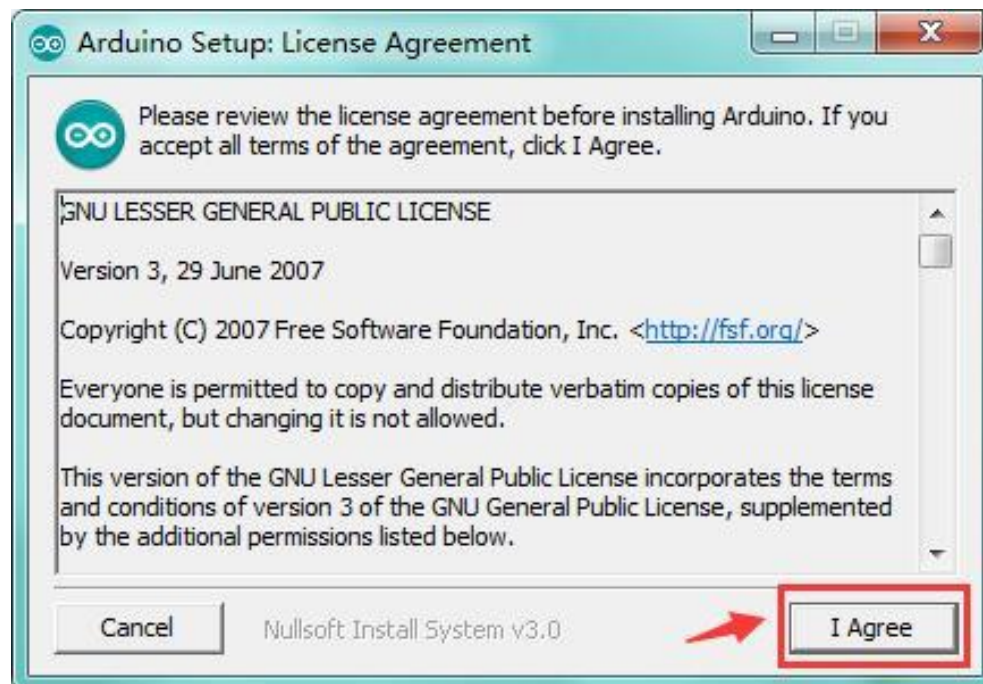
JUST DOWNLOAD **CONTRIBUTE & DOWNLOAD**

This way you just need to click JUST DOWNLOAD, then click the downloaded file to install it.

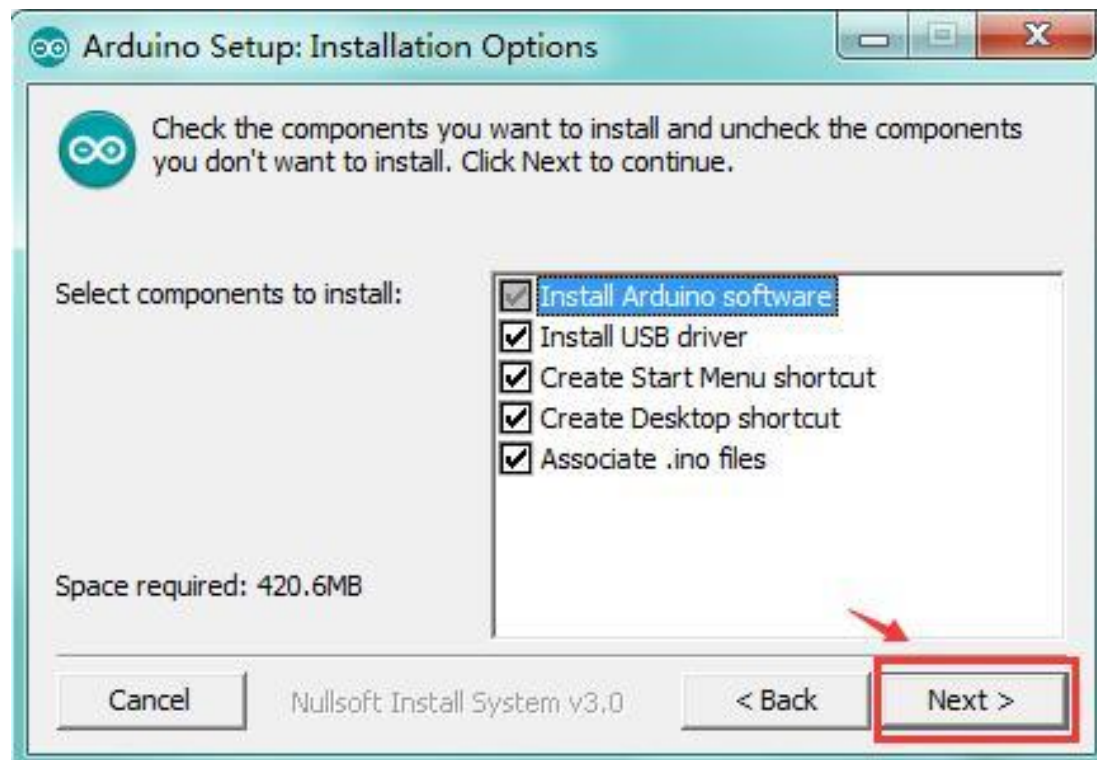
For non-installed version, first click Windows ZIP file, you will also get the pop-up interface as the above figure. Click JUST DOWNLOAD, and when the ZIP file is downloaded well to your computer, you can directly unzip the file and then click the icon of ARDUINO program to start it.

Installing Arduino (Windows)

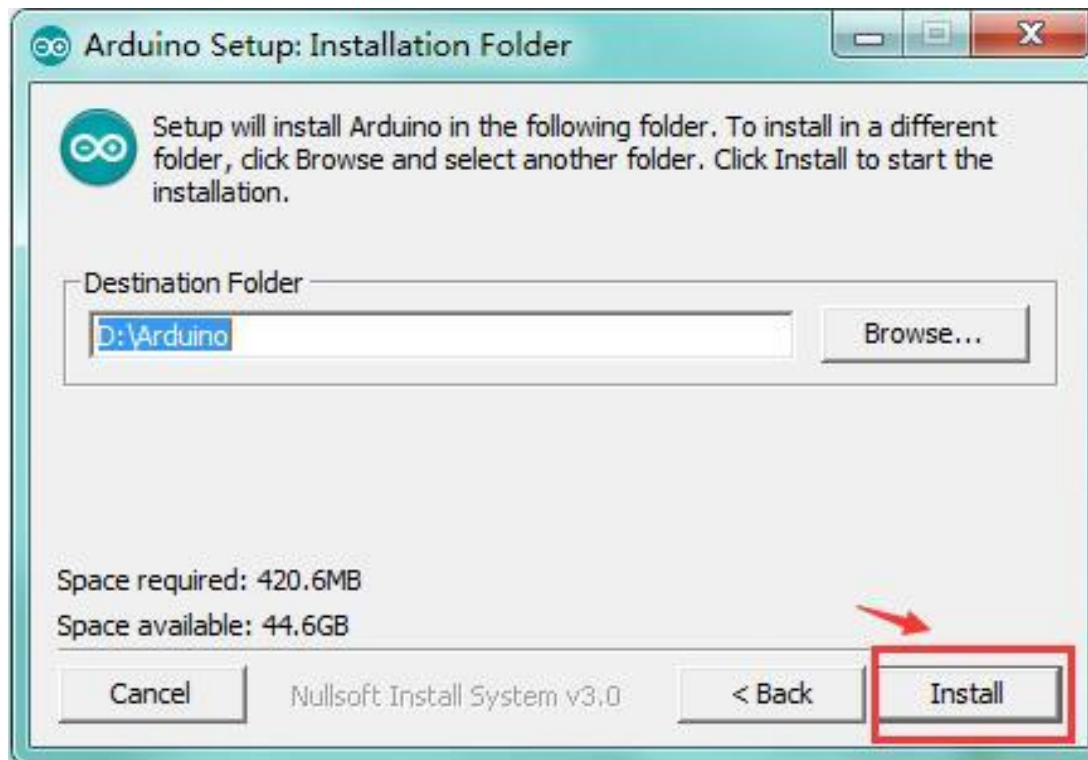
Install Arduino with the exe. Installation package



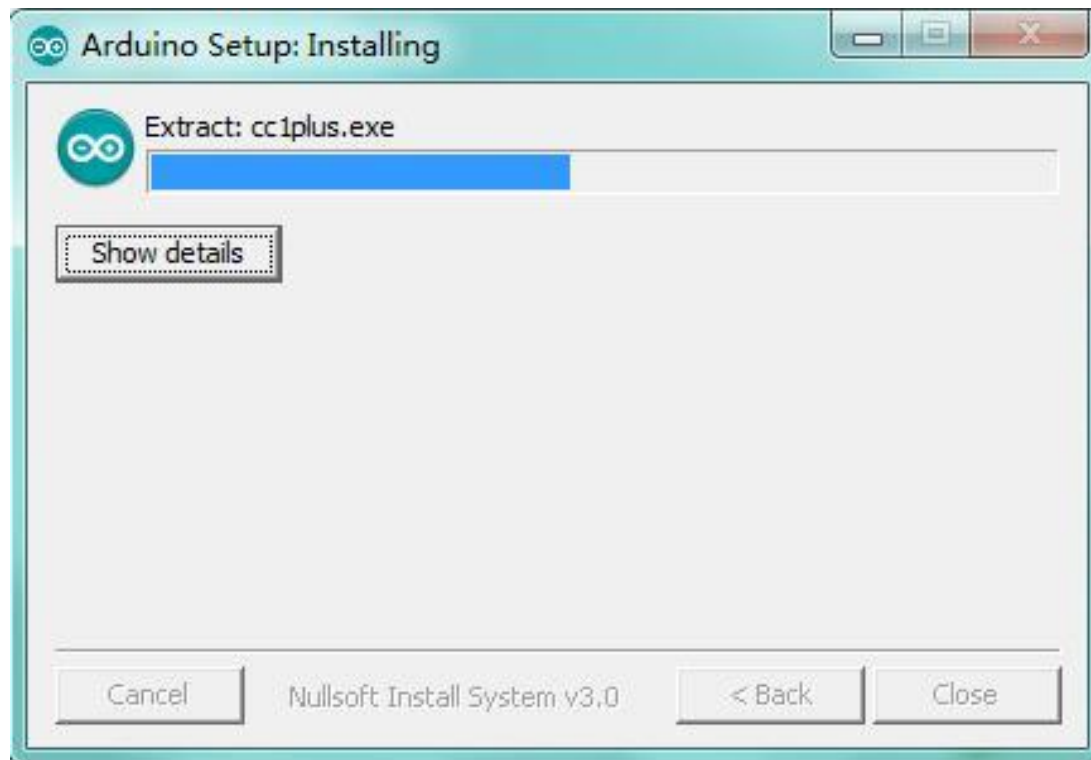
Click "*I Agree*" to see the following interface.



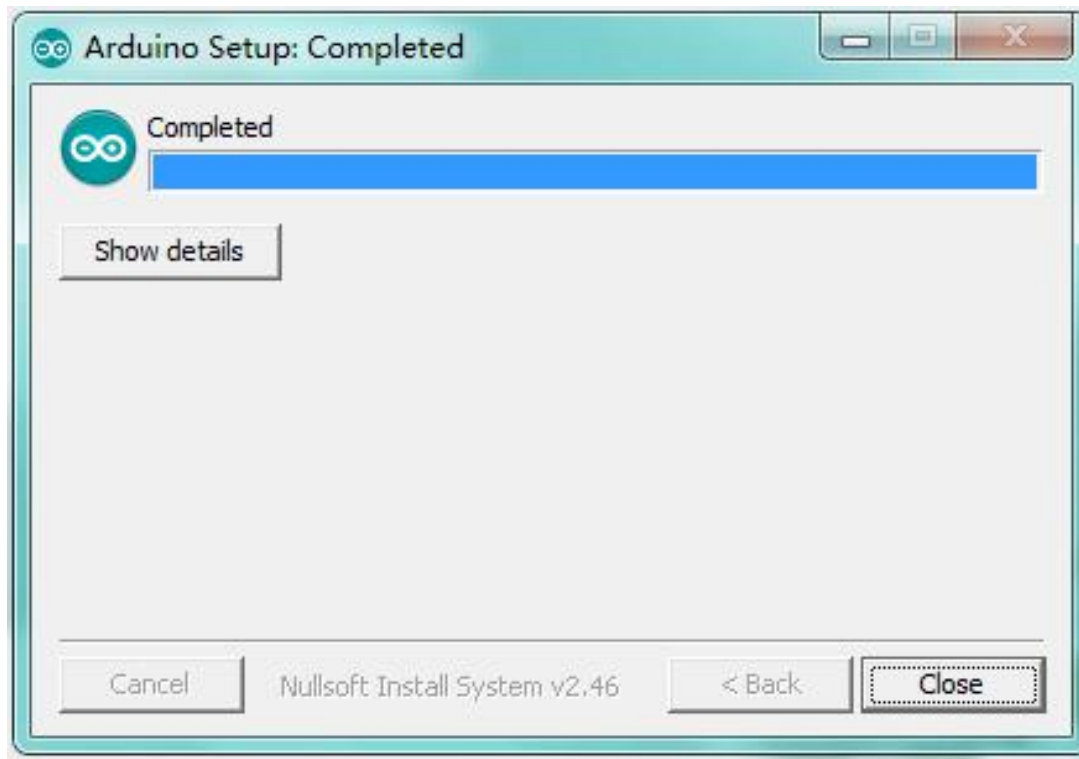
Click "*Next*". Pop up the interface below.



You can press Browse... to choose an installation path or directly type in the directory you want. Then click "Install" to initiate installation.



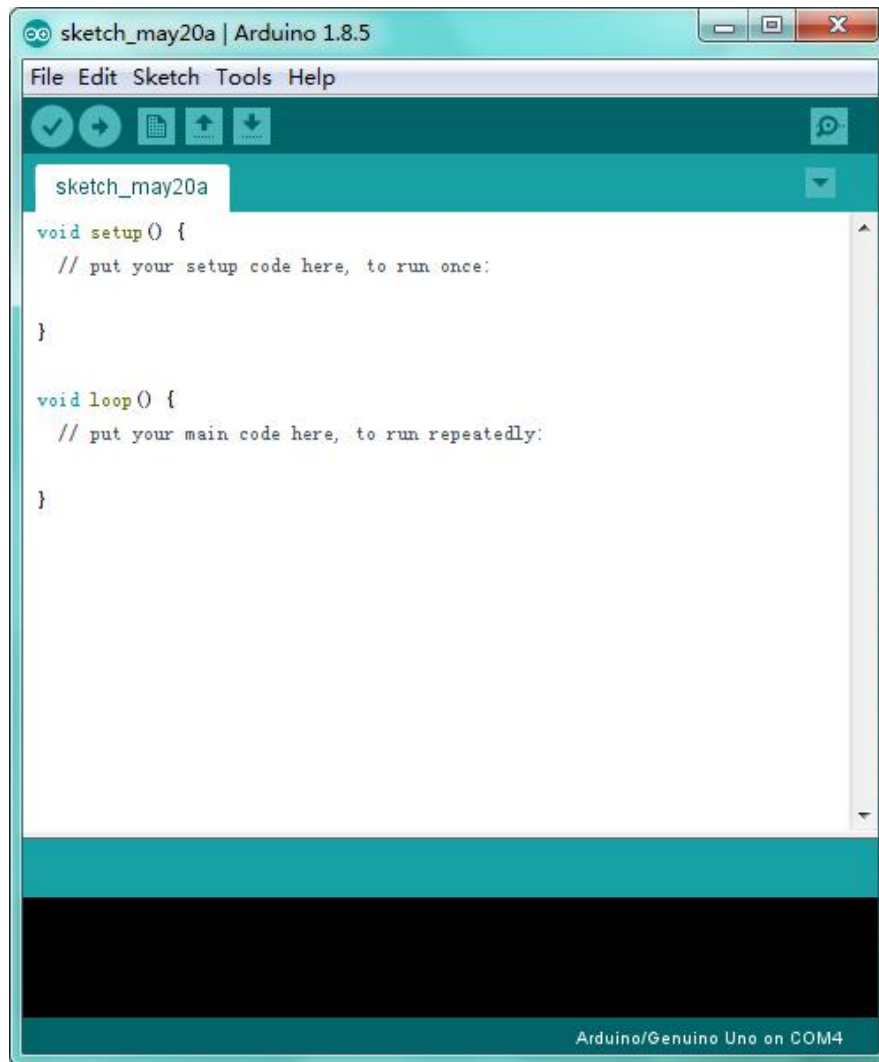
Wait for the installing process, if appear the interface of Window Security, just continue to click Install to finish the installation.



All right, up to now, you have completed the Arduino setup! The following icon will appear on your PC desktop.



Double-click the icon of Arduino to enter the desired development environment shown as below.



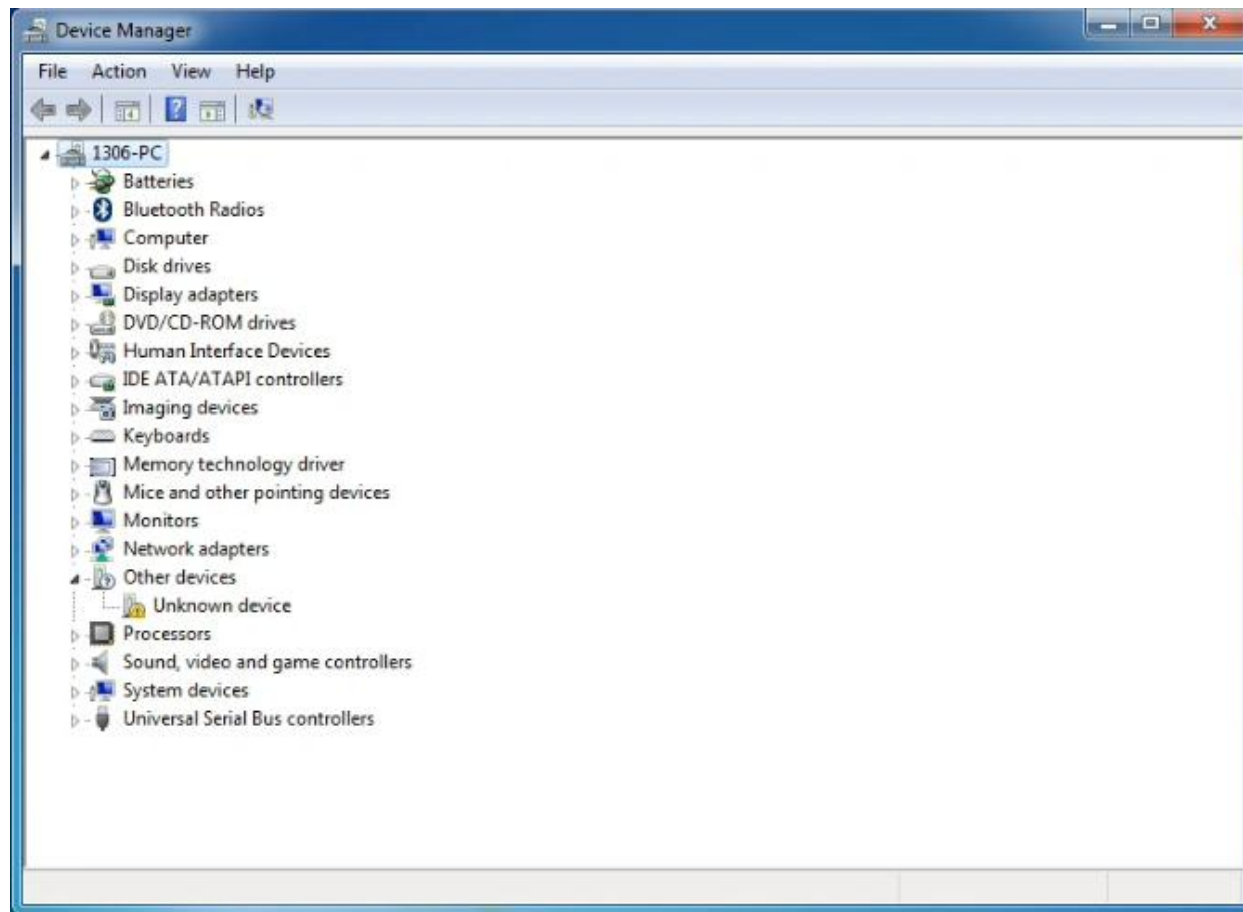
Installing Driver

Next, we will introduce the board driver installation. The driver installation may have slight difference in different computer systems. So in the following let's move on to the driver installation in the WIN 7 system.

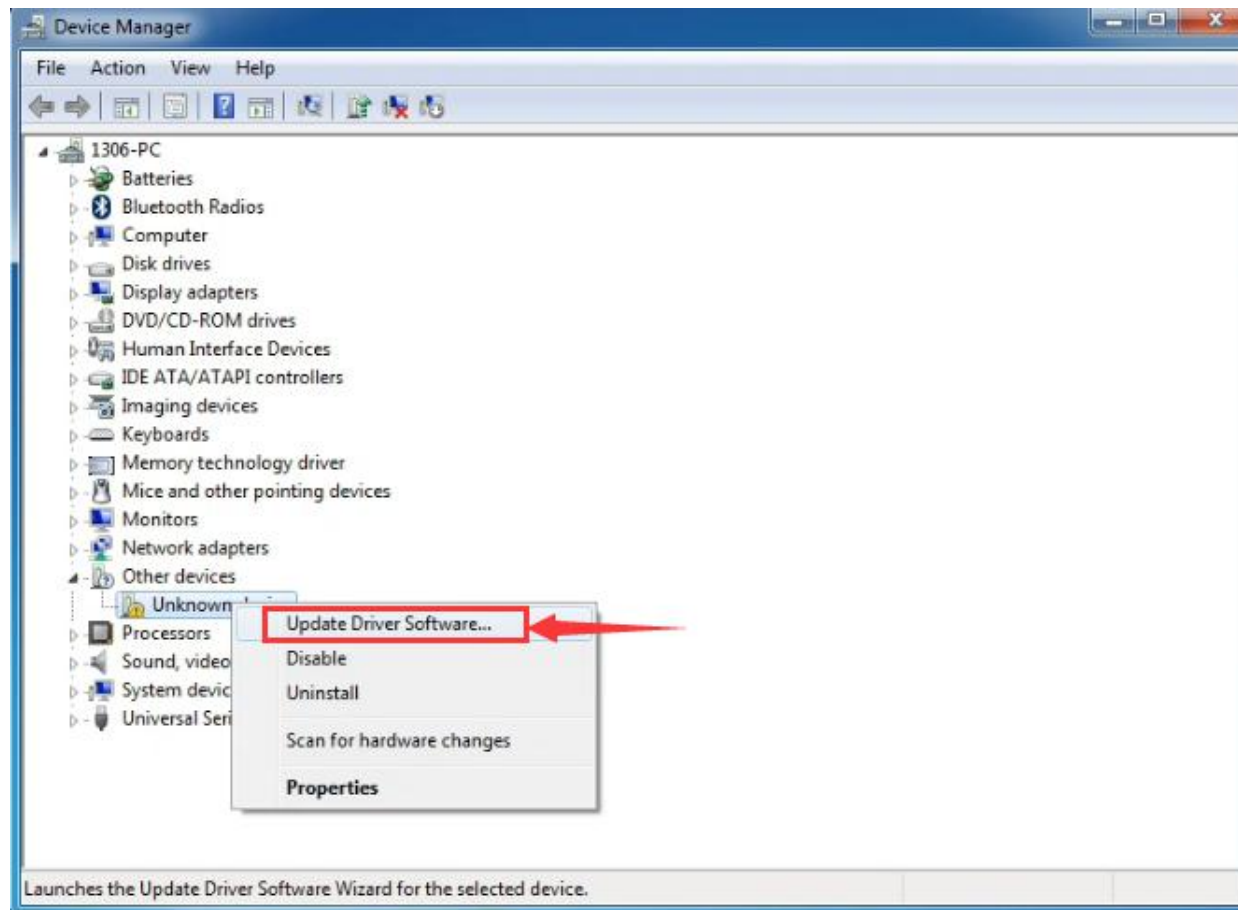
The Arduino folder contains both the Arduino program itself and the drivers that allow the Arduino to be connected to your computer by a USB cable. Before we launch the Arduino software, you are going to install the USB drivers.

Plug one end of your USB cable into the board and the other into a USB socket on your computer.

When you connect the board to your computer at the first time, right click the icon of your *"Computer"* —> *for "Properties"*—> *click the "Device manager"*, under "Other Devices", you should see an icon for "Unknown device" with a little yellow warning triangle next to it.

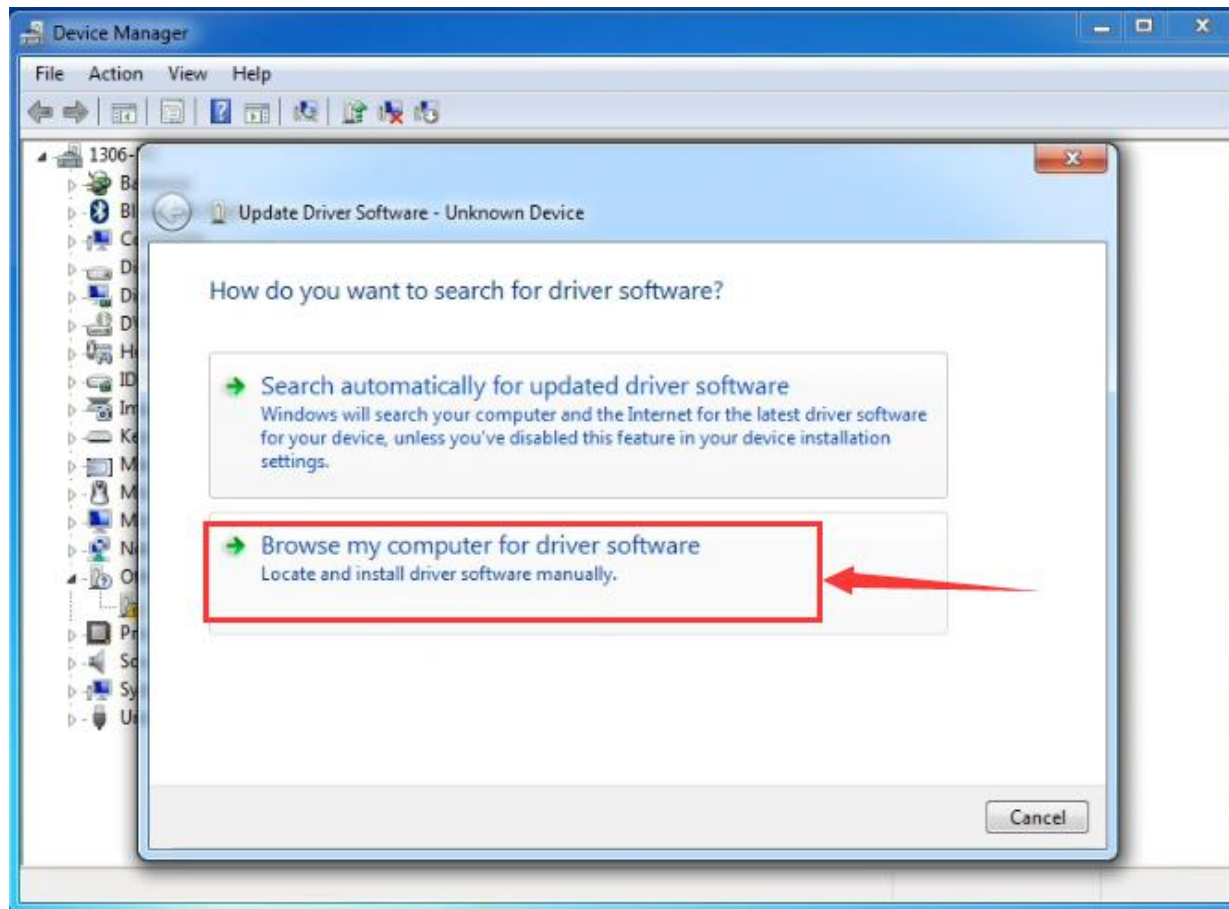


Then right-click on the device and select the top menu option (Update Driver Software...) shown as the figure below.

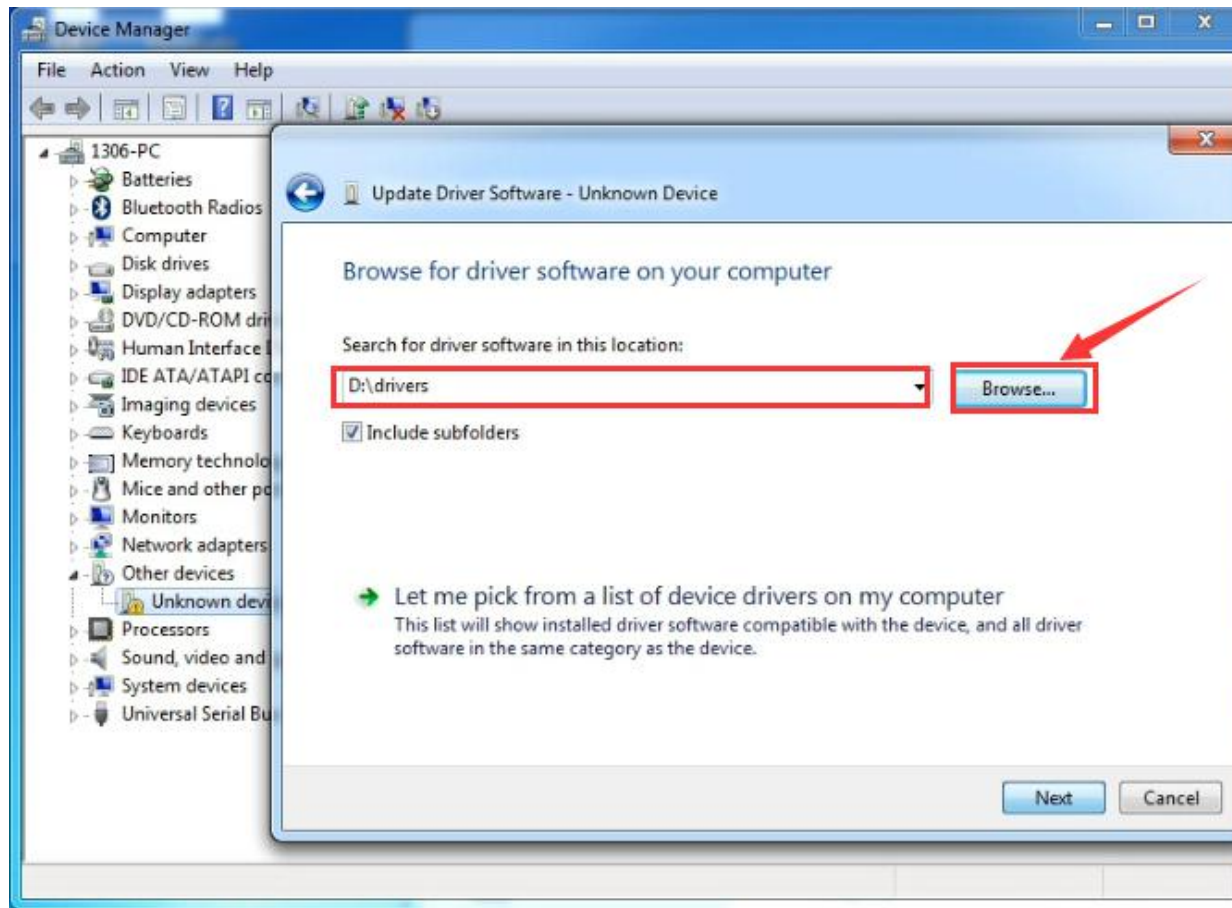


It will then be prompted to either "Search Automatically for updated driver software" or "Browse my computer for

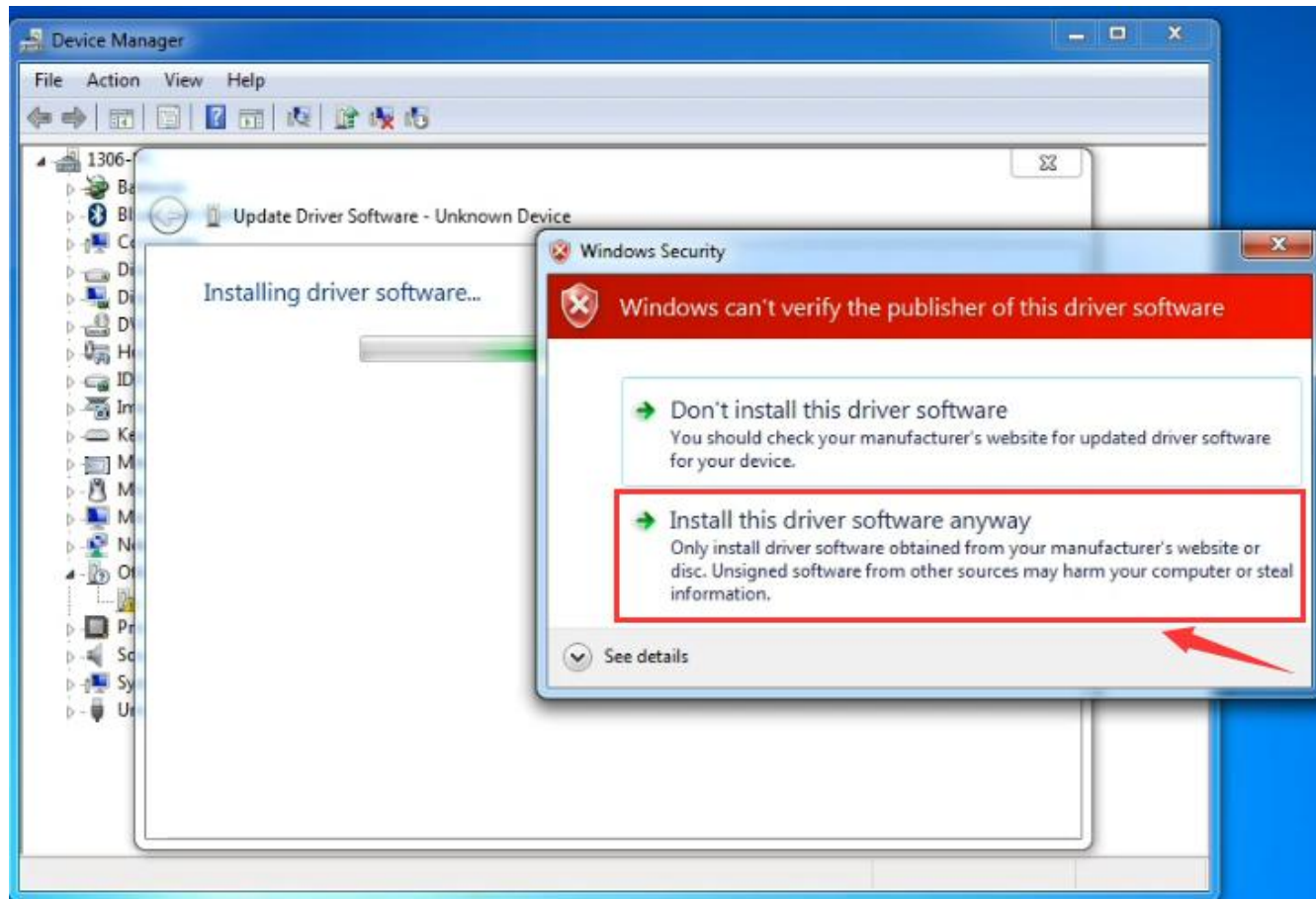
driver software". Shown as below. In this page, select "Browse my computer for driver software".



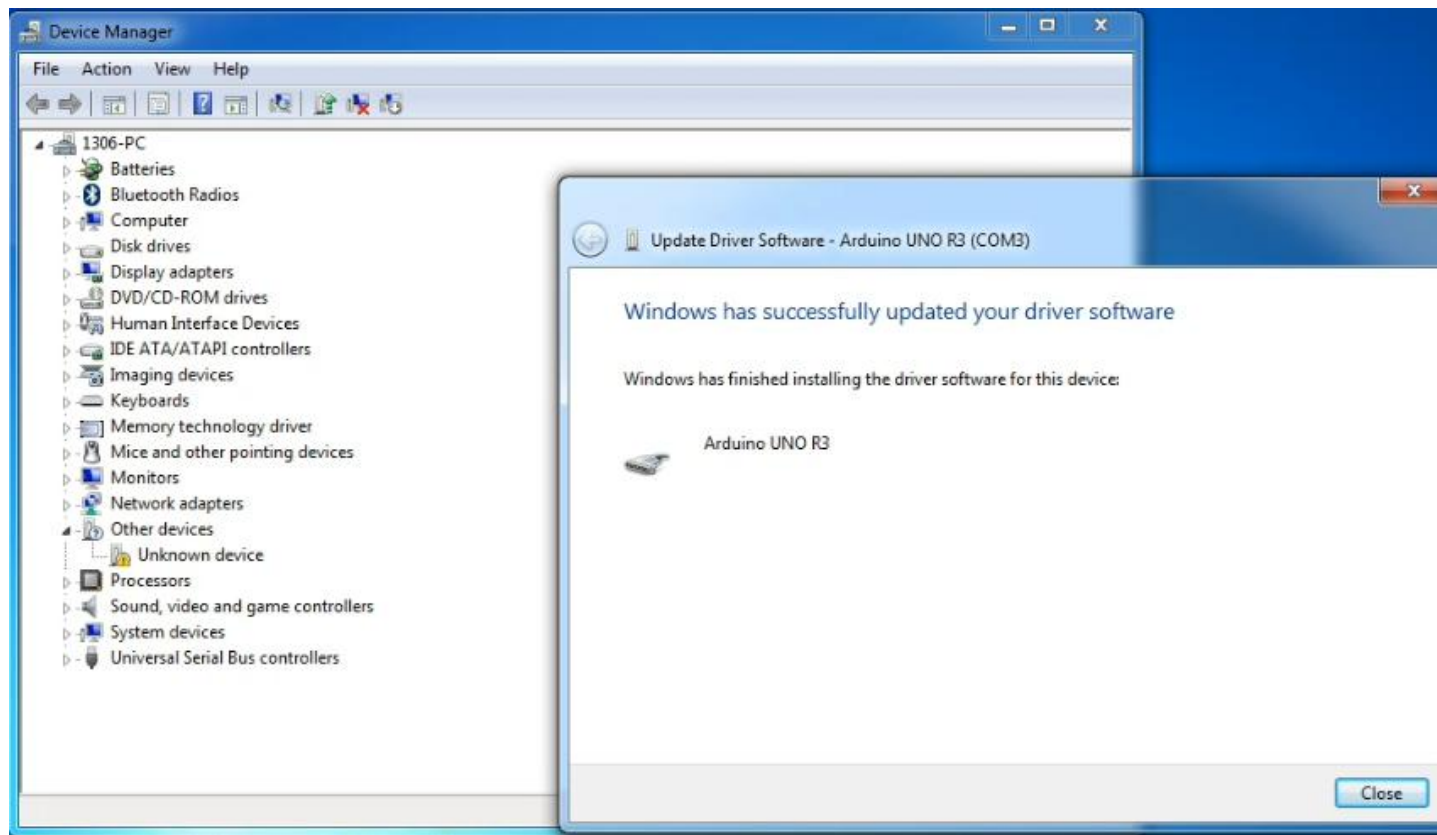
After that, select the option to browse and navigate to the "drivers" folder of Arduino installation.



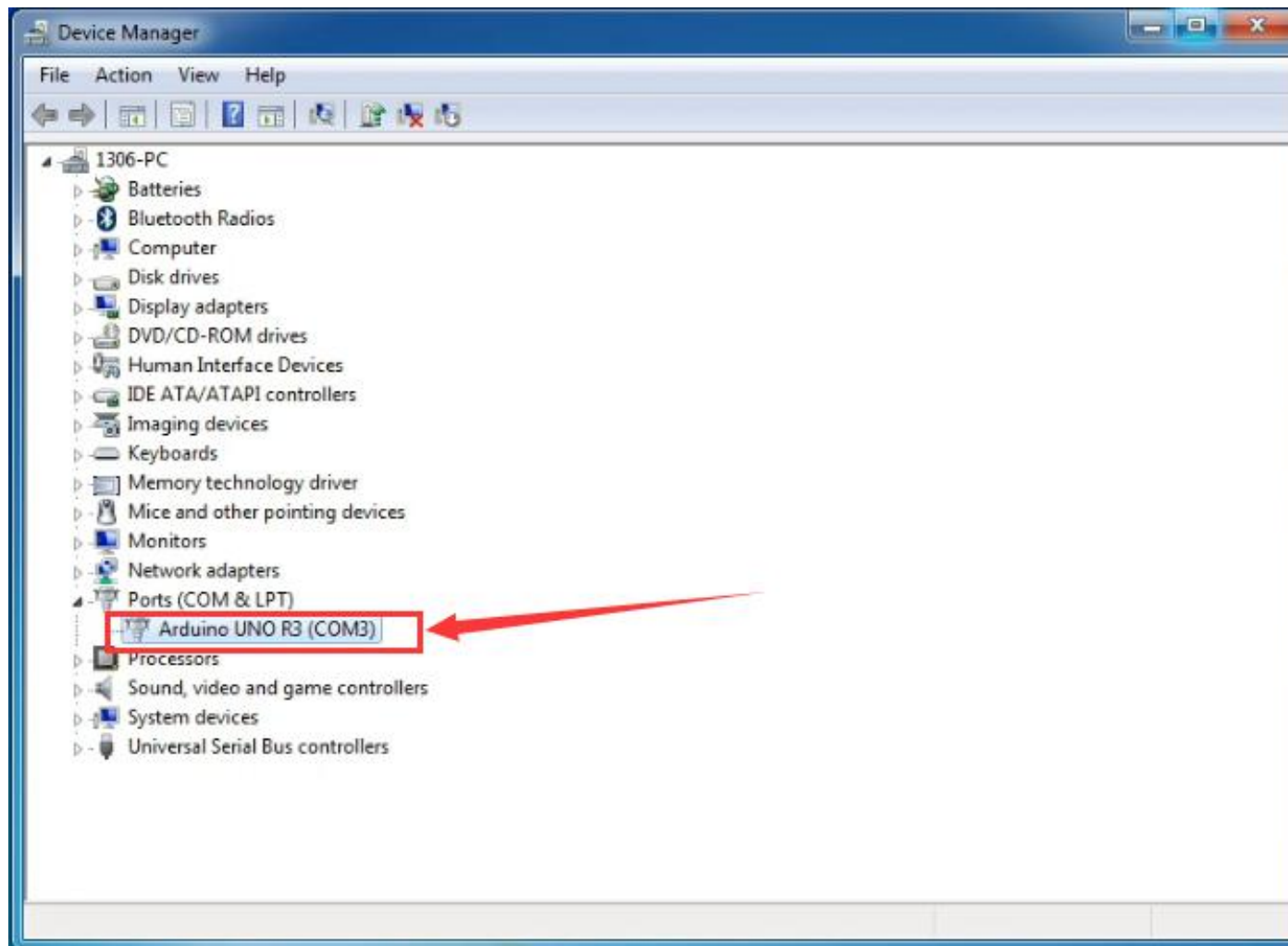
Click "Next" and you may get a security warning, if so, allow the software to be installed. Shown as below.



Once the software has been installed, you will get a confirmation message. Installation completed, click "Close".



Up to now, the driver is installed well. Then you can right click "*Computer*" —> "*Properties*" —> "*Device manager*", you should see the device as the figure shown below.

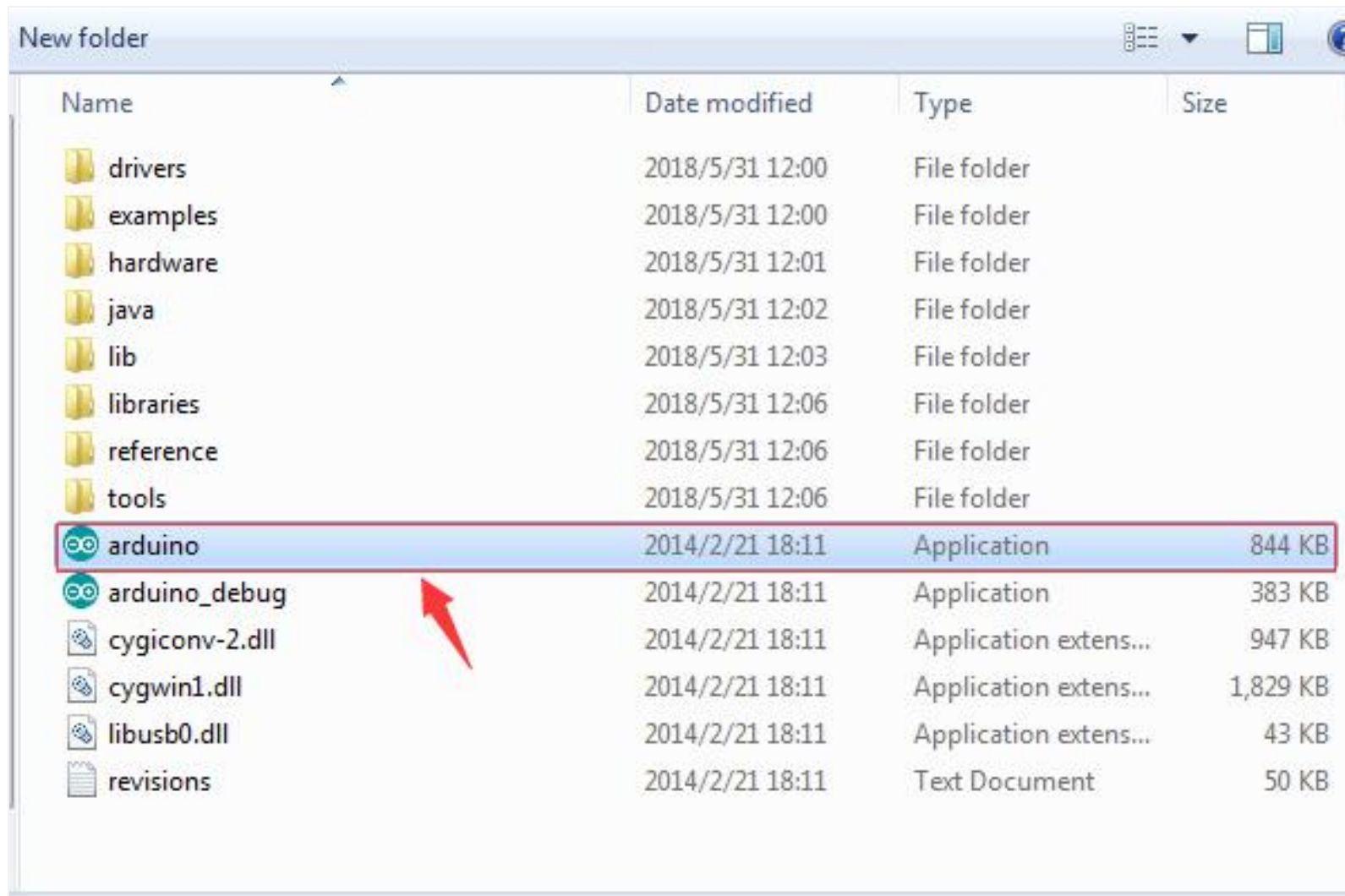


2) Example Use of ARDUINO IDE

STEP 1: Open Arduino IDE

In the previous, we have introduced the Arduino installation. So this time let's first have basic understanding of the ARDUINO development environment. After that, you will learn how to upload the program to Arduino board.

First of all, open the unzipped folder of ARDUINO development software and click icon of ARDUINO to open the software, as the figure shown below.



New folder

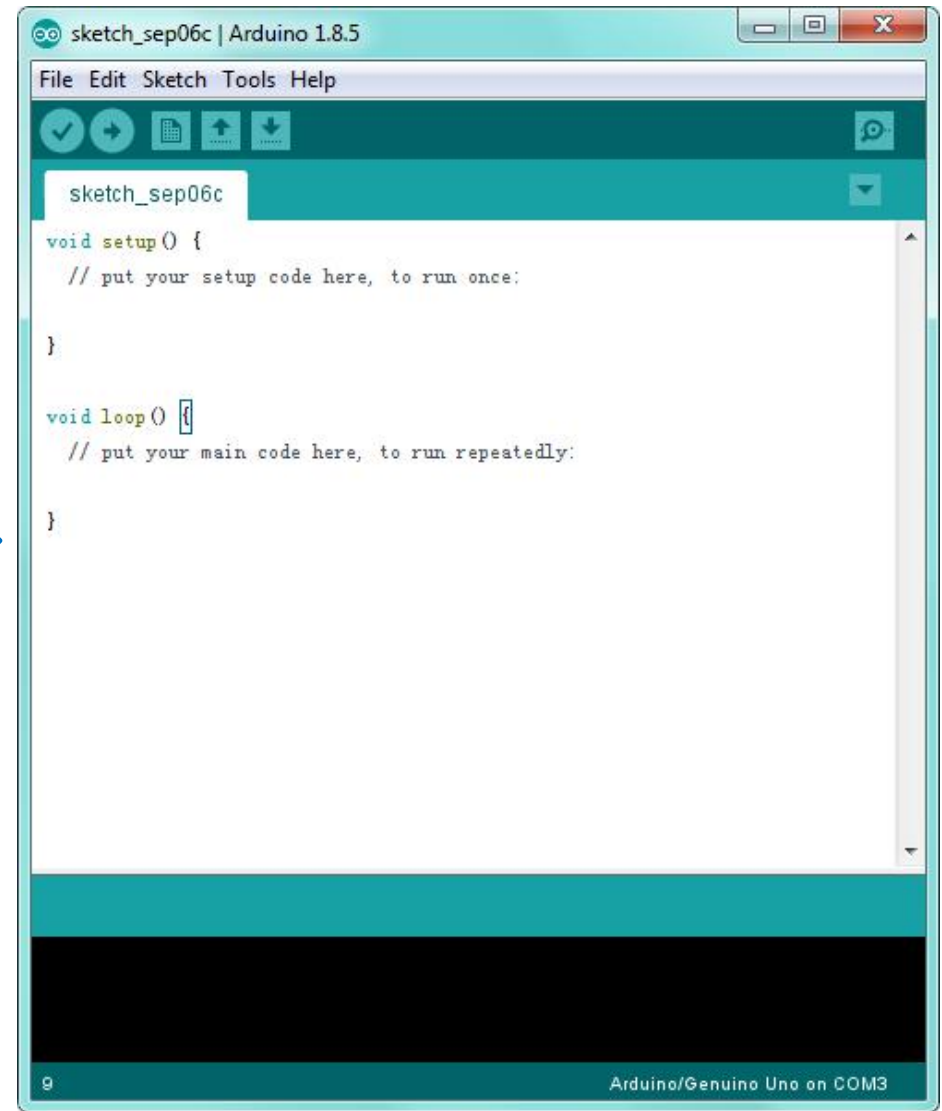
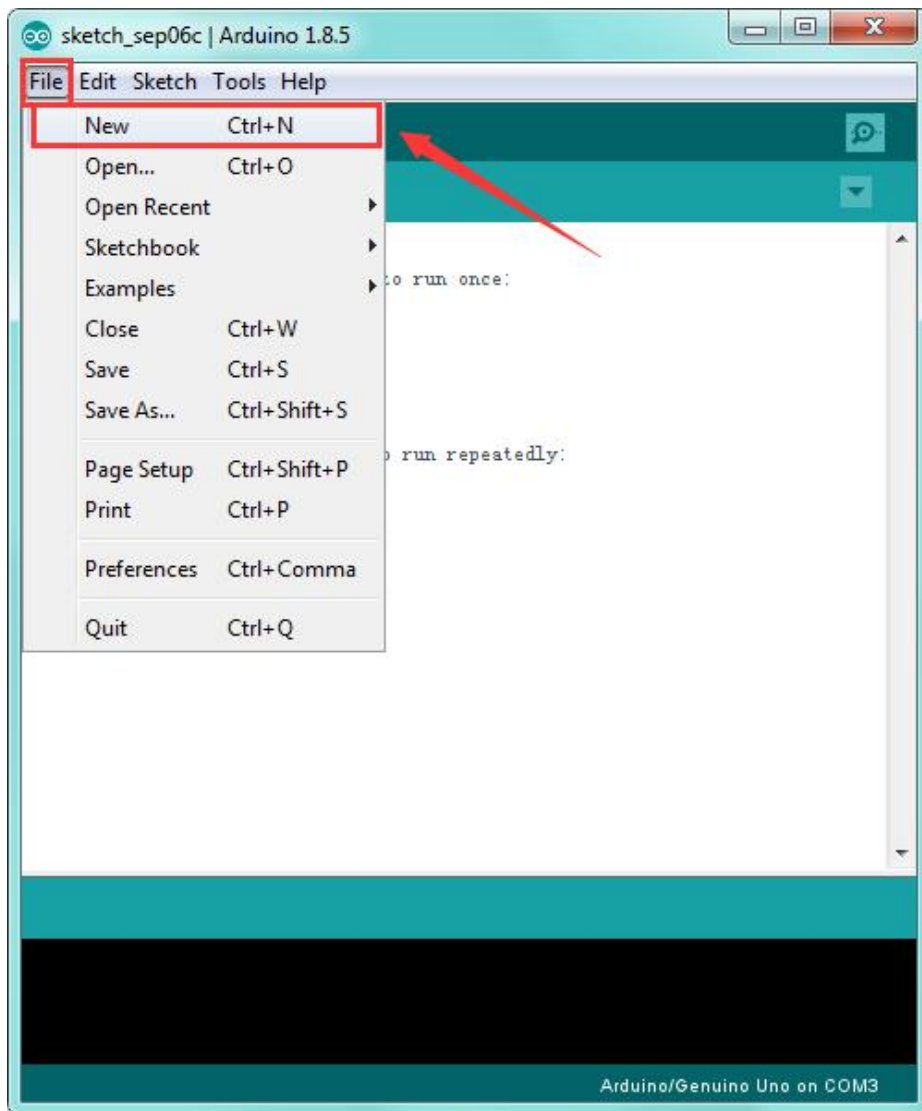
Name	Date modified	Type	Size
drivers	2018/5/31 12:00	File folder	
examples	2018/5/31 12:00	File folder	
hardware	2018/5/31 12:01	File folder	
java	2018/5/31 12:02	File folder	
lib	2018/5/31 12:03	File folder	
libraries	2018/5/31 12:06	File folder	
reference	2018/5/31 12:06	File folder	
tools	2018/5/31 12:06	File folder	
arduino	2014/2/21 18:11	Application	844 KB
arduino_debug	2014/2/21 18:11	Application	383 KB
cygiconv-2.dll	2014/2/21 18:11	Application extens...	947 KB
cygwin1.dll	2014/2/21 18:11	Application extens...	1,829 KB
libusb0.dll	2014/2/21 18:11	Application extens...	43 KB
revisions	2014/2/21 18:11	Text Document	50 KB

STEP 2: Build Projects

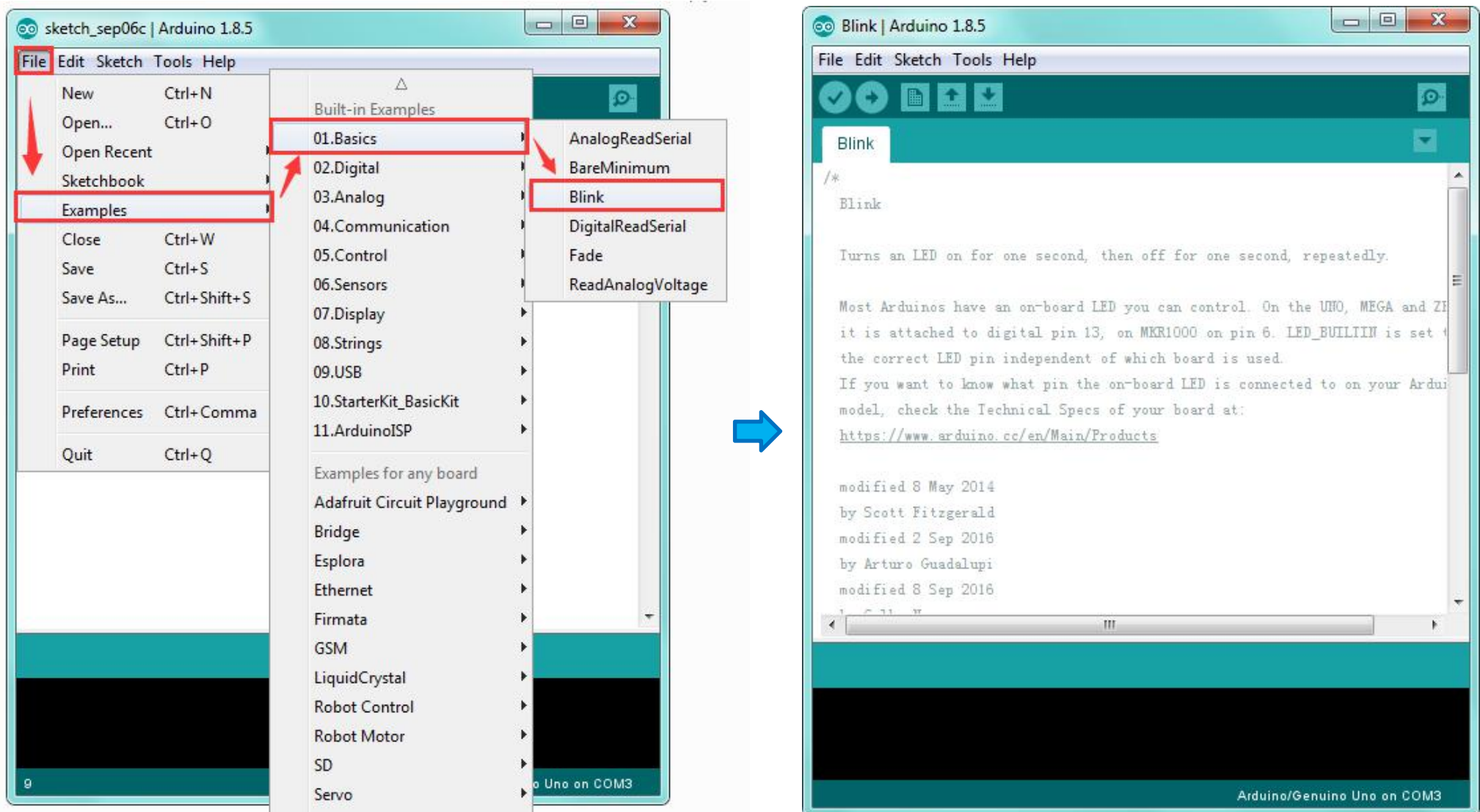
When open the Arduino software, you will have two options as below:

- Build a new project
- Open an exiting project example

If you want to build a new project, please select “File”→then click “New”, you will see the software interface as follows.

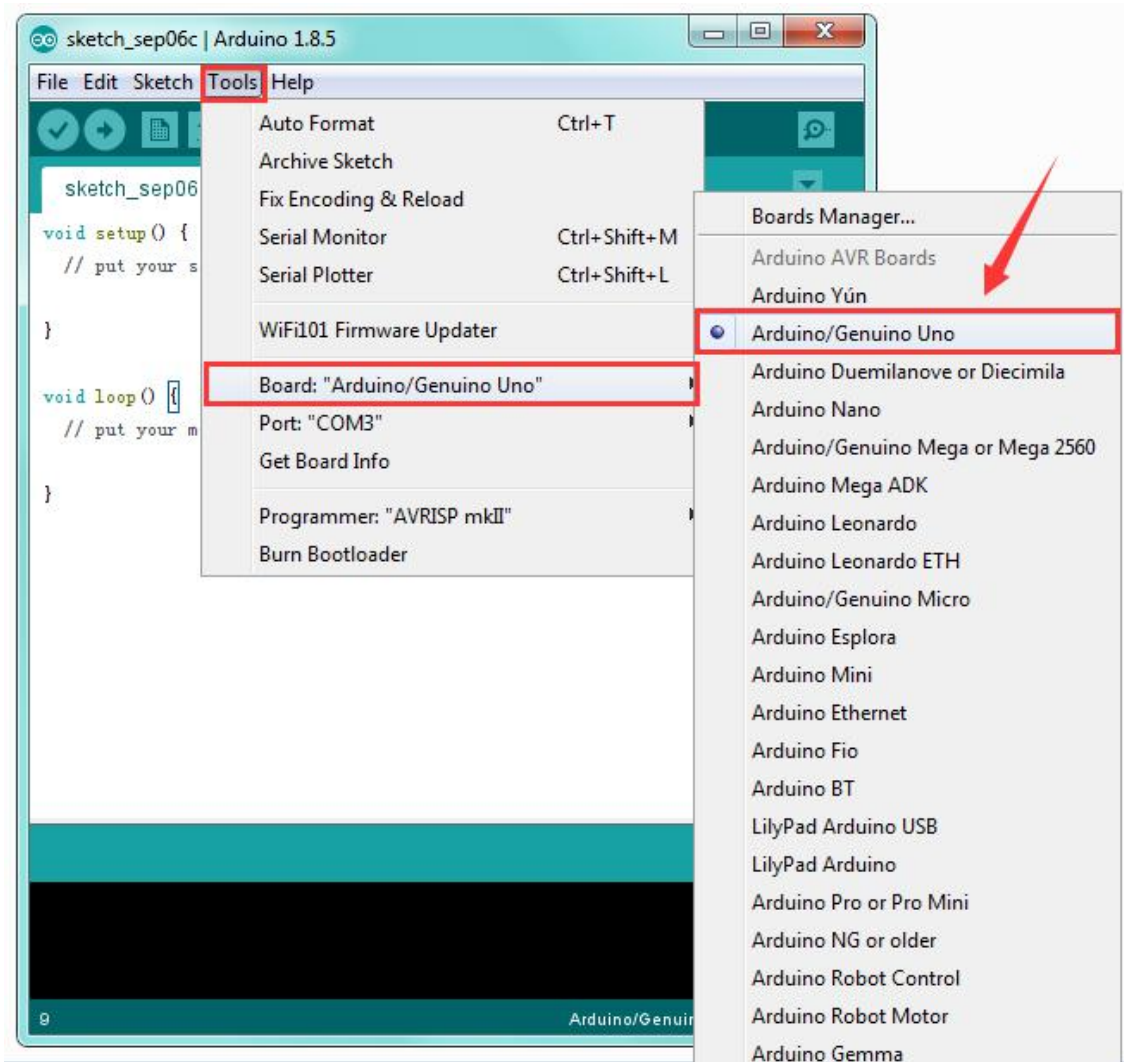


If you want to open an example project, please select *File*→*Example*→*Basics*→*Blink*. Shown below.



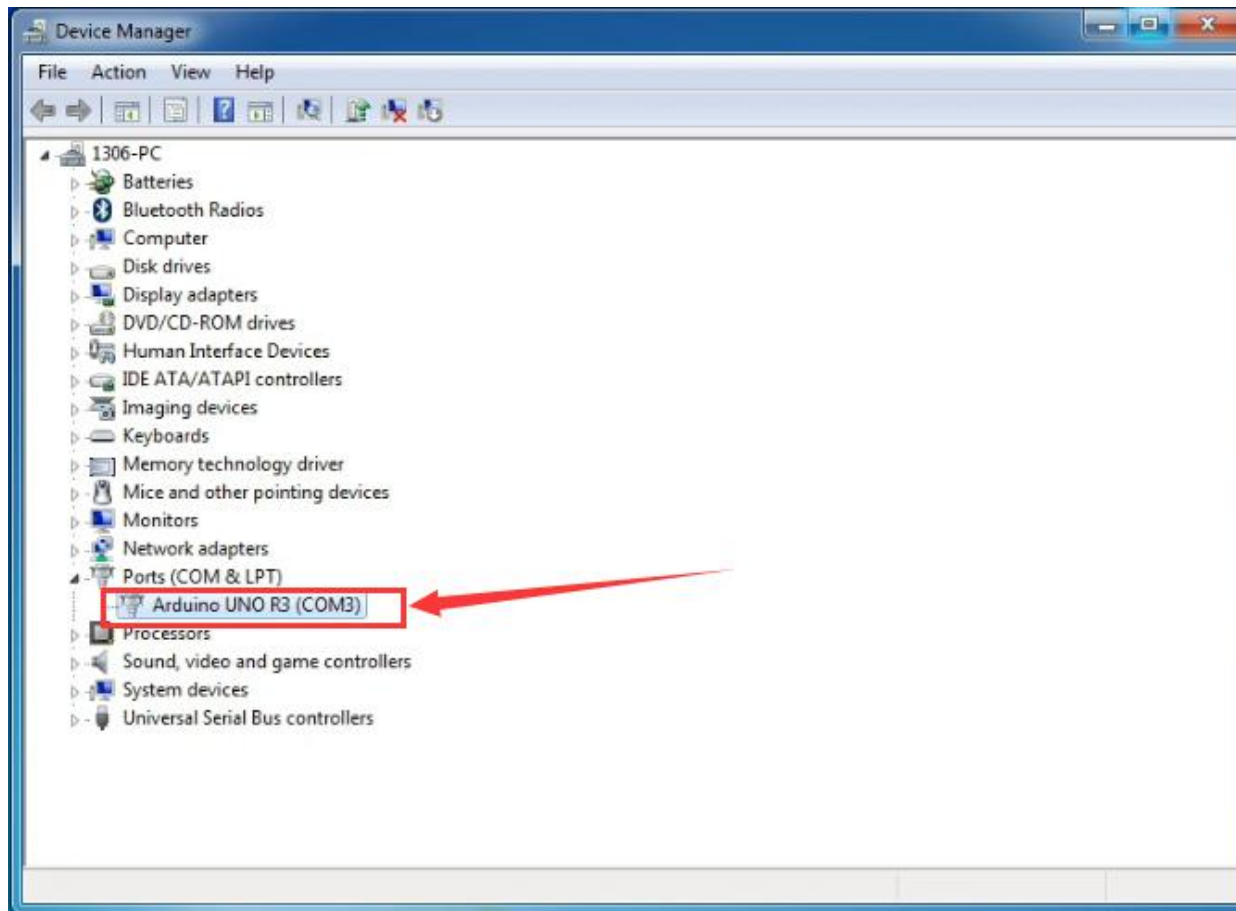
STEP 3: Select Arduino Board

On the Arduino software, you should click *Tools*→*Board* , select the correct board. Here in our tutorial we should select Arduino Uno. Shown as below.

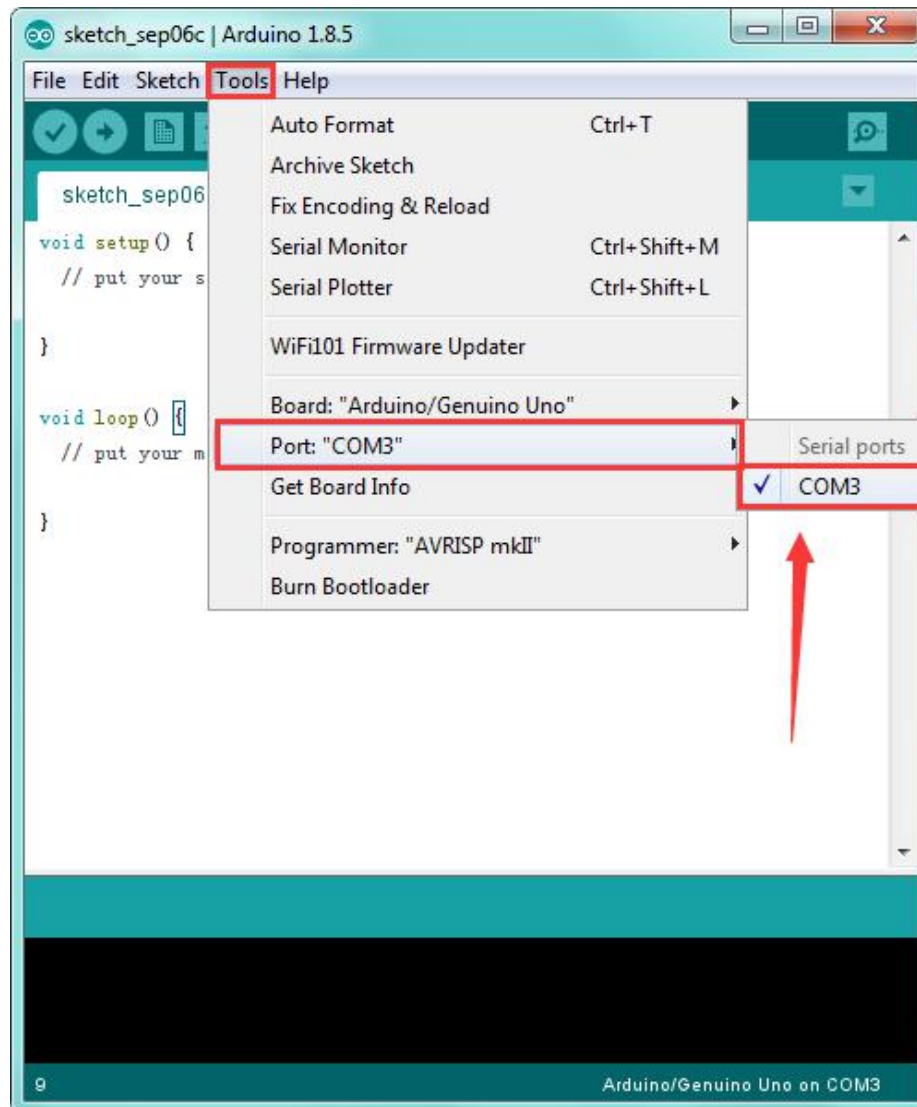


STEP 4: Select Serial Port

If you are not sure which port is correct, at first directly open the Control Panel of your computer, then click to open *Device Manager*, you can check the COM port here. Shown as below.



Then you should click *Tools*→*Serial Port*. It may be COM3 or higher.



STEP 5: Upload the Code to Your Board

Before showing you how to upload the code to your board, you can check the function of each icon on the Tool bar of Arduino IDE listed below:



 Verify/Compile	Check the code for errors
 Upload	Upload the current Sketch to the Arduino
 New	Create a new blank Sketch

 Open	Show a list of Sketches
 Save	Save the current Sketch
 Serial Monitor	Display the serial data being sent from the Arduino

Project 2: Joint Rotation and Pin Control

1) Joint Rotation and Servo Angle Settings

Name	0°	180°
Servo 1 (baseplate)	Rotate toward the rightmost	Rotate toward the leftmost
Servo 2 (right side)	Rocker arm connected to Servo 2 draws back	stretch out
Servo 3 (left side)	Rocker arm connected to Servo 3 stretches out	draw back
Servo 4 (clamp claw)	closed	opened

2) Pin Control

Name	IO Pin
Servo 1 (baseplate)	A1
Servo 2 (right side)	A0
Servo 3 (left side)	6
Servo 4 (clamp claw)	9
Right Joystick X	A2
Right Joystick Y	A5
Right Joystick Z (key)	7
Left Joystick X	A3
Left Joystick Y	A4
Left Joystick Z	8

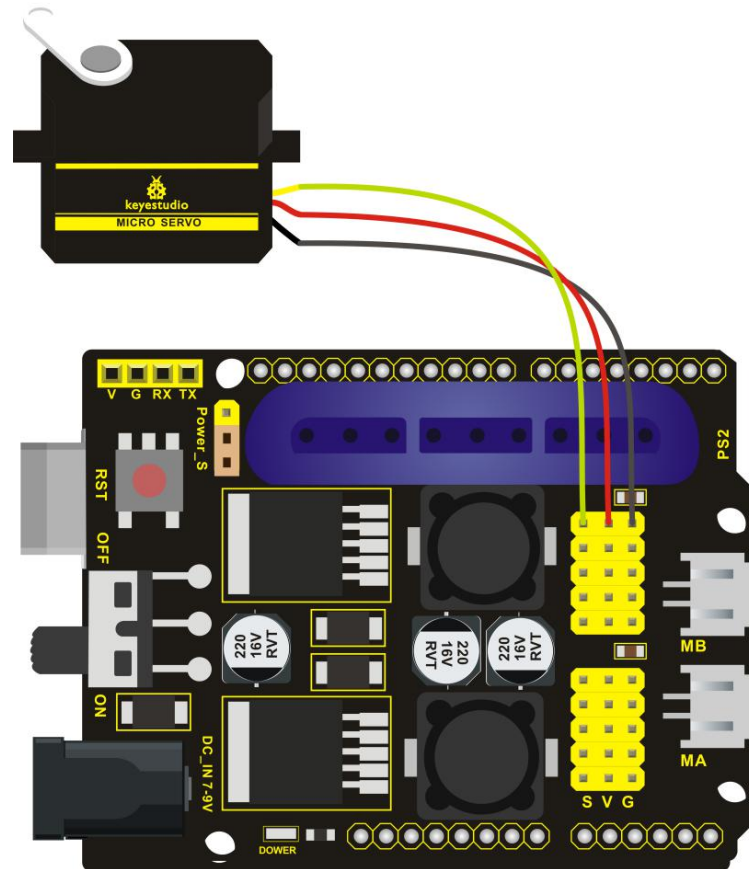
D1/DAT of PS2	12
D0/CMD of PS2	11
CE/SEL of PS2	10
CLK of PS2	13

Project 3: JoyStick Controlled Robot Arm

3.1 Servo Control

❖ How to adjust the Servo Angel

Hookup Guide:



Code to Note:

In the previous assembly, we have set the square wave and servo angle. So we should use the servo libraries to control the angle of each servo. The test result is the same. Before using, place the Servo folder inside the libraries folder of Arduino IDE directory, then open the Arduino IDE, and the libraries are able to use.

You can download all the libraries for test code from the link:

<https://drive.google.com/open?id=1HtyzuNT59Av5QP8nO0kv6syQsoK1fUOw>

Test Code 1:

```
#include <Servo.h>
Servo myservo; // create servo object to control a servo

void setup()
{
  Serial.begin(9600);
  delay(1000);
}

void loop()
{
```

```
myservo.attach(A0); // modify each pin to adjust
myservo.write(0); // angle value
delay(1000);
}
```

Test Result:

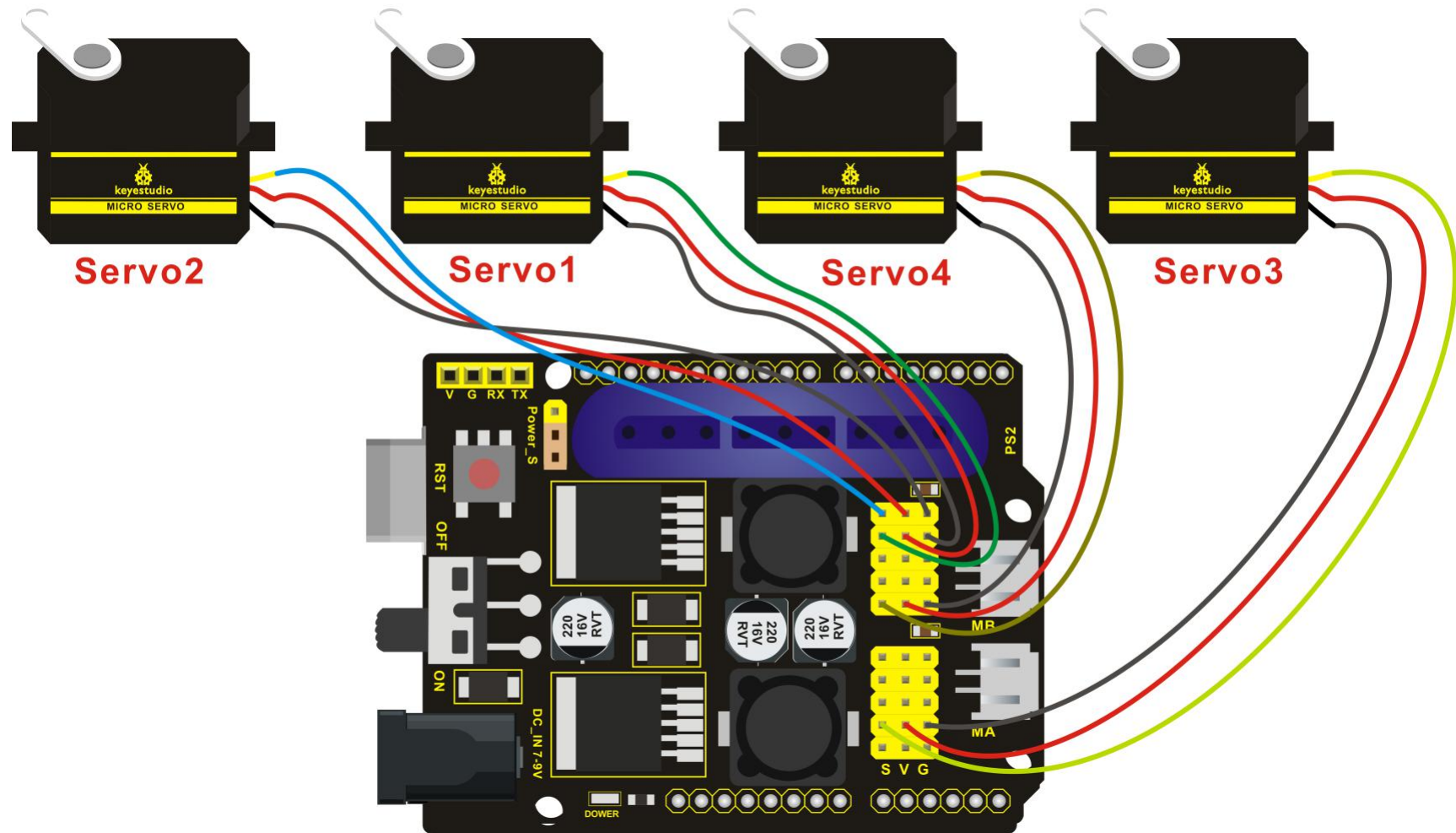
Stack the drive shield onto UNO R3 board and connect the servo motor, upload well the code. Powered on, press the reset button, the servo will automatically rotate to 0° .

❖ Automatic Turning

Description:

In the previous section, you have learned to set the Servo angle. In fact, we just need to continually change the angle of 4 Servo motors, thus make the 4DOF robot arm operate different motions.

Hookup Guide:



Test Code 2:

```
*****
#include <Servo.h>
Servo myservol; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;
int pos1=80, pos2=60, pos3=130, pos4=0;

void setup()
{
  myservol.attach(A1); // attaches the servo on pin 9 to the servo object
  myservo2.attach(A0);
  myservo3.attach(6);
  myservo4.attach(9);

  myservol.write(pos1);
  delay(1000);
  myservo2.write(pos2);
  myservo3.write(pos3);
  myservo4.write(pos4);
  delay(1500);
}
```

```
void loop()
{
  // turn right
  for(pos1;pos1>0;pos1--)
  {
    myservo1.write(pos1);
    delay(5);      // delay 5ms (used to adjust the servo speed)
  }
  delay(1000);

  // open the claw
  for(pos4;pos4<100;pos4++)
  {
    myservo4.write(pos4);
  }
  delay(1000);

  // right servo rotates to 100 degrees
  for(pos2;pos2>50;pos2--)
  {
    myservo2.write(pos2);
    delay(5);
  }

  // left servo rotates to 5 degrees
  for(pos3;pos3>50;pos3--)
```

```
{
  myservo3.write(pos3);
  delay(5);
}
delay(1500);

// close the claw
for(pos4;pos4>0;pos4--)
{
  myservo4.write(pos4);
}
delay(1000);

// left servo rotates to 100 degrees, rocker arm lifts.
for(pos3;pos3<120;pos3++)
{
  myservo3.write(pos3);
  delay(5);
}
delay(1000);

// turn to left
for(pos1;pos1<180;pos1++)
{
  myservo1.write(pos1);
  delay(5);
}
```

```
}  
delay(1000);  
  
// Lower the arm  
for(pos3;pos3>50;pos3--)  
{  
    myservo3.write(pos3);  
    delay(5);  
}  
delay(1000);  
  
// open the claw  
for(pos4;pos4<100;pos4++)  
{  
    myservo4.write(pos4);  
}  
delay(1000);  
  
// lift up the arm  
for(pos3;pos3<120;pos3++)  
{  
    myservo3.write(pos3);  
    delay(5);  
}  
delay(1000);
```

```
// close the claw
for(pos4;pos4>0;pos4--)
{
    myservo4.write(pos4);
}
delay(1000);
}
```

Test Result:

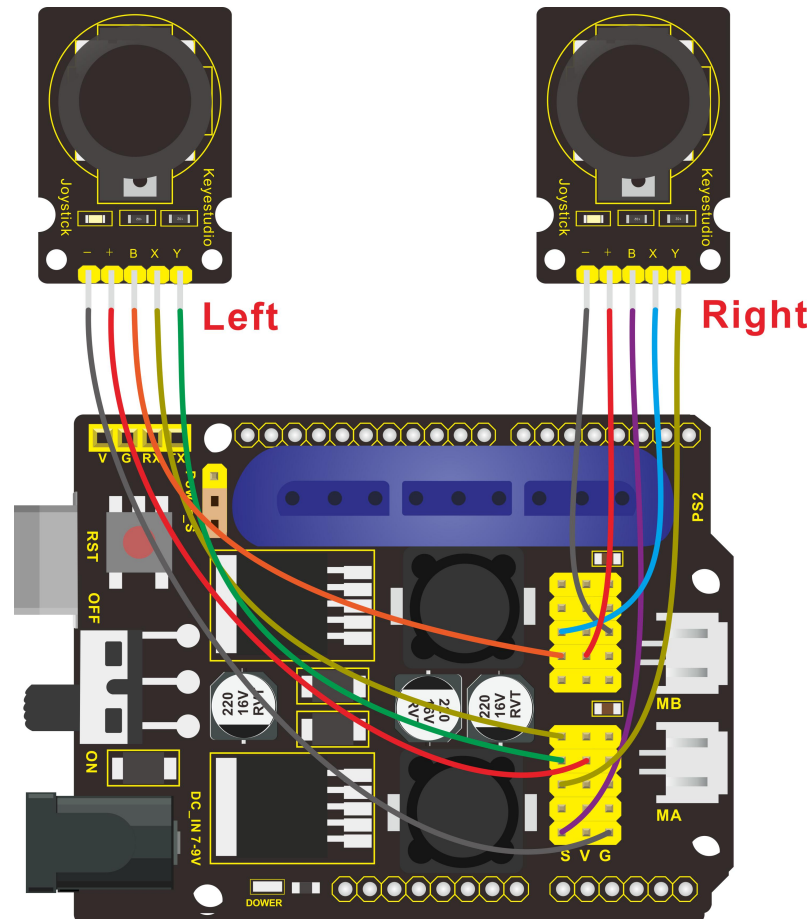
Stack the drive shield onto UNO R3 board and connect the servo motor, upload well the code. Powered on, press the reset button, the robot arm will realize a cyclic motion. Turn to the right, stretch out the arm, lower, the claw is clamped; then the arm is retracted, raised, turned to the left, stretched out, lowered and the claw is opened. After that, return to the right and continue to repeat the motion.

3.2 Read the JoyStick Value

Description:

The sensor's pin X, Y are for analog sensor, so directly read the measured analog value. Pin Z is a digital button, first should set the pin to Input status and then read the measured value 1 (pressed down) or 0 (not press). Check out the value printed on the serial monitor.

Hookup Guide:



Test Code 3:

```
*****
const int right_X = A2; // define the right X pin to A2
const int right_Y = A5; // define the right Y pin to A5
const int right_key = 7; //define the right key pin to 7 (that is the value Z)

const int left_X = A3; //define the left X pin to A3
const int left_Y = A4; // define the left Y pin to A4
const int left_key = 8; //define the left key pin to 8 (that is the value Z)

void setup()
{
  pinMode(right_key, INPUT); // set the right/left key to INPUT
  pinMode(left_key, INPUT);
  Serial.begin(9600); // set the baud rate to 9600
}

void loop()
{
  int x1,y1,z1; // define the variable, used to save the joystick value it reads
  int x2,y2,z2;

  x1 = analogRead(right_X); // read the value of right X
```

```
y1 = analogRead(right_Y); // read the value of right Y
z1 = digitalRead(right_key); //// read the value of right Z

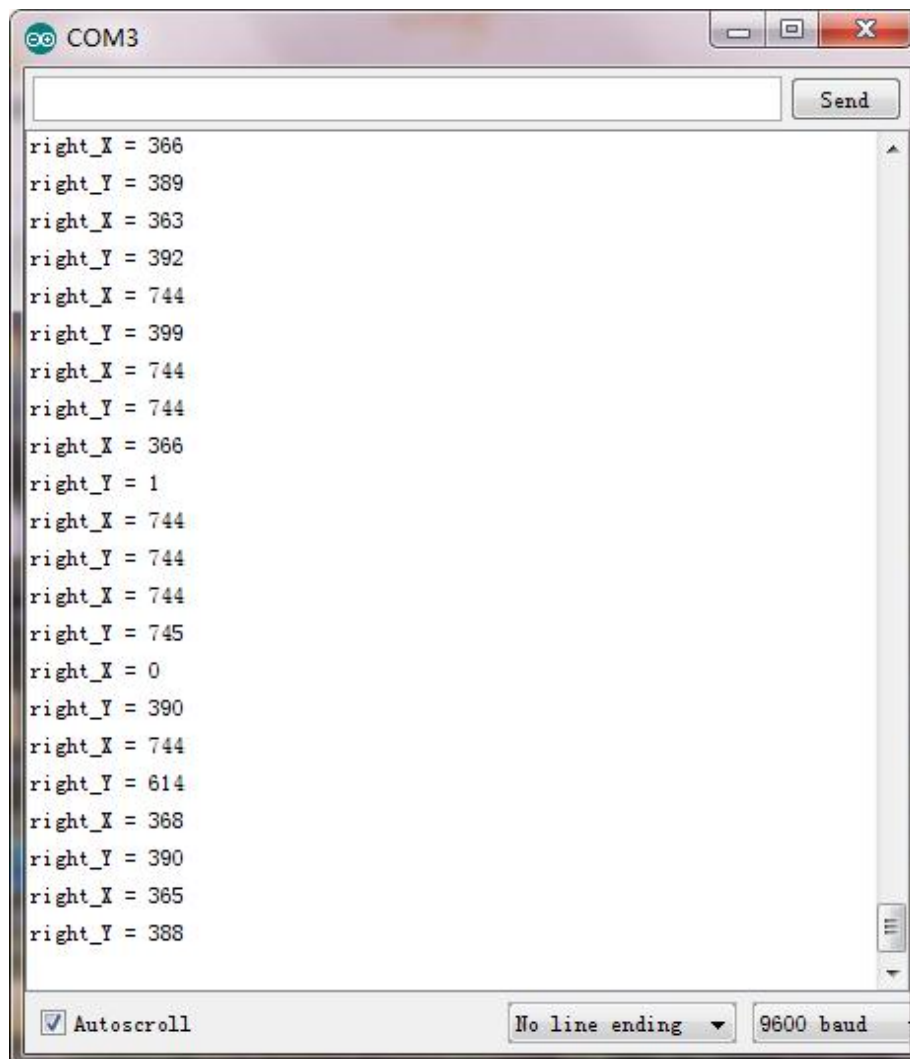
x2 = analogRead(left_X); // read the value of left X
y2 = analogRead(left_Y); // read the value of left Y
z2 = digitalRead(left_key); // read the value of left Z

Serial.print("right_X = "); // on the serial monitor, print out right_X =
Serial.println(x1 ,DEC); // print out the value of right X and line wrap
Serial.print("right_Y = ");
Serial.println(y1 ,DEC);
//Serial.print("right_key = ");
//Serial.println(z1 ,DEC);
// Serial.println("*****right*****");
/*Serial.print("left_X = ");
Serial.println(x2 ,DEC);
Serial.print("left_Y = ");
Serial.println(y2 ,DEC);
Serial.print("left_key = ");
Serial.println(z2 ,DEC);
Serial.println("*****left*****");*/
delay(200);
}

*****
```

Test Result:

Hook it up and upload well the code. Connect the UNO R3 to computer using a USB cable, then open the serial monitor and set the baud rate to 9600, you should see the analog value of the right Joystick pin X,Y.



The screenshot shows the Serial Monitor window for COM3. The window has a title bar with 'COM3' and standard window controls. Below the title bar is a text input field and a 'Send' button. The main area displays a list of joystick coordinates. At the bottom, there are settings: 'Autoscroll' is checked, 'No line ending' is selected from a dropdown, and '9600 baud' is set.

```
right_X = 366
right_Y = 389
right_X = 363
right_Y = 392
right_X = 744
right_Y = 399
right_X = 744
right_Y = 744
right_X = 366
right_Y = 1
right_X = 744
right_Y = 744
right_X = 744
right_Y = 745
right_X = 0
right_Y = 390
right_X = 744
right_Y = 614
right_X = 368
right_Y = 390
right_X = 365
right_Y = 388
```

☒ Autoscroll No line ending ▼ 9600 baud

3.3 Dual-JoyStick Controlling

Description:

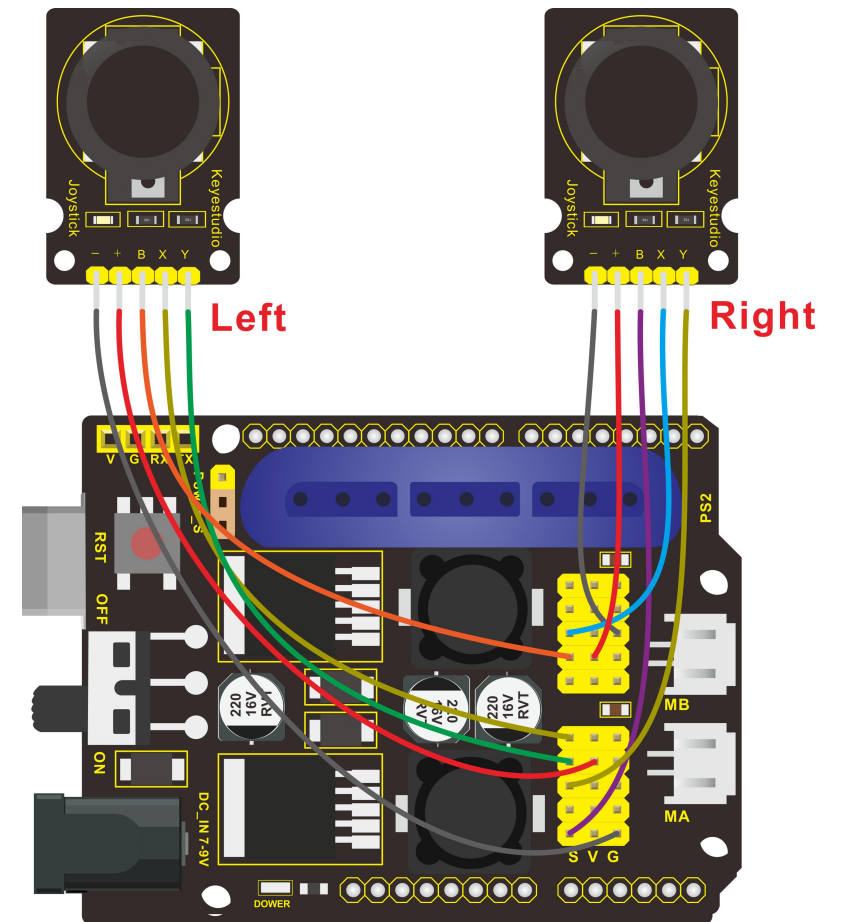
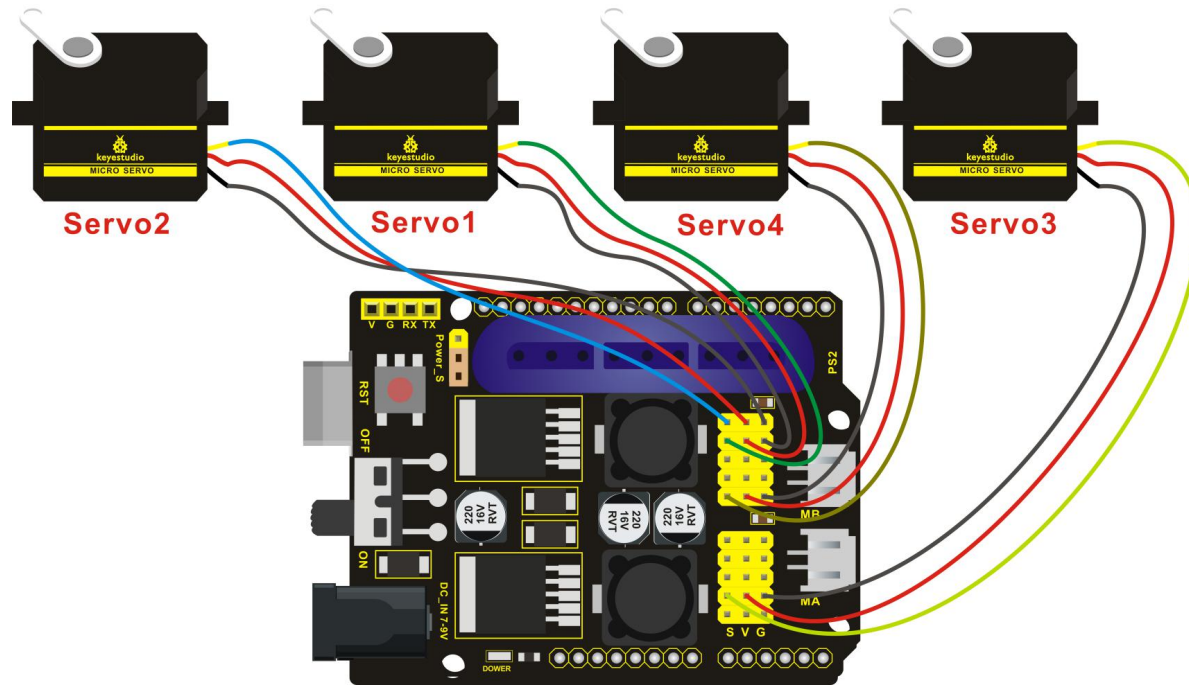
In the previous section, we have introduced how to use 4 Servo to control the robot arm. Next, combine those two experiments. Use two Joystick modules to control 4DOF robot arm realize different motions.

At first, set the boot posture. The Joystick control is shown as below table.

Right Joystick	Servo	Left Joystick	Servo
$X1 < 50$	Servo 1 gradually reduces to 0° (push the right joystick to the right, the servo that controls the arm rotation turns right, and stops at 0°)	$X2 < 50$	Servo 4 gradually reduces to 0° (push the left joystick to the right, the claw is closed)
$X1 > 1000$	Servo 1 gradually increases to 180° (push the right joystick to the left, the servo that controls the arm rotation turns left, and stops at 180°)	$X2 > 1000$	Servo 4 gradually increases to 180° (push the left joystick to the left, the claw opens)

Y1>1000	Servo 2 gradually reduces to 0° (that is, lift up the robot upper arm)	Y2>1000	Servo 3 gradually reduces to 35° (that is, stretch out the robot lower arm)
Y1<50	Servo 2 gradually reduces to 180° (that is, lower the robot upper arm)	Y2<50	Servo 3 gradually increases to 180° (that is, draw back the robot lower arm)

Hookup Guide:



Test Code 4:

```
*****
```

```
#include <Servo.h> // add the servo libraries
```

```
Servo myservo1; // create servo object to control a servo
```

```
Servo myservo2;
```

```
Servo myservo3;
```

```
Servo myservo4;
```

```
int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle, and assign the initial value (that is the boot posture angle value)
```

```
const int right_X = A2; // define the right X pin to A2
```

```
const int right_Y = A5; // define the right Y pin to A5
```

```
const int right_key = 7; // define the right key pin to 7 (that is the value of Z)
```

```
const int left_X = A3; // define the left X pin to A3
```

```
const int left_Y = A4; // define the left X pin to A4
```

```
const int left_key = 8; //define the left key pin to 8 (that is the value of Z)
```

```
int x1,y1,z1; // define the variable, used to save the joystick value it read.
```

```
int x2,y2,z2;
```

```
void setup()
{

    // boot posture
    myservo1.write(pos1);
    delay(1000);
    myservo2.write(pos2);
    myservo3.write(pos3);
    myservo4.write(pos4);
    delay(1500);

    pinMode(right_key, INPUT); // set the right/left key to INPUT
    pinMode(left_key, INPUT);
    Serial.begin(9600); // set the baud rate to 9600
}

void loop()
{
    myservo1.attach(A1); // set the control pin of servo 1 to A1
    myservo2.attach(A0); // set the control pin of servo 2 to A0
    myservo3.attach(6); //set the control pin of servo 3 to D6
    myservo4.attach(9); // set the control pin of servo 4 to D9

    x1 = analogRead(right_X); //read the right X value
    y1 = analogRead(right_Y); // read the right Y value
    z1 = digitalRead(right_key); //// read the right Z value
```

```
x2 = analogRead(left_X); //read the left X value
y2 = analogRead(left_Y); //read the left Y value
z2 = digitalRead(left_key); // read the left Z value
//delay(5); // lower the speed overall

// claw
zhuazi();
// rotate
zhuandong();
// upper arm
xiaobi();
//lower arm
dabi();
}

//claw
void zhuazi()
{
    //claw
    if(x2<50) // if push the left joystick to the right
    {
        pos4=pos4-2; //current angle of servo 4 subtracts 2 (change the value you subtract, thus change the closed speed of claw)
        //Serial.println(pos4);
        myservo4.write(pos4); // servo 4 operates the action, claw is gradually closed.
        delay(5);
```

```
    if(pos4<2) // if pos4 value subtracts to 2, the claw in 37 degrees we have tested is closed.
    {
        // (should change the value based on the fact)
        pos4=2; // stop subtraction when reduce to 2
    }
}
if(x2>1000) //// if push the left joystick to the left
{
    pos4=pos4+8; // current angle of servo 4 plus 8 (change the value you plus, thus change the open speed of claw)
    //Serial.println(pos4);
    myservo4.write(pos4); // servo 4 operates the motion, the claw gradually opens.
    delay(5);
    if(pos4>108) // limit the largest angle when open the claw
    {
        pos4=108;
    }
}
}
//*****
// turn
void zhuandong()
{
    if(x1<50) // if push the right joystick to the right
    {
        pos1=pos1-1; //pos1 subtracts 1
        myservo1.write(pos1); // servo 1 operates the motion, the arm turns right.
        delay(5);
    }
}
```

```
    if(pos1<1)    // limit the angle when turn right
    {
        pos1=1;
    }
}
if(x1>1000)  // if push the right joystick to the let
{
    pos1=pos1+1;  //pos1 plus 1
    myservol.write(pos1);  // arm turns left
    delay(5);
    if(pos1>180)  // limit the angle when turn left
    {
        pos1=180;
    }
}
}

//*****/
//upper arm
void xiaobi()
{
    if(y1>1000)  // if push the right joystick upward
    {
        pos2=pos2-1;
        myservo2.write(pos2);  // the upper arm will lift
        delay(5);
    }
}
```

```
    if(pos2<0)  // limit the lifting angle
    {
        pos2=0;
    }
}
if(y1<50)  // if push the right joystick downward
{
    pos2=pos2+1;
    myservo2.write(pos2);  // upper arm will go down
    delay(5);
    if(pos2>180)  // limit the angle when go down
    {
        pos2=180;
    }
}
}

//*****/
// lower arm
void dabi()
{
    if(y2<50)  // if push the left joystick upward
    {
        pos3=pos3+1;
        myservo3.write(pos3);  // lower arm will stretch out
        delay(5);
    }
}
```

```
    if(pos3>180)    // limit the stretched angle
    {
        pos3=180;
    }
}
if(y2>1000)  // if push the left joystick downward
{
    pos3=pos3-1;
    myservo3.write(pos3);  // lower arm will draw back
    delay(5);
    if(pos3<35)  // limit the retracted angle
    {
        pos3=35;
    }
}
}
```

Test Result:

Upload the code to main board and stack the shield onto it. Wire it up, 4DOF robot arm will keep the initial position, push the two Joysticks to operate the robot arm.

3.4 Add Memory Function

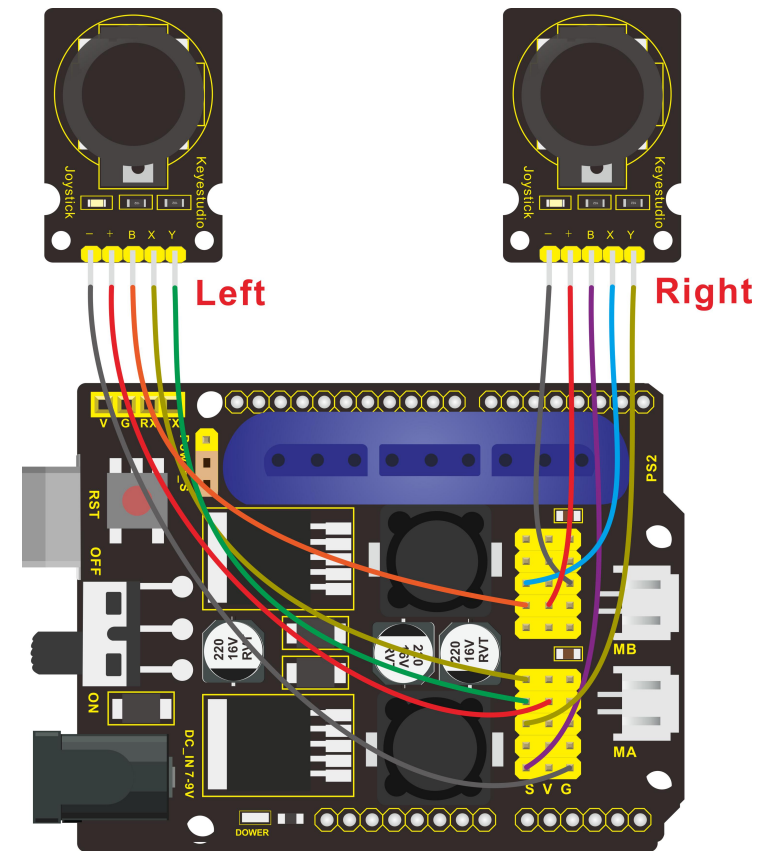
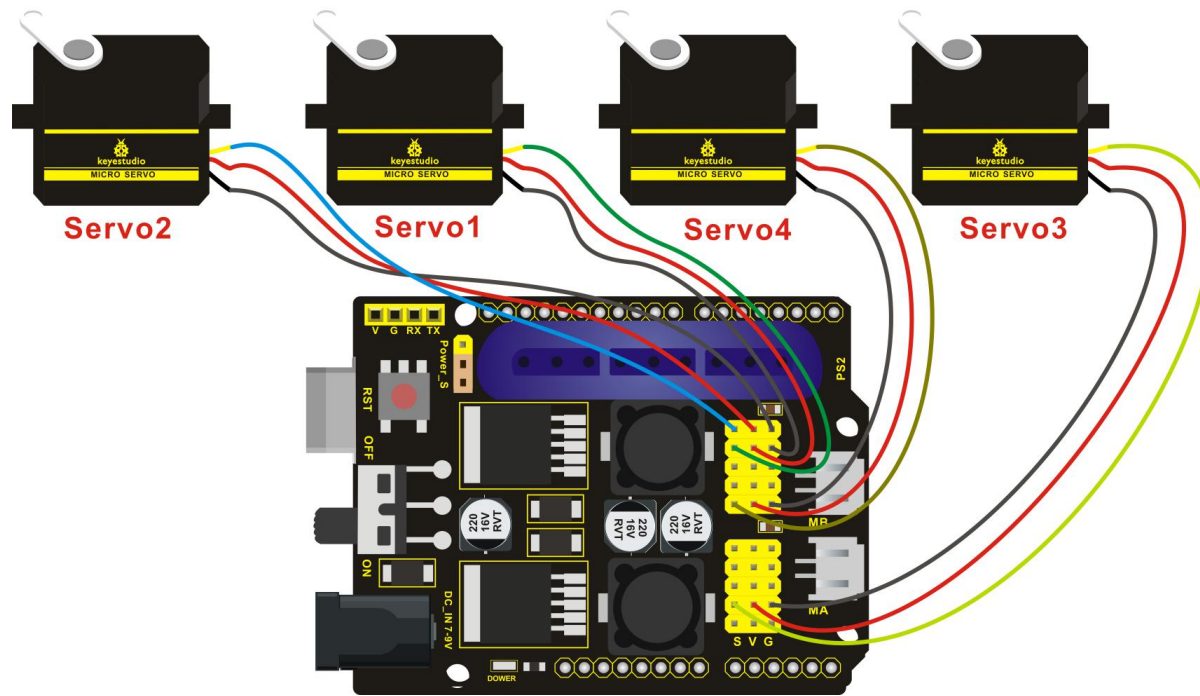
❖ Memorize One Posture

Description:

In the previous section, use the analog value of pin X,Y of 2 Joystick modules to control the robot arm.

In the following experiment, we add a memory function for the robot arm, making it remember a posture then operate. Set 4 variables for saving the angle value of 4 servos, use the Joystick to control a posture. Press the key Z1 of right Joystick to save the angle value of 4 servos; press the key Z2 of left Joystick to make the servo operate a posture saved in the variable.

Hookup Guide:



Test Code 5:

```
*****
#include <Servo.h> // add servo libraries
Servo myservol; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;
int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture
    angle value)

const int right_X = A2; // define the right X pin to A2
const int right_Y = A5; // define the right Y pin to A3
const int right_key = 7; // define the right key pin to 7 (that is Z value)

const int left_X = A3; // define the left X pin to A3
const int left_Y = A4; // define the left Y pin to A4
const int left_key = 8; // define the left key pin to 8 (that is Z value)

int x1,y1,z1; // define the variable, used to save the joystick value.
int x2,y2,z2;
int s1,s2,s3,s4;

void setup()
{
    // boot posture
    myservol.write(pos1);
    delay(1000);
}
```

```
myservo2.write(pos2);
myservo3.write(pos3);
myservo4.write(pos4);
delay(1500);

pinMode(right_key, INPUT); // set the right/left key to INPUT
pinMode(left_key, INPUT);
Serial.begin(9600); // set the baud rate to 9600
}

void loop()
{
  myservo1.attach(A1); // set the control pin of servo 1 to A1
  myservo2.attach(A0); // set the control pin of servo 2 to A0
  myservo3.attach(6); //set the control pin of servo 3 to D6
  myservo4.attach(9); //set the control pin of servo 4 to D9

  x1 = analogRead(right_X); // read the right X value
  y1 = analogRead(right_Y); // read the right Y value
  z1 = digitalRead(right_key); //// read the right key Z value

  x2 = analogRead(left_X); // read the left X value
  y2 = analogRead(left_Y); //read the left Y value
  z2 = digitalRead(left_key); //read the left key Z value
  //delay(5); // reduce the speed overall
```

```
if(z1==1) // if the right joystick key is pressed
{
    delay(10); // delay for eliminating shake
    if(z1==1) // judge again if the right key is pressed
    {
        s1=myservo1.read(); // read the angle value of each servo
        s2=myservo2.read();
        s3=myservo3.read();
        s4=myservo4.read();
    }
}
if(z2==1) // if the left key is pressed
{
    delay(10);
    if(z2==1)
    {
        pos1=myservo1.read(); // record the angle value of 4 servos in current posture
        pos2=myservo2.read();
        pos3=myservo3.read();
        pos4=myservo4.read();

        if(pos1<s1) // if angle of servo 1 is smaller than variable s1 value
        {
            while(pos1<s1) //while loop, rotate the servo to the position of the value stored in the array.
            {
                myservo1.write(pos1); // servo 1 operates the motion
```

```
    pos1++; //pos1 plus 1
    delay(5); // delay for 5ms, controlling the rotation speed of servo.
}

}

else // if angle of servo 1 is greater than the value stored in array 1.
{
    while(pos1>s1) //while loop,rotate the servo to the position of the value stored in the array.
    {
        myservo1.write(pos1); // servo 1 operates the motion
        pos1--; //pos1 subtracts 1
        delay(5); // delay for 5ms, controlling the rotation speed of servo.
    }
}

//*****
// the explanation is the same as servo 1
if(pos2<s2)
{
    while(pos2<s2)
    {
        myservo2.write(pos2);
        pos2++;
        delay(5);
    }
}

else
```

```
{
  while(pos2>s2)
  {
    myservo2.write(pos2);
    pos2--;
    delay(5);
  }
}

//*****
// the explanation is the same as servo 1
if(pos3<s3)
{
  while(pos3<s3)
  {
    myservo3.write(pos3);
    pos3++;
    delay(5);
  }
}
else
{
  while(pos3>s3)
  {
    myservo3.write(pos3);
    pos3--;
    delay(5);
  }
}
```

```
    }  
  }  
  //*****  
  // the explanation is the same as servo 1  
  if(pos4<s4)  
  {  
    while(pos4<s4)  
    {  
      myservo4.write(pos4);  
      pos4++;  
      delay(5);  
    }  
  }  
  else  
  {  
    while(pos4>s4)  
    {  
      myservo4.write(pos4);  
      pos4--;  
      delay(5);  
    }  
  }  
}  
}
```



```
//claw
zhuazi();
//turn
zhuandong();
// upper arm
xiaobi();
// lower arm
dabi();
}
```

```
//claw
void zhuazi()
{
    //claw
    if(x2<50) // if push the left joystick to the right
    {
        pos4=pos4-2; // current angle of servo 4 subtracts 2 (change the value you subtract, thus change the closed speed of claw)
        //Serial.println(pos4);
        myservo4.write(pos4); //servo 4 operates the action, claw is gradually closed
        delay(5);
        if(pos4<2) // if pos4 value subtracts to 2, the claw in 37 degrees we have tested is closed.)
        {
            // (should change the value based on the fact)
            pos4=2; // stop subtraction when reduce to 2
        }
    }
}
```

```
if(x2>1000) //// if push the left joystick to the left
{
    pos4=pos4+8; // current angle of servo 4 plus 8 (change the value you plus, thus change the open speed of claw)
    //Serial.println(pos4);
    myservo4.write(pos4); // servo 4 operates the motion, the claw gradually opens.
    delay(5);
    if(pos4>90) // limit the largest angle when open
    {
        pos4=90;
    }
}
}
//*****
// turn
void zhuandong()
{
    if(x1<50) // if push the right joystick to the right
    {
        pos1=pos1-1; //pos1 subtracts 1
        myservo1.write(pos1); // servo 1 operates the motion, the arm turns right.
        delay(5);
        if(pos1<1) // limit the angle when turn right
        {
            pos1=1;
        }
    }
}
```

```
if(x1>1000) // if push the right joystick to the left
{
    pos1=pos1+1; //pos1 plus 1
    myservo1.write(pos1); // robot arm turns left
    delay(5);
    if(pos1>180) // limit the angle when turn left
    {
        pos1=180;
    }
}
}

//*****/
// upper arm
void xiaobi()
{
    if(y1>1000) // if push the right joystick upward
    {
        pos2=pos2-1;
        myservo2.write(pos2); // the upper arm will lift
        delay(5);
        if(pos2<0) // limit the lifting angle
        {
            pos2=0;
        }
    }
}
```

```
if(y1<50) // if push the right joystick downward
{
    pos2=pos2+1;
    myservo2.write(pos2); // the upper arm will go down
    delay(5);
    if(pos2>180) // limit the angle when go down
    {
        pos2=180;
    }
}
}

//*****/
// lower arm
void dabi()
{
    if(y2>1000) // if push the left joystick upward
    {
        pos3=pos3-1;
        myservo3.write(pos3); // the lower arm will stretch out
        delay(5);
        if(pos3<35) // limit the stretched angle
        {
            pos3=35;
        }
    }
}
```

```
if(y2<50) // if push the left joystick downward
{
    pos3=pos3+1;
    myservo3.write(pos3); // the lower arm will draw back
    delay(5);
    if(pos3>180) // limit the retracted angle
    {
        pos3=180;
    }
}
```

Test Result:

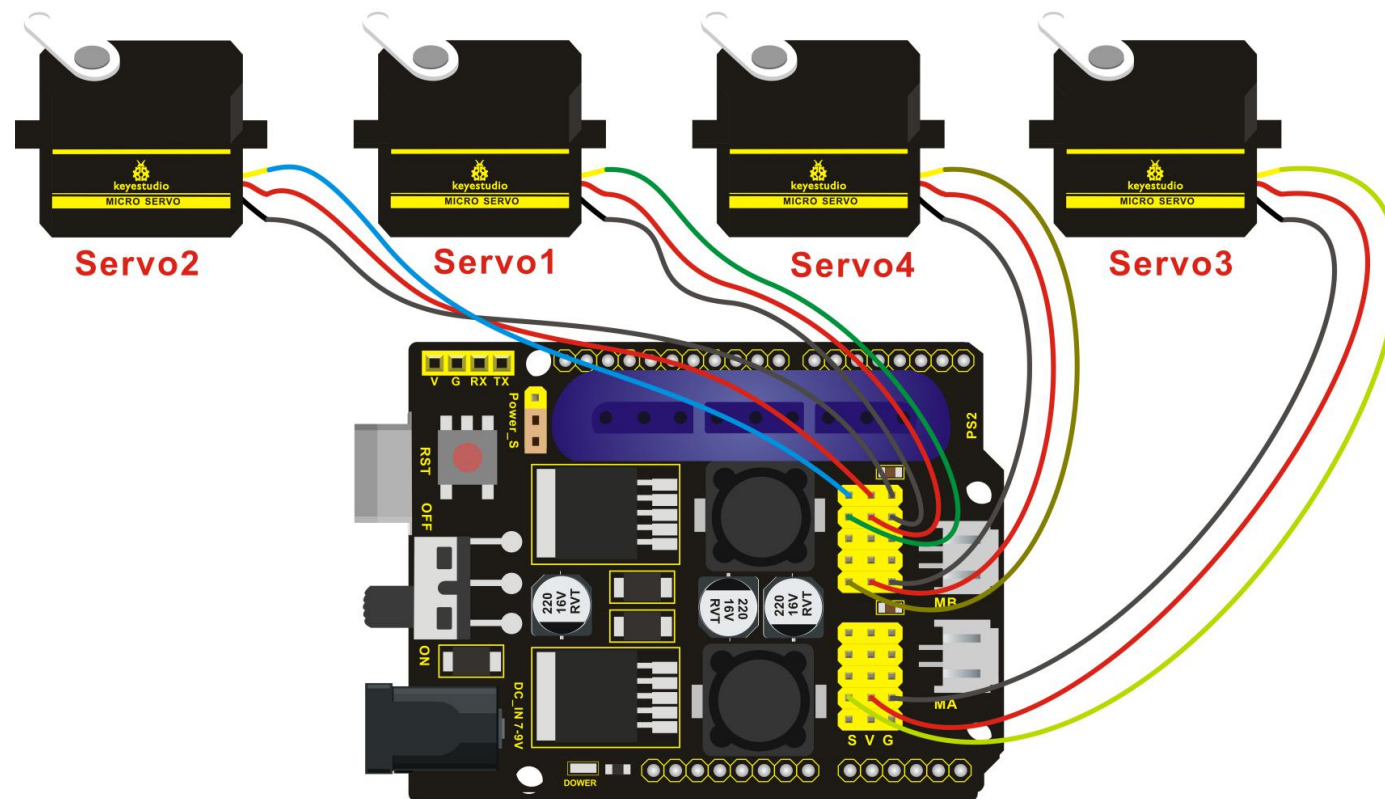
Wire it up, stack the shield onto UNO R3, upload the code. Powered on, press the key Z1 of right Joystick to save the angle value of 4 servos control. Press the key Z2 of left Joystick to operate a servo posture saved in the variable.

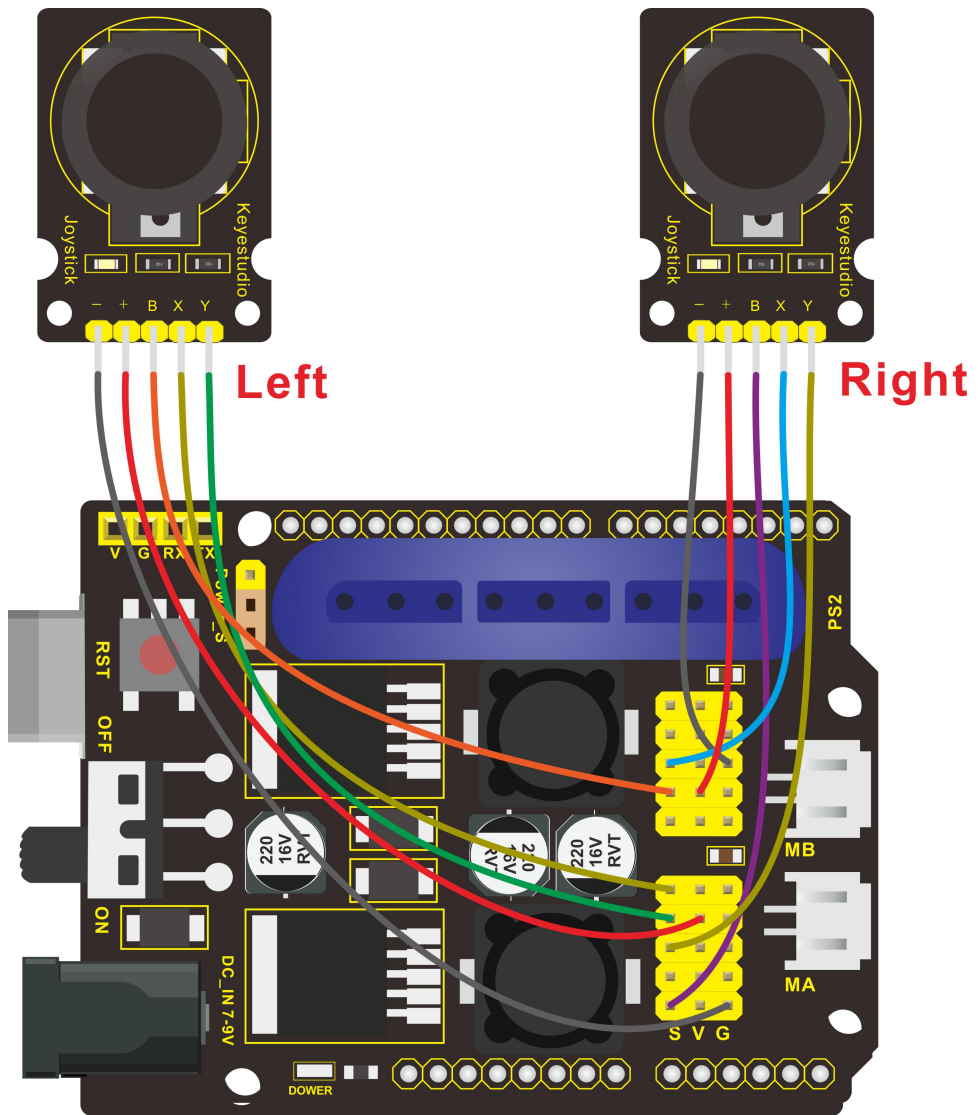
❖ Memorize Several Postures

Description:

In the previous section, we have set the angle of 4 servos to make the robot arm remember and operate a posture. To extend the experiment, next make it remember several postures, at most 10 (you can set it in the code), then make 4DOF robot arm continually operate the posture in memory. That is, make robot arm memorize a group of actions, and you can set the memorizing speed in the code.

Hookup Guide:





Test Code 6:

```

*****
#include <Servo.h> // add the servo libraries
Servo myservo1; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;
int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture
angle value)

const int right_X = A2; // define the right X pin to A2
const int right_Y = A5; // define the right Y pin to A5
const int right_key = 7; // define the right key pin to 7 (that is Z value)

const int left_X = A3; // define the left X pin to A3
const int left_Y = A4; // define the left Y pin to A4
const int left_key = 8; // define the left key pin to 8 (that is Z value)

int x1,y1,z1; //define the variable, used to save the joystick value.
int x2,y2,z2;
int s1,s2,s3,s4;
int jiyi1[10]; // define 4 array, separately used to save the angle of four servo.
int jiyi2[10]; // (array length is 10, namely can save angle data of 0~10 servo )

```

```
int jiyi3[10]; // if need to save more data, just change the number 10 to be more larger number.
int jiyi4[10];
int i=0; // for loop
int j=0; // save the last value of i

void setup()
{
    // boot posture
    myservol.write(pos1);
    delay(1000);
    myservo2.write(pos2);
    myservo3.write(pos3);
    myservo4.write(pos4);
    delay(1500);

    pinMode(right_key, INPUT); // set the right/left key to INPUT
    pinMode(left_key, INPUT);
    Serial.begin(9600); // set baud rate to 9600
}

void loop()
{
    myservol.attach(A1); // set the control pin of servo 1 to A1
    myservo2.attach(A0); // set the control pin of servo 2 to A0
    myservo3.attach(6); // set the control pin of servo 3 to D6
```

```
myservo4.attach(9);    // set the control pin of servo 4 to D9

x1 = analogRead(right_X); // read the right X value
y1 = analogRead(right_Y); // read the right Y value
z1 = digitalRead(right_key); // read the right Z value

x2 = analogRead(left_X); // read the left X value
y2 = analogRead(left_Y); // read the left Y value
z2 = digitalRead(left_key); // read the left Z value
//delay(5); // reduce the speed overall
if(z1==1)    // if the right joystick key is pressed
{
    delay(10);    // delay for eliminating shake
    if(z1==1)    // judge again if the right key is pressed
    {
        s1=myservo1.read();    // read the angle value of each servo
        delay(100);
        Serial.println(s1);
        s2=myservo2.read();
        delay(100);
        Serial.println(s2);
        s3=myservo3.read();
        delay(100);
        Serial.println(s3);
        s4=myservo4.read();
        delay(100);
```

```
Serial.println(s4);

jiyi1[i]=s1;    // Save the read servo value to the array sequentially
jiyi2[i]=s2;
jiyi3[i]=s3;
jiyi4[i]=s4;
i++;           //i value plus 1
j=i;           // assign the last value of i to j
delay(100);
Serial.println(i); // on the serial monitor, print out the value i
}
}

if(z2==1) // if the left joystick key is pressed
{
    delay(10);
    if(z2==1) // judge again if the left key is pressed
    {
        i=0; // assign i to 0, prepare for the next memory
        pos1 = myservo1.read(); // memorize the angle value of 4 servo posture
        pos2 = myservo2.read();
        pos3 = myservo3.read();
        pos4 = myservo4.read();

        for(int k=0;k<j;k++) // loop for j times, perform all actions saved.
        {
```

```
if(pos1<jiyi1[k])    // if the current servo 1 angle is less than the value stored in array 1.
{
  while(pos1<jiyi1[k])  //while loop, make servo turn to the position of value stored in the array.
  {
    myservol.write(pos1); // servo 1 performs the action
    delay(5);    // delay 5ms, controlling the servo rotating speed
    pos1++; //pos1 plus 1
    //Serial.println(pos1);
  }
}
else    // if the current servo 1 angle is greater than the value stored in array 1.
{
  while(pos1>jiyi1[k])    //while loop, make servo turn to the position of value stored in the array.
  {
    myservol.write(pos1); // servo 1 performs the action
    delay(5);    //delay 5ms, controlling the servo rotating speed
    pos1--;    //pos1 subtracts 1
    //Serial.println(pos1);
  }
}

//*****
//the explanation is the same as the previous servo
if(pos2<jiyi2[k])
{
  while(pos2<jiyi2[k])
  {
```

```
        myservo2.write(pos2);
        delay(5);
        pos2++;
        //Serial.println(pos1);
    }
}
else
{
    while(pos2>jiyi2[k])
    {
        myservo2.write(pos2);
        delay(5);
        pos2--;
        //Serial.println(pos1);
    }
}

//*****
// the explanation is the same as the previous servo

if(pos3<jiyi3[k])
{
    while(pos3<jiyi3[k])
    {
        myservo3.write(pos3);
        delay(5);
        pos3++;
    }
}
```

```
        //Serial.println(pos1);
    }
}
else
{
    while(pos3>jiyi3[k])
    {
        myservo3.write(pos3);
        delay(5);
        pos3--;
        //Serial.println(pos1);
    }
}
//*****
//the explanation is the same as the previous servo

if(pos4<jiyi4[k])
{
    while(pos4<jiyi4[k])
    {
        myservo4.write(pos4);
        delay(5);
        pos4++;
        //Serial.println(pos1);
    }
}
```

```
    else
    {
        while(pos4>jiyi4[k])
        {
            myservo4.write(pos4);
            delay(5);
            pos4--;
            //Serial.println(pos1);
        }
    }
}

//claw
zhuazi();
//turn
zhuandong();
//upper arm
xiaobi();
// lower arm
dabi();
}

//claw
```

```
void zhuazi()
{
    //claw
    if(x2<50) // if push the left joystick to the right
    {
        pos4=pos4-2; // angle of servo 4, subtract 2 (change the value you subtract, thus change the closed speed of claw)
        //Serial.println(pos4);
        myservo4.write(pos4); // servo 4 operates the motion and claw is gradually closed.
        delay(5);
        if(pos4<2) // if pos4 value subtracts to 2, the claw in 37 degrees we have tested is closed.)
        {
            // (should change the value based on the fact)
            pos4=2; // stop subtraction when reduce to 2
        }
    }
    if(x2>1000) //// if push the left joystick to the left
    {
        pos4=pos4+8; // current angle of servo 4 plus 8 (change the value you plus, thus change the open speed of claw)
        //Serial.println(pos4);
        myservo4.write(pos4); // servo 4 operates the action, claw gradually opens.
        delay(5);
        if(pos4>90) // limit the largest angle opened
        {
            pos4=90;
        }
    }
}
```

```
//*****
// turn
void zhuandong()
{
    if(x1<50)  // if push the right joystick to the right
    {
        pos1=pos1-1;  //pos1 subtracts 1
        myservol.write(pos1);  // servo 1 operates the motion and robot arm turns right
        delay(5);
        if(pos1<1)  // limit the angle when turn right
        {
            pos1=1;
        }
    }
    if(x1>1000)  // if push the right joystick to the left
    {
        pos1=pos1+1;  //pos1 plus 1
        myservol.write(pos1);  // robot arm turns left
        delay(5);
        if(pos1>180)  // limit the angle when turn left
        {
            pos1=180;
        }
    }
}
```

```

//*****
// upper arm
void xiaobi()
{
    if(y1>1000) // if push the right joystick upward
    {
        pos2=pos2-1;
        myservo2.write(pos2); // the upper arm will lift
        delay(5);
        if(pos2<0) // limit the lifting angle
        {
            pos2=0;
        }
    }
    if(y1<50) // if push the right joystick downward
    {
        pos2=pos2+1;
        myservo2.write(pos2); // the upper arm will go down
        delay(5);
        if(pos2>180) // limit the declining angle
        {
            pos2=180;
        }
    }
}

```

```
/******  
// lower arm  
void dabi()  
{  
  if(y2>1000) // if push the left joystick upward  
  {  
    pos3=pos3-1;  
    myservo3.write(pos3); // the lower arm will stretch out  
    delay(5);  
    if(pos3<35) // limit the stretched angle  
    {  
      pos3=35;  
    }  
  }  
  
  if(y2<50) // if push the left joystick downward  
  {  
    pos3=pos3+1;  
    myservo3.write(pos3); // the lower arm will draw back  
    delay(5);  
    if(pos3>180) // limit the retracted angle  
    {  
      pos3=180;  
    }  
  }  
}
```

Test Result:

Wire it up, stack the shield onto UNO R3, upload the code. Powered on, press the key Z1 of right Joystick to save the angle value of 4 servos. Press down the key Z1 to memorize different postures, at most 10 postures in the code. If need to memorize more postures, you can set it in the code. When memorizing successfully, press down the key Z2 of left Joystick to make the robot arm carry out several postures stored successively.

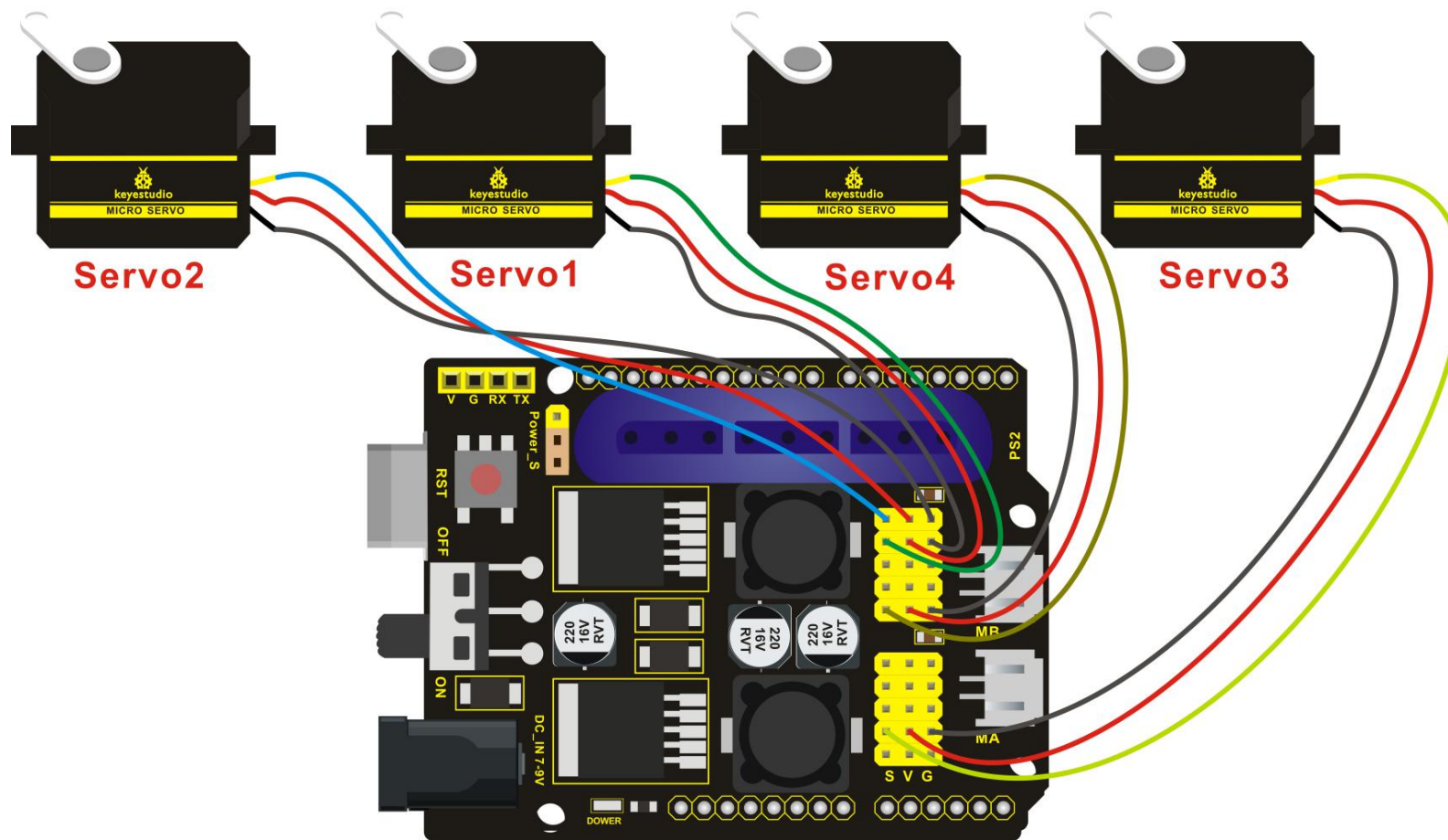
❖ Memorize Several Postures And Loop

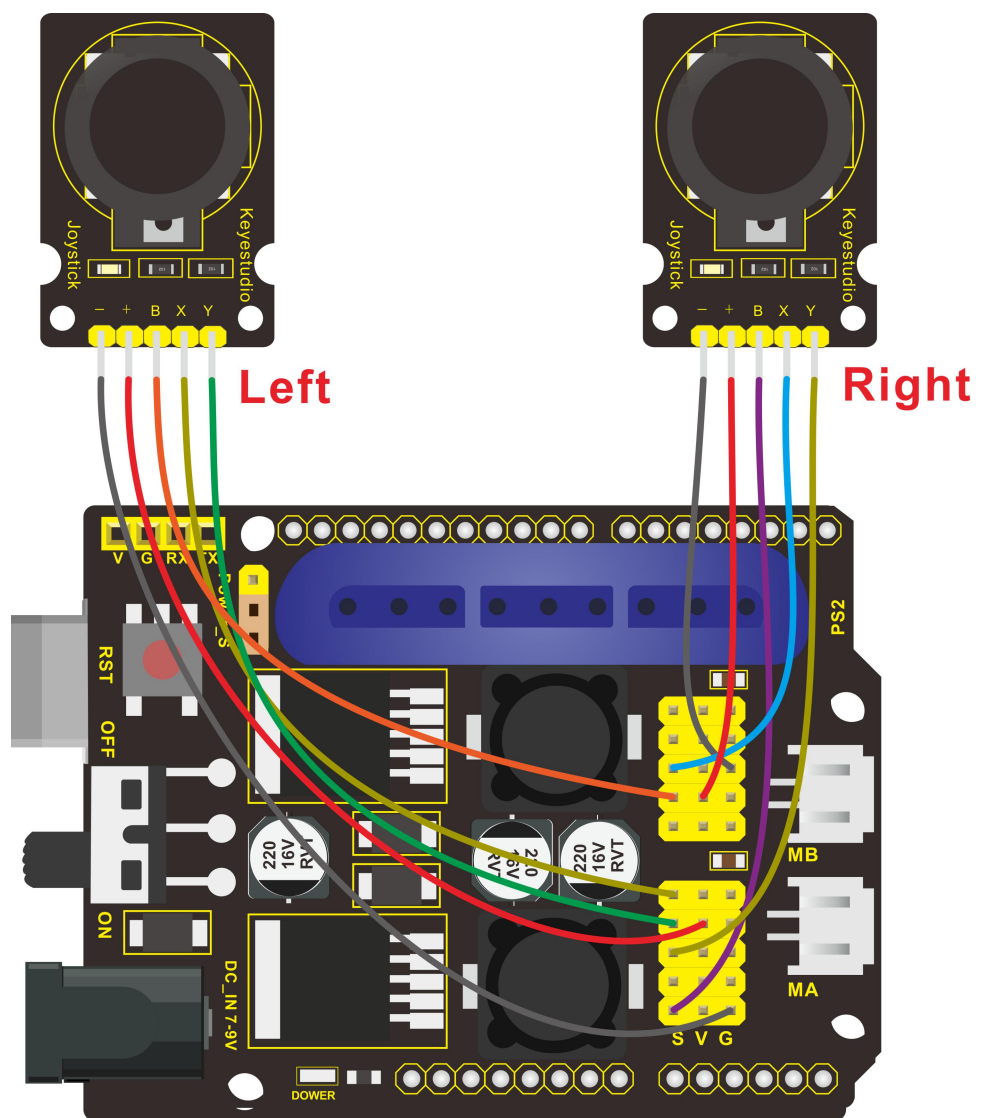
Description:

In the previous section, we have introduced how to make 4DOF robot arm to memorize and perform a group of posture. Furthermore, let's extend one more loop function. That is, when robot arm performs all the memorized actions, it will not stop, and continue to repeat those actions.

In the following experiment, press the key Z1, 4DOF robot arm will exit the looping action. Press the key Z1 again, start to memorize the posture, after that, press the key Z2 to loop the memorized actions.

Hookup Guide:





Test Code 7:

```
#include <Servo.h> // add the servo libraries
Servo myservo1; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;
int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture
angle value)
const int right_X = A2; // define the right X pin to A2
const int right_Y = A5; // define the right Y pin to A5
const int right_key = 7; // define the right key pin to 7 (that is Z value)

const int left_X = A3; //define the left X pin to A3
const int left_Y = A4; // define the left Y pin to A4
const int left_key = 8; // define the left key pin to 8 (that is Z value)

int x1,y1,z1; // define the variable, used to save the joystick value.
int x2,y2,z2;
int s1,s2,s3,s4;
int jiyi1[20]; //define 4 array, separately used to save the angle of four servo.
int jiyi2[20]; // (array length is 20, namely can save angle data of 0~20 servo)
int jiyi3[20]; //if need to save more data, just change the number 20 to be more larger number.
```

```
int jiyi4[20];
int i=0; // for loop
int j=0; // save the last value of i

void setup()
{
    // boot posture
    myservo1.write(pos1); //turn servo 1 to 90 degrees
    delay(1000);
    myservo2.write(pos2); // turn servo 2 to 90 degrees
    myservo3.write(pos3); // turn servo 3 to 120 degrees
    myservo4.write(pos4); // turn servo 4 to 35 degrees
    delay(1500);

    pinMode(right_key, INPUT); // set the right/left key to INOUT
    pinMode(left_key, INPUT);
    Serial.begin(9600); // set the baud rate to 9600
}

void loop()
{
    myservo1.attach(A1); // set the control pin of servo 1 to A1
    myservo2.attach(A0); // set the control pin of servo 2 to A0
    myservo3.attach(6); //set the control pin of servo 3 to D6
    myservo4.attach(9); // set the control pin of servo 4 to D9
```

```
x1 = analogRead(right_X); // read the right X value
y1 = analogRead(right_Y); //read the right Y value
z1 = digitalRead(right_key); //read the right Z value

x2 = analogRead(left_X); // read the left X value
y2 = analogRead(left_Y); // read the left Y value
z2 = digitalRead(left_key); // read the left Z value
//delay(5); // delay, used to reduce the joystick value read, that is reduce the whole speed.
if(z1==1) // if the joystick right key is pressed
{
    delay(10); // delay for eliminating shake
    if(z1==1) // judge again if the right key is pressed
    {
        s1=myservo1.read(); // read the angle value of servo 1 and assign it to s1
        delay(100);
        Serial.println(s1); // print out the angle value of servo 1 on the serial monitor
        s2=myservo2.read(); // read the angle value of servo 2 and assign it to s2
        delay(100);
        Serial.println(s2);
        s3=myservo3.read(); // read the angle value of servo 3 and assign it to s3
        delay(100);
        Serial.println(s3);
        s4=myservo4.read(); // read the angle value of servo 4 and assign it to s4
        delay(100);
        Serial.println(s4);
```

```
    jiyi1[i]=s1; // Save the read servo value to the array sequentially
    jiyi2[i]=s2;
    jiyi3[i]=s3;
    jiyi4[i]=s4;
    i++; //i plus 1
    j=i; // assign the last value of i to j
    delay(100); // delay 100ms
    Serial.println(i); // print out the value i
}
}
if(z2==1) // if the left joystick key is pressed
{
    delay(10); // delay for eliminating shake
    if(z2==1) //judge again if the left key is pressed
    {
        pos1 = myservo1.read(); // memorize the angle value of 4 servo posture
        pos2 = myservo2.read();
        pos3 = myservo3.read();
        pos4 = myservo4.read();

        while(z2==1) // loop, make the arm repeat the action.
        {
            for(int k=1;k<j;k++) //for loop, perform all the stored actions.
            {
                if(pos1<jiyi1[k]) // if the current servo 1 angle is less than the value stored in array 1.
```

```
{
  while(pos1<jiyi1[k]) //while loop, make servo turn to the position of value stored in the array.
  {
    myservo1.write(pos1); //servo 1 performs the action
    delay(5); //delay 5ms, controlling the servo rotating speed.
    pos1++; //pos1 plus 1
    //Serial.println(pos1);
  }
}
else //if the current servo 1 angle is greater than the value stored in array 1.
{
  while(pos1>jiyi1[k]) //while loop, make servo turn to the position of value stored in the array.
  {
    myservo1.write(pos1); //servo 1 performs the action
    delay(5); //delay 5ms, controlling the servo rotating speed.
    pos1--; //pos1 subtracts 1
    //Serial.println(pos1);
  }
}

//*****
//the explanation is the same as the previous servo.
if(pos2<jiyi2[k])
{
  while(pos2<jiyi2[k])
  {
    myservo2.write(pos2);
```

```
        delay(5);
        pos2++;
        //Serial.println(pos1);
    }
}
else
{
    while(pos2>jiyi2[k])
    {
        myservo2.write(pos2);
        delay(5);
        pos2--;
        //Serial.println(pos1);
    }
}
//*****
//the explanation is the same as the previous servo.
    if(pos3<jiyi3[k])
    {
        while(pos3<jiyi3[k])
        {
            myservo3.write(pos3);
            delay(5);
            pos3++;
            //Serial.println(pos1);
        }
    }
```

```
    }
    else
    {
        while(pos3>jiyi3[k])
        {
            myservo3.write(pos3);
            delay(5);
            pos3--;
            //Serial.println(pos1);
        }
    }

//*****
//the explanation is the same as the previous servo.
    if(pos4<jiyi4[k])
    {
        while(pos4<jiyi4[k])
        {
            myservo4.write(pos4);
            delay(5);
            pos4++;
            //Serial.println(pos1);
        }
    }
    else
    {
        while(pos4>jiyi4[k])
```

```

        {
            myservo4.write(pos4);
            delay(5);
            pos4--;
            //Serial.println(pos1);
        }
    }
}

//*****
// for exiting the loop
    z1 = digitalRead(right_key); // read the right Z value
    if(z1==1) // if the right key is pressed
    {
        delay(10); //eliminate the shake
        if(z1==1) // if the key z1 is pressed
        {
            pos1=jiyi1[(j-1)]; // assign the last angle value saved in array to pos
            pos2=jiyi2[(j-1)]; // for exiting the loop, still access to joystick control.
            pos3=jiyi3[(j-1)];
            pos4=jiyi4[(j-1)];
            i=0; // assign i as 0, prepare for saving the angle value using array
            z2=0; // assign z2 as 0, for exiting the while loop
            break; //exit the current loop
        }
    }
}

//*****

```

```
    }
  }
}

//claw
zhuazi();
//turn
zhuandong();
//upper arm
xiaobi();
//lower arm
dabi();
}

//claw
void zhuazi()
{
  //claw
  if(x2<50) // if push the left joystick to the right
  {
    pos4=pos4-2; // angle of servo 4, subtract 2 (change the value you subtract, thus change the closed speed of claw)
    //Serial.println(pos4);
    myservo4.write(pos4); // servo 4 operates the motion and claw is gradually closed.
    delay(5);
    if(pos4<2) // if pos4 value subtracts to 2, the claw in 37 degrees we have tested is closed.)
```

```
    {          // (should change the value based on the fact)
      pos4=2;  //stop subtraction when reduce to 2
    }
  }
if(x2>1000) ////if push the left joystick to the left
{
  pos4=pos4+8; //current angle of servo 4 plus 8 (change the value you plus, thus change the open speed of claw)
  //Serial.println(pos4);
  myservo4.write(pos4); // servo 4 operates the action, claw gradually opens.
  delay(5);
  if(pos4>90) //limit the largest angle opened
  {
    pos4=90;
  }
}
}
//*****
//turn
void zhuandong()
{
  if(x1<50) //if push the right joystick to the right
  {
    pos1=pos1-1; //pos1 subtracts 1
    myservo1.write(pos1); // servo 1 performs the action, the robot arm turns right.
    delay(5);
    if(pos1<1) // limit the right turning angle
```

```
{
    pos1=1;
}
}
if(x1>1000) // if push the right joystick to the left
{
    pos1=pos1+1; //pos1 plus 1
    myservo1.write(pos1); //the robot arm turns left.
    delay(5);
    if(pos1>180) //limit the left turning angle
    {
        pos1=180;
    }
}
}

//*****/
// upper arm
void xiaobi()
{
    if(y1>1000) // if push the right joystick upward
    {
        pos2=pos2-1;
        myservo2.write(pos2); // the robot arm will lift
        delay(5);
        if(pos2<0) // limit the lifting angle
```

```
{
    pos2=0;
}
}
if(y1<50) // if push the right joystick downward
{
    pos2=pos2+1;
    myservo2.write(pos2); // the robot arm will go down
    delay(5);
    if(pos2>180) // limit the declining angle
    {
        pos2=180;
    }
}
}

//*****/
// lower arm
void dabi()
{
    if(y2>1000) // if push the left joystick upward
    {
        pos3=pos3-1;
        myservo3.write(pos3); // the lower arm will stretch out
        delay(5);
        if(pos3<35) // limit the stretched angle
```

```
{
  pos3=35;
}

if(y2<50) // if push the right joystick downward
{
  pos3=pos3+1;
  myservo3.write(pos3); // the lower arm will draw back
  delay(5);
  if(pos3>180) // limit the retraction angle
  {
    pos3=180;
  }
}
}
```

Test Result:

Wire it up, stack the shield onto UNO R3, upload the code. Powered on, press the key Z1 of right Joystick to save the angle value of 4 servos. Press down the key Z1 to memorize different postures, at most 10 postures in the code. If need to memorize more postures, can set it in the code.

When memorizing successfully, press down the key Z2 of left Joystick to make the robot arm carry out several postures stored successively, looping.

Long press the key Z1, 4DOF robot arm will exit the looping action. Press the key Z1 again, start to memorize the posture, after that, press the key Z2 to loop the memorized actions.

Project 4: Bluetooth Controlled Robot Arm

4.1 Principle of Bluetooth Control

Description:

Bluetooth, as the name implies, blue teeth, and he is not used to bite people, but a wireless data transmission method. Bluetooth technology is a wireless standard technology that enables short-range data exchange among fixed devices, mobile devices, and personal area networks of buildings (UHF radio waves in the ISM band of 2.4 to 2.485 GHz).

There are two kinds of commonly used Bluetooth module on the market, HC-05 and HC-06 models.

The difference between them is that the HC-05 is a master-slave one. It can not only make small reports to its own “master”, but also can receive the command given to it. The HC-06 can only work in slave mode, which can only accept the superior command. For instance, in many cases you may want to be an overbearing man, letting subordinates obey the order without any nonsense. In such situation, it is enough to use



the HC-06 module.

Note: HC-06 Bluetooth module is only compatible with Android system, not support Mac.

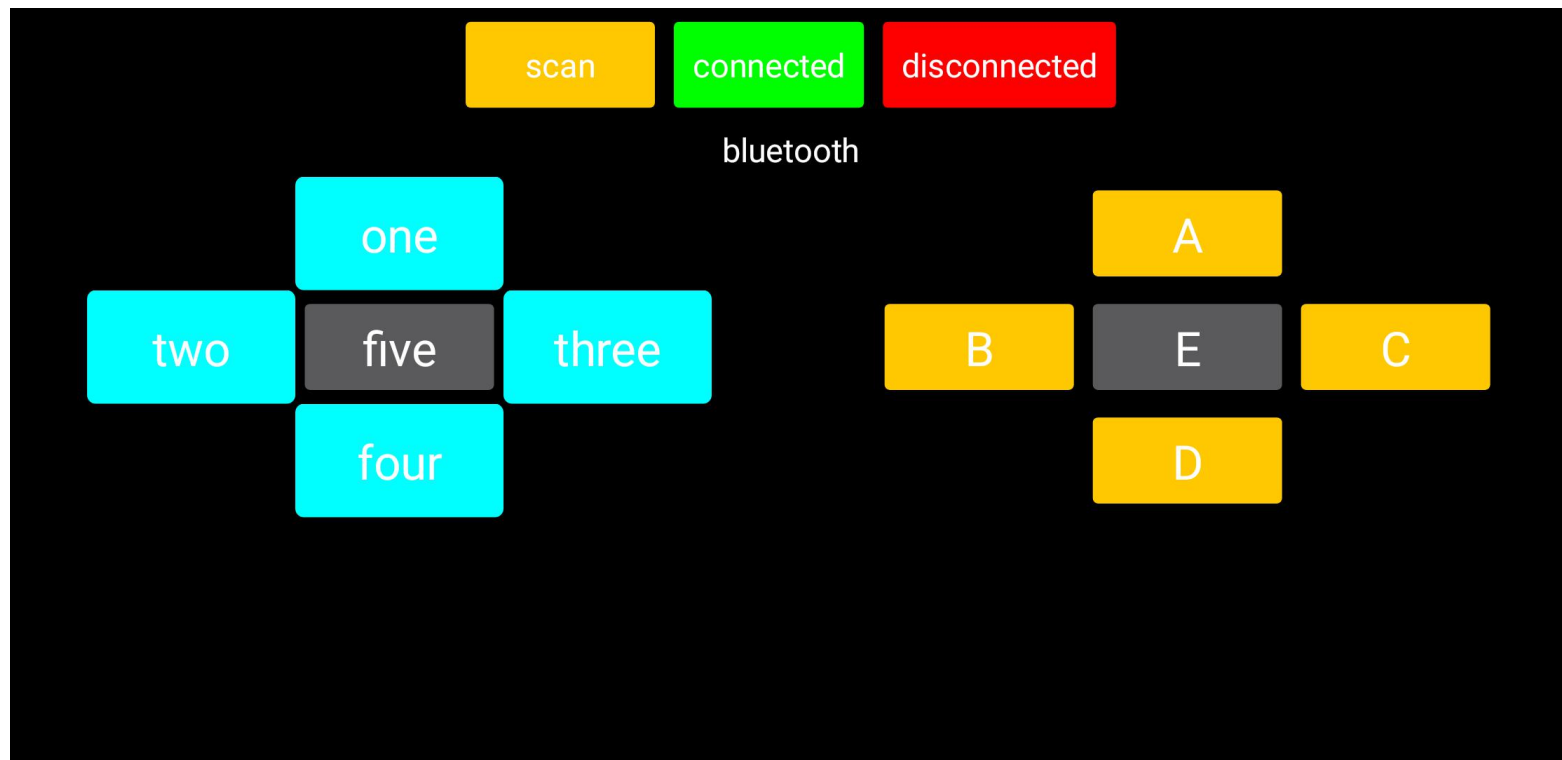
Specification Parameters:

- Bluetooth Protocol: Bluetooth 2.1+ EDR Standard
- USB Protocol: USB v1.1/2.0
- Operating Frequency: 2.4GHz ISM Frequency Band
- Modulation Mode: Gauss Frequency Shift Keying
- Transmit Power: $\leq 4\text{dBm}$, Second Stage
- Sensitivity: $\leq -84\text{dBm}$ at 0.1% Bit Error Rate
- Transmission Speed: 2.1Mbps(Max)/160 kbps(Asynchronous); 1Mbps/1Mbps(Synchronous)
- Safety Feature: Authentication and Encryption
- Supported Configuration: Bluetooth Serial Port (major and minor)
- Supply Voltage: DC 5V
- Operating Temperature: -20°C to 55°C

4.2 Bluetooth Control Key Test

Description:

Next, we are going to introduce the use method for HC-06 Bluetooth module. To easily use the HC-06 Bluetooth module to control the robot arm, we particularly design the APP control. Shown below.



You can download the APP from the link:

<https://drive.google.com/open?id=10U7EWyCyb5bGUxWm3DXRXGKEm1UQGU4Q>

There are 10 control keys on the App. When connect well the HC-06 Bluetooth module to Android phone using our APP, press the control key, Android phone will receive a corresponding value.

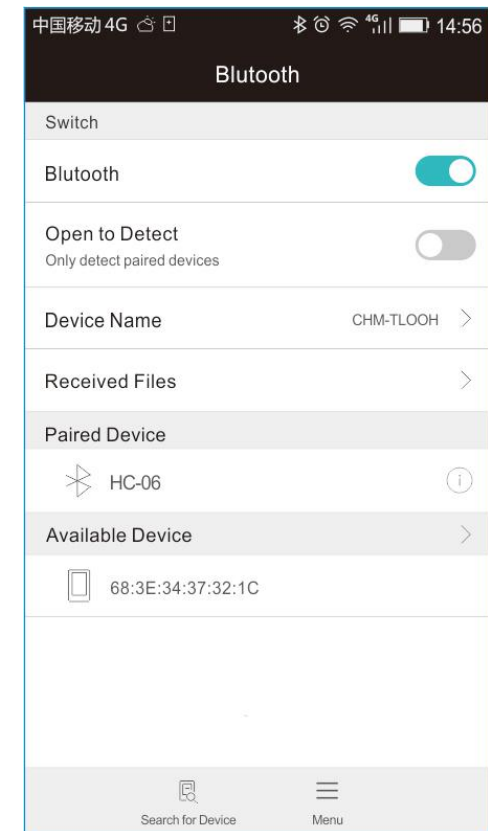
When programming, you can set the function for the corresponding value. So in the experiment, we will test each key to get the corresponding value.

Pay close attention to:

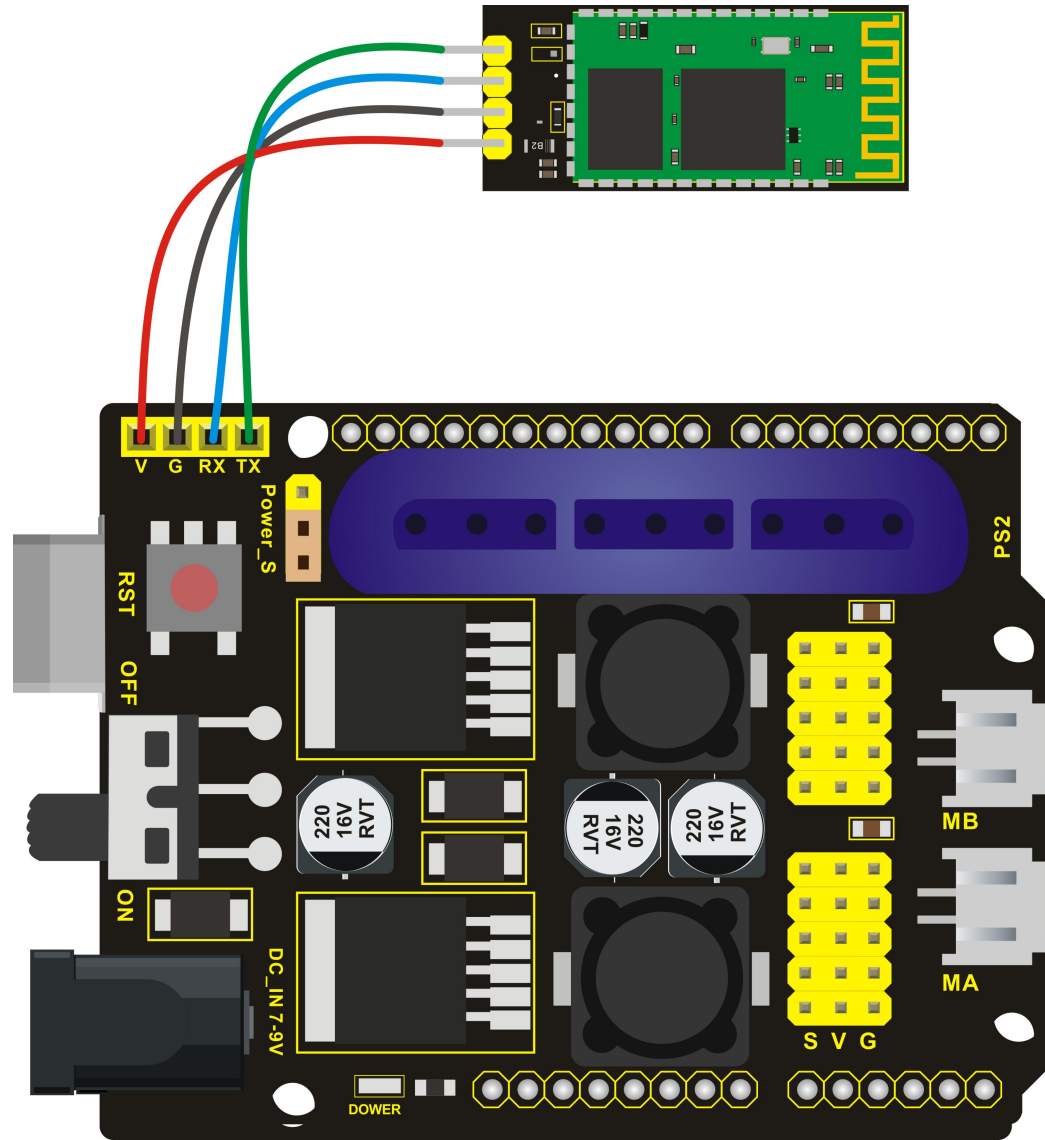
1) First should pair Bluetooth in the phone settings. Connect the Bluetooth to shield, powered up, LED on the Bluetooth module flashes, then open the mobile Bluetooth, search for a Bluetooth device.

If find a new address, pair and enter 1234, paired successfully, you should see the address name change to HC-06.

2) Must first upload the test program successfully, then connect the Bluetooth, pair and connect the App. Otherwise, fail to upload the program.



Hookup Guide:



Test Code 8:

```
*****  
void setup()  
{  
  Serial.begin(9600);    // set the serial baud rate to 9600  
}  
  
void loop()  
{  
  char val;    // define a variable, used to receive the value read from Bluetooth.  
  if(Serial.available())    // if receive the value  
  {  
    val = Serial.read();    // assign the value read to val  
    Serial.println(val);  
  }  
}  
*****
```

Test Result:

Upload the code, hookup well and power up. After connecting the Bluetooth APP, open the serial monitor and set the baud rate to 9600, press down the control key on the APP, you should see the corresponding value. Shown

below.

key	value
one	1
two	2
three	3
four	4
five	5
A	A
B	B
C	C
D	D
E	E

4.3 Bluetooth Controlling Arm

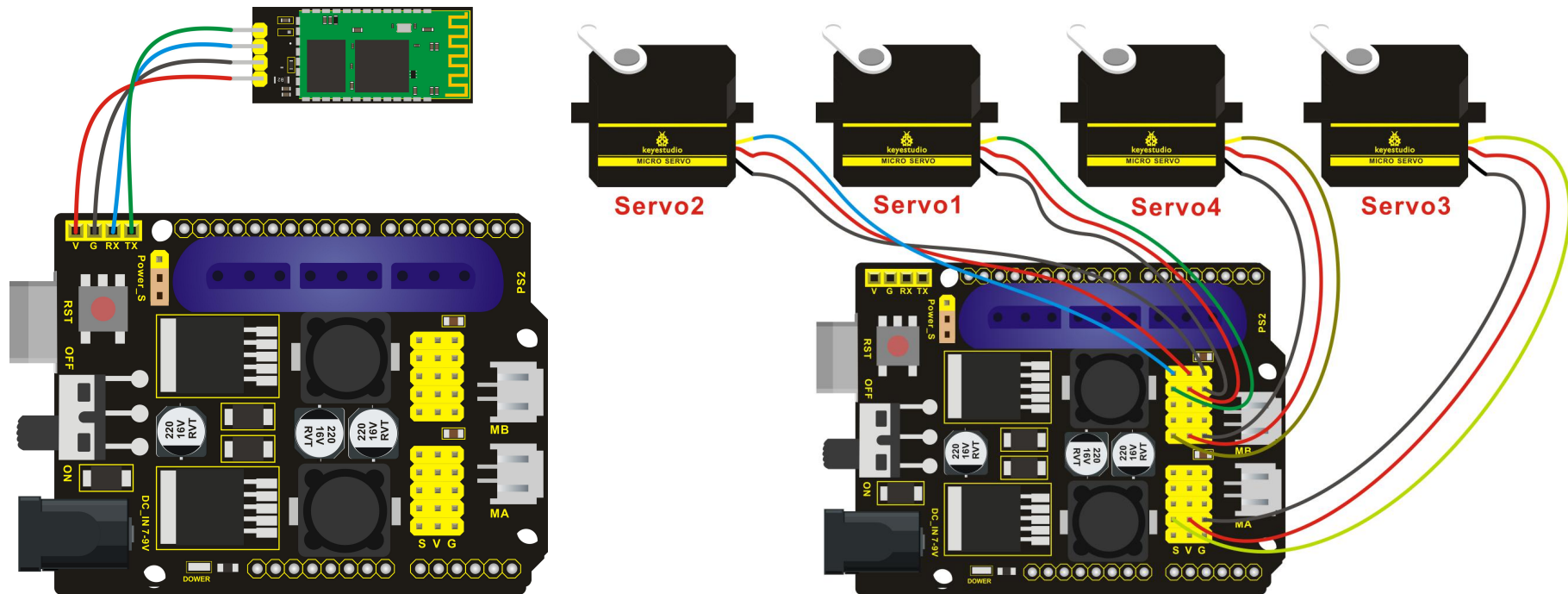
Description:

In this experiment, we combine a Android APP and HC-06 Bluetooth module to control the robot arm. The controlling method are shown below.

Key	Corresponding arm action
one	The lower arm stretches out
two	The clamp claw opens
three	The clamp claw is closed
four	The lower arm draws back
five	-
A	The upper arm lifts up
B	The arm turns left
C	The arm turns right

D	The upper arm goes down
E	-

Hookup Guide:



Test Code 9:

```
#include <Servo.h> // add the servo libraries
Servo myservo1; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;
int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture
angle value)
char val;

void setup()
{
    // boot posture
    myservo1.write(pos1);
    delay(1000);
    myservo2.write(pos2);
    myservo3.write(pos3);
    myservo4.write(pos4);
    delay(1500);

    Serial.begin(9600); // set the baud rate to 9600
}

void loop()
{
```

```
myservo1.attach(A1); // set the control pin of servo 1 to A1
myservo2.attach(A0); // set the control pin of servo 2 to A0
myservo3.attach(6);  // set the control pin of servo 3 to D6
myservo4.attach(9);  // set the control pin of servo 4 to D9

if(Serial.available()) // if receive the data
{
    val=Serial.read();  // read the received data
    Serial.println(val);

    switch(val)
    {
        case 'B': T_left(); break;  // execute the corresponding function when receive the value
        case 'C': T_right(); break;
        case 'A': RF(); break;
        case 'D': RB(); break;
        case '2': ZK(); break;
        case '3': ZB(); break;
        case '1': LF(); break;
        case '4': LB(); break;
    }
}

//*****
// turn left
void T_left()
```

```
{
    pos1=pos1+1;
    myservol.write(pos1);
    delay(5);
    if(pos1>180)
    {
        pos1=180;
    }
}
//turn right
void T_right()
{
    pos1=pos1-1;
    myservol.write(pos1);
    delay(5);
    if(pos1<1)
    {
        pos1=1;
    }
}
//*****
//open the claw
void ZK()
{
    pos4=pos4-2;
    Serial.println(pos4);
}
```

```
    myservo4.write(pos4);
    delay(5);
    if(pos4<2)
    {
        pos4=0;
    }
}
// close the claw
void ZB()
{
    pos4=pos4+8;
    Serial.println(pos4);
    myservo4.write(pos4);
    delay(5);
    if(pos4>108)
    {
        pos4=108;
    }
}

//*****
// the upper arm will lift up
void RF()
{
    pos2=pos2-1;
    myservo2.write(pos2);
```

```
    delay(5);
    if(pos2<0)
    {
        pos2=0;
    }
}
// the upper arm will go down
void RB()
{
    pos2=pos2+1;
    myservo2.write(pos2);
    delay(5);
    if(pos2>180)
    {
        pos2=180;
    }
}

//*****
// the lower arm will stretch out
void LB()
{
    pos3=pos3+1;
    myservo3.write(pos3);
    delay(5);
    if(pos3>180)
```

```
    {  
        pos3=180;  
    }  
}  
// the lower arm will draw back  
void LF()  
{  
    pos3=pos3-1;  
    myservo3.write(pos3);  
    delay(5);  
    if(pos3<35)  
    {  
        pos3=35;  
    }  
}  
*****
```

Test Result:

Upload the code, connect it up and power on, after connecting the Bluetooth APP, press the key to control the robot arm do commanded motions.

Project 5: PS2 Controlled Robot Arm (Extension)

5.1 PS2 Joypad Key Test

Description:

On the drive shield there is a PS2 Joystick connector, which is easy for you to control the 4DOF robot arm using the PS2 Joypad. But you need to purchase it by yourself because the PS2 Joypad is not included in the kit.

When use the PS2 Joypad to control the robot arm, first need to get the corresponding character of each key on the PS2 Joypad.

So this experiment will help you test the character of each key on the PS2 Joypad.

After connecting the Joypad, should upload the test program on Arduino IDE. But before testing, should place the **PS2X_lib** folder inside the libraries folder of Arduino IDE directory.

Uploading the code, open the serial monitor, connect the PS2 Joypad. When press down the key, you should see the corresponding character on the monitor.

Test Code 10:

```
#include <PS2X_lib.h> //for v1.6
```

```
/******
```

```
 * set pins connected to PS2 controller:
```

```
 * - 1e column: original
```

```
 * - 2e colmun: Stef?
```

```
 * replace pin numbers by the ones you use
```

```
*****/
```

```
#define PS2_DAT      13  //14
```

```
#define PS2_CMD      11  //15
```

```
#define PS2_SEL      10  //16
```

```
#define PS2_CLK      12  //17
```

```
/******
```

```
 * select modes of PS2 controller:
```

```
 * - pressures = analog reading of push-butttons
```

```
 * - rumble     = motor rumbling
```

```
 * uncomment 1 of the lines for each mode selection
```

```
*****/
```

```
//#define pressures  true
```

```
#define pressures  false
```

```
//#define rumble      true
#define rumble      false

PS2X ps2x; // create PS2 Controller Class

//right now, the library does NOT support hot pluggable controllers, meaning
//you must always either restart your Arduino after you connect the controller,
//or call config_gamepad(pins) again after connecting the controller.

int error = 0;
byte type = 0;
byte vibrate = 0;

void setup() {

  Serial.begin(57600);

  delay(300); //added delay to give wireless ps2 module some time to startup, before configuring it

  //CHANGES for v1.6 HERE!!! *****PAY ATTENTION*****

  //setup pins and settings: GamePad(clock, command, attention, data, Pressures?, Rumble?) check for error
  error = ps2x.config_gamepad(PS2_CLK, PS2_CMD, PS2_SEL, PS2_DAT, pressures, rumble);

  if(error == 0) {
    Serial.print("Found Controller, configured successful ");
```

```
    Serial.print("pressures = ");
    if (pressures)
        Serial.println("true ");
    else
        Serial.println("false");
    Serial.print("rumble = ");
    if (rumble)
        Serial.println("true");
    else
        Serial.println("false");
        Serial.println("Try out all the buttons, X will vibrate the controller, faster as you press harder;");
        Serial.println("holding L1 or R1 will print out the analog stick values.");
        Serial.println("Note: Go to www.billporter.info for updates and to report bugs.");
    }
    else if(error == 1)
        Serial.println("No controller found, check wiring, see readme.txt to enable debug. visit www.billporter.info for troubleshooting tips");

    else if(error == 2)
        Serial.println("Controller found but not accepting commands. see readme.txt to enable debug. Visit www.billporter.info for troubleshooting tips");

    else if(error == 3)
        Serial.println("Controller refusing to enter Pressures mode, may not support it. ");

//  Serial.print(ps2x.Analog(1), HEX);
```

```
type = ps2x.readType();
switch(type) {
    case 0:
        Serial.print("Unknown Controller type found ");
        break;
    case 1:
        Serial.print("DualShock Controller found ");
        break;
    case 2:
        Serial.print("GuitarHero Controller found ");
        break;
    case 3:
        Serial.print("Wireless Sony DualShock Controller found ");
        break;
}

void loop() {
    /* You must Read Gamepad to get new values and set vibration values
       ps2x.read_gamepad(small motor on/off, larger motor strenght from 0-255)
       if you don't enable the rumble, use ps2x.read_gamepad(); with no values
       You should call this at least once a second
    */
    if(error == 1) //skip loop if no controller found
        return;
```

```
if(type == 2){ //Guitar Hero Controller
    ps2x.read_gamepad();          //read controller

    if(ps2x.ButtonPressed(GREEN_FRET))
        Serial.println("Green Fret Pressed");
    if(ps2x.ButtonPressed(RED_FRET))
        Serial.println("Red Fret Pressed");
    if(ps2x.ButtonPressed(YELLOW_FRET))
        Serial.println("Yellow Fret Pressed");
    if(ps2x.ButtonPressed(BLUE_FRET))
        Serial.println("Blue Fret Pressed");
    if(ps2x.ButtonPressed(ORANGE_FRET))
        Serial.println("Orange Fret Pressed");

    if(ps2x.ButtonPressed(STAR_POWER))
        Serial.println("Star Power Command");

    if(ps2x.Button(UP_STRUM))          //will be TRUE as long as button is pressed
        Serial.println("Up Strum");
    if(ps2x.Button(DOWN_STRUM))
        Serial.println("DOWN Strum");

    if(ps2x.Button(PSB_START))          //will be TRUE as long as button is pressed
        Serial.println("Start is being held");
    if(ps2x.Button(PSB_SELECT))
```

```
    Serial.println("Select is being held");

    if(ps2x.Button(ORANGE_FRET)) {    // print stick value IF TRUE
        Serial.print("Wammy Bar Position:");
        Serial.println(ps2x.Analog(WHAMMY_BAR), DEC);
    }
}
else { //DualShock Controller
    ps2x.read_gamepad(false, vibrate); //read controller and set large motor to spin at 'vibrate' speed

    if(ps2x.Button(PSB_START))        //will be TRUE as long as button is pressed
        Serial.println("Start is being held");
    if(ps2x.Button(PSB_SELECT))
        Serial.println("Select is being held");

    if(ps2x.Button(PSB_PAD_UP)) {    //will be TRUE as long as button is pressed
        Serial.print("Up held this hard: ");
        Serial.println(ps2x.Analog(PSAB_PAD_UP), DEC);
    }
    if(ps2x.Button(PSB_PAD_RIGHT)) {
        Serial.print("Right held this hard: ");
        Serial.println(ps2x.Analog(PSAB_PAD_RIGHT), DEC);
    }
    if(ps2x.Button(PSB_PAD_LEFT)) {
        Serial.print("LEFT held this hard: ");
        Serial.println(ps2x.Analog(PSAB_PAD_LEFT), DEC);
    }
}
```

```
}
if(ps2x.Button(PSB_PAD_DOWN)) {
    Serial.print("DOWN held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_DOWN), DEC);
}

vibrate = ps2x.Analog(PSAB_CROSS); //this will set the large motor vibrate speed based on how hard you press the blue (X) button
if (ps2x.NewButtonState()) {      //will be TRUE if any button changes state (on to off, or off to on)
    if(ps2x.Button(PSB_L3))
        Serial.println("L3 pressed");
    if(ps2x.Button(PSB_R3))
        Serial.println("R3 pressed");
    if(ps2x.Button(PSB_L2))
        Serial.println("L2 pressed");
    if(ps2x.Button(PSB_R2))
        Serial.println("R2 pressed");
    if(ps2x.Button(PSB_TRIANGLE))
        Serial.println("Triangle pressed");
}

if(ps2x.ButtonPressed(PSB_CIRCLE))          //will be TRUE if button was JUST pressed
    Serial.println("Circle just pressed");
if(ps2x.NewButtonState(PSB_CROSS))          //will be TRUE if button was JUST pressed OR released
    Serial.println("X just changed");
if(ps2x.ButtonReleased(PSB_SQUARE))          //will be TRUE if button was JUST released
    Serial.println("Square just released");
```

```
if(ps2x.Button(PSB_L1) || ps2x.Button(PSB_R1)) { //print stick values if either is TRUE
  Serial.print("Stick Values:");
  Serial.print(ps2x.Analog(PSS_LY), DEC); //Left stick, Y axis. Other options: LX, RY, RX
  Serial.print(",");
  Serial.print(ps2x.Analog(PSS_LX), DEC);
  Serial.print(",");
  Serial.print(ps2x.Analog(PSS_RY), DEC);
  Serial.print(",");
  Serial.println(ps2x.Analog(PSS_RX), DEC);
}
}
delay(50);
}
```

Test Result:

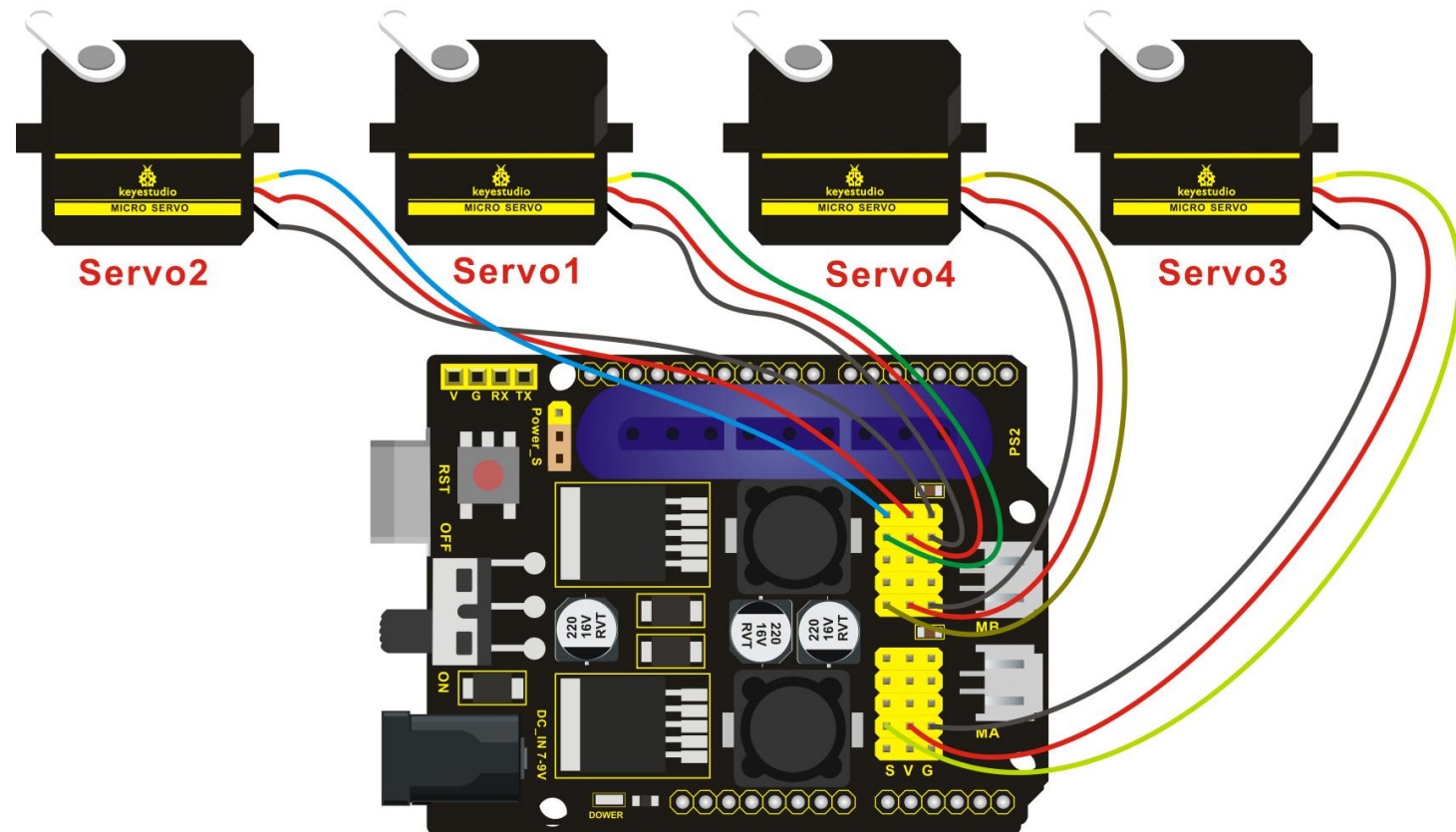
Stack the drive shield onto UNO R3 and upload the code. Connecting the PS2 Joypad, open the serial monitor and set the baud rate to 57600. When press down the key or push the rocker, you should see the corresponding character showed on the monitor.

5.2 PS2 Joypad Control

Description:

In the previous section, we have showed how to use Joystick module to control the robot arm. It is almost the same for you to control the 4DOF robot arm using the PS2 Joypad.

Hookup Guide:



Test Code 11:

```
#include <PS2X_lib.h>
```

```
PS2X ps2x; // create PS2 Controller Class
```

```
//right now, the library does NOT support hot pluggable controllers, meaning  
//you must always either restart your Arduino after you connect the controller,  
//or call config_gamepad(pins) again after connecting the controller.
```

```
int error = 0;
```

```
byte vibrate = 0;
```

```
#include <Servo.h> // add the servo libraries
```

```
Servo myservo1; // create servo object to control a servo
```

```
Servo myservo2;
```

```
Servo myservo3;
```

```
Servo myservo4;
```

```
int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture  
angle value)
```

```
void setup() {
```

```
  Serial.begin(57600);
```

```
// boot posture
myservol.write(pos1);
delay(1000);
myservo2.write(pos2);
myservo3.write(pos3);
myservo4.write(pos4);
delay(1500);

error = ps2x.config_gamepad(13,11,10,12); //setup GamePad(clock, command, attention, data) pins, check for error

if(error == 0){
    Serial.println("Found Controller, configured successful");
    Serial.println("Try out all the buttons, X will vibrate the controller, faster as you press harder;");
    Serial.println("holding L1 or R1 will print out the analog stick values.");
    Serial.println("Go to www.billporter.info for updates and to report bugs.");
}

else if(error == 1)
    Serial.println("No controller found, check wiring, see readme.txt to enable debug. visit www.billporter.info for troubleshooting tips");

else if(error == 2)
    Serial.println("Controller found but not accepting commands. see readme.txt to enable debug. Visit www.billporter.info for troubleshooting tips");
```

```
//Serial.print(ps2x.Analog(1), HEX);

ps2x.enableRumble();           //enable rumble vibration motors
ps2x.enablePressures();        //enable reading the pressure values from the buttons.

}

void loop() {
  /* You must Read Gamepad to get new values
  Read GamePad and set vibration values
  ps2x.read_gamepad(small motor on/off, larger motor strenght from 0-255)
  if you don't enable the rumble, use ps2x.read_gamepad(); with no values

  you should call this at least once a second
  */

  myservol.attach(A1); // set the control pin of servo 1 to A1
  myservo2.attach(A0); // set the control pin of servo 2 to A0
  myservo3.attach(6);  // set the control pin of servo 3 to D6
  myservo4.attach(9);  // set the control pin of servo 4 to D9

  if(error != 0)
    return;
```

```
ps2x.read_gamepad(false, vibrate);          //read controller and set large motor to spin at 'vibrate' speed

if(ps2x.Button(PSB_START))                  //will be TRUE as long as button is pressed
    Serial.println("Start is being held");
if(ps2x.Button(PSB_SELECT))
    Serial.println("Select is being held");

if(ps2x.Button(PSB_PAD_UP)) {               //will be TRUE as long as button is pressed
    Serial.print("Up held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_UP), DEC);
}
if(ps2x.Button(PSB_PAD_RIGHT)) {
    Serial.print("Right held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_RIGHT), DEC);
}
if(ps2x.Button(PSB_PAD_LEFT)) {
    Serial.print("LEFT held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_LEFT), DEC);
}
if(ps2x.Button(PSB_PAD_DOWN)) {
    Serial.print("DOWN held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_DOWN), DEC);
}
```

```
vibrate = ps2x.Analog(PSAB_BLUE);           //this will set the large motor vibrate speed based on
                                              //how hard you press the blue (X) button

if (ps2x.NewButtonState())                  //will be TRUE if any button changes state (on to off, or off to on)
{
    if(ps2x.Button(PSB_R3))

        Serial.println("R3 pressed");

    if(ps2x.Button(PSB_L3))
        Serial.println("L3 pressed");

    if(ps2x.Button(PSB_L2))
        Serial.println("L2 pressed");
    if(ps2x.Button(PSB_R2))
        Serial.println("R2 pressed");
    if(ps2x.Button(PSB_GREEN))
        Serial.println("Triangle pressed");

}

if(ps2x.ButtonPressed(PSB_RED))              //will be TRUE if button was JUST pressed
    Serial.println("Circle just pressed");
```

```
if(ps2x.ButtonReleased(PSB_PINK))           //will be TRUE if button was JUST released
    Serial.println("Square just released");

if(ps2x.NewButtonState(PSB_BLUE))           //will be TRUE if button was JUST pressed OR released
    Serial.println("X just changed");

//转动
zhuandong();
//瓜子
zhuazi();
//大臂
dabi();
//小臂
xiaobi();

if(ps2x.Button(PSB_L1) || ps2x.Button(PSB_R1)) // print stick values if either is TRUE
{
    Serial.print("Stick Values:");
    Serial.print(ps2x.Analog(PSS_LY), DEC); //Left stick, Y axis. Other options: LX, RY, RX
    Serial.print(",");
    Serial.print(ps2x.Analog(PSS_LX), DEC);
    Serial.print(",");
    Serial.print(ps2x.Analog(PSS_RY), DEC);
    Serial.print(",");
    Serial.println(ps2x.Analog(PSS_RX), DEC);
```

```
}
delay(5);
}
//*****
// turn
void zhuandong()
{
//turn right
if(ps2x.Analog (PSS_RX) > 200) // if push the right joystick to the right
{
//Serial.println(ps2x.Analog(PSS_RX), DEC);
pos1=pos1-1; //pos1 subtracts 1
myservol.write(pos1); // servo 1 executes the action, the arm will turn right.
// delay(5);
if(pos1<1) // limit the right turning angle
{
pos1=1;
}
}
// turn left
if(ps2x.Analog (PSS_RX) < 50) // if push the right joystick to the left
{
//Serial.println(ps2x.Analog(PSS_RX), DEC);
pos1=pos1+1; //pos1 plus 1
myservol.write(pos1); // the arm turns left
// delay(5);
```

```
        if(pos1>180)    // limit the left turning angle
        {
            pos1=180;
        }
    }
}

//*****
// upper arm
void xiaobi()
{
    //upper arm front
    if(ps2x.Analog(PSS_RY)<50)  // if push the right joystick upward
    {
        pos2=pos2-1;
        myservo2.write(pos2);    // the upper arm will lift
        delay(5);
        if(pos2<0)    // limit the lifting angle
        {
            pos2=0;
        }
    }
    //upper arm back
    if(ps2x.Analog(PSS_RY)>200)    // if push the right joystick downward
    {
        pos2=pos2+1;
        myservo2.write(pos2);    // the upper arm will go down
    }
}
```

```
    delay(5);
    if(pos2>180) // limit the declining angle
    {
        pos2=180;
    }
}

//*****
void zhuazi()
{
    // close the claw
    if(ps2x.Analog(PSS_LX)>220) // if push the left joystick to the right
    {
        pos4=pos4-1;
        Serial.println(pos4);
        myservo4.write(pos4); // servo 4 carries out the action and the claw is gradually closed.
        delay(5);
        if(pos4<0) // if pos4 value subtracts to 37, the claw in 37 degrees we have tested is closed.)
        {
            pos4=0;
        }
    }
    // open the claw
    if(ps2x.Analog(PSS_LX)<10) // if push the left joystick to the left
    {
        pos4=pos4+8;
```

```
    Serial.println(pos4);
    myservo4.write(pos4);  // servo 4 carries out the action and the claw is gradually opened
    delay(5);
    if(pos4>108)    // limit the maximum opening angle
    {
        pos4=108;
    }
}

//*****
void dabi()
{
    // lower arm front
    if(ps2x.Analog(PSS_LY)>200)  // if push the left joystick upward
    {
        pos3=pos3+1;
        myservo3.write(pos3);  // the lower arm will stretch out
        delay(5);
        if(pos3>180)    // limit the stretched angle
        {
            pos3=180;
        }
    }
    if(ps2x.Analog(PSS_LY)<10)    //if push the left joystick downward
    {
        pos3=pos3-1;
```



```
myservo3.write(pos3); // the lower arm will draw back
delay(5);
if(pos3<35) // limit the retracted angle
{
    pos3=35;
}
}
```

Test Result:

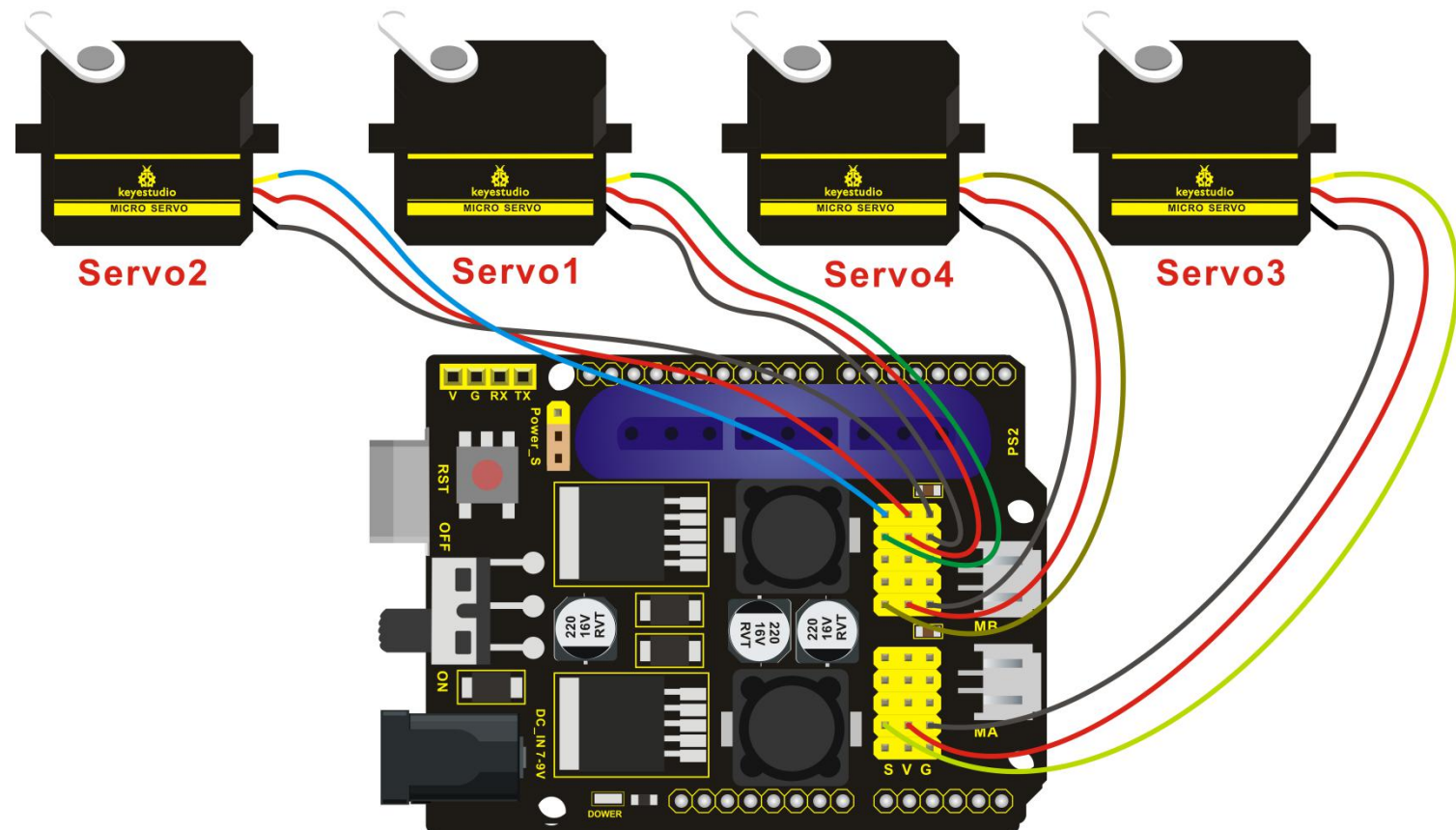
Stack the shield onto UNO R3 and upload the code. Powered on and connected the PS2 Joypad, you can use the PS2 Joypad to control the robot arm actions.

5.3 PS2 Controlling Posture Memory

Description:

In the previous experiment, we have showed how to use Joystick module to control the robot arm memorize several postures. Now we replace the joystick module with PS2 Joypad. The program thought is almost the same.

Hookup Guide:



Test Code 12:

```
#include <PS2X_lib.h>
#include <Servo.h> // add the servo libraries
Servo myservo1; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;

int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture
angle value)

PS2X ps2x; // create PS2 Controller Class

//right now, the library does NOT support hot pluggable controllers, meaning
//you must always either restart your Arduino after you conect the controller,
//or call config_gamepad(pins) again after connecting the controller.
int error = 0;
byte vibrate = 0;

int s1,s2,s3,s4;
int jiyi1[20]; // define four array, separately used to save the angle of 4 servos.
int jiyi2[20];
int jiyi3[20];
```

```
int jiyi4[20];
int i=0;
int j=0;

void setup()
{
  Serial.begin(57600);

  // boot posture
  myservol.write(pos1);
  delay(1000);
  myservo2.write(pos2);
  myservo3.write(pos3);
  myservo4.write(pos4);
  delay(1500);

  error = ps2x.config_gamepad(13,11,10,12);  //setup GamePad(clock, command, attention, data) pins, check for error

  if(error == 0){
    Serial.println("Found Controller, configured successful");
    Serial.println("Try out all the buttons, X will vibrate the controller, faster as you press harder;");
    Serial.println("holding L1 or R1 will print out the analog stick values.");
    Serial.println("Go to www.billporter.info for updates and to report bugs.");
  }
```

```
    else if(error == 1)
        Serial.println("No controller found, check wiring, see readme.txt to enable debug. visit www.billporter.info for troubleshooting tips");

    else if(error == 2)
        Serial.println("Controller found but not accepting commands. see readme.txt to enable debug. Visit www.billporter.info for troubleshooting tips");

    //Serial.print(ps2x.Analog(1), HEX);

    ps2x.enableRumble();           //enable rumble vibration motors
    ps2x.enablePressures();       //enable reading the pressure values from the buttons.

}

void loop()
{
    myservo1.attach(A1); // set the control pin of servo 1 to A1
    myservo2.attach(A0); // set the control pin of servo 2 to A0
    myservo3.attach(6);  // set the control pin of servo 3 to D6
    myservo4.attach(9);  // set the control pin of servo 4 to D9

    if(error != 0)
```

```
return;

ps2x.read_gamepad(false, vibrate);           //read controller and set large motor to spin at 'vibrate' speed

if(ps2x.Button(PSB_START))                   //will be TRUE as long as button is pressed
    Serial.println("Start is being held");
if(ps2x.Button(PSB_SELECT))
    Serial.println("Select is being held");


if(ps2x.Button(PSB_PAD_UP)) {                //will be TRUE as long as button is pressed
    Serial.print("Up held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_UP), DEC);
}
if(ps2x.Button(PSB_PAD_RIGHT)) {
    Serial.print("Right held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_RIGHT), DEC);
}
if(ps2x.Button(PSB_PAD_LEFT)) {
    Serial.print("LEFT held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_LEFT), DEC);
}
if(ps2x.Button(PSB_PAD_DOWN)) {
    Serial.print("DOWN held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_DOWN), DEC);
}
```

```
vibrate = ps2x.Analog(PSAB_BLUE);           //this will set the large motor vibrate speed based on
                                              //how hard you press the blue (X) button

if (ps2x.NewButtonState())                  //will be TRUE if any button changes state (on to off, or off to on)
{
    if(ps2x.Button(PSB_R3))
    {
        //Serial.println("R3 pressed");
        // record
        s1=myservo1.read();
        delay(100);
        Serial.println(s1);
        s2=myservo2.read();
        delay(100);
        Serial.println(s2);
        s3=myservo3.read();
        delay(100);
        Serial.println(s3);
        s4=myservo4.read();
        delay(100);
        Serial.println(s4);

        jiyi1[i]=s1; // save the servo value read in the array sequentially
        jiyi2[i]=s2;
```

```
    jiyi3[i]=s3;
    jiyi4[i]=s4;
    i++;
    j=i;
    // delay(100);
    Serial.println(i);
}

if(ps2x.Button(PSB_L3))
{
    //Serial.println("L3 pressed");
    i=0;
//执行
    pos1 = myservo1.read();
    pos2 = myservo2.read();
    pos3 = myservo3.read();
    pos4 = myservo4.read();

    for(int k=0;k<j;k++) //for loop, to execute all the stored actions
    {
        if(pos1<jiyi1[k]) //if the current servo 1 angle is less than the value stored in array 1.
        {
            while(pos1<jiyi1[k]) //while loop, make servo turn to the position of value stored in the array.
            {
                myservo1.write(pos1); // servo 1 executes the action
                delay(5); // delay 5ms, controlling the rotating speed of servo
```

```
        pos1++;
        //Serial.println(pos1);
    }
}
else //if the current servo 1 angle is greater than the value stored in array 1.
{
    while(pos1>jiyi1[k]) //while loop, make servo turn to the position of value stored in the array.
    {
        myservo1.write(pos1); // servo 1 executes the action
        delay(5); //delay 5ms, controlling the rotating speed of servo
        pos1--;
        //Serial.println(pos1);
    }

//*****
// the same analysis as the previous servo
    if(pos2<jiyi2[k])
    {
        while(pos2<jiyi2[k])
        {
            myservo2.write(pos2);
            delay(5);
            pos2++;
            //Serial.println(pos1);
        }
    }
}
```

```
else
{
  while(pos2>jiyi2[k])
  {
    myservo2.write(pos2);
    delay(5);
    pos2--;
    //Serial.println(pos1);
  }
}
//*****
//the same analysis
if(pos3<jiyi3[k])
{
  while(pos3<jiyi3[k])
  {
    myservo3.write(pos3);
    delay(5);
    pos3++;
    //Serial.println(pos1);
  }
}
else
{
  while(pos3>jiyi3[k])
  {
```

```
        myservo3.write(pos3);
        delay(5);
        pos3--;
        //Serial.println(pos1);
    }
}

//*****
//the same analysis
    if(pos4<jiyi4[k])
    {
        while(pos4<jiyi4[k])
        {
            myservo4.write(pos4);
            delay(5);
            pos4++;
            //Serial.println(pos1);
        }
    }
else
{
    while(pos4>jiyi4[k])
    {
        myservo4.write(pos4);
        delay(5);
        pos4--;
        //Serial.println(pos1);
    }
}
```

```
    }  
  }  
}  
}  
  
if(ps2x.Button(PSB_L2))  
  Serial.println("L2 pressed");  
if(ps2x.Button(PSB_R2))  
  Serial.println("R2 pressed");  
if(ps2x.Button(PSB_GREEN))  
  Serial.println("Triangle pressed");  
  
}  
  
if(ps2x.ButtonPressed(PSB_RED))           //will be TRUE if button was JUST pressed  
  Serial.println("Circle just pressed");  
  
if(ps2x.ButtonReleased(PSB_PINK))          //will be TRUE if button was JUST released  
  Serial.println("Square just released");  
  
if(ps2x.NewButtonState(PSB_BLUE))          //will be TRUE if button was JUST pressed OR released  
  Serial.println("X just changed");
```

```
// turn
zhuandong();
// claw
zhuazi();
// lower arm
dabi();
// upper arm
xiaobi();

if(ps2x.Button(PSB_L1) || ps2x.Button(PSB_R1)) // print stick values if either is TRUE
{
    Serial.print("Stick Values:");
    Serial.print(ps2x.Analog(PSS_LY), DEC); //Left stick, Y axis. Other options: LX, RY, RX
    Serial.print(",");
    Serial.print(ps2x.Analog(PSS_LX), DEC);
    Serial.print(",");
    Serial.print(ps2x.Analog(PSS_RY), DEC);
    Serial.print(",");
    Serial.println(ps2x.Analog(PSS_RX), DEC);
}

delay(5);
}

//*****
// turn
```

```
void zhuandong()
{
//turn right
if(ps2x.Analog (PSS_RX) > 200) // if push the right joystick to the right
{
    //Serial.println(ps2x.Analog(PSS_RX), DEC);
    pos1=pos1-1;    //pos1 subtracts 1
    myservol.write(pos1);    // servo 1 carries out the action and the arm will turn right
    // delay(5);
    if(pos1<1)    // limit the right turning angle
    {
        pos1=1;
    }
}
//左转
if(ps2x.Analog (PSS_RX) < 50)    //if push the right joystick to the left
{
    //Serial.println(ps2x.Analog(PSS_RX), DEC);
    pos1=pos1+1;    //pos1 plus 1
    myservol.write(pos1);    // the arm will turn left
    // delay(5);
    if(pos1>180)    // limit the left turning angle
    {
        pos1=180;
    }
}
```

```
}
//*****
// upper arm
void xiaobi()
{
  //upper arm front
  if(ps2x.Analog(PSS_RY)<50)  // if push the right joystick upward
  {
    pos2=pos2-1;
    myservo2.write(pos2);    // the upper arm will lift up
    delay(5);
    if(pos2<0)    // limit the lifting angle
    {
      pos2=0;
    }
  }
  // upper arm back
  if(ps2x.Analog(PSS_RY)>200)    //if push the right joystick downward
  {
    pos2=pos2+1;
    myservo2.write(pos2);    // the upper arm will go down
    delay(5);
    if(pos2>180)  // limit the declining angle
    {
      pos2=180;
    }
  }
}
```

```
    }  
}  
//*****  
void zhuazi()  
{  
// close the claw  
    if(ps2x.Analog(PSS_LX)>220)    // if push the left joystick to the right  
    {  
        pos4=pos4-1;  
        Serial.println(pos4);  
        myservo4.write(pos4); // servo 4 carries out the action and the claw is gradually closed.  
        delay(5);  
        if(pos4<0)    // if pos4 value reduces to 37 (the claw we test in 37degrees is closed)  
        {  
            pos4=0;  
        }  
    }  
// open the claw  
    if(ps2x.Analog(PSS_LX)<10)    // if push the left joystick to the left  
    {  
        pos4=pos4+8;  
        Serial.println(pos4);  
        myservo4.write(pos4); // servo 4 carries out the action and the claw is gradually opened  
        delay(5);  
        if(pos4>108)    // limit the maximum angle opened  
        {
```

```
        pos4=108;
    }
}

//*****
void dabi()
{
    // lower arm front
    if(ps2x.Analog(PSS_LY)>200)  // if push the left joystick upward
    {
        pos3=pos3+1;
        myservo3.write(pos3);  // the lower arm will stretch out
        delay(5);
        if(pos3>180)  // limit the stretched angle
        {
            pos3=180;
        }
    }
    if(ps2x.Analog(PSS_LY)<10)  // if push the left joystick downward
    {
        pos3=pos3-1;
        myservo3.write(pos3);  //the lower arm will retract
        delay(5);
        if(pos3<35)  // limit the retracted angle
        {
            pos3=35;
        }
    }
}
```

```
}  
}  
}
```

Test Result:

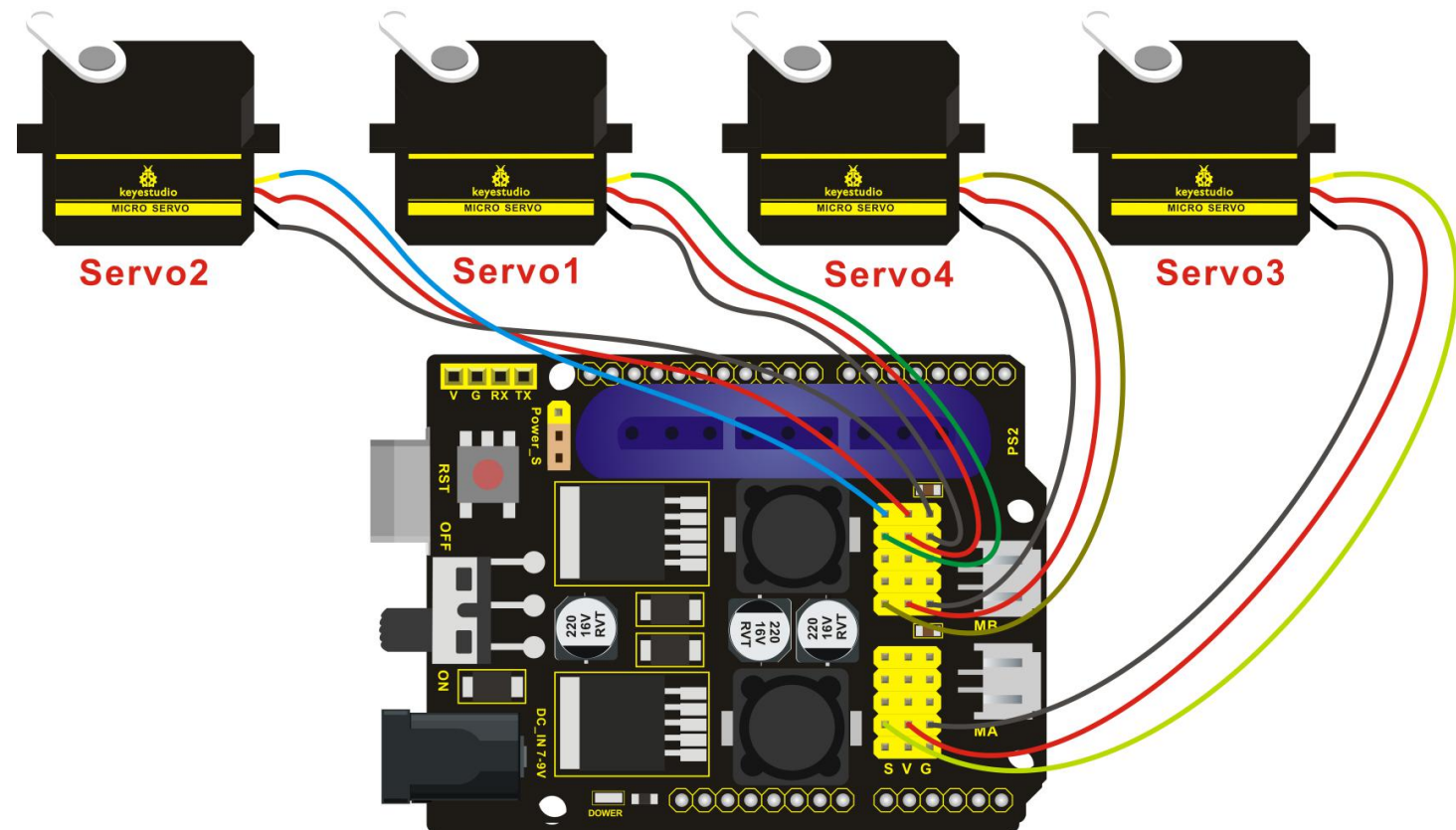
Stack the shield onto UNO R3 and upload the code. Powered on and connected the PS2 Joypad, you can use the PS2 Joypad to control the robot arm memorize several postures.

5.4 PS2 Controlling Posture Memory And Loop

Description:

In the previous experiment, we have showed how to use Joystick module to control the robot arm memorize several postures and loop. Now we replace the Joystick module with the PS2 Joypad. The program is almost the same.

Hookup Guide:



Test Code 13:

```
#include <PS2X_lib.h>

#include <Servo.h> // add the servo libraries
Servo myservo1; // create servo object to control a servo
Servo myservo2;
Servo myservo3;
Servo myservo4;

int pos1=80, pos2=60, pos3=130, pos4=0; // define the variable of 4 servo angle and assign the initial value( that is the boot posture
angle value)

PS2X ps2x; // create PS2 Controller Class

//right now, the library does NOT support hot pluggable controllers, meaning
//you must always either restart your Arduino after you conect the controller,
//or call config_gamepad(pins) again after connecting the controller.
int error = 0;
byte vibrate = 0;

int s1,s2,s3,s4;
int jiyil[30]; //define four array, separately used to save the angle of 4 servos.
```

```
int jiyi2[30];
int jiyi3[30];
int jiyi4[30];
int i=0;
int j=0,tt=0;
void setup()
{
  Serial.begin(57600);

  // boot posture
  myservol.write(pos1);
  delay(1000);
  myservo2.write(pos2);
  myservo3.write(pos3);
  myservo4.write(pos4);
  delay(1500);

  error = ps2x.config_gamepad(13,11,10,12);  //setup GamePad(clock, command, attention, data) pins, check for error

  if(error == 0){
    Serial.println("Found Controller, configured successful");
    Serial.println("Try out all the buttons, X will vibrate the controller, faster as you press harder;");
    Serial.println("holding L1 or R1 will print out the analog stick values.");
    Serial.println("Go to www.billporter.info for updates and to report bugs.");
  }
```

```
    else if(error == 1)
        Serial.println("No controller found, check wiring, see readme.txt to enable debug. visit www.billporter.info for troubleshooting tips");

    else if(error == 2)
        Serial.println("Controller found but not accepting commands. see readme.txt to enable debug. Visit www.billporter.info for troubleshooting tips");

    //Serial.print(ps2x.Analog(1), HEX);

    ps2x.enableRumble();           //enable rumble vibration motors
    ps2x.enablePressures();       //enable reading the pressure values from the buttons.
}

void loop()
{
    myservo1.attach(A1); // set the control pin of servo 1 to A1
    myservo2.attach(A0); //set the control pin of servo 2 to A0
    myservo3.attach(6);  //set the control pin of servo 3 to D6
    myservo4.attach(9);  //set the control pin of servo 4 to D9

    if(error != 0)
        return;
```

```
ps2x.read_gamepad(false, vibrate);           //read controller and set large motor to spin at 'vibrate' speed

if(ps2x.Button(PSB_START))                   //will be TRUE as long as button is pressed
    Serial.println("Start is being held");
if(ps2x.Button(PSB_SELECT))
    Serial.println("Select is being held");


if(ps2x.Button(PSB_PAD_UP)) {                 //will be TRUE as long as button is pressed
    Serial.print("Up held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_UP), DEC);
}
if(ps2x.Button(PSB_PAD_RIGHT)) {
    Serial.print("Right held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_RIGHT), DEC);
}
if(ps2x.Button(PSB_PAD_LEFT)) {
    Serial.print("LEFT held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_LEFT), DEC);
}
if(ps2x.Button(PSB_PAD_DOWN)) {
    Serial.print("DOWN held this hard: ");
    Serial.println(ps2x.Analog(PSAB_PAD_DOWN), DEC);
}
```

```
vibrate = ps2x.Analog(PSAB_BLUE);    //this will set the large motor vibrate speed based on
                                     //how hard you press the blue (X) button

if (ps2x.NewButtonState())           //will be TRUE if any button changes state (on to off, or off to on)
{
    if(ps2x.Button(PSB_R3))
    {
        //Serial.println("R3 pressed");
        //record
        s1=myservo1.read();
        delay(100);
        Serial.println(s1);
        s2=myservo2.read();
        delay(100);
        Serial.println(s2);
        s3=myservo3.read();
        delay(100);
        Serial.println(s3);
        s4=myservo4.read();
        delay(100);
        Serial.println(s4);

        jiyi1[i]=s1; //save the servo value read in the array sequentially
        jiyi2[i]=s2;
        jiyi3[i]=s3;
        jiyi4[i]=s4;
```

```
    i++;
    j=i;
    // delay(100);
    Serial.println(i);
}

// carry out
if(ps2x.Button(PSB_L3))
{
    //Serial.println("L3 pressed");
    i=0;
    tt=1;

    pos1 = myservo1.read(); // record the angle value of 4 servo posture
    pos2 = myservo2.read();
    pos3 = myservo3.read();
    pos4 = myservo4.read();

while(tt==1) // repeat the actions
{
    for(int k=0;k<j;k++) //for loop, to execute all the stored actions.
    {
        if(pos1<jiyi1[k]) // if the current servo 1 angle is less than the value stored in array 1.
        {
            while(pos1<jiyi1[k]) //while loop, make servo turn to the position of value stored in the array.
            {
```

```
    myservo1.write(pos1);    //servo 1 executes the action
    delay(5);               //delay 5ms, controlling the rotating speed of servo.
    pos1++;
    //Serial.println(pos1);
}
}
else    //if the current servo 1 angle is greater than the value stored in array 1.
{
    while(pos1>jiyi1[k])    //while loop, make servo turn to the position of value stored in the array.
    {
        myservo1.write(pos1);    //servo 1 executes the action
        delay(5);               //delay 5ms, controlling the rotating speed of servo.
        pos1--;
        //Serial.println(pos1);
    }
}

//*****
// the same analysis as the previous servo
if(pos2<jiyi2[k])
{
    while(pos2<jiyi2[k])
    {
        myservo2.write(pos2);
        delay(5);
        pos2++;
        //Serial.println(pos1);
    }
}
```

```
    }
  }
  else
  {
    while(pos2>jiyi2[k])
    {
      myservo2.write(pos2);
      delay(5);
      pos2--;
      //Serial.println(pos1);
    }
  }
  //*****
  // the same analysis as the previous servo
  if(pos3<jiyi3[k])
  {
    while(pos3<jiyi3[k])
    {
      myservo3.write(pos3);
      delay(5);
      pos3++;
      //Serial.println(pos1);
    }
  }
  else
  {
```

```
    while(pos3>jiyi3[k])
    {
        myservo3.write(pos3);
        delay(5);
        pos3--;
        //Serial.println(pos1);
    }
}

//*****
// the same analysis as the previous servo
    if(pos4<jiyi4[k])
    {
        while(pos4<jiyi4[k])
        {
            myservo4.write(pos4);
            delay(5);
            pos4++;
            //Serial.println(pos1);
        }
    }
    else
    {
        while(pos4>jiyi4[k])
        {
            myservo4.write(pos4);
            delay(5);
```

```

        pos4--;
        //Serial.println(pos1);
    }
}
}
}
//*****
// exit the looping
ps2x.enableRumble();           //enable rumble vibration motors
ps2x.enablePressures();
ps2x.read_gamepad(false, vibrate);
vibrate = ps2x.Analog(PSAB_BLUE);
if (ps2x.NewButtonState())      //will be TRUE if any button changes state (on to off, or off to on)
{
    if(ps2x.Button(PSB_R3))
    {
        tt=0;
        i=0;
        break;
    }
}
//*****
}
}

if (ps2x.Button(PSB_L2))

```

```
    Serial.println("L2 pressed");
    if(ps2x.Button(PSB_R2))
        Serial.println("R2 pressed");
    if(ps2x.Button(PSB_GREEN))
        Serial.println("Triangle pressed");

}

if(ps2x.ButtonPressed(PSB_RED))           //will be TRUE if button was JUST pressed
    Serial.println("Circle just pressed");

if(ps2x.ButtonReleased(PSB_PINK))          //will be TRUE if button was JUST released
    Serial.println("Square just released");

if(ps2x.NewButtonState(PSB_BLUE))          //will be TRUE if button was JUST pressed OR released
    Serial.println("X just changed");

//turn
zhuandong();
//claw
zhuazi();
//lower arm
dabi();
//upper arm
xiaobi();
```

```
if(ps2x.Button(PSB_L1) || ps2x.Button(PSB_R1)) // print stick values if either is TRUE
{
    Serial.print("Stick Values:");
    Serial.print(ps2x.Analog(PSS_LY), DEC); //Left stick, Y axis. Other options: LX, RY, RX
    Serial.print(",");
    Serial.print(ps2x.Analog(PSS_LX), DEC);
    Serial.print(",");
    Serial.print(ps2x.Analog(PSS_RY), DEC);
    Serial.print(",");
    Serial.println(ps2x.Analog(PSS_RX), DEC);
}

delay(5);
}

//*****
// turn
void zhuandong()
{
    // turn right
    if(ps2x.Analog (PSS_RX) > 200) // if push the right joystick to the right
    {
        //Serial.println(ps2x.Analog(PSS_RX), DEC);
        pos1=pos1-1;    //pos1 subtracts 1
        myservol.write(pos1);    // servo 1 carries out the action, the robot arm turns right.
```

```
// delay(5);
if(pos1<1)  // limit the right turning angle
{
    pos1=1;
}
}
// turn left
if(ps2x.Analog (PSS_RX) < 50)    // if push the right joystick to the left
{
    //Serial.println(ps2x.Analog(PSS_RX), DEC);
    pos1=pos1+1;  //pos1 plus 1
    myservol.write(pos1);    // the robot arm turns left
    // delay(5);
    if(pos1>180)    // limit the left turning angle
    {
        pos1=180;
    }
}
}
//*****
// the upper arm
void xiaobi()
{
    // upper arm front
    if(ps2x.Analog(PSS_RY)<50)  // if push the right joystick upward
    {
```

```
    pos2=pos2-1;
    myservo2.write(pos2);    // the upper arm will lift up
    delay(5);
    if(pos2<0)    // limit the lifting angle
    {
        pos2=0;
    }
}
// upper arm back
if(ps2x.Analog(PSS_RY)>200)    //if push the right joystick to downward
{
    pos2=pos2+1;
    myservo2.write(pos2);    // the robot arm will go down
    delay(5);
    if(pos2>180)    // limit the declining angle
    {
        pos2=180;
    }
}
}
//*****
void zhuazi()
{
    // close the claw
    if(ps2x.Analog(PSS_LX)>220)    // if push the left joystick to the right
    {
```

```
pos4=pos4-1;
Serial.println(pos4);
myservo4.write(pos4); // servo 4 carries out the action and claw is gradually closed
delay(5);
if(pos4<0) // if pos4 value subtracts to 37, the claw in 37 degrees we have tested is closed.)
{
    pos4=0;
}
}
// open the claw
if(ps2x.Analog(PSS_LX)<10) // if push the left joystick to the left
{
    pos4=pos4+8;
    Serial.println(pos4);
    myservo4.write(pos4); // servo 4 carries out the action and claw is gradually opened
    delay(5);
    if(pos4>108) // limit the maximum angle opened
    {
        pos4=108;
    }
}
}
//*****
void dabi()
{
    // lower arm front
```

```
if(ps2x.Analog(PSS_LY)>200) // if push the left joystick upward
{
    pos3=pos3+1;
    myservo3.write(pos3); // the lower arm will stretch out
    delay(5);
    if(pos3>180) // limit the stretched angle
    {
        pos3=180;
    }
}
if(ps2x.Analog(PSS_LY)<10) // if push the left joystick downward
{
    pos3=pos3-1;
    myservo3.write(pos3); // the lower arm will draw back
    delay(5);
    if(pos3<35) // limit the retracted angle
    {
        pos3=35;
    }
}
}
```

Test Result:

Stack the shield onto UNO R3 and upload the code. Powered on and connected the PS2 Joypad, you can use the PS2 Joypad to control the robot arm memorize several postures, looping.

6. More Resources:

You can get more reference from the links below:

KEYESTUDIO WIKI: <http://wiki.keyestudio.com/>

ARDUINO Software: <https://www.arduino.cc/en/Main/OldSoftwareReleases#1.5.x>

Detailed User Guide: <https://drive.google.com/open?id=1R2ZPdKgTu1U1Xq4ZJEyIQewvL4wWueWJ>

Bluetooth APP Download: <https://drive.google.com/open?id=1Ogl37dZywPXqLIN86H3D60R7omOLMky1>

Libraries Download: <https://drive.google.com/open?id=1mkCRozLVgOBwiv85KXTu61Df2Jn7rmM4>

Arduino Project Code: https://drive.google.com/open?id=1iHGFJnnBRoq9ODAcodIXZxtUzLQM_ous

Servo Test Code: <https://drive.google.com/open?id=1qmQ8FsJNvxwqYV4vVaIYT5ytA1TU37Ga>

Assembly Video Link: <http://www.keyestudio.com/wp/ks0198/>

Our Tutorial

This tutorial is designed for everyone to DIY robot arm. You will learn all the basic information about how to control this robot arm with Joystick and Bluetooth. Just enjoy your time!

Great! It's just the beginning of ARDUINO programming journey. There are more and more awesome projects for you to explore. Furthermore, our KEYESTUDIO research and development team will continue to explore on this path, walking you through the basics up to complex projects. Hope that you can enjoy our works!

About keystudio

Located in Shenzhen, the Silicon Valley of China, KEYES DIY ROBOT CO.,LTD is a thriving technology company dedicated to open-source hardware research and development, production and marketing. Keystudio is a best-selling brand owned by KEYES Corporation, our product lines range from Arduino boards, shields, sensor modules, Raspberry Pi, micro:bit extension boards and smart car to complete starter kits designed for customers of any level to learn Arduino knowledge.

All of our products comply with international quality standards and are greatly appreciated in a variety of different markets throughout the world. For more details of our products, you can check it from the links below.

Official Website: <http://www.keyestudio.com/>

US Amazon storefront: <http://www.amazon.com/shops/A26TCVWBQE4D9T>

CA Amazon storefront: <http://www.amazon.ca/shops/A26TCVWBQE4D9T>

UK Amazon storefront: <http://www.amazon.co.uk/shops/A39F7KX4U3W9JH>

DE Amazon storefront: <http://www.amazon.de/shops/A39F7KX4U3W9JH>

FR Amazon storefront: <http://www.amazon.de/shops/A39F7KX4U3W9JH>

ES Amazon storefront: <http://www.amazon.de/shops/A39F7KX4U3W9JH>

IT Amazon storefront: <http://www.amazon.de/shops/A39F7KX4U3W9JH>

US Amazon storefront: <http://www.amazon.com/shops/APU90DTITU5DG>

CA Amazon storefront: <http://www.amazon.ca/shops/APU90DTITU5DG>

JP Amazon storefront: <http://www.amazon.jp/shops/AE9VWCCXQIC6J>

Customer Service

As a continuous and fast growing technology company, we keep striving our best to offer you excellent products and quality service as to meet your expectation. We look forward to hearing from you and any of your critical

comment or suggestion would be much valuable to us.

You can reach out to us by simply drop a line at keyestudio@126.com

Thank you in advance.

